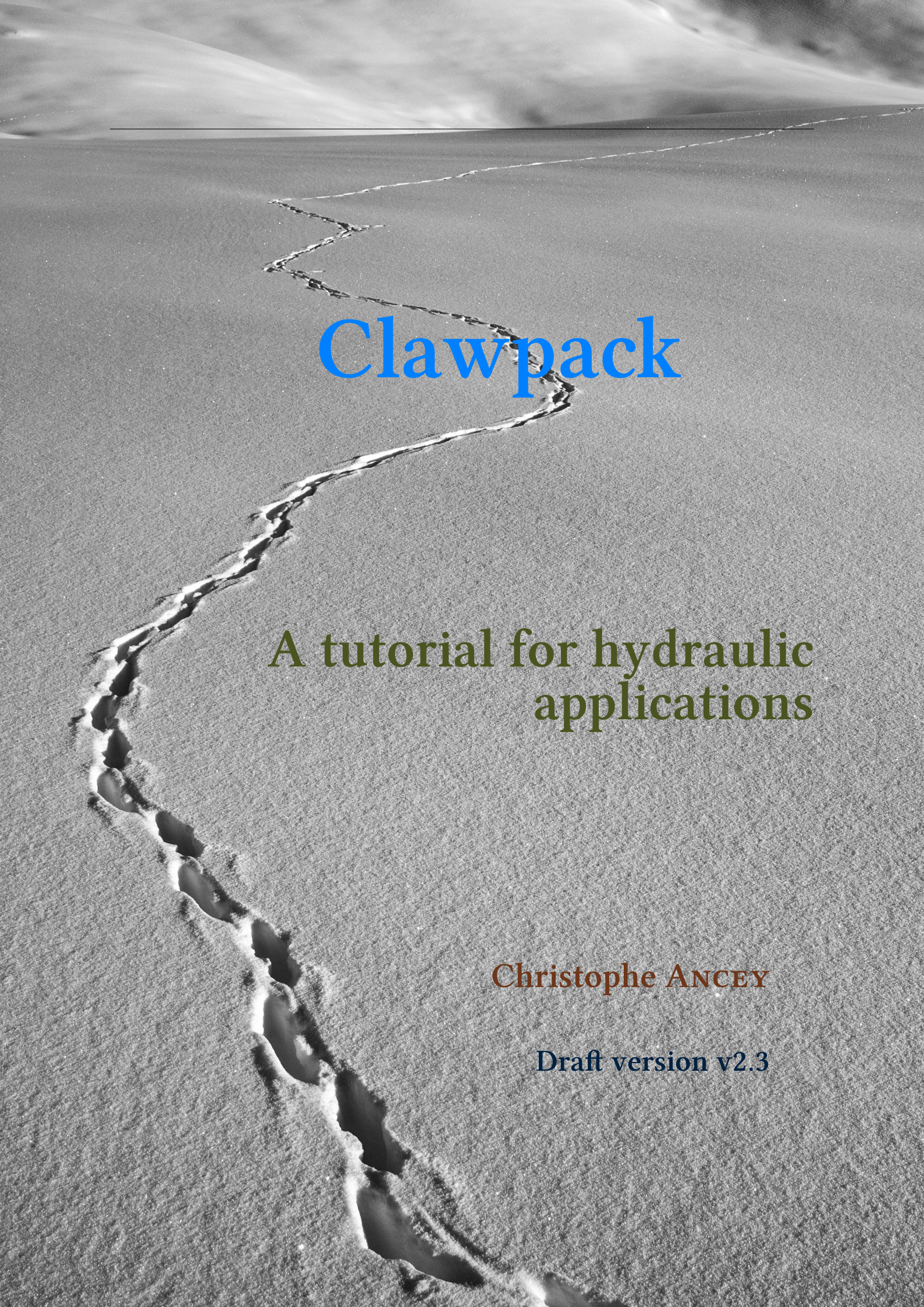




Clawpack

A tutorial for hydraulic
applications

Christophe ANCEY

Draft version v2.3

Christophe ANCEY,
 EPFL, ENAC/IIC/LHE,
 Ecublens, CH-1015 Lausanne, Switzerland
christophe.ancey@epfl.ch, en.ancey.ch, lhe.epfl.ch

Clawpack tutorial / C. ANCEY

version 1.1 from 17 June 2020, Lausanne

version 1.2 from 20 April 2021, Lausanne

version 1.3 from 31 July 2023, Lausanne

version 1.4 from 3 January 2024, Lausanne

version 1.5 from 2 April 2024, Lausanne

version 2.1 from 24 April 2024, Lausanne

version 2.2 from 17 September 2024, Lausanne

version 2.3 from 10 June 2025, Lausanne



Attribution: no commercial use, no modification, 3.0. [Creative Common License 3.0](https://creativecommons.org/licenses/by-nc-nd/3.0/).
 This work is subject to copyright. All rights are reserved; any copy, partial or complete, must be authorized by the author. The typographic management was done using the package *efrench* by Bernard GAULLE (updated and managed today by Raymond JULLIERAT). All pictures are by Christophe ANCEY unless otherwise stated.

Picture credit. **Cover page** : en route vers les monts Tellier (VS). **Chapter 1**: William Turner, *The Falls of the Rhine at Schaffhausen* (Courtauld collection, London). **Chapter 2**: William Turner, *Waves Breaking on a Lee Shore at Margate* (Tate Britain, London). **Chapter 3**: William Turner, *dawn after the wreck*(Courtauld collection, London). **Chapter 4**: William Turner, *Lausanne (cathedral and bridge)* (Tate Britain, London). **Chapter 5**: William Turner, *The fall of anarchy* (Tate Britain, London). **Chapter 6**: William Turner, *The Dogano, San Giorgio, Citella, from the Steps of the Europa hotel* (Tate Britain, London). **Chapter 7**: William Turner, *Sea disaster* (Tate Britain, London). **Chapter 8**: William Turner, *The Parting of Hero and Leander* (National Gallery, London). **Bibliography**: William Turner, *Lausanne (War: The Exile and the Rock Limpet* (Tate Britain, London). **Index**: William Turner, *Lausanne (Two Figures on a Beach with a Boat* (Tate Britain, London).

Table of Contents

Table of Contents

iii

1	Hyperbolic equations	1
1.1	Riemann problems for linear hyperbolic equations	1
1.1.1	Linear system	1
1.1.2	Diagonalisation	1
1.1.3	Mathematical complement	2
1.1.4	Characteristic form and solution to the Cauchy problem for homogenous equations	4
1.1.5	Simple wave	5
1.1.6	Riemann problem: definition	6
1.1.7	Phase plane representation for $m = 2$ equations	7
1.2	Nonlinear scalar problem	8
1.2.1	Characteristic form	9
1.2.2	Rankine–Hugoniot equation	9
1.2.3	Riemann problem	11
1.3	Nonlinear systems	12
1.3.1	Riemann variables	13
1.3.2	Shock wave and the Lax entropy condition	14
1.3.3	Rarefaction wave	15
2	The Saint-Venant equations	19
2.1	Governing equations	19
2.1.1	Tensorial form	19
2.1.2	Characteristic form	20
2.2	Energy balance equation	20
2.2.1	Governing equation	20
2.2.2	Associated Rankine–Hugoniot equation	21
2.2.3	Shock admissibility	22
2.3	Shock wave	22
2.3.1	Hugoniot loci	23
2.3.2	Physical admissibility based on energy balance	24
2.3.3	Physical admissibility based on regularisation	25
2.3.4	Example	28
2.4	Rarefaction wave	29
2.4.1	Riemann invariants	29
2.4.2	Rarefaction wave equation	30

2.4.3	Graphical representation	31
2.5	General solution to the Riemann problem	33
2.5.1	Governing equations	33
2.5.2	Computational strategy	33
2.6	Dam break problem	39
2.6.1	Wet bed	39
2.6.2	Dry bed	44
3	Finite volume methods	47
3.1	General formulation	47
3.1.1	Finite-volume averaging	47
3.1.2	Upwind method	48
3.2	Godunov's method for linear systems	50
3.3	Wave decomposition for linear systems	51
3.3.1	Introductory example	51
3.3.2	General formulation	53
3.3.3	Interface flux	54
3.4	Approximate Riemann solvers for nonlinear problems	57
3.4.1	Scalar problems	57
3.4.2	Systems of equations	60
3.5	Roe solver	62
3.6	Two-wave solver: HLL solver	63
3.7	Alternative: the f-wave method	63
3.8	High-resolutions methods	64
3.8.1	Scalar problems	64
3.8.2	Linear systems	73
3.8.3	Extension to nonlinear equations	76
3.9	Source term	77
3.9.1	Fractional-step method	77
3.9.2	Strang splitting method	78
3.9.3	Stiff source terms and implicit methods	78
3.9.4	Well-balanced algorithms	79
3.10	Boundary conditions	79
3.10.1	Periodic conditions	80
3.10.2	Extrapolation	80
3.10.3	Solid Wall	80
3.10.4	Inflow boundary conditions	83
3.11	Implementation in Clawpack	84
3.11.1	Clawpack installation	84
3.11.2	Legacy Clawpack	86
3.11.3	AMRClaw	88
3.11.4	Pyclaw	88
3.11.5	Development version	90
3.11.6	Implementation of boundary conditions in Clawpack	90
4	Linear advection	93
4.1	Linear advection equation	93
4.1.1	Governing equation and solution	93
4.1.2	Implementation in Clawpack	94
4.1.3	Example	95
4.1.4	Implementation in AMRClaw	97
4.1.5	Implementation in Pyclaw	99

4.2	Acoustic waves	107
4.2.1	Governing equations	107
4.2.2	Implementation	108
4.2.3	Implementation in Pyclaw	109
4.3	Spatially-varying advection equation	117
4.3.1	Theory	117
4.3.2	Implementation in Pyclaw	118
4.3.3	Application	118
4.4	Conservative spatially-varying advection equation	121
4.4.1	Theory	121
4.4.2	Implementation in Clawpack	123
4.4.3	Implementation in Pyclaw	123
4.4.4	Application	125
4.4.5	Alternative	128
5	Burgers' equations	133
5.1	Theory	133
5.2	Approximate solvers	134
5.2.1	Implementation in Clawpack	135
5.2.2	Implementation in Pyclaw	135
5.3	Examples	136
5.3.1	Solution to the Riemann problem	136
5.3.2	Solution to an initial value problem	140
5.4	Nonlinear advection equation with a diffusive term	144
5.4.1	Theoretical considerations	144
5.4.2	Numerical implementation in Clawpack	145
5.4.3	Numerical implementation in Pyclaw	148
5.5	Nonlinear advection equation with a source term and boundary conditions	154
5.5.1	Theoretical considerations	154
5.5.2	Numerical implementation	155
5.6	Nonlinear advection equation with a linear source term	159
5.6.1	Example	159
5.6.2	Numerical implementation	160
5.6.3	Application	160
5.7	Nonlinear advection equation with a nonlinear source term	162
5.7.1	Example	162
5.7.2	Numerical implementation	163
5.7.3	Application	164
6	Solving the Saint-Venant equations	167
6.1	Governing equations	167
6.1.1	Horizontal frictionless bed	167
6.2	Exact solver for the Riemann problem	168
6.2.1	Initial-value problem	168
6.2.2	Examples	168
6.3	Approximate solver: the Roe solver	175
6.3.1	Derivation	175
6.3.2	Wave form	177
6.3.3	Sonic entropy fix	178
6.3.4	Implementation in Clawpack	179
6.3.5	Implementation in Pyclaw	182
6.4	HLLE solver	183

6.4.1	Principle	183
6.4.2	Implementation in Pyclaw	184
6.5	f-wave formulation	186
6.5.1	Principle	186
6.5.2	Implementation in Pyclaw	186
6.6	Example: dam break	189
7	Solving the Saint-Venant equations with scalar transport	191
7.1	Theory	191
7.2	Roe solver	192
7.2.1	Derivation	192
7.2.2	Implementation in Clawpack	193
7.2.3	Implementation in Pyclaw	195
7.3	HLLC Solver	196
7.3.1	Principle	196
7.3.2	Implementation in Pyclaw	198
7.4	F-wave formulation	199
7.4.1	Principle	199
7.4.2	Implementation in Pyclaw	200
7.5	Example: dam break	201
8	Solving the Saint-Venant equations with a source term	203
8.1	Dam-break problem on a sloping bed	203
8.1.1	Governing equations	203
8.1.2	Dimensionless Saint-Venant equations	203
8.1.3	Characteristic form in the fixed frame	204
8.1.4	Change of frame	204
8.1.5	Characteristic form in the moving frame	204
8.1.6	Saint-Venant equations in the hodograph plane	205
8.1.7	Riemann problem	206
8.1.8	Hodograph plane	208
8.1.9	Characteristic diagram	208
8.1.10	Riemann function	210
8.1.11	Solution to the governing equation for t	212
8.2	Coulomb dam-break problem on a horizontal bottom	213
8.2.1	Governing equations	213
8.2.2	Dimensionless Saint-Venant equations	213
8.2.3	Saint-Venant equations in the hodograph plane	214
8.2.4	Riemann problem	214
8.2.5	Riemann function	217
8.2.6	Summary	222
8.3	Flow resistance	225
	Bibliography	227
	Index	231

Foreword

This tutorial is primarily based on the material written by Randall LeVeque and his collaborators.

References

Hyperbolic equation theory is described in a few books, including:

- Numerical Methods for Conservation Laws ([LeVeque, 1992](#))
- Finite Volume Methods for Hyperbolic Problems ([LeVeque, 2002](#))
- Riemann Problems and Jupyter Solutions ([Ketcheson *et al.*, 2020](#))

Some review papers can give an overview of the Clawpack approach ([Berger *et al.*, 2011](#); [Ketcheson *et al.*, 2012, 2013](#); [Mandli *et al.*, 2016](#)).

Online material

Clawpack can be downloaded from its official website www.clawpack.org. It can also be downloaded from github: github.com/clawpack.

The new book based on jupyter notebooks by David Ketcheson *et al.* offers a convenient way of learning Clawpack and the theoretical fundamentals through a series of examples. Many of these examples will be used here. The html version of this book is available [online](#).

Jupyter notebook viewers: cocalc.com and nbviewer.jupyter.org.

Jupyter can be installed under different operating systems (see jupyter.org/install). I personally use the [Anaconda distribution](#) or [code visual studio](#) and their user graphics interface. Registered EPFL students can use jupyter notebooks online by accessing the [noto](#) platform.

I place my jupyter notebooks in my github page: github.com/cancey/.

Additional ressources

There are alternatives to Clawpack.

- Shaltop ([Peruzzetto *et al.*, 2021](#))
- [Basilisk](#)

- [Iber](#) (Bladé *et al.*, 2014; Cea & Bladé, 2015)
- [Basement](#)
- [OpenFoam](#)
- [DassFlow](#)

I tested some of them.

Notation

The notation used in this tutorial differs from that used by Randall LeVeque. I use the classic tensorial notation: vectors and tensors are denoted by boldface symbols. I also use the operator $\cdot$ to refer to the contracted product (“produit une fois contracté” in French) and $:$ for the double-contracted product. The vertical bar in $f(a|b)$ means that f is a function of the variable a and depends on a parameter b that is supposed to be known.

With this notation, we introduce the following operations between tensors A and B whose matrix representation is A_{ij} (i row, j column index) and B_{ij} , respectively, v and w two vectors of coordinates v_i and w_i in a given basis:

$$\begin{aligned} A \cdot B &= \sum_j A_{ij} B_{jk} \\ A : B &= \sum_{i,j} A_{ij} B_{ji} \\ A \cdot v &= \sum_j A_{ij} v_j \\ w \cdot v &= \sum_i w_i v_i \end{aligned}$$

Sometimes indices are in superscript when there is no possible confusion with exponents of a power function and the subscript position is already in use.

Nomenclature

Roman symbols

Variable	Signification
A	Jacobian of F
B	width
C	Chézy coefficient
c	wave speed
E	energy
F	flux function
g	accélération de la gravité
h	flow depth
h_n	normal depth
h_c	critical depth
K	Manning–Strickler’s coefficient
$\dot{m}$	mass flux
n	unit normal
p	pressure
Q	flow rate
q	flow rate per unit with
r	Riemann constant
s	Riemann variable
t	time
u	velocity
$\bar{u}$	depth-averaged velocity
u	velocity
v	velocity in the y -direction
v	velocity
v	left eigenvector of A
w	right eigenvector of A

Greek symbols

Variable	Signification
χ	wetted perimeter
δ	Dirac function
δ	small variation
δ	difference or jump
ϵ	aspect ratio
κ	von Kármán’s constant
λ	eigenvalue
μ	dynamic viscosity
μ	integrating factor
ϱ	density
σ	stress
$\dot{\sigma}$	shock speed
σ	shock position
θ	slope angle
τ	shear stress

Variable	Signification
ξ	similarity variable

Hyperbolic equations

Let us start with linear hyperbolic systems. Nonlinear equations are more complex, but the solutions to the Riemann problem have a similar structure to that exhibited by linear systems. Furthermore, finite-volume numerical solvers involve approximate (linearised) solutions to this Riemann problem.

1.1 Riemann problems for linear hyperbolic equations

1.1.1 Linear system

For one-dimensional problems, a linear hyperbolic equation is defined by an equation of the form

$$\frac{\partial}{\partial t} q + A \cdot \frac{\partial}{\partial x} q = S, \quad (1.1)$$

where q is a vector with m components representing the unknowns q_i , A is an $m \times m$ matrix whose eigenvalues λ_i are assumed to be real and distinct, and S is a vector (of dimension m) called the *source term*, x is the spatial coordinate, and t is time. For the moment, we assume that $S = 0$ (the equation is said to be *homogenous*). The matrix A has m real eigenvalues λ_i , which are associated with m left v_i and m right eigenvectors w_i :

$$A \cdot w_i = \lambda_i w_i \text{ and } v_i \cdot A = \lambda_i v_i. \quad (1.2)$$

In the following, the eigenvalues are ranked in ascending order: $\lambda_1 < \lambda_2 < \dots < \lambda_m$.

1.1.2 Diagonalisation

Equation (1.1) is a system of coupled partial differential equations. It is easier to solve uncoupled equations than coupled equations, and thus our first task is to see how we can uncouple Eq. (1.1).

If we multiply Eq. (1.1) by v_i , we obtain:

$$v_i \cdot \frac{\partial}{\partial t} q + v_i \cdot A \cdot \frac{\partial}{\partial x} q = v_i \cdot S. \quad (1.3)$$

We introduce the *characteristic variable* or *Riemann variable* s_i ($1 \leq i \leq m$):

$$s_i = v_i \cdot q \text{ and the vector } s = (s_1, \dots, s_m), \quad (1.4)$$

and the diagonal matrix $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_m)$. With this notation, we transform Eq. (1.3) into a system of m uncoupled equations:

$$\frac{\partial}{\partial t} s + \Lambda \cdot \frac{\partial}{\partial x} s = L \cdot S, \quad (1.5)$$

where L is a matrix whose rows are made of the left eigenvectors: $L = [v_1, \dots, v_m]^T$.

1.1.3 Mathematical complement

Similarly to what we did with L , we define the matrix R whose columns are made of the right eigenvectors: $R = [w_1, \dots, w_m]$. The following relationships hold true

$$A \cdot R = R \cdot \Lambda, \quad (1.6)$$

$$L \cdot A = \Lambda \cdot L. \quad (1.7)$$

R is sometimes called the *modal matrix*, while Λ is called the *spectral matrix*. We also have:

$$A = R \cdot \Lambda \cdot R^{-1}, \quad (1.8)$$

$$A = L^{-1} \cdot \Lambda \cdot L. \quad (1.9)$$

Because when taking the transpose of $v_i \cdot A = \lambda_i v_i$ we have

$$(v_i \cdot A)^T = A^T \cdot v_i^T = \lambda_i v_i^T, \quad (1.10)$$

the left eigenvectors v_i of A is also the right eigenvector of A^T .

Multiplying Eq. (1.6) by L and Eq. (1.7) by R , we get

$$L \cdot A \cdot R = L \cdot R \cdot \Lambda = \Lambda \cdot L \cdot R. \quad (1.11)$$

When two matrices M and D (where D is diagonal) satisfy $D \cdot M = M \cdot D$, then M is diagonal. This means here that $M = L \cdot R$ is diagonal. There is no unique choice for R and L as any multiple of an eigenvector is also an eigenvector. As a consequence, the product $L \cdot R$ is a diagonal matrix whose entries can take any value.

We can always define the right eigenvectors such that:

$$R = L^{-1}. \quad (1.12)$$

A geometrical interpretation of R and L is the following: as we have $R \cdot L = L^{-1} \cdot L = 1$ (where 1 denotes the identity matrix), then the left and right eigenvectors are orthogonal two by two: $v_i \cdot w_i \neq 0$ and $v_i \cdot w_k = 0$ for $k \neq i$.

In practice, we determine the right eigenvectors w_i . The left eigenvectors are the right eigenvectors of the transpose of A . The resulting matrices R and L satisfy: $R \cdot L^T = \text{diag}(w_k \cdot v_k)_{1 \leq k \leq m}$. Furthermore, by normalising the right eigenvectors ($\tilde{w}_i = w_i/|w_i|$), we can enforce $L = R^{-1}$, a relationship that turns out to be helpful thereafter.

Example Let us consider the 3×3 matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 2 & 8 & 2 \end{pmatrix}.$$

The eigenvalues are $\lambda_1 = -3$, $\lambda_2 = -1$, $\lambda_3 = 12$ associated with the right eigenvectors

$$w_1 = \begin{pmatrix} -3 \\ 0 \\ 2 \end{pmatrix}, w_2 = \begin{pmatrix} -1 \\ -1 \\ 2 \end{pmatrix}, w_3 = \begin{pmatrix} 11 \\ 26 \\ 23 \end{pmatrix},$$

and the left eigenvectors are determined by seeking the right eigenvectors of the transpose of A

$$v_1 = \begin{pmatrix} 5 \\ -3 \\ 1 \end{pmatrix}, v_2 = \begin{pmatrix} 4 \\ -7 \\ 6 \end{pmatrix}, v_3 = \begin{pmatrix} 2 \\ 4 \\ 3 \end{pmatrix}.$$

It can be checked that $A \cdot R - R \cdot \Lambda$ is the null matrix. Note that the order with which the product is made is important. For instance, $A \cdot R - \Lambda \cdot R$ provides a matrix whose diagonal entries are zero, but the off-diagonal entries are nonzero

$$A \cdot R - \Lambda \cdot R = \begin{pmatrix} 0 & 2 & 143 \\ 0 & 0 & 390 \\ -26 & -30 & 0 \end{pmatrix}$$

and similarly $L \cdot A - \Lambda \cdot L$ provides the null matrix.

We can also check that $L \cdot R$ is a diagonal matrix:

$$L \cdot R = \begin{pmatrix} -13 & 0 & 0 \\ 0 & 15 & 0 \\ 0 & 0 & 195 \end{pmatrix}.$$

```

1 #import the numpy libray and its linear algebra sublibrary
2 import numpy as np
3 from numpy import linalg as la
4 eps=1e-8

5 #examples
6 A=np.array([ [1,2,3], [4,5,6], [2,8,2] ])

7 #computing the right and left eigenvalues and vectors
8 # @ refers to the simple contracted product and .T provides the transpose
9 # R: matrix whose columns are right eigenvectors.
10 # L: matrix whose rows are left eigenvectors.
11 # LAMB: diagonal matrix whose entries are the eigenvalues
12 lamb1, R = la.eig(A)
13 lamb2, L = la.eig(A.T)
14 L=L.T
15 print("R = \n",R)
16 print("L = \n",L)
17 LAM = np.diag(lamb1)
18 print("[lambda] = \n",LAM)

R =
[[-3.02079270e-01 -8.32050294e-01 -4.08248290e-01]
 [-7.14005547e-01  2.07143878e-16 -4.08248290e-01]
 [-6.31620292e-01  5.54700196e-01  8.16496581e-01]]
L =
[[-0.37139068 -0.74278135 -0.55708601]
 [-0.84515425  0.50709255 -0.16903085]
 [ 0.39801488 -0.69652603  0.59702231]]
[lambda] =
[[12.  0.  0.]
 [ 0. -1.  0.]
 [ 0.  0. -3.]]

31 #check that w is made by the right eigenvectors. We compute C = A.R-R.LAM.
    We replace small values by 0.
32 # The diagonal of C provides A.w_i-lambda_i w_i
33 # We check that this diagonal is zero
34 C=A@R-R@LAM
35 C[abs(C)<eps]=0
36 print(C)

```

```
[ [0. 0. 0.]
  [0. 0. 0.]
  [0. 0. 0.] ]
```

```
41 #We do the same with thee left eigenvectors and check that C=L.A- LAM.L is
    zero.
42 C=L@A -LAM@L
43 C[abs(C)<eps]=0
44 print(C)
```

```
[ [0. 0. 0.]
  [0. 0. 0.]
  [0. 0. 0.] ]
```

```
49 #Check that the product L.R gives a diagonal matrix
50 lr=L@R
51 lr[lr<eps]=0
52 print(lr)
```

```
[ [0.99440626 0. 0. ]
  [0. 0.6094494 0. ]
  [0. 0. 0.60933335] ]
```

```
56 #left and right eigenvectors are perpendicular when not associated with the
    same eigenvalue
57 w1 = R[:,0] # First column is the first right eigenvector
58 w2 = R[:,1]
59 w3 = R[:,2]
60
61 v1 = L[0,:] # First row is the first left eigenvector
62 v2 = L[1,:]
63 v3 = L[2,:]
64
65 # v_1.A-lambda_1v_1 = 0 ?
66 result=v1@A-lamb[0]*v1
67 result[result<eps]=0
68
69 print("v_1.A-lambda_1*v_1 = ",result)
70 # v_1 and w_1 normal?
71 vw11=v1@w1;
72 vw12=v1@w3;
73 print("v_1.w_1 = ",vw11)
74 print("v_1.w_2 = ",vw12)
```

```
v_1.A-lambda_1*v_1 = [0. 0. 0.]
v_1.w_1 = 0.9944062617398504
v_1.w_2 = 0.0
```

```
[ [0.99440626 0. 0. ]
  [0. 0.6094494 0. ]
  [0. 0. 0.60933335] ]
```

1.1.4 Characteristic form and solution to the Cauchy problem for homogenous equations

When we have seen that when we want to solve a hyperbolic problem in the form (1.1), the strategy is to uncouple the original equations by making a change of variable $q \rightarrow s = L \cdot q$. The original equation

(1.1) becomes

$$\frac{\partial}{\partial t} s + \Lambda \cdot \frac{\partial}{\partial x} s = L \cdot S. \quad (1.13)$$

Each uncoupled equation of the system (1.13) can be put into a characteristic form

$$\frac{\partial s_i}{\partial t} + \lambda_i \frac{\partial s_i}{\partial x} = v_i^T \cdot S \Leftrightarrow \frac{ds_i}{dt} = v_i \cdot S \text{ along the straight line } \frac{dx}{dt} = \lambda_i. \quad (1.14)$$

For a homogenous problem, this means that s_i is constant along the line $x = \lambda_i t + x_0$. If we know the initial value $q_0 = q(x, t = 0)$, then we can deduce the initial condition for s : $s_0 = s(x, t = 0) = L \cdot q_0$. For a homogenous equation, the solution to Eq. (1.14) is

$$s_i(x, t) = s_{i,0}(x - \lambda_i t), \quad (1.15)$$

and thus the solution to the initial-value (Cauchy) problem is

$$q(x, t) = L^{-1} \cdot s = R \cdot s \quad (1.16)$$

$$= \sum_{i=1}^m s_i(x, t) w_i, \quad (1.17)$$

$$= \sum_{i=1}^m s_{0,i}(x - \lambda_i t) w_i, \quad (1.18)$$

$$= \sum_{i=1}^m (v_i \cdot q_0 (x - \lambda_i t)) w_i. \quad (1.19)$$

The solution q is a combination of the right eigenvectors. In other words, the initial conditions propagate along the directions w_i .

This propagation is a consequence of the travelling-wave structure. Indeed, the linear hyperbolic system (1.1) is invariant to the travelling wave group. If we seek a solution in the form $s(x, t) = s(\xi)$ where $\xi = x - at$ and a is the wave velocity, then Eq. (1.1) leads to:

$$-a \frac{d}{d\xi} s + A \cdot \frac{d}{d\xi} s = 0. \quad (1.20)$$

This shows that s' is an eigenvector of A and a must be one of the eigenvalues, say λ_i . Substituting the Cauchy solution Eq. (1.16) into Eq. (1.20) shows that this condition is met. For strictly hyperbolic systems (i.e., when all eigenvalues are real and distinct), the right eigenvectors form a basis, and the decomposition (1.16) is unique.

The solution to the Cauchy problem is the superposition of m waves, each is advected independently at the velocity λ_i along the direction w_i , with no change in shape when the system is homogenous.

1.1.5 Simple wave

When the initial conditions are constant for all but one value k

$$s_{i,0}(x) = s_i \text{ for } i \neq k \text{ and } s_{k,0}(x) = s_{k,0}(x - \lambda_k t), \quad (1.21)$$

then the solution

$$q(x, t) = q_0(x - \lambda_k t) = s_{k,0}(x - \lambda_k t) w_k + \sum_{i \neq k} s_i w_i \quad (1.22)$$

is called a *simple wave*. Propagation concerns the direction k alone.

1.1.6 Riemann problem: definition

A Riemann problem is an initial-value problem for which the initial value is piecewise constant with a single jump discontinuity at some point, by default at $x = 0$:

$$q = \begin{cases} q_l & \text{if } x < 0, \\ q_r & \text{if } x > 0, \end{cases} \quad (1.23)$$

where q_l and q_r are called the left and right states, respectively. Because the right eigenvectors form a basis, we can decompose q_l and q_r in this basis

$$q_l = \sum_{i=1}^m \alpha_i^l w_i \text{ and } q_r = \sum_{i=1}^m \alpha_i^r w_i, \quad (1.24)$$

where α_i^l and α_i^r are two constants. Each Riemann variable $s_i = v \cdot q$ satisfies the initial condition

$$s_{i,0} = \begin{cases} s_i^l = v_i \cdot q_l = \alpha_i^l & \text{if } x < 0, \\ s_i^r = v_i \cdot q_r = \alpha_i^r & \text{if } x > 0. \end{cases} \quad (1.25)$$

The solution to the individual Riemann problem shows that the initial discontinuity propagates with speed λ_i along the direction w_i :

$$s_i(x, t) = \begin{cases} \alpha_i^l & \text{if } x < \lambda_i t, \\ \alpha_i^r & \text{if } x > \lambda_i t. \end{cases} \quad (1.26)$$

Using Eq. (1.16), we deduce that the solution to the Riemann problem is

$$q(x, t) = R \cdot s = \sum_{i=1}^m s_i(x, t) w_i, \quad (1.27)$$

We can interpret this solution in the following way. Let us consider a point P at (x, t) . We refer to I as the maximum index i for which $x > \lambda_i t$. As illustrated by the example of Fig. 1.1, we can decompose the solution into two parts, either reflecting the left or right initial conditions

$$q = \sum_{i=1}^I \alpha_i^r w_i + \sum_{i=I+1}^m \alpha_i^l w_i. \quad (1.28)$$

When crossing the i th characteristic, there is a jump from α_i^l to α_i^r while the other coefficients remain constant. As illustrated in Fig. 1.2(a), the plane is split into different wedges separated by characteristic lines of slope λ_i . Across the i th characteristic, the solution q experiences a jump:

$$\Delta q = (\alpha_i^r - \alpha_i^l) w_i, \quad (1.29)$$

which can be written as

$$\Delta q = \alpha_i w_i \text{ with } \alpha_i = \alpha_i^r - \alpha_i^l. \quad (1.30)$$

For linear hyperbolic systems, a strategy of solving the Riemann problem is to decompose the initial jump $\Delta q = q_r - q_l$ in the right eigenvector basis

$$\Delta q = \sum_{i=1}^m \alpha_i w_i, \quad (1.31)$$

which requires determining the coefficient α_i

$$\Delta q = R \cdot \alpha \Rightarrow \alpha = R^{-1} \cdot \Delta q = L \cdot \Delta q. \quad (1.32)$$

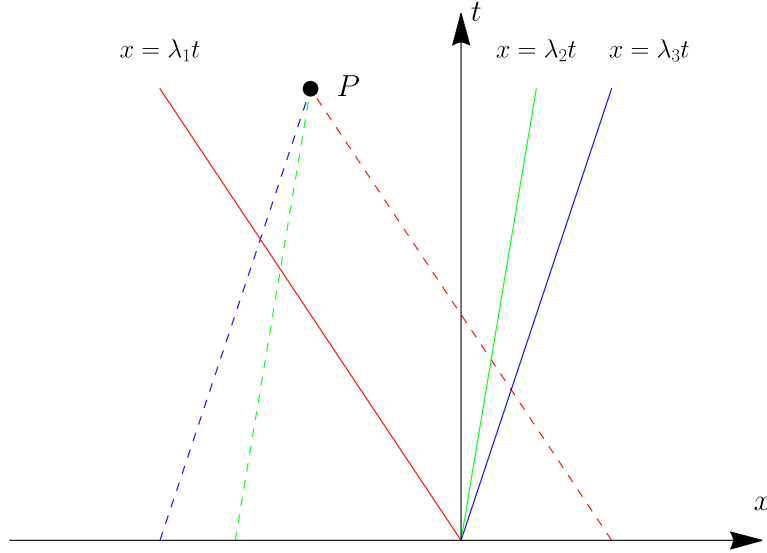


Figure 1.1 Characteristic lines emanating from the origin point (solid lines) and joining P (dashed lines). In this figure, we have $I = 1$: P is on the right of the first characteristic curve $x = \lambda_1 t$ (and thus the P coordinates satisfy $x > \lambda_1 t$), and on the left of the two others. Here we have $q = \alpha_1^r w_1 + \alpha_2^l w_2 + \alpha_3^l w_3$.

As this decomposition is central to Clawpack, we give further information about this decomposition. Let us introduce the wave

$$W_i = \alpha_i w_i. \quad (1.33)$$

The solution to the Riemann problem can thus be written

$$q = \sum_{i=1}^m s_i w_i \text{ with } s_i = v_i \cdot q_0, \quad (1.34)$$

$$q = q_l + \sum_{i=1}^I W_i, \quad (1.35)$$

$$q = q_r - \sum_{i=I+1}^m W_i, \quad (1.36)$$

$$q = q_l + \sum_{i=1}^m H(x - \lambda_i t) W_i, \quad (1.37)$$

where H is the Heaviside function. Equation (1.35) can also be written

$$q = q_l + \sum_{\lambda_i < x/t} W_i, \quad (1.38)$$

which can be interpreted as follows (see the example in Fig. 1.1): at time t and position x , the state q is the left initial state to which contributions from the right initial state are added if this point is on the right of the characteristic $x = \lambda_i t$ (that is, when $x > \lambda_i t$).

1.1.7 Phase plane representation for $m = 2$ equations

For a linear system of two hyperbolic equations, the solution consists of two discontinuities $x = \lambda_1 t$ and $x = \lambda_2 t$, and within the wedge formed by these two discontinuities, there is an intermediate

(constant) state

$$q_* = \alpha_1^r w_1 + \alpha_2^l w_2. \quad (1.39)$$

As shown by Fig. 1.2, the intermediate state can be expressed in different ways:

$$\begin{aligned} q_* &= \alpha_1^r w_1 + \alpha_2^l w_2, \\ &= q_l + (\alpha_1^r - \alpha_1^l) w_1 = q_l + W_1, \\ &= q_r - (\alpha_2^r - \alpha_2^l) w_2 = q_r - W_2. \end{aligned}$$

The intermediate q_* is related to the left state q_l by the shock wave $W_1 = (\alpha_1^r - \alpha_1^l) w_1$, and to the right state q_r by the shock wave $-W_2 = -(\alpha_2^r - \alpha_2^l) w_2$.

We can also plot the solution in the phase plane (q_1, q_2) : starting from the left state q_l , we follow the direction w_1 to reach the intermediate state q_* , and finally the direction w_2 to reach the right state q_r , shown by Fig. 1.2(b). As information is propagated along the characteristic curves at the velocities $\lambda_1 < \lambda_2$, the intermediate state is necessarily connected to the left state by the 1-shock wave (velocity λ_1 and direction w_1), and it is connected to the right state by the 2-shock wave (velocity λ_2 and direction w_2).

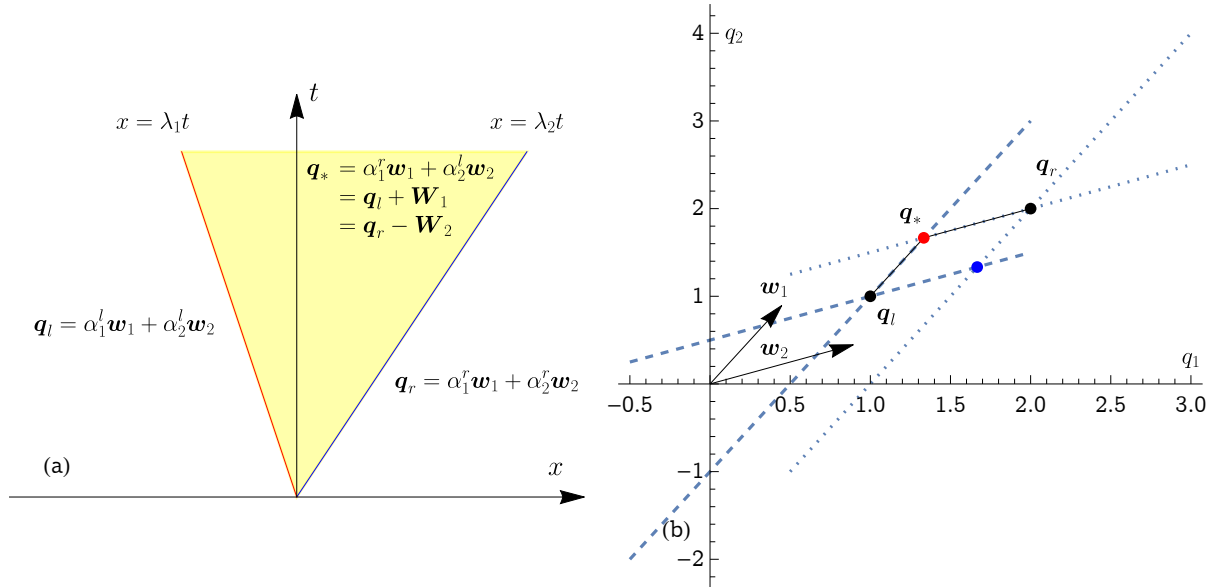


Figure 1.2 (a) General solution to a two-dimensional Riemann problem in the (x, t) plane. The intermediate state q_* is separated from the left and right states by the shocks $W_i = \alpha_i w_i$ with $i = 1$ or 2 . (b) Phase plane representation.

1.2 Nonlinear scalar problem

Let us consider the following nonlinear hyperbolic equation called the *nonlinear advection equation*, which can be cast in two different forms called the conservative (left equation) and non-conservative (right equation) forms:

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = S(q, x, t) \Leftrightarrow \frac{\partial}{\partial t} q + c(q) \frac{\partial}{\partial x} q = S(q, x, t), \quad (1.40)$$

where f is the flux function (a function of q , and possibly of x and t), q is the unknown, S is the source term, and $c = f'(q)$ is the celerity. We assume that the celerity is an increasing function of q , which

implies that the flux function is convex ($f'' > 0$). Nonconvex functions are possible, but they lead to difficulties that we will not address here. The equation is said to be *homogeneous* when the source term s is zero

$$\frac{\partial}{\partial t}q + \frac{\partial}{\partial x}f(q) = 0. \quad (1.41)$$

1.2.1 Characteristic form

Equation (1.40) can be put into the characteristic form

$$\frac{d}{dt}q = S(q, x, t) \text{ along the curve } \frac{dx}{dt} = c(q). \quad (1.42)$$

When the source term is zero ($S = 0$), then q is constant along the characteristic curve, which is therefore a straight line of slope c , whose value is fixed by the initial or boundary condition.

1.2.2 Rankine–Hugoniot equation

As the celerity $c(q)$ is function of q , the characteristic curves are not parallel like in the linear case, and may intersect. As multivalued functions are not possible (this would otherwise break the assumptions of smoothness and uniqueness of the solution), then a shock takes place and connects two continuous branches of the solution. By taking a control volume around the shock position $x = \sigma(t)$, we can deduce that its velocity $\dot{\sigma}$ is given by the Rankine–Hugoniot equation:

$$\dot{\sigma} = \frac{[f(q)]}{[q]}, \quad (1.43)$$

where the double brackets denote the flux jump across the shock wave

$$[f(q)] = \lim_{x \rightarrow \sigma, x > \sigma} f(q) - \lim_{x \rightarrow \sigma, x < \sigma} f(q).$$

Proof. Let us integrate Eq. (1.40) over the interval $[x_l, x_r]$ with a discontinuity at $x = \sigma(t)$ (with $x_l < \sigma < x_r$):

$$\begin{aligned} \int_{x_l}^{x_r} \frac{\partial}{\partial t} q(x, t) dx &= - \int_{x_l}^{x_r} \frac{\partial}{\partial x} f[q(x, t)] dx \\ &= f(q(x_l, t)) - f(q(x_r, t)). \end{aligned}$$

We would like to swap spatial integration and time differentiation while paying attention to the discontinuity at $x = \sigma(t)$. By breaking down the interval $[x_l, x_r]$ into two parts, we can write

$$\frac{d}{dt} \int_{x_l}^{x_r} q(x, t) dx = \frac{d}{dt} \left(\int_{x_l}^{\sigma} q(x, t) dx + \int_{\sigma}^{x_r} q(x, t) dx \right),$$

and by differentiating with respect to t and using Leibniz rule, we deduce:

$$\frac{d}{dt} \int_{x_l}^{x_r} q(x, t) dx = \int_{x_l}^{\sigma} \frac{\partial}{\partial t} q(x, t) dx + \int_{\sigma}^{x_r} \frac{\partial}{\partial t} q(x, t) dx + \dot{\sigma} q(x_r, t) - \dot{\sigma} q(x_l, t).$$

The integral version of Eq. (1.40) is thus

$$-\frac{d}{dt} \int_{x_l}^{x_r} q(x, t) dx + \dot{\sigma} q(x_r, t) - \dot{\sigma} q(x_l, t) = f(q(x_r, t)) - f(q(x_l, t)).$$

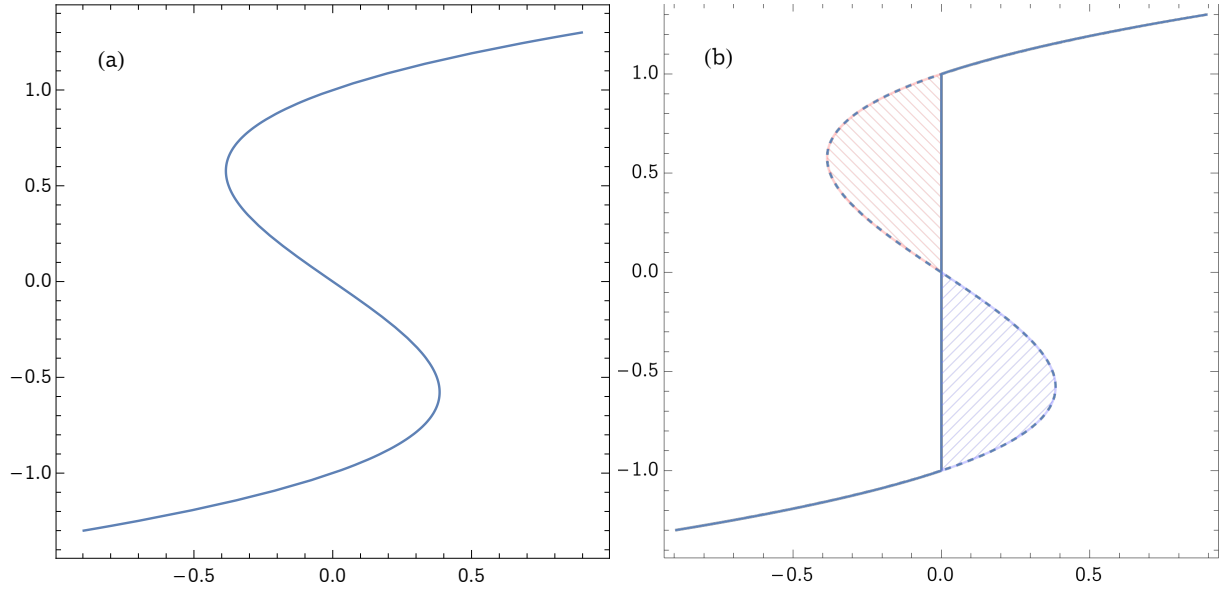


Figure 1.3 (a) Multivalued function. (b) The multivalued part is replaced by a discontinuity. The areas of the two lobes are identical.

Taking the limit $x_r \rightarrow \sigma$ and $x_l \rightarrow \sigma$, we eventually find the desired expression:

$$\dot{\sigma}[[q]] = [[f(q)]].$$

□

One problem with the Rankine–Hugoniot equation (1.43) is that it can provide solutions that are mathematically correct, but physically unrealistic because they violate the principle of energy dissipation (shocks dissipate energy) (Smoller, 1982; Holden & Risebro, 2015). One method for determining whether a solution is physically admissible is to consider a regularised version of the governing equation (1.41)

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = \mu \frac{\partial^2}{\partial x^2} q, \quad (1.44)$$

where μ is a constant viscosity ($\mu > 0$). We would like to solve the regularised equation (1.40) in the limit of small viscosities $\mu \rightarrow 0$. We are seeking a travelling-wave solution in the form

$$q(x, t) = U(\xi) \text{ with } \xi = \frac{x - at}{\mu} \quad (1.45)$$

where $a > 0$ is the wave velocity, and U satisfies the boundary conditions

$$\begin{cases} U \rightarrow q_l \text{ when } \xi \rightarrow -\infty, \\ U \rightarrow q_r \text{ when } \xi \rightarrow +\infty. \end{cases} \quad (1.46)$$

Substituting the travelling wave (1.45) into Eq. (1.44) and making the change of variable

$$\frac{\partial \cdot}{\partial t} = \frac{\partial \cdot}{\partial \xi} \frac{\partial \xi}{\partial t} = -\frac{a}{\mu} \frac{d \cdot}{d \xi} \text{ and } \frac{\partial \cdot}{\partial x} = \frac{\partial \cdot}{\partial \xi} \frac{\partial \xi}{\partial x} = \frac{1}{\mu} \frac{d \cdot}{d \xi},$$

we obtain the following equation

$$\ddot{U}(\xi) = -a \dot{U} + \frac{d}{d \xi} f(U),$$

whose integration provides

$$\dot{U}(\xi) = -aU + f(U) + b,$$

where b is a constant of integration. By looking at the boundary conditions (1.46), we find that this constant satisfies the following condition (1.46)

$$b = aq_r - f(q_r) = aq_l - f(q_l), \quad (1.47)$$

which is possible only if a satisfied

$$a = \frac{f(q_r) - f(q_l)}{q_r - q_l}. \quad (1.48)$$

It can be observed that the wave speed a coincides with the shock speed $\dot{\sigma}$ predicted by Eq. (1.43): $a = \dot{\sigma}$. We thus have to solve the equation

$$\dot{U} = V(U) = -\dot{\sigma}(U - q_l) + f(U) - f(q_l) = -\dot{\sigma}(U - q_r) + f(U) - f(q_r).$$

We note that q_l and q_r are equilibrium points of $V(U)$: $V(q_l) = V(q_r) = 0$. To determine the behaviour of the solution near equilibrium point, we linearise V . We assume that $q_l > q_r$ and we start with V near q_l :

$$\dot{U} = V(U) = -\dot{\sigma}(U - q_l) - f(q_l) + f(q_l) + (U - q_l)f'(q_l) + \dots,$$

and introducing $\eta = U - q_l$, we have to first order in η

$$\frac{d\eta}{d\xi} = (f'(q_l) - \dot{\sigma})\eta,$$

whose solution is $U = q_l + u_0 \exp[(f'(q_l) - \dot{\sigma})\xi]$ where u_0 is a constant of integration. When $\xi \rightarrow -\infty$, the boundary condition (1.46) imposes that $f'(q_l) - \dot{\sigma} > 0$ and thus

$$f'(q_l) > \dot{\sigma}.$$

Reiterating the same procedure for the right state q_r , we obtain that $f'(q_r) < \dot{\sigma}$. We then obtain a relation that is called the *Lax entropy condition*

$$f'(q_r) < \dot{\sigma} < f'(q_l). \quad (1.49)$$

This condition is consistent with the flux convexity ($f'' > 0$ implies that f' is an increasing function). This would have not been the case if we had initially assumed that $q_r > q_l$.

1.2.3 Riemann problem

Let us consider the Riemann problem for a homogeneous hyperbolic equation:

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = 0, \quad (1.50)$$

subject to the initial condition

$$q(x, 0) = q_0(x) = \begin{cases} q_l & \text{if } x < 0, \\ q_r & \text{if } x > 0, \end{cases}$$

where q_l and q_r are constant. When the flux function $f(q)$ is convex, two solutions are possible depending on these constants:

- rarefaction waves,
- shock waves.

Let us start with rarefaction waves. The governing equation (1.50) is invariant to the transformation $x \rightarrow \alpha x$ and $t \rightarrow \alpha t$. A solution can thus be sought in the form $q(\xi)$ with $\xi = x/t$. Substituting this form into Eq. (1.50):

$$(f'(q(\xi)) - \xi) q' = 0.$$

Apart from the trivial solution $q = 0$ (note that it does not satisfy the initial condition), the solution is

$$q(x, t) = f'^{(-1)}(\xi)$$

where $f'^{(-1)}$ is the inverse of f' . The whole solution is the piecewise continuous function:

$$q(x, t) = \begin{cases} q_l & \text{if } \frac{x}{t} \leq f'(q_l), \\ f'^{(-1)}(\xi) & \text{if } f'(q_l) \leq \frac{x}{t} \leq f'(q_r), \\ q_r & \text{if } \frac{x}{t} \geq f'(q_r). \end{cases}$$

Let us now consider a shock wave. Its position is $x = \sigma(t) = \dot{\sigma}t$. The Rankine–Hugoniot equation (1.43) gives $[[f(q)]] = \dot{\sigma}[[q]]$. The whole solution is:

$$q(x, t) = \begin{cases} q_l & \text{if } x < \dot{\sigma}t, \\ q_r & \text{if } x > \dot{\sigma}t. \end{cases}$$

The shock velocity $\dot{\sigma}$ is given by:

$$\dot{\sigma} = \frac{[[f(q)]]}{[[q]]} = \frac{f(q_l) - f(q_r)}{q_l - q_r}.$$

Let us summarise the two possible solutions (see Fig. 1.4): Recall that when $f'' > 0$, the celerity $c(q) = f'(q)$ is an increasing function of q , which is also the slope of the characteristic curves (straight lines):

- $q_r < q_l$. The two families of characteristic curves cross at any time $t > 0$. The shock wave moves at speed $c(q_r) < \dot{\sigma} < c(q_l)$. This condition is called the *Lax condition*, which defines whether a shock is physically admissible.
- $q_r > q_l$, $c(q_r) > c(q_l)$. At time $t = 0$, the two families of characteristic curves fan out. Equation $\xi = f'(U(\xi))$ is an implicit solution over the interval $c(q_r) > \xi > c(q_l)$.

1.3 Nonlinear systems

Let us now consider the nonlinear case for one-dimensional problems in its conservative and non-conservative forms

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = S \Leftrightarrow \frac{\partial}{\partial t} q + A(q) \cdot \frac{\partial}{\partial x} q = S, \quad (1.51)$$

where q is a vector with m components representing the unknowns, f is the flux function, $A = \nabla f$ is its Jacobian (the gradient involves the derivatives with respect to the q components). We assume that A is $m \times m$ matrix whose eigenvalues λ_i are assumed to be real and distinct over a certain domain—like for the linear case.

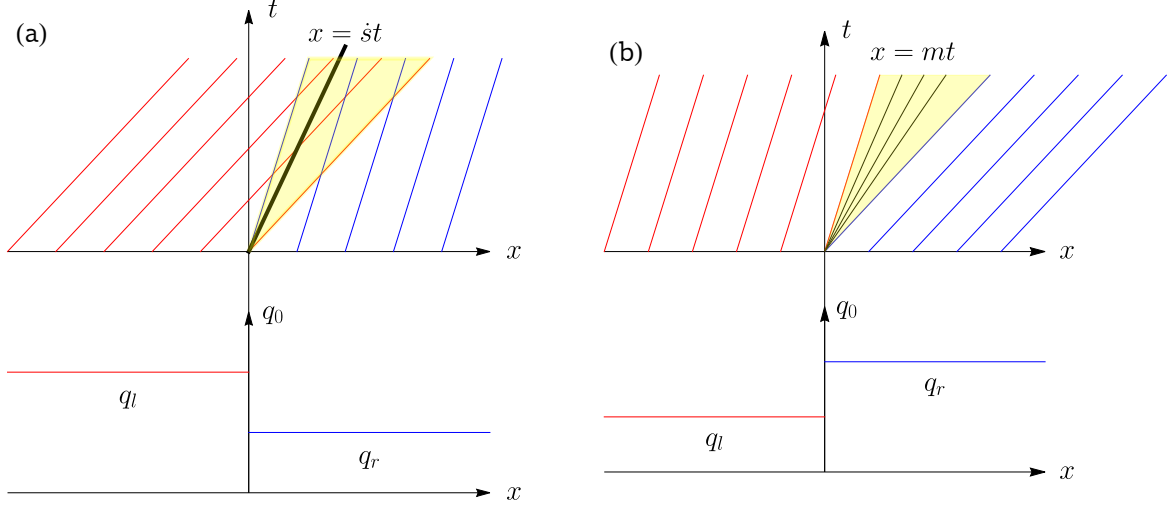


Figure 1.4 (a) When $q_r > q_l$, the solution is a shock wave $x = \dot{s}t$ with $\dot{s} = \llbracket f(q) \rrbracket / \llbracket q \rrbracket$ separating the two constant states q_l and q_r . The coloured area shows the region in which characteristic lines intersect. (b) When $q_r < q_l$, the solution is a rarefaction wave. The characteristic curves are the straight lines $x = mt$ with $c(q_r) > m > c(q_l)$. The coloured area shows the region in which the characteristic curves fan out.

1.3.1 Riemann variables

The computational strategy closely follows the one taken for the linear case. It relies on the concept of differential invariants. Let us illustrate this concept for $m = 2$. The unknown vector q has components (q_1, q_2) . We seek a new variable $s = \{s_1, s_2\}$ such that:

$$v_1 \cdot dq = \mu_1 ds_1,$$

$$v_2 \cdot dq = \mu_2 ds_2,$$

where μ_i are the integrating factors such that ds_i are exact differentials. By expanding the differential ds_1 , we get:

$$\mu_1 ds_1 = \mu_1 \left(\frac{\partial s_1}{\partial q_1} dq_1 + \frac{\partial s_1}{\partial q_2} dq_2 \right) = v_{11} dq_1 + v_{12} dq_2.$$

Upon identification with the former equation, we deduce:

$$\frac{\partial s_1}{\partial q_1} = \frac{v_{11}}{\mu_1},$$

and

$$\frac{\partial s_1}{\partial q_2} = \frac{v_{12}}{\mu_1}.$$

We deduce the governing equations for s_1 and μ_1 . By dividing the two equations above, we obtain:

$$\frac{\partial s_1}{\partial q_1} = \frac{v_{11}}{v_{12}} \frac{\partial s_1}{\partial q_2}, \quad (1.52)$$

while the integrating factor is obtained by applying the Schwarz theorem

$$\frac{\partial}{\partial q_1} \frac{v_{12}}{\mu_1} = \frac{\partial}{\partial q_2} \frac{v_{11}}{\mu_1}.$$

We have seen above that the left and right eigenvectors are orthogonal two by two, which means here that $v_1 \cdot w_2 = 0$ (that is, $v_{12} = w_{21}$ and $-v_{11} = w_{22}$). We can then transform Eq. (1.52) into

$$w_{21} \frac{\partial s_1}{\partial q_1} + w_{22} \frac{\partial s_1}{\partial q_2} = 0 \Rightarrow w_2 \cdot \nabla s_1 = 0. \quad (1.53)$$

Caveat. Later, we will introduce Riemann invariants r_k (see § 1.3.3). These invariants will be defined as $w_k \cdot \nabla r_k = 0$. Although they are closely related to the Riemann variables s_k , they differ from each other: we have $s_1 = r_2$ and $s_2 = r_1$.

Equation (1.52) can be cast in the form

$$\frac{dq_1}{v_{12}} = \frac{dq_2}{v_{11}} = \frac{ds_1}{0},$$

whose integration provides s_1 . Equation (1.51) leads to:

$$v_1 \cdot \frac{dq}{dt} \Big|_{x=X_1(t)} + v_1 \cdot S = 0,$$

where the characteristic curve $x = X_1(t)$ satisfies $dX_1/dt = \lambda_1$. It is called the 1-characteristic. We have:

$$\mu_1 \frac{ds_1}{dt} \Big|_{x=X_1(t)} = v_1 \cdot S.$$

Similarly for s_2 :

$$\mu_2 \frac{ds_2}{dt} \Big|_{x=X_2(t)} = v_2 \cdot S.$$

The compact form of Eq. (1.51) after the change of variable is:

$$\frac{ds}{dt} \Big|_{s=X(t)} = L \cdot S \text{ along } \frac{dX}{dt} = \Lambda, \quad (1.54)$$

where $L = [v_1, v_2]^T$, $\Lambda = \{\lambda_1, \lambda_2\}$ and $s = \{s_1, s_2\}$.

1.3.2 Shock wave and the Lax entropy condition

The Rankine–Hugoniot equation holds for systems of nonlinear hyperbolic equations (1.51). The shock velocity is

$$\dot{\sigma} = \frac{[f(q)]}{[q]}. \quad (1.55)$$

This provides a system of m equations. By eliminating $\dot{\sigma}$, we obtain $m - 1$ equations, and thus there are at least $m - 1$ curves in the phase plane, called the *Hugoniot locus*. These equations may, however, define more than $m - 1$ one-parameter family of curves, as will be illustrated below with the Saint-Venant equations (see Chap. 2).

We have seen in § 1.2.3 that a shock wave occurs when the characteristic curves intersect. In other words, for a right-going shock, the characteristic speed (which is also the eigenvalue) on the left is higher than the one on the right. It can also be shown by considering energy balance or by regularising the homogenous equation that only some characteristic speeds lead to physically admissible solutions. Overall, the constraint on the characteristic speed leads to the *Lax entropy condition*: a shock exists between two states q_l and q_r if there is one index k for which the shock velocity $\dot{\sigma}$ satisfies:

$$\lambda_k(q_l) > \dot{\sigma} > \lambda_k(q_r), \quad (1.56)$$

whereas for the other indices, the characteristic curves do not cross the shock curve:

$$\lambda_i(q_l) < \dot{\sigma} \text{ and } \lambda_i(q_r) < \dot{\sigma} \text{ for } i < k, \quad (1.57)$$

and

$$\lambda_i(q_l) > \dot{\sigma} \text{ and } \lambda_i(q_r) > \dot{\sigma} \text{ for } i > k, \quad (1.58)$$

When working on the Saint-Venant equations (see Chap. 2), we will see how to derive these relations by generalising what has been done with one-dimensional equations in §1.2.3.

If we take the limit of $\llbracket q \rrbracket \rightarrow 0$ around a state q_* , then

$$\lim_{\llbracket q \rrbracket \rightarrow 0} \llbracket f(q) \rrbracket = \nabla f(q_*) \cdot \llbracket q \rrbracket.$$

Making use of the Rankine–Hugoniot equation (1.55), we find that

$$\nabla f(q_*) \cdot \llbracket q \rrbracket = \dot{\sigma} \llbracket q \rrbracket,$$

which shows that shock waves in the close vicinity of q_* are right eigenvectors of the Jacobian $\nabla f(q_*)$. In the next section, we will find that rarefaction waves are tangent to the right eigenvector field. As a consequence, in the close vicinity of q_* , rarefaction and shock waves are tangent to the same integral curves. This remarkable property will be used by approximate Riemann solvers, which substitute the Jacobian $\nabla f(q_*)$ by a constant matrix (see §3.4.2). Figure 2.1 shows an example with the shallow water equations.

1.3.3 Rarefaction wave

Definition of simple waves

Before defining rarefaction waves for nonlinear hyperbolic systems, we need a more general concept that extends the concept of *simple wave* seen in §1.1.5. A simple wave is a special solution to the homogeneous hyperbolic system:

$$\frac{\partial}{\partial t} q + A(q) \cdot \frac{\partial}{\partial x} q = 0, \quad (1.59)$$

where the solution $q(x, t)$ is sought in the form:

$$q(x, t) = q(\xi(x, t)),$$

where $\xi(x, t)$ is a function of x and t . If we substitute $q(x, t)$ with $q(\xi)$ into Eq. (1.59), we then obtain:

$$\frac{\partial \xi}{\partial t} q' + \frac{\partial \xi}{\partial x} A(q) \cdot q' = 0.$$

It is possible to solve this equation if we further assume that a simple wave is the integral curve of one of the right eigenvectors w_k , or in other words, we assume that q' and w_k are collinear:

$$\frac{d}{d\xi} q = \alpha w_k, \quad (1.60)$$

where α is a proportionality factor (possibly a function of ξ). When q is an integral curve of w_k , the function $\xi(x, t)$ satisfies the equation

$$\frac{\partial \xi}{\partial t} + \lambda_k(q) \frac{\partial \xi}{\partial x} = 0. \quad (1.61)$$

Note that this equation is a nonlinear hyperbolic equation and thus may develop discontinuities even though $\xi(x, t)$ is initially continuous.

Note that the characteristic form of Eq. (1.61) is

$$\frac{d\xi}{dt} = 0 \text{ along the curve } \frac{dx}{dt} = \lambda_k(q),$$

which implies that ξ is constant along the characteristic curve $x'(t) = \lambda_k(q(\xi))$, but since the characteristic speed $\lambda_k(q(\xi))$ is also constant, then the characteristic curves are straight lines. Along each of these characteristic curves, $q(\xi)$ is constant. A simple wave can be interpreted as a wave that carries some constant information at a constant velocity.

If we assume that $\lambda_k(q(x, 0))$ is initially monotonically increasing in x at time $t = 0$, the characteristic curves will not cross each other at later times, and thus a continuous solution will exist for all time. In the opposite case, if $\lambda_k(q(x, 0))$ is locally decreasing in x at time $t = 0$, the wave undergoes compression, and thus an initially smooth simple wave may compress to the point of forming a shock in the region where the characteristic speed λ_k decreases. To determine whether the characteristic speed λ_k is a monotonically increasing function, we can compute

$$\frac{d}{d\xi} \lambda_k(q(\xi)) = \nabla \lambda_k \cdot q'. \quad (1.62)$$

Since q' is a right eigenvector of A , q' is collinear with the right eigenvector w_k , and thus the monotonic character is provided by the sign of $\nabla \lambda_k \cdot w_k$. When

$$\nabla \lambda_k \cdot w_k \neq 0 \quad (1.63)$$

the characteristic speed is said to be a *genuinely nonlinear field*. When

$$\nabla \lambda_k \cdot w_k = 0 \quad (1.64)$$

for all q , the field is said to be *linearly degenerate*.

To sum up, we can define simple waves as curves $q(\xi(x, t))$ where q is an integral curve of the right eigenvector w_k . The function ξ must satisfy the nonlinear advection equation (1.61). Characteristic curves in the $x-t$ plane are thus straight lines of slope $\lambda_k(q(\xi))$. A geometric interpretation is that the curve $q(\xi)$ is tangent to the vector field w_k . Note that if we seek a function $r(q)$ that remains constant along this integral curve, then we recover the definition (1.52) of the Riemann invariant:

$$\frac{d}{d\xi} r(q) = 0 \Rightarrow \nabla r \cdot q' = 0,$$

and since $q' = \alpha w_k$, then the invariance condition is:

$$\nabla r \cdot w_k = 0. \quad (1.65)$$

As indicated in § 1.3.1, this property gives an operational way of defining the k -Riemann invariant: it is a function whose gradient is normal to the right eigenvector w_k at each point q . In general, we can find $m - 1$ distinct k -Riemann invariants for each family k : the solution to $w_k \cdot \nabla r_k = 0$ gives m equations, but eliminating ξ reduces by one the number of independent solutions (see the example of the Saint-Venant equations in the next section).

Definition of rarefaction waves

A (centred) rarefaction wave is a particular case of simple wave for which $\xi = x/t$. The interest of this special solution is obvious when considering that the governing equation (1.59) is invariant to the

stretching group $x \rightarrow \alpha x$ and $t \rightarrow \alpha t$. The function ξ is thus the similarity variable $\xi = x/t$. We are seeking a similarity solution $q = q(\xi)$. Substituting this form into Eq. (1.59) gives

$$-\xi q' + A \cdot q' = 0,$$

which shows that q' is a right eigenvector of A , imposing $\xi = \lambda_k$ and q' collinear with the right eigenvector w_k .

The condition $\xi = \lambda_k(q(\xi))$ gives us the possibility to derive a formal definition of the integral curve $q(\xi)$. By differentiating this condition with respect to ξ , we obtain

$$1 = \nabla \lambda_k \cdot q'(\xi)$$

where the gradient $\nabla \lambda_k$ is obtained by differentiating the characteristic speed λ_k with respect to q . Since $q'(\xi)$ is collinear with the right eigenvector w_k , there is a scalar α such that $q' = \alpha w_k$. Substituting this form in the equation above, we obtain

$$1 = \alpha \nabla \lambda_k \cdot w_k,$$

and we eventually deduce

$$\alpha = \frac{1}{\nabla \lambda_k \cdot w_k},$$

and the integral curve thus satisfies the differential equation

$$q'(\xi) = \frac{w_k}{\nabla \lambda_k \cdot w_k}, \quad (1.66)$$

whose solution is denoted by $\tilde{q}$.

The characteristics are straight lines $x = mt$, where m is a real factor satisfying $\xi_l < m < \xi_r$ with $\xi_l \lambda_k(q_l)$ and $\xi_r = \lambda_k(q_r)$. Equation (1.66) holds for $\xi_l < \xi < \xi_r$ and satisfies the boundary conditions

$$q(\xi_l) = q_l \text{ and } q(\xi_r) = q_r.$$

The rarefaction wave's structure is then

$$q(x, t) = \begin{cases} q_l & \text{if } x/t \leq \xi_1, \\ \tilde{q}(x/t) & \text{if } \xi_1 \leq x/t \leq \xi_2, \\ q_r & \text{if } x/t \geq \xi_2, \end{cases} \quad (1.67)$$

where

$$\xi_1 = \lambda_k(q_l) \text{ and } \xi_2 = \lambda_k(q_r).$$

Since a k -rarefaction wave is a simple wave, the k -Riemann invariant given by (1.65) is constant along this wave.

The Saint-Venant equations

2.1 Governing equations

For water waves over horizontal frictionless beds in one space direction (here, the x -direction), the governing equations are given by the Saint-Venant equations (also called shallow water equations) (Saint-Venant, 1871):

$$\frac{\partial h}{\partial t} + \frac{\partial h\bar{u}}{\partial x} = 0, \quad (2.1)$$

$$\frac{\partial h\bar{u}}{\partial t} + \frac{\partial h\bar{u}^2}{\partial x} + gh \frac{\partial h}{\partial x} = 0, \quad (2.2)$$

where h denotes the flow depth, $q = h\bar{u}$ is the water discharge per unit width, g is gravity acceleration, and $\bar{u}$ is the depth-averaged velocity.

2.1.1 Tensorial form

The Saint-Venant equations can be cast in the tensorial form (1.59):

$$\frac{\partial}{\partial t} q + A(q) \cdot \frac{\partial}{\partial x} q = 0, \quad (2.3)$$

with:

$$q = \begin{pmatrix} h \\ q \end{pmatrix} \text{ and } A = f' = \begin{pmatrix} 0 & 1 \\ -q^2/h^2 + gh & 2q/h \end{pmatrix}. \quad (2.4)$$

By integrating $f' = A$, we can determine the flux function f :

$$f = \begin{pmatrix} q \\ \frac{q^2}{h} + \frac{1}{2}gh^2 \end{pmatrix}. \quad (2.5)$$

The eigenvalues of the Jacobian matrix A are:

$$\lambda_1 = \bar{u} - c \text{ and } \lambda_2 = \bar{u} + c, \quad (2.6)$$

where $c = \sqrt{gh}$ and the right eigenvectors are:

$$w_1 = \begin{pmatrix} 1 \\ \bar{u} - c \end{pmatrix} \text{ and } w_2 = \begin{pmatrix} 1 \\ \bar{u} + c \end{pmatrix}. \quad (2.7)$$

2.1.2 Characteristic form

If we define the 1-Riemann variable s_1 as $\nabla s_1 \cdot w_2 = 0$ (see § 1.3.1), then s_1 is the solution to:

$$\frac{\partial s_1}{\partial h} + (\bar{u} + c) \frac{\partial s_1}{\partial q} = 0 \Leftrightarrow \frac{dh}{1} = \frac{dq}{\bar{u} + c} = \frac{ds_1}{0}.$$

Integrating the first pair of equations gives:

$$dq = (\bar{u} + c)dh \Rightarrow \bar{u}dh + h d\bar{u} = (\bar{u} + c)dh.$$

After simplification and integration, we obtain:

$$d\bar{u} = \sqrt{\frac{g}{h}} dh \Rightarrow \bar{u} = 2\sqrt{gh} + a,$$

where a is a constant of integration. As s_1 is an arbitrary function of a , we select the simplest form and thus set $a = s_1 - 2\sqrt{gh}$. The first Riemann variable s_1 is thus defined as:

$$s_1 = \bar{u} - 2\sqrt{gh}. \quad (2.8)$$

Similarly for the 2-variable s_2 , we find:

$$s_2 = \bar{u} + 2\sqrt{gh} \quad (2.9)$$

The Saint-Venant equations (2.1)–(2.2) are thus equivalent to:

$$\begin{cases} \frac{ds_1}{dt} = 0 \text{ along } \frac{dx}{dt} = \lambda_1 = \bar{u} - c, \\ \frac{ds_2}{dt} = 0 \text{ along } \frac{dx}{dt} = \lambda_2 = \bar{u} + c. \end{cases} \quad (2.10)$$

This form is named the *characteristic form*. It has the advantage of uncoupling the shallow water equations. This form has long served as the basis for numerical methods of solving Saint-Venant equations until finite-volume methods began to become widespread in the 1990s (Whitham, 1974; Cunge *et al.*, 1980). It is still of paramount significance to understand how informed is propagated along the characteristic curves.

2.2 Energy balance equation

2.2.1 Governing equation

The *specific energy* per unit mass E (here merely called energy) is the sum of the kinetic energy and hydrostatic pressure in a water column of depth h :

$$E = \frac{1}{2}h\bar{u}^2 + \frac{1}{2}gh^2. \quad (2.11)$$

By multiplying the momentum balance equation (2.2) by the depth-averaged velocity $\bar{u}$, we obtain:

$$\bar{u} \frac{\partial h\bar{u}}{\partial t} + \bar{u} \frac{\partial}{\partial x} \left(h\bar{u}^2 + g \frac{h^2}{2} \right) = 0. \quad (2.12)$$

Using the Leibniz rule (product rule), we have:

$$\bar{u} \frac{\partial h\bar{u}}{\partial t} = \bar{u} \frac{\partial h\bar{u}^2}{\partial t} - h\bar{u} \frac{\partial \bar{u}}{\partial t}.$$

To determine the last term on the right side of the equation above, we use the momentum balance equation (2.2) in non-conservative form:

$$\frac{\partial \bar{u}}{\partial t} + \bar{u} \frac{\partial \bar{u}}{\partial x} + g \frac{\partial h}{\partial x} = 0.$$

Equation (2.12) becomes;

$$\frac{\partial h \bar{u}^2}{\partial t} + \bar{u} \frac{\partial}{\partial x} \left(h \bar{u}^2 + g \frac{h^2}{2} \right) + h \bar{u}^2 \frac{\partial \bar{u}}{\partial x} + g h \bar{u} \frac{\partial h}{\partial x} = 0. \quad (2.13)$$

The idea is to put this equation in conservative form. By rearranging the terms in Eq. (2.13), we obtain:

$$\frac{\partial h \bar{u}^2}{\partial t} + \frac{\partial}{\partial x} (h \bar{u}^3) + 2gh \bar{u} \frac{\partial h}{\partial x} = 0. \quad (2.14)$$

We can express the flow-depth gradient as follows:

$$2gh \bar{u} \frac{\partial h}{\partial x} = 2g \left(\frac{\partial h^2 \bar{u}}{\partial x} - h \frac{\partial h \bar{u}}{\partial x} \right).$$

From the mass balance equation (2.1), we get:

$$h \frac{\partial h \bar{u}}{\partial x} = -\frac{1}{2} \frac{\partial h^2}{\partial t}.$$

Substituting this equation into Eq. (2.14), we eventually arrive at the energy balance equation:

$$\frac{\partial}{\partial t} \left(\frac{1}{2} h \bar{u}^2 + \frac{1}{2} g h^2 \right) + \frac{\partial}{\partial x} \left(\frac{1}{2} h \bar{u}^3 + g h^2 \bar{u} \right) = 0. \quad (2.15)$$

This equation can be cast in the generic form:

$$\frac{\partial E}{\partial t} + \frac{\partial F}{\partial x} = 0.$$

where the flux of E is denoted by F :

$$F = \frac{1}{2} h \bar{u}^3 + g h^2 \bar{u}.$$

2.2.2 Associated Rankine–Hugoniot equation

Equation (2.15) is associated with a Rankine–Hugoniot equation that describes the energy jump across a hydraulic jump:

$$[[F]] - \dot{\sigma} [[E]] = 0. \quad (2.16)$$

There is a consensus to state that energy is not conserved across a discontinuity, but should decrease because of internal energy dissipation (Rayleigh, 1914; Stoker, 1957; Whitham, 1974; Antuono, 2010; Richard & Gavriluk, 2013; Kalisch *et al.*, 2017; Paulsen & Kalisch, 2020). Let us define the energy jump:

$$\Delta E = [[F]] - \dot{\sigma} [[E]]. \quad (2.17)$$

Energy should be dissipated across a jump, which implies the following inequality:

$$\Delta E < 0, \quad (2.18)$$

which can be regarded as the equivalence of the entropy condition in gas dynamics. The difference $[[E]]$ is calculated here by considering that the flow goes from left to right.

The definition of total energy E in Eq. (2.17) can be simplified a great deal. We need to introduce the relative velocity (with respect to the shock speed $\dot{\sigma}$):

$$v_r = u_r - \dot{\sigma} \text{ and } v_l = u_l - \dot{\sigma},$$

and the mass flux across the discontinuity $x = \sigma(t)$:

$$\dot{m} = h_l v_l = h_r v_r \quad (2.19)$$

which is the Rankine–Hugoniot equation (2.24) expressed in the frame attached to the discontinuity (see below). From the Rankine–Hugoniot equation (2.24) associated with momentum balance, we obtain:

$$h_l v_l (v_r - v_l) = -\frac{1}{2} g (h_r^2 - h_l^2), \quad (2.20)$$

and thus by solving Eqs (2.19)–(2.20), we deduce the relationship between relative velocities and flow depths:

$$v_l^2 = \frac{g}{2} \frac{h_r}{h_l} (h_l + h_r) \text{ and } v_r^2 = \frac{g}{2} \frac{h_l}{h_r} (h_l + h_r). \quad (2.21)$$

With these notations, we can express the total energy jump across the discontinuity (2.17):

$$\begin{aligned} \Delta E &= \llbracket F \rrbracket - \dot{\sigma} \llbracket E \rrbracket, \\ &= \left\llbracket \frac{1}{2} h \bar{u}^2 v \right\rrbracket + \left\llbracket g h^2 \bar{u} - \frac{\dot{\sigma}}{2} g h^2 \right\rrbracket, \\ &= \left\llbracket \frac{1}{2} \dot{m} \bar{u}^2 \right\rrbracket + \left\llbracket \frac{1}{2} g h^2 (\bar{u} + v) \right\rrbracket, \\ &= \left\llbracket \frac{1}{2} \dot{m} (v + \dot{\sigma})^2 \right\rrbracket + \left\llbracket \frac{1}{2} g h^2 (2v + \dot{\sigma}) \right\rrbracket, \\ &= \dot{m} \left\llbracket \frac{1}{2} v^2 + g h \right\rrbracket, \end{aligned}$$

which finds an easy interpretation: ΔE is the jump in the quantity $\Psi = \dot{m}(\frac{1}{2}v^2 + gh)$ representing the product of the mass flux $\dot{m}$ and head $\frac{1}{2}v^2 + gh$ (per unit mass). By using the relations (2.20), we eventually find:

$$\Delta E = -\dot{m} g \frac{(h_r - h_l)^3}{4h_l h_r}. \quad (2.22)$$

2.2.3 Shock admissibility

The inequality in Eq. (2.18) implies that a shock is physically admissible if and only if:

$$\Delta E < 0 \Rightarrow \begin{cases} \text{if } \dot{m} > 0, \text{ then } h_r > h_l, \\ \text{if } \dot{m} < 0, \text{ then } h_r < h_l. \end{cases} \quad (2.23)$$

This condition is called the *entropy condition* for the Saint-Venant equations. If this inequality is not satisfied, then the shock is not physically admissible.

2.3 Shock wave

From the Rankine–Hugoniot equation (1.55), we can determine the shock velocity and its features for the Saint-Venant equations (2.1)–(2.2):

$$\dot{\sigma} \begin{bmatrix} h \\ q \end{bmatrix} = \begin{bmatrix} q \\ \frac{q^2}{h} + \frac{1}{2} g h^2 \end{bmatrix}. \quad (2.24)$$

This is a system of two (nonlinear) equations. By solving it for the shock speed $\dot{\sigma}$, we obtain two curves $q(h)$ named the *Hugoniot loci*.

2.3.1 Hugoniot loci

Let us determine the Hugoniot locus, that is, the locus of points that are connected to a given point (h_*, q_*) in the phase plane, and for which the Rankine–Hugoniot equation (2.24) is satisfied. The mass balance equation of Eq. (2.24) leads to:

$$\dot{\sigma} = \frac{q - q_*}{h - h_*}. \quad (2.25)$$

Substituting this relationship into the momentum balance of Eq. (2.24) leads to the equation:

$$q - q_* = \frac{h - h_*}{q - q_*} \left(\frac{q^2}{h} + \frac{1}{2}gh^2 - \frac{q_*^2}{h_*} - \frac{1}{2}gh_*^2 \right).$$

The solution to this quadratic equation is:

$$q(h|h_*, q_*) = \frac{h}{h_*}q_* \pm (h - h_*)\sqrt{\frac{1}{2}g\frac{h}{h_*}(h + h_*)}.$$

This equation defines two curves:

- The 1-shock curve

$$q(h|h_*, q_*) = \frac{h}{h_*}q_* - (h - h_*)\sqrt{\frac{1}{2}g\frac{h}{h_*}(h + h_*)}. \quad (2.26)$$

- The 2-shock curve

$$q(h|h_*, q_*) = \frac{h}{h_*}q_* + (h - h_*)\sqrt{\frac{1}{2}g\frac{h}{h_*}(h + h_*)}. \quad (2.27)$$

From Eq. (2.25), we deduce that the shock moves at the velocity:

$$\dot{\sigma} = u_* \pm \sqrt{gh\frac{h_* + h}{2h_*}}. \quad (2.28)$$

Example. Figure 2.1 shows the two shock curves issuing from point P with coordinate $(h_*, q_*) = (2, 1)$. One remarkable property is that at point P , the shock curves are tangent to the eigenvectors w_i ($i = 1, 2$) of the Jacobian matrix A . Indeed, in the limit $q \rightarrow q_*$, the Rankine–Hugoniot equation (1.55) can be linearised:

$$\dot{\sigma}[[q]] = f'(q_*) \cdot [[q]] \quad (2.29)$$

$$= A(q_*) \cdot [[q]], \quad (2.30)$$

which shows that in this limit, $[[q]]$ is an eigenvector of A , and thus is collinear to either w_1 or w_2 . As a consequence, rarefaction curves are also tangent to the shock curves at point P . This property is used in numerical algorithms (see Chap. 3).

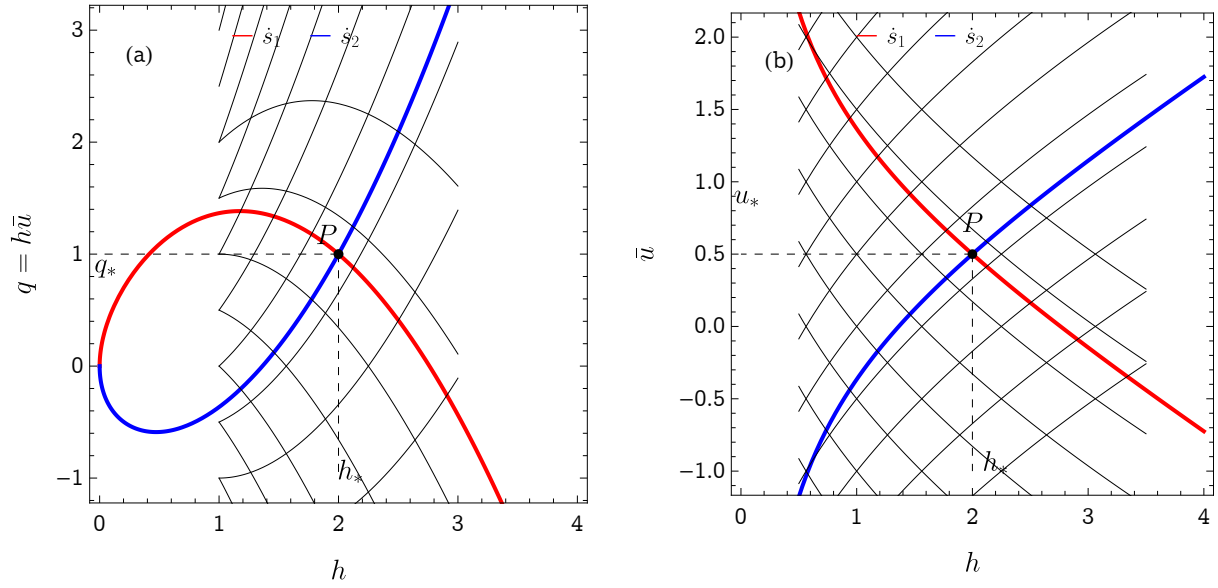


Figure 2.1 Hugoniot locus of all points connected to point P with coordinate $(h_*, q_*) = (2, 1)$ through the 1-shock curve (2.26) or 2-shock curve (2.27). The red curve shows the 1-shock wave (2.26), while the blue curve shows the 2-shock wave (2.27). The thin lines show the contour plot of the Riemann invariants r_1 and r_2 given by Eqs. (2.8) and (2.9), respectively. (a) Curves in the $(h, q = h\bar{u})$ plane. (b) Curves in the $(h, \bar{u})$ plane.

2.3.2 Physical admissibility based on energy balance

Not all points on the shock curves are physically admissible. We have seen that a shock is said to be *physically admissible* when it dissipates energy. If it creates energy, it is not admissible. This condition has been called the *entropy condition*. One way of looking at the physical relevance of the solution is to compute the energy change across the shock. The energy jump is given by Eq. (2.22):

$$\Delta E = \dot{m} \left[\frac{1}{2} \bar{v}^2 + gh \right] = -\dot{m} g \frac{(h_r - h_l)^3}{4h_r h_l},$$

where $v = u - \dot{\sigma}$ is the relative velocity and $\dot{m} = h_l v_l = h_r u_r$ is the mass flux across the discontinuity $x = \sigma(t)$. We have seen that shocks are physically admissible if

$$\Delta E < 0.$$

The sign of ΔE depends on the mass flux $\dot{m}$ and the difference $h_r - h_l$. To determine the sign of $\dot{m}$ depending on the left and right states, we need to do some algebraic manipulations. Let us note that the shock speed (2.25) can be cast in the following form:

$$\begin{aligned} \dot{\sigma} &= \frac{h_r u_r - h_l u_l}{h_r - h_l}, \\ &= \frac{h_r(u_r - u_l) + (h_r - h_l)u_l}{h_r - h_l}, \\ &= \frac{[u]}{[h]} h_r + u_l, \end{aligned}$$

and thus, we deduce:

$$v_l = u_l - \dot{\sigma} = -\frac{[u]}{[h]} h_r.$$

Similarly for the relative velocity v_r , we have:

$$\begin{aligned}\dot{\sigma} &= \frac{u_r(h_r - h_l) + (u_r - u_l)h_l}{h_r - h_l}, \\ &= \frac{[u]}{[h]}h_l + u_r,\end{aligned}$$

and thus, we deduce:

$$v_r = u_r - \dot{\sigma} = -\frac{[u]}{[h]}h_l.$$

As shown by Fig. 2.2, we consider a point P which can be the left or right state. For each possibility, we seek the Hugoniot loci of states that are physically admissible. To that end, we determine the sign of ΔE by looking at the sign of $\dot{m} = h_l v_l$ (or $\dot{m} = h_r v_r$) and that of $h_r - h_l$. For each considered point, we can split the plane into four quadrants, and evaluate the sign of $\dot{m}$ and $h_r - h_l$. The solid lines in Fig. 2.2 show the parts of the 1- and 2-shock curves for which $\Delta E < 0$, whereas the dashed lines show the non-physical parts ($\Delta E > 0$).

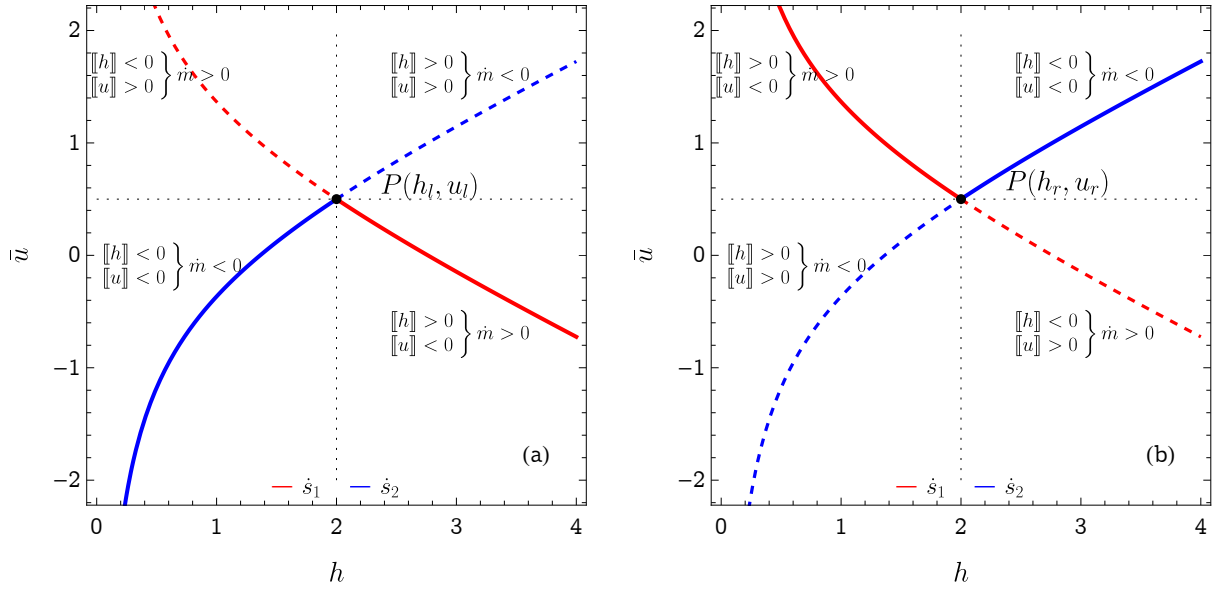


Figure 2.2 Determination of the admissible states. (a) When the left state (h_l, u_l) is fixed, we look for which part of the curve the relationship $\Delta E < 0$ is satisfied. (b) When the right state (h_r, u_r) is fixed, we look for which part of the curve the relationship $\Delta E < 0$ is satisfied. The red curve shows the 1-shock wave (2.26), while the blue curve shows the 2-shock wave (2.27).

2.3.3 Physical admissibility based on regularisation

Like for one-dimensional problems seen in § 1.2.2, we can determine whether a shock is physically admissible by assuming that this shock is the idealisation of a wave¹ with sharp but smooth variations in q . To that end, we consider the regularised version of the Saint-Venant equation by adding a viscous term:

$$\frac{\partial}{\partial t}q + \frac{\partial}{\partial x}f(q) = \mu \frac{\partial^2}{\partial x^2}q \quad (2.31)$$

¹This wave is sometimes referred to as a *monoclinical wave* (Whitham, 1974; Ferrick, 2005; Fowler, 2011).

where μ is the viscosity ($\mu > 0$ and we will take $\mu \rightarrow 0$) and

$$q = \begin{pmatrix} h \\ q \end{pmatrix} \text{ and } f = \begin{pmatrix} q \\ \frac{q^2}{h} + \frac{1}{2}gh^2 \end{pmatrix}.$$

We are seeking a travelling-wave solution the regularised Saint-Venant equations (2.31):

$$q(x, t) = U(\xi) \text{ with } \xi = \frac{x - \dot{\sigma}t}{\mu} \quad (2.32)$$

where $\dot{\sigma}$ is the wave velocity given by the Rankine–Hugoniot equation (2.24), and U satisfies the boundary conditions:

$$\begin{cases} U \rightarrow q_l \text{ when } \xi \rightarrow -\infty, \\ U \rightarrow q_r \text{ when } \xi \rightarrow +\infty. \end{cases} \quad (2.33)$$

Substituting the form (2.32) into the regularised Saint-Venant equations (2.31) gives:

$$-\dot{\sigma} \frac{d}{d\xi} U + \frac{d}{d\xi} f(U) = \frac{d^2}{d\xi^2} U,$$

which, after integration and determining the constant of integration from the boundary conditions (2.33), yields:

$$\frac{d}{d\xi} U = V(U) = -\dot{\sigma}(U - q_l) + f(U) - f(q_l) \quad (2.34)$$

$$= -\dot{\sigma}(U - q_r) + f(U) - f(q_r). \quad (2.35)$$

We note that q_l and q_r are equilibrium points of $V(U)$. To determine the behaviour near these points, we linearise $V(U)$. For instance, near q_l , we have to first order:

$$V(U) = (-\dot{\sigma}1 + A) \cdot (U - q_l) = B \cdot (U - q_l)$$

where the matrix B (evaluated at q_l) is:

$$B = A - \dot{\sigma}1 = \begin{bmatrix} -\dot{\sigma} & 1 \\ gh - \frac{q^2}{h^2} & 2\frac{q}{h} - \dot{\sigma} \end{bmatrix}.$$

This matrix has the following eigenvalues: $\eta_i = -\dot{\sigma} + \bar{u}_l \pm \sqrt{gh_l}$ or, written differently, $\eta_i = -\dot{\sigma} + \lambda_i(q_l)$. Similarly, near q_r , we have to first order:

$$V(U) = (-\dot{\sigma}1 + A) \cdot (U - q_r) = B \cdot (U - q_r)$$

where the matrix B is now evaluated at q_r and has the eigenvalues $\eta_i = -\dot{\sigma} + \lambda_i(q_r)$.

We will show that travelling wave solutions to the regularised Saint-Venant equations exist only if we constrain the eigenvalues η_i of B . Here we provide a simple proof based on the expected behaviour of the solutions in the phase plane; the reader is referred to technical books for more formal proofs (Smoller, 1982; Holden & Risebro, 2015).

Let us first consider the 1-shock curve connecting the left (L) and right (R) states and associated with the eigenvalue $\eta_1 = -\dot{\sigma} + \lambda_1$. For $\xi \rightarrow -\infty$, the integral curve of Eq. (2.32) starts from L. When seeking local solutions to Eq. (2.32) near L in the form $U = q_l + U_0 e^{\eta_1 \xi}$ (with U_0 the initial condition which must be collinear to the eigenvector of $B(q_l)$ associated with the eigenvalue η_1), then the condition $\eta_1 > 0$ at L must be met for the left boundary condition (2.33) to be satisfied. As $\eta_2 > \eta_1$, then we deduce that the second eigenvalue is also positive at L: $\eta_2 > 0$. We then infer the conditions:

$$\dot{\sigma} < \lambda_1(q_l) \text{ and } \dot{\sigma} < \lambda_2(q_l) \quad (2.36)$$

As the two eigenvalues η_1 and η_2 are positive, the left state L is a node (source). For the same reason, when looking at the local behaviour near R by seeking solutions in the $U = q_r + U_0 e^{\eta_1 \xi}$ when $\xi \rightarrow \infty$, then we must have $\eta_1 < 0$ at R, and thus:

$$\dot{\sigma} < \lambda_1(q_l).$$

The right state R must be a saddle point to prevent an integral path issuing from L by (being tangent to the second eigenvector at L) from reaching R. This implies that the second eigenvalue must be positive at R. In the end, we thus have the conditions:

$$\dot{\sigma} > \lambda_1(q_r) \text{ and } \dot{\sigma} < \lambda_2(q_r) \quad (2.37)$$

Figure 2.3(a) shows a simplified phase plane where the integral path (also called here a *heteroclinic orbit*) connects the source point L to the saddle point R.

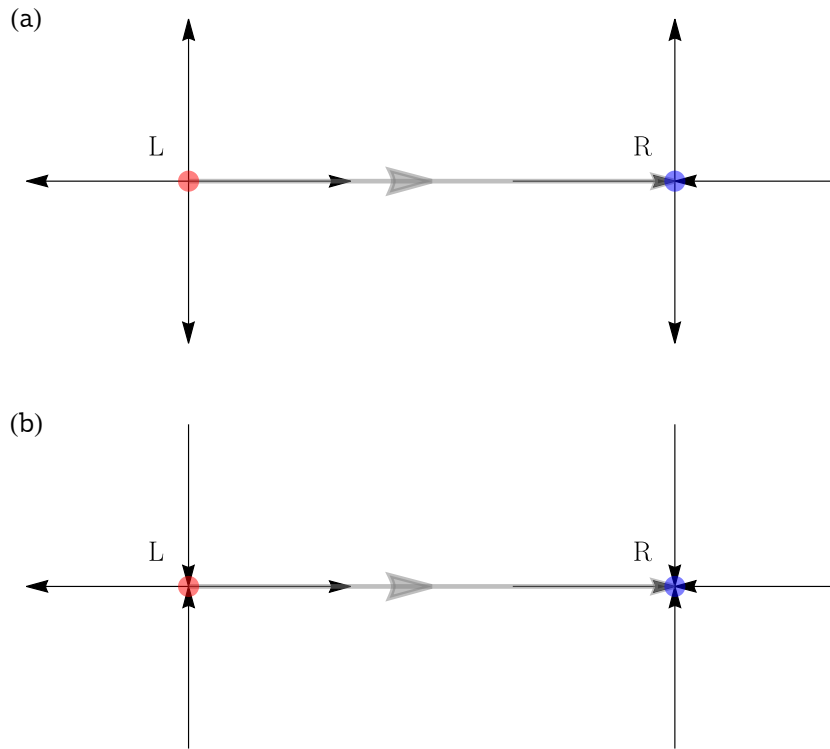


Figure 2.3 Sketch of the phase plane and the behaviour near the left (L) and right (R) states. (a) When considering the 1-shock curve, the left state is a node (source) and the right state is a saddle point. (b) When considering the 2-shock curve, the left state is a saddle point and the right state is a node (sink). The grey line shows the heteroclinic orbit connecting the left and right states.

Recombining Eqs. (2.36) and (2.37), we deduce that a travelling wave (mimicking the 1-shock wave) exists only when the Lax condition is satisfied:

$$\lambda_1(q_r) < \dot{\sigma} < \lambda_1(q_l), \quad (2.38)$$

$$\dot{\sigma} < \lambda_2(q_l) \text{ and } \dot{\sigma} < \lambda_2(q_r). \quad (2.39)$$

We could reiterate the reasoning for the 2-shock curve. In that case, we obtain that the left state L must be saddle point whereas the right state R must be a node (sink). A travelling wave (mimicking

the 2-shock wave) exists only when the Lax condition is satisfied:

$$\lambda_2(q_r) < \dot{\sigma} < \lambda_2(q_l), \quad (2.40)$$

$$\dot{\sigma} > \lambda_1(q_l) \text{ and } \dot{\sigma} > \lambda_1(q_r). \quad (2.41)$$

Figure 2.3(b) shows the simplified phase plane for the 2-shock curve.

2.3.4 Example

Figure 2.4 shows an example of shocks between two states $q_l = (h_l, q_l) = (2, 1)$ and $q_r = (h_r, q_r) = (2, -0.436)$. The 1-characteristics $x = x_0 + \lambda_1 t$ intersect each other, and thus there is a 1-shock curve separating the two states. The characteristics are parallel straight lines in each wedge where q is constant.

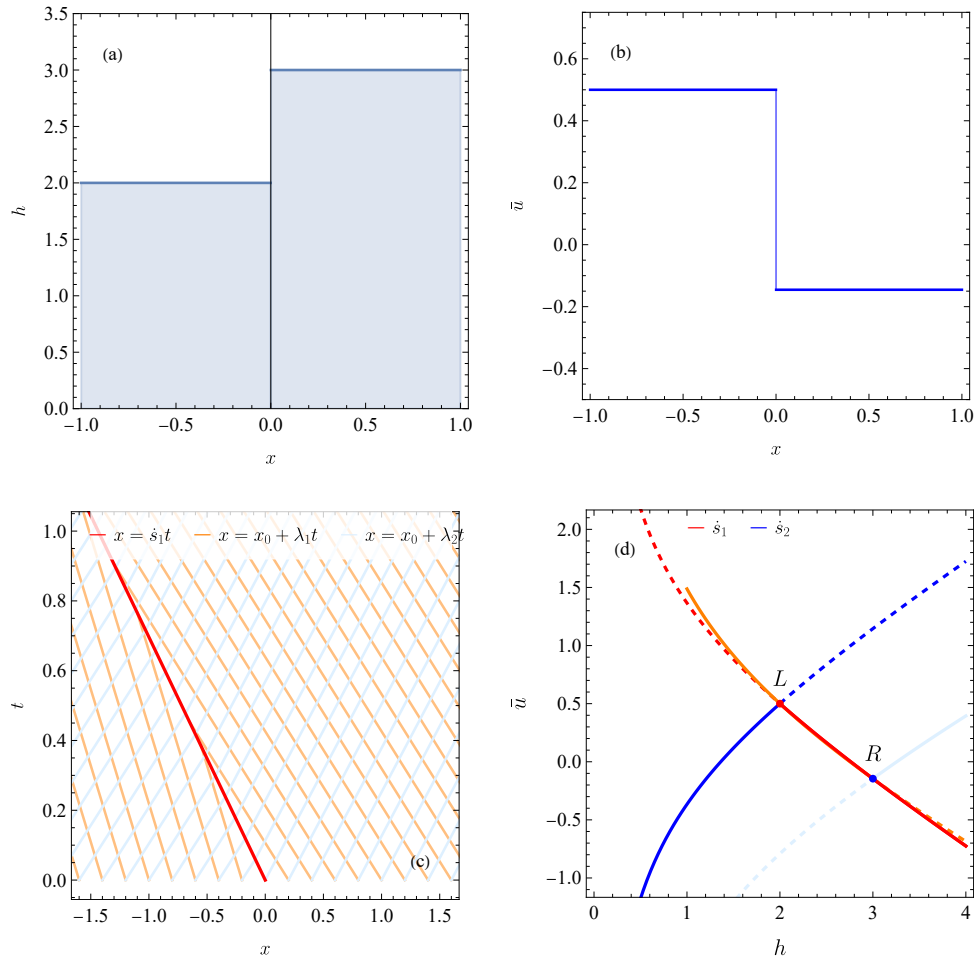


Figure 2.4 Example of Riemann problem with a 1-shock curve as the solution. Left state: $q_l = (h_l, q_l) = (2, 1)$. Right state: $q_r = (h_r, q_r) = (2, -0.436)$. (a) Initial depth profile. (b) Initial velocity profile. (c) Characteristic diagram. (d) Phase plane.

2.4 Rarefaction wave

Let us recall that a rarefaction wave is a special case of simple waves, which are defined as integral curves of right eigenvectors. There are thus as many rarefaction waves as right eigenvectors, and these waves are labelled as k -rarefaction waves when they are associated with the right eigenvector w_k . Here are their main features:

- A k -rarefaction wave is the similarity solution $q(\xi)$ (with $\xi = x/t$) to the governing equation (1.59) (see § 1.3.3).
- The characteristic curves are straight lines $\xi = x/t = m$ where the slope m ranges from $\lambda(q_l)$ to $\lambda(q_r)$.
- The k -Riemann variable r_k defined by Eq. (1.65) is constant along the k -rarefaction wave.

2.4.1 Riemann invariants

The k -Riemann variable r_k is defined by Eq. (1.65) ($\nabla r_k \cdot w_k = 0$). We will show here that it can also be calculated by using the properties of simple waves.

Let us start with the integral curve of an eigenvector w_1 . Integrating Eq. (1.60) with the definition of the right eigenvector w_1 given by Eq. (2.7) gives:

$$\frac{d}{d\xi} \begin{pmatrix} h \\ q \end{pmatrix} = \begin{pmatrix} 1 \\ q/h - \sqrt{gh} \end{pmatrix}. \quad (2.42)$$

The first equation tells us that $h' = 1$, and thus we obtain $h = \xi + a$. We assume that $a = 0$ such that ξ and h coincide. The second equation then becomes:

$$\frac{dq}{dh} = q/h - \sqrt{gh}. \quad (2.43)$$

Let us assume that at $h = h_*$, $q = q_* = h_* \bar{u}_*$. Integrating this equation gives:

$$q(h) = h \left(\frac{q_*}{h_*} + 2\sqrt{gh_*} - 2\sqrt{gh} \right) = h\bar{u}_* + 2h(\sqrt{gh_*} - \sqrt{gh}). \quad (2.44)$$

Expressed in terms of velocity, this equation is equivalent to:

$$\bar{u}(h) + 2\sqrt{gh} = \bar{u}_* + 2\sqrt{gh_*},$$

and we then deduce that the quantity:

$$r_1 = \bar{u} + 2\sqrt{gh} \quad (2.45)$$

is constant along the 1-rarefaction wave. This is the *1-Riemann invariant*.

Similarly, we can calculate the 2-rarefaction wave associated with w_2 , and we obtain:

$$q(h) = h \left(\frac{q_*}{h_*} + 2\sqrt{gh_*} - 2\sqrt{gh} \right) = h\bar{u}_* + 2h(-\sqrt{gh_*} + \sqrt{gh}). \quad (2.46)$$

The 2-Riemann invariant is:

$$r_2 = \bar{u} - 2\sqrt{gh}. \quad (2.47)$$

Notes:

1. The equations used for characterising the simple waves are not sufficient to determine the structure of rarefaction waves. Indeed, the k -rarefaction wave is defined from Eq. (1.66) as $q' = \alpha w_k$ where $\alpha = (\nabla \lambda_k \cdot w_k)^{-1}$, but in the calculations just above, we integrated $q' = \alpha w_k$ with $\alpha = 1$. This assumption simplified the calculation of r_k but did not allow us to derive the rarefaction wave's exact structure.
2. Recall that the 1-Riemann variable $s_1 = \bar{u} - 2\sqrt{gh}$ used in the characteristic form (2.10) of the Saint-Venant equations is the 2-Riemann invariant $r_2 = \bar{u} - 2\sqrt{gh}$ (as indicated in § 1.3.1). Similarly, the 2-Riemann variable s_2 matches the 1-Riemann invariant:

$$s_1 = r_2 \text{ and } s_2 = r_1.$$

2.4.2 Rarefaction wave equation

To compute the rarefaction wave's exact structure, we use the formal definition (1.66) of the rarefaction wave given in § 1.3.3:

$$q'(\xi) = \begin{pmatrix} h' \\ q' \end{pmatrix} = \frac{w_1}{\nabla \lambda_1 \cdot w_1} = -\frac{2}{3} \sqrt{\frac{h}{g}} \begin{pmatrix} 1 \\ \bar{u} - c \end{pmatrix} \quad (2.48)$$

since

$$\nabla \lambda_k \cdot w_1 = \begin{pmatrix} -\bar{u}/h - c/2 \\ 1/h \end{pmatrix} \cdot \begin{pmatrix} 1 \\ \bar{u} - c \end{pmatrix} = -\frac{3}{2} \sqrt{\frac{g}{h}} = -\frac{3}{2} \frac{c}{h}.$$

As $h > 0$, the scalar product $\nabla \lambda_k \cdot w_1$ is always negative, and the 1-characteristic is said to be a *genuinely nonlinear field*. The first equation can be readily integrated:

$$h' = -\frac{2}{3} \sqrt{\frac{h}{g}} \Rightarrow h(\xi) = \frac{(3a - 2\xi)^2}{36g}, \quad (2.49)$$

where a is a constant of integration. Since the boundary condition imposes $h = h_l$ at $\xi = \lambda_l = u_l - \sqrt{gh_l}$, we have:

$$3a = 2u_l + 4\sqrt{gh_l},$$

and similarly for the right boundary condition, we have:

$$3a = 2u_r + 4\sqrt{gh_r},$$

which is possible only if:

$$u_r + 2\sqrt{gh_r} = u_l + 2\sqrt{gh_l}, \quad (2.50)$$

or in other words, if the Riemann invariant r_1 is constant. Using this result in Eq. (2.49), we finally obtain:

$$h(\xi) = \frac{(u_r + 2\sqrt{gh_r} - 2\xi)^2}{9g} = \frac{(u_l + 2\sqrt{gh_l} - 2\xi)^2}{9g} \quad (2.51)$$

The second equation is:

$$q' = -\frac{2}{3}(\bar{u} - c)\frac{h}{c}.$$

Formally, we could solve this equation by substituting h with the solution $h(\xi)$ found previously. This leads to intricate calculations. Note first that q can be determined directly without solving any equation by recalling that the 1-Riemann invariant is constant, and thus $q/h + 2\sqrt{gh}$ is a constant. Otherwise,

we can integrate the equation above by making a change of variable $q(\xi) \rightarrow q(h)$ and making use of Eq. (2.49):

$$q' = \frac{dq}{d\xi} = \frac{dq}{dh} \frac{dh}{d\xi} = -\frac{2}{3} \sqrt{\frac{h}{g}} \frac{dq}{dh},$$

and thus

$$\frac{dq}{dh} = \frac{q}{h} - \sqrt{gh} \Rightarrow q(h) = h(a - 2\sqrt{gh}),$$

where a is a constant of integration. The boundary conditions impose that:

$$a = u_l + 2\sqrt{gh_l} = u_r + 2\sqrt{gh_r}.$$

This leads to the desired result.

2.4.3 Graphical representation

Figure 2.5(a) shows the phase portrait of Eq. (2.43) and one integral curve (2.44) (in red). The black curves show the vector field w_1 , and the red curve shows the integral curve related to a 1-rarefaction wave (integral curve of w_1) associated with the initial condition $q_*(h_*) = 1$ for $h_* = 1$. The integral curve related to the 1-rarefaction wave is the contour line (or level curve) of r_1 : we have $r_1 = \bar{u}_* + 2\sqrt{gh_*} = 3$ (when we take $g = 1 \text{ m}\cdot\text{s}^{-2}$).

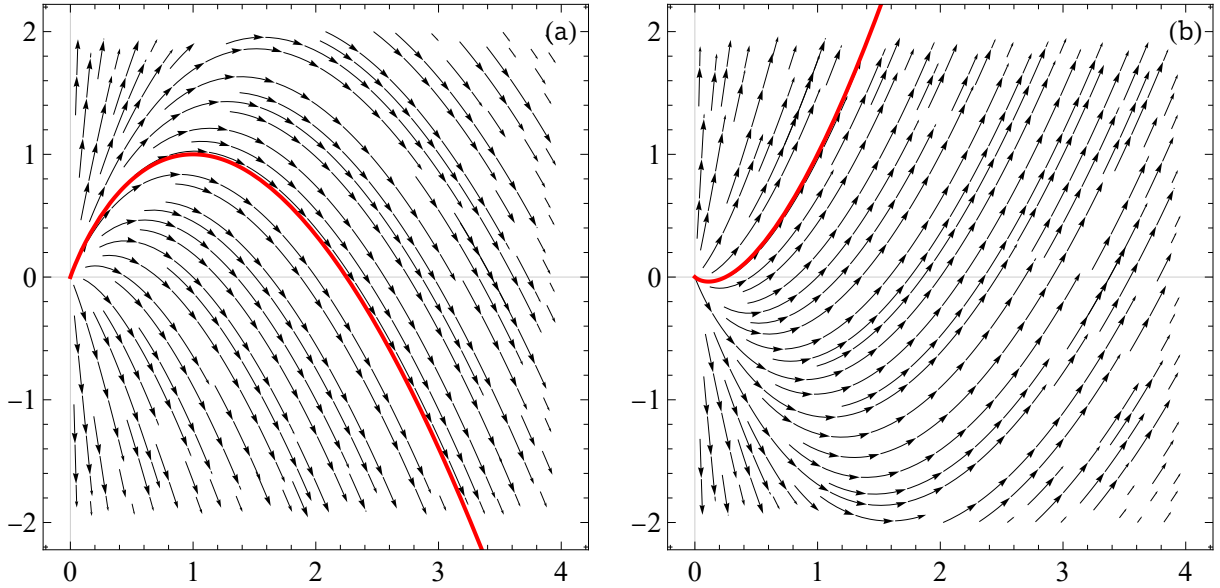


Figure 2.5 Phase portrait (a) associated with w_1 ($q'(h) = q/h - \sqrt{gh}$). (b) associated with w_2 ($q'(h) = q/h + \sqrt{gh}$). The red curves show the integral curve (a) $q(h) = h\bar{u}_* + 2h(\sqrt{gh_*} - \sqrt{gh})$ (1-rarefaction wave) with $h_* = 1$ and $q_* = 1$, and (b) $h\bar{u}_* + 2h(\sqrt{gh_*} + \sqrt{gh})$ (2-rarefaction wave). We take $g = 1 \text{ m}\cdot\text{s}^{-2}$.

Figure 2.6(a) shows one integral curve (related to $r_1 = 2$) and the contour lines of the eigenvalues. As the field is genuinely nonlinear, the characteristic speed λ_1 varies monotonically along any integral curve. If we take the decreasing part of the integral path (red curve), we see that the eigenvalue λ_1 decreases when one goes from left to right. This has important consequences to determine which part of the rarefaction waves is physically admissible. Let us assume that we start from a left state $q_l = (h_l, q_l)$

and we seek where a point $q_* = (h_*, q_*)$ connected to the left state lies on the rarefaction wave. As $\lambda_1(q_l) < \lambda_1(q_*)$, the physically admissible part of the integral path is the one along which λ_1 increases.

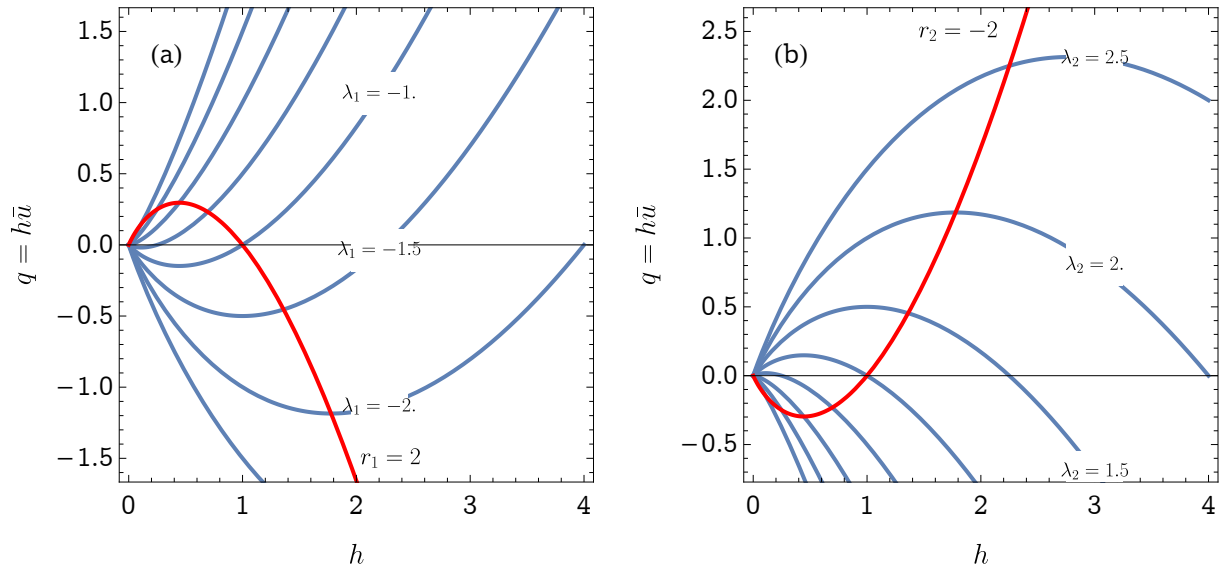


Figure 2.6 Contour lines of the eigenvalues λ and rarefaction waves $r = cst$. (a) $\lambda = -2.5, 1, 0.5$. (b) $\lambda = -1, 2.5, 0.5$. The red curve shows the integral curves related to 1-rarefaction and 2-rarefaction waves (here $r_1 = 2$ and $r_2 = -2$) whereas the blue curves show the contour lines for the eigenvalues λ_1 and λ_2 .

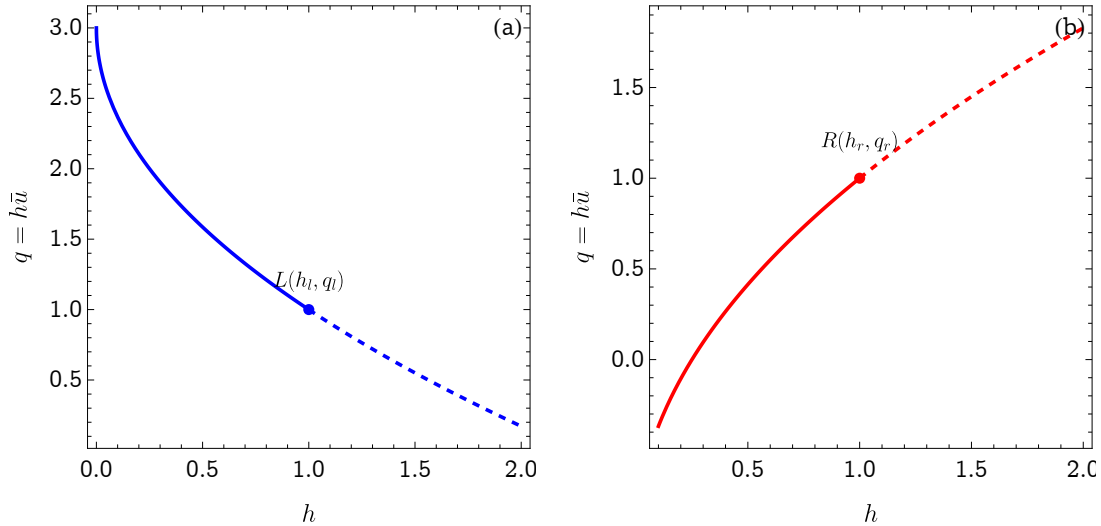


Figure 2.7 (a) Integral curve related to the 1-rarefaction wave issuing from the left state. (a) Integral curve related to the 2-rarefaction wave connecting to the right state. The solid lines show the physically admissible states, whereas the dashed lines show the part of the rarefaction wave that is not physically admissible

2.5 General solution to the Riemann problem

2.5.1 Governing equations

We are now armed to work out the general solution to the Riemann problem for the Saint-Venant equations:

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = 0 \Leftrightarrow \frac{\partial}{\partial t} q + A(q) \cdot \frac{\partial}{\partial x} q = 0, \quad (2.52)$$

where the unknown vector and the Jacobian matrix A are:

$$q = \begin{pmatrix} h \\ q \end{pmatrix} \text{ and } A = f' = \begin{pmatrix} 0 & 1 \\ -q^2/h^2 + gh & 2q/h \end{pmatrix}, \quad (2.53)$$

and the flux function f is:

$$f = \begin{pmatrix} q \\ \frac{q^2}{h} + \frac{1}{2}gh^2 \end{pmatrix}. \quad (2.54)$$

For a Riemann problem, the initial condition takes the form:

$$q(x, 0) = \begin{cases} q_l & \text{for } x < 0, \\ q_r & \text{for } x > 0, \end{cases} \quad (2.55)$$

where q_l and q_r are the left and right states, respectively.

Unless the left and right states can be connected by a single rarefaction or shock curve, the solution usually involves a combination of two waves, usually a rarefaction wave and a shock wave, but sometimes, depending on the particular values of q_l and q_r , two shocks or two rarefaction waves. One of these waves connects the left state q_l with an intermediate state q_* , while the other connects q_* to q_r . Note that possibly, the intermediate state corresponds to the dry state $h = 0$. The problem thus lies in the calculation of this intermediate state q_* and the type of connecting waves.

2.5.2 Computational strategy

Intermediate state equation

We have seen that:

- Shocks have to satisfy the Lax entropy condition (1.56)–(1.58), which imposes that a 1-shock wave connecting the left and right states has to satisfy:

$$\lambda_1(q_r) < \dot{\sigma} < \lambda_1(q_l), \quad (2.56)$$

$$\dot{\sigma} < \lambda_2(q_l) \text{ and } \dot{\sigma} < \lambda_2(q_r). \quad (2.57)$$

The Lax condition for a 2-shock wave is:

$$\lambda_2(q_r) < \dot{\sigma} < \lambda_2(q_l), \quad (2.58)$$

$$\dot{\sigma} > \lambda_1(q_l) \text{ and } \dot{\sigma} > \lambda_1(q_r). \quad (2.59)$$

- If there is an intermediate state q_* , the wave from q_l to q_* move more slowly than the wave from q_* to q_r . As a consequence, the wave issuing from the left state q_l is necessarily a 1-wave, whereas the wave issuing from the intermediate state q_* is a 2-wave.

Let us now use this information to calculate the features of the 1-shock wave connecting q_l to q_* . Equation (2.56) imposes:

$$\bar{u}_* - \sqrt{gh_*} < \dot{\sigma} < \bar{u}_l - \sqrt{gh_l},$$

and recalling that the shock velocity can also be expressed [see Eq. (2.28)] in the form:

$$\dot{\sigma} = u_* - \sqrt{gh_l \frac{h_* + h_l}{2h_*}} = u_l - \sqrt{gh_* \frac{h_* + h_l}{2h_l}}.$$

we find that the Lax entropy condition implies:

$$h_* > h_l \quad (2.60)$$

for the 1-shock wave. By using the same procedure for the 2-shock wave, we deduce that the Lax entropy condition for this wave is:

$$h_r > h_* \quad (2.61)$$

These conditions are consistent with the criteria established in § 2.4 (see also Fig. 2.7).

We define two functions ϕ_l and ϕ_r :

$$\phi_l(h) = \begin{cases} u_l + 2(\sqrt{gh_l} - \sqrt{gh}) & \text{if } h < h_l \\ u_l - (h - h_l) \sqrt{\frac{g}{2} \left(\frac{1}{h_l} + \frac{1}{h} \right)} & \text{if } h > h_l \end{cases} \quad (2.62)$$

and

$$\phi_r(h) = \begin{cases} u_r - 2(\sqrt{gh_r} - \sqrt{gh}) & \text{if } h < h_r \\ u_r + (h - h_r) \sqrt{\frac{g}{2} \left(\frac{1}{h_r} + \frac{1}{h} \right)} & \text{if } h > h_r \end{cases} \quad (2.63)$$

The intermediate state is found by solving:

$$\phi_l(h_*) = \phi_r(h_*). \quad (2.64)$$

Given that we know the depth h , the function ϕ_l yields the value of the velocity u such that the state (h, hu) is connected to the left state $(h_l, h_l u_l)$ by a 1-wave (rarefaction or shock), whereas ϕ_r provides the value of the velocity u such that the state (h, hu) is connected to the right state $(h_r, h_r u_r)$ by a 2-wave (rarefaction or shock).

Intermediate state's existence and uniqueness

To study the existence and uniqueness of the solution to Eq. (2.64), let us follow [Toro \(2001\)](#) and separate the variables u and h by introducing the functions f_l :

$$f_l(h|h_l) = \begin{cases} 2(\sqrt{gh} - \sqrt{gh_l}) & \text{if } h \leq h_l, \\ (h - h_l) \sqrt{\frac{g}{2} \left(\frac{1}{h_l} + \frac{1}{h} \right)} & \text{if } h > h_l, \end{cases} \quad (2.65)$$

and f_r

$$f_r(h|h_r) = \begin{cases} 2(\sqrt{gh} - \sqrt{gh_r}) & \text{if } h \leq h_r, \\ (h - h_r) \sqrt{\frac{g}{2} \left(\frac{1}{h_r} + \frac{1}{h} \right)} & \text{if } h > h_r, \end{cases} \quad (2.66)$$

such that:

$$\phi_l(h|h_l, u_l) = u_l - f_l(h|h_l) \text{ and } \phi_r(h|h_r, u_r) = u_r + f_r(h|h_r).$$

Solving Eq. (2.64) is thus equivalent to solving:

$$f(h) = f_l(h) + f_r(h) + \Delta u = 0 \text{ with } \Delta u = u_r - u_l. \quad (2.67)$$

Although the function f is made of three branches, it is concave, continuous and continuously differentiable. If it exists, the solution $f(h_*) = 0$ is thus unique. Figure 2.8 shows an example of f variations with h .

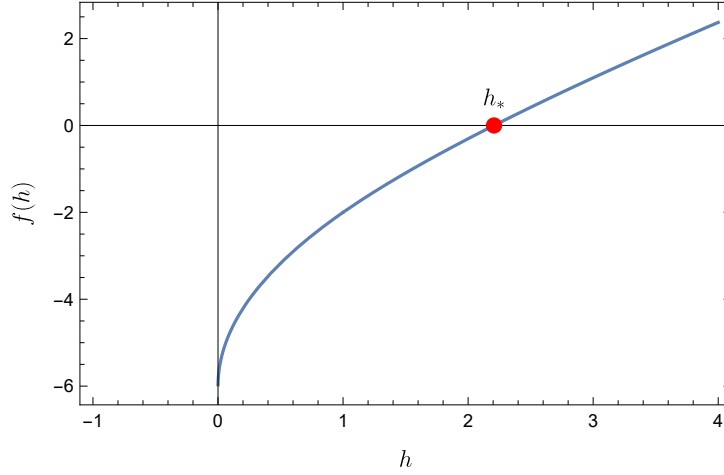


Figure 2.8 Variation in f with h . Parameters: $q_l = (4, 0)$, $q_r = (1, 0)$, $g = 1 \text{ m}^2/\text{s}$. The solution to $f(h) = 0$ provides $h_* = 2.206 \text{ m}$.

Wave structure

We must have $h \geq 0$ (f is undefined for $h < 0$). We note that when $h = 0$, then h is necessarily connected to h_l and h_r by rarefaction waves, and thus $f_l = -2\sqrt{gh_l}$, $f_r = -2\sqrt{gh_r}$, and consequently:

$$f = f_l + f_r + \Delta u = -2\sqrt{gh_l} - 2\sqrt{gh_r} + \Delta u. \quad (2.68)$$

For the solution $f = 0$ to have a solution, then $\Delta u = u_r - u_l$ must not exceed the critical value:

$$(\Delta u)_{cr} = 2\sqrt{gh_l} + 2\sqrt{gh_r} \quad (2.69)$$

This condition is called the *depth positivity condition*. There are two possibilities:

- If $\Delta u < (\Delta u)_{cr}$ or said differently if $u_r - 2\sqrt{gh_r} < u_l + 2\sqrt{gh_l}$, then we can find an intermediate depth h_* to the sixth-order polynomial

$$f(h_*) = 0. \quad (2.70)$$

There is no analytical solution to this equation. The intermediate velocity is given by:

$$u_* = \frac{u_l + u_r}{2} + \frac{f_r(h_*|h_r) - f_l(h_*|h_l)}{2}. \quad (2.71)$$

- If $\Delta u > (\Delta u)_{cr}$ (or equivalently $u_l + 2\sqrt{gh_l} < u_r - 2\sqrt{gh_r}$), then the only possibility is that $h_* = 0$ and a dry cavity takes place between the left and right states. The dry cavity is connected to left and right states by rarefaction waves.

Let us assume that $\Delta u < (\Delta u)_{cr}$. There is an intermediate state (h_*, u_*) with $h_* > 0$. Let us define

$$h_0 = \min(h_l, h_r) \text{ and } h_1 = \max(h_l, h_r)$$

Then we can deduce the structure of the solution:

- $0 < h_* < h_0$, the solution comprises two rarefaction waves.
- $h_0 < h_* < h_1$, the solution consists of one rarefaction wave and one shock wave.
- $h_1 < h_*$, the solution comprises two shock waves.

Initially dry state

There is an intermediate state that is the solution to Eq. (2.64) and which is connected to the left and right states by two waves if and only if one of these states is nonzero. If the left or right state is zero, then there is a single wave connecting the left and right state and $h_* = 0$.

Right-going wave. If the right state is zero ($h_r = \bar{u}_r = 0$) whereas the left state is nonzero ($h_l > 0$), then the only possibility is the integral curve related to the 1-rarefaction wave connecting L to the intermediate I which coincides with the right state in the $(h, h\bar{u})$ plane (see Fig. 2.9(a)). If we plot the curve ϕ_l in the $(h, \bar{u})$ plane (see Fig. 2.9(b)), then the intermediate state is $I(0, u_* = 2\sqrt{gh_l})$ and differs from the right state; this peculiarity is explained by the fact that $h = 0$ and thus any velocity is admissible (even though it makes no physical sense).

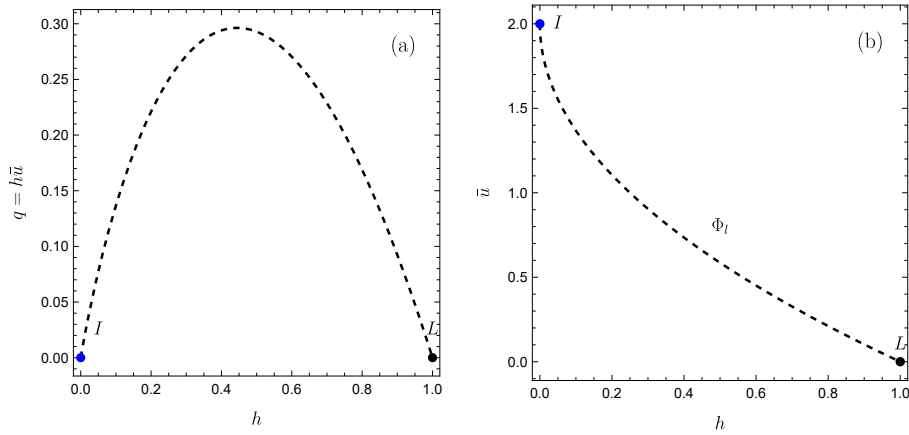


Figure 2.9 Dam break with an initially dry state on the right: $h_l = 1, h_r = 0, \bar{u}_l = \bar{u}_r = 0$. (a) Integral curve related to the 1-rarefaction wave connecting L to I in the $(h, h\bar{u})$ plane. (b) Integral curve related to the 1-rarefaction wave in the $(h, \bar{u})$ plane.

We can now calculate the solution. Figure 2.10 shows the characteristic curves emanating from the initial states. At the origin, the 1-characteristic curves are straight lines pivoting around the origin point:

$$\xi = \frac{x}{t} = \lambda_1 = \bar{u} - \sqrt{gh}, \quad (2.72)$$

and at the same time, we know that the 1-Riemann invariant $r_1 = \bar{u} + 2\sqrt{gh}$ is constant throughout the rarefaction fan. The value taken by r_1 is fixed by the left state:

$$r_1 = \bar{u} + 2\sqrt{gh} = \bar{u}_l + 2\sqrt{gh_l}. \quad (2.73)$$

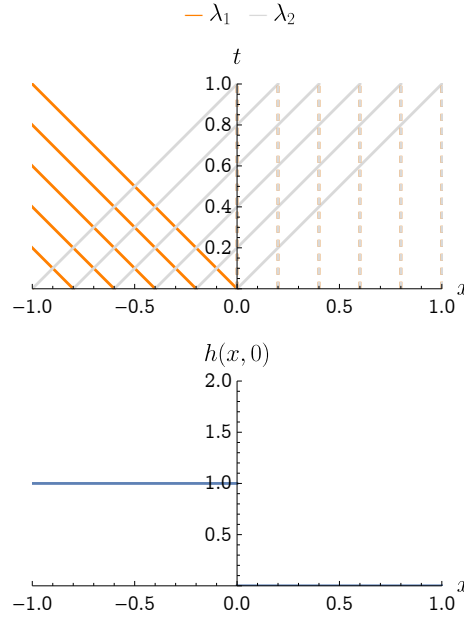


Figure 2.10 Characteristic diagram for the initial-value problem shown in Fig. 2.9 at early times. Initially, the characteristic curves are straight lines in the form $x = x_0 + \lambda_i^l t$ where $\lambda_i^l = \bar{u}_l \pm \sqrt{gh_l}$. To determine their behaviour at later times, we first need to compute the solution.

Solving Eqs. (2.72)–(2.73) for $\bar{u}$, we find:

$$\bar{u}(x, t) = \frac{r_1 + 2\xi}{3}. \quad (2.74)$$

We deduce that the flow depth is:

$$h(x, t) = \frac{(\bar{u} - \xi)^2}{g} = \frac{(r_1 - \xi)^2}{9g}. \quad (2.75)$$

There is a front x_f such that $h(x_f, t) = 0$. Equation (2.75) tells us that this occurs for:

$$\xi_f = r_1 \Rightarrow x_f = r_1 t = (\bar{u}_l + 2\sqrt{gh_l})t.$$

The rarefaction fan is thus the set of straight lines:

$$x = \lambda_1 t,$$

with $\lambda_1^l \leq \lambda_1 \leq \lambda_1^f$ where $\lambda_1^f = \bar{u}_l + 2\sqrt{gh_l}$ and $\lambda_1^l = \bar{u}_l - \sqrt{gh_l}$.

We can now deduce the 2-characteristic curves. At early times, they are straight lines:

$$x = x_0 + \lambda_2^l t \quad (2.76)$$

where $x_0 < 0$ and $\lambda_2^l = \bar{u}_l + \sqrt{gh_l}$. At later times, once they have crossed the left 1-rarefaction wave $x = \lambda_1^l t$ (where $\lambda_1^l = \bar{u}_l - \sqrt{gh_l}$) at time:

$$t_0 = \frac{x_0}{\lambda_1^l - \lambda_2^l},$$

the characteristic equation becomes nonlinear for $t > t_0$:

$$\frac{dx}{dt} = \lambda_2 = \bar{u} + \sqrt{gh} = \frac{2r_1 + \xi}{3}.$$

Solving this ordinary differential equation leads to:

$$x = r_1 t + at^{1/3}, \quad (2.77)$$

where a is the constant of integration, whose value is set by the continuity at $t = t_0$ with the solution (2.76) at early times:

$$r_1 t_0 + at_0^{1/3} = x_0 + \lambda_2^l t_0 = \frac{\lambda_1^l}{\lambda_1^l - \lambda_2^l} x_0.$$

Figure 2.11 shows the 1- and 2-characteristic curves at any time. We note that 2-characteristic curves tend to be parallel to the 1-rarefaction line near the front. This is an example of *contact discontinuity*.

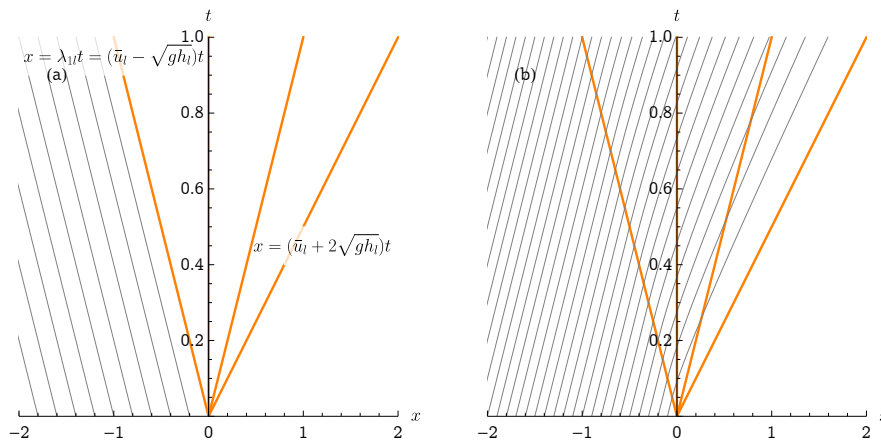


Figure 2.11 Characteristic diagram for the initial-value problem shown in Fig. 2.9. (a) 1-characteristic curves. (b) 2-characteristic curves. The orange straight lines stand for the 1-rarefaction fan $x = \lambda_1 t$ with $\lambda_1^l \leq \lambda_1 \leq \lambda_1^f = \bar{u}_l + 2\sqrt{gh_l}$.

Left-going wave. If the left state is zero ($h_l = \bar{u}_l = 0$) whereas the right state is nonzero ($h_r > 0$), then the only possibility is a 2-rarefaction wave connecting R to the intermediate I which coincides with the left state in the $(h, h\bar{u})$ plane. Using the same reasoning as above, we find the velocity function:

$$\bar{u}(x, t) = \frac{r_2 + 2\xi}{3}, \quad (2.78)$$

where $r_2 = \bar{u}_r - 2\sqrt{gh_r}$. We deduce that the flow depth is

$$h(x, t) = \frac{(\bar{u} - \xi)^2}{g} = \frac{(r_2 - \xi)^2}{9g}. \quad (2.79)$$

There is a front x_f such that $h(x_f, t) = 0$. Equation (2.79) tells us that this occurs for:

$$\xi_f = r_2 \Rightarrow x_f = r_2 t = (\bar{u}_r - 2\sqrt{gh_r})t.$$

The rarefaction fan is thus the set of straight lines:

$$x = \lambda_2 t,$$

with $\lambda_2^f \leq \lambda_2 \leq \lambda_2^r$ where $\lambda_2^f = \bar{u}_r - 2\sqrt{gh_r}$ and $\lambda_2^r = \bar{u}_r + \sqrt{gh_r}$.

2.6 Dam break problem

A direct application of the methodology developed in § 2.5 is the solution to the dam break problem, which is of great significance in hydraulic engineering.

2.6.1 Wet bed

Initial conditions

Let us consider the dam break problem on a wet bed (that is, the initial flow depth is nonzero everywhere):

$$h(x, 0) = \begin{cases} h_l & \text{for } x < 0, \\ h_r & \text{for } x > 0, \end{cases} \quad (2.80)$$

and $u(x, 0) = 0$ everywhere, and we assume that $h_l > h_r$. The solution structure is shown by Fig. 2.12. There is an intermediate state $q = (h_*, q_*)$ separated from the left initial state q_l by a rarefaction wave, and from the right initial state q_r by a shock wave.

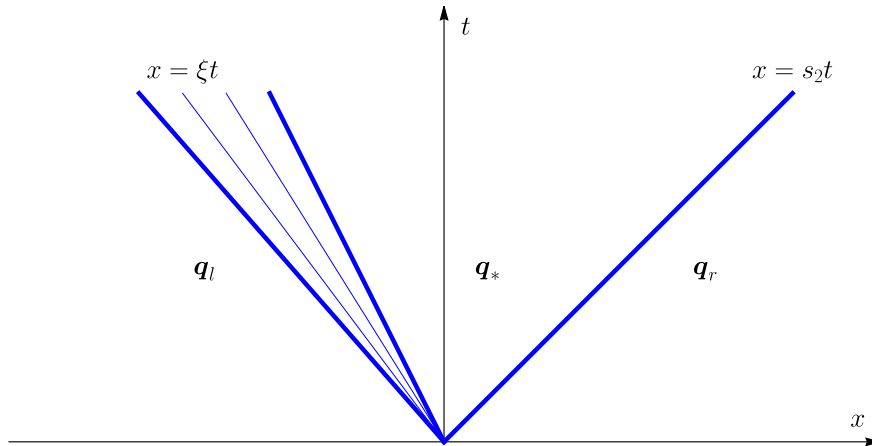


Figure 2.12 Dam break with an intermediate state separated from the left initial state by a rarefaction wave, and from the right initial state by a shock wave.

Intermediate state

The intermediate state satisfies the Rankine–Hugoniot condition:

$$\dot{\sigma}(q_* - q_r) = f(q_*) - f(q_r), \quad (2.81)$$

which implies that the shock speed is

$$\dot{\sigma} = \frac{q_* - q_r}{h_* - h_r} = u_* \mp \sqrt{gh_r \frac{h_r + h_*}{2h_*}},$$

By eliminating the shock speed $\dot{\sigma}$ in Eq. (2.81), we find that the flow rate q_* depends on the initial rate q_r (which is 0 in the example here, but we will keep it here to outline the general case) and depth h_r :

$$q_* = q_r + (h_* - h_r) \left(u_r \pm \sqrt{gh_r \left(1 + \frac{h_* - h_r}{h_r} \right) \left(1 + \frac{h_* - h_r}{2h_r} \right)} \right), \quad (2.82)$$

or in terms of the velocity u_* :

$$u_* = u_r \pm (h_* - h_r) \sqrt{\frac{g}{2} \left(\frac{1}{h_r} + \frac{1}{h_*} \right)}. \quad (2.83)$$

The intermediate state is also connected to the left state by the rarefaction wave. Let us remember that a rarefaction wave is a similarity solution $q(\xi)$ with $\xi = x/t$ (see § 2.4). Substituting this form into the hyperbolic system (2.3) gives:

$$-\xi q'(\xi) + f'(q) \cdot q'(\xi) = 0,$$

which shows that $q'(\xi)$ is a right eigenvector of the Jacobian matrix f' , and thus there exists a scalar function $\alpha(\xi)$ such that

$$q'(\xi) = \alpha(\xi) w_k,$$

with $k = 1, 2$. Let us assume that $\alpha = 1$ and take $k = 1$ (that is, we are looking for the 1-rarefaction wave). We have to solve:

$$q'(\xi) = \begin{pmatrix} h' \\ q' \end{pmatrix} = \begin{pmatrix} 1 \\ u - \sqrt{gh} \end{pmatrix}.$$

By setting the first constant of integration to zero, we find:

$$h(\xi) = \xi,$$

and

$$q' = \frac{q}{\xi} - \sqrt{g\xi} \Rightarrow q(\xi) = a\xi - 2\xi\sqrt{g\xi},$$

where a is a constant of integration. As we have $h = \xi$, this means that we also have $q(h) = ah - 2h\sqrt{gh}$. We impose that the intermediate state lies on the rarefaction wave, and thus:

$$q_* = ah_* - 2h_*\sqrt{gh_*} \Rightarrow a = u_* + 2h_*\sqrt{gh_*}.$$

The 1-rarefaction is thus the curve:

$$q(h) = hu_* + 2h(\sqrt{gh_*} - \sqrt{gh}). \quad (2.84)$$

In the (u, h) coordinate system, the 1-rarefaction wave satisfies:

$$u + 2\sqrt{gh} = u_* + 2\sqrt{gh_*}. \quad (2.85)$$

Example

Let us consider the following example with $q_l = (4, 0)$ and $q_r = (1, 0)$. Figure 2.14(a) shows the initial depth profile, and Fig. 2.14(b) shows the characteristic lines emanating from the initial condition. The 1-characteristic curves spread out, and thus there is apparently an empty wedge in the left quadrant. This wedge will be occupied by a rarefaction wave (for this reason, we will refer to it as the *rarefaction fan*). The 2-characteristic curves cross each other, which indicates the formation of a shock wave in the

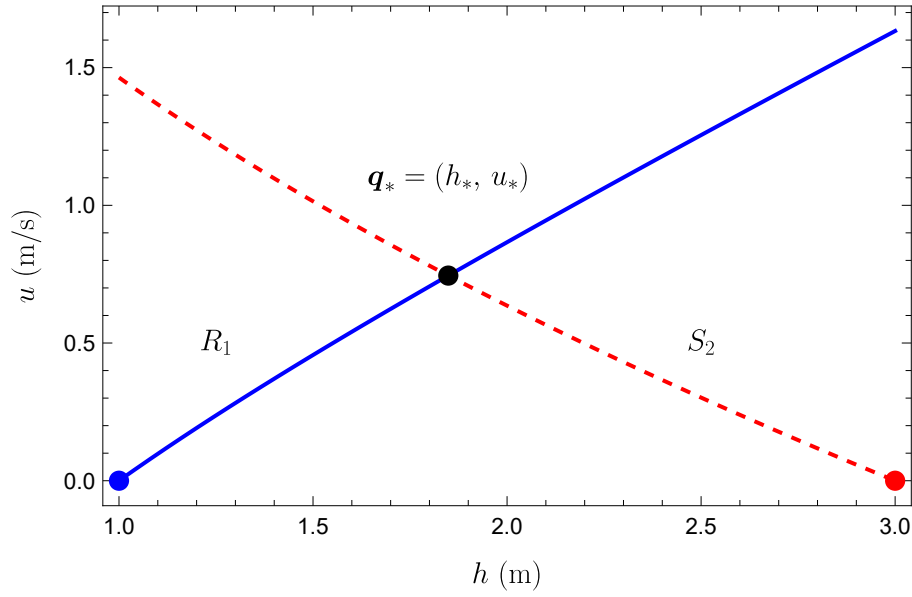


Figure 2.13 Phase plane for $h_l = 3$ and $h_r = 1$ (with $g = 1 \text{ m/s}^2$).

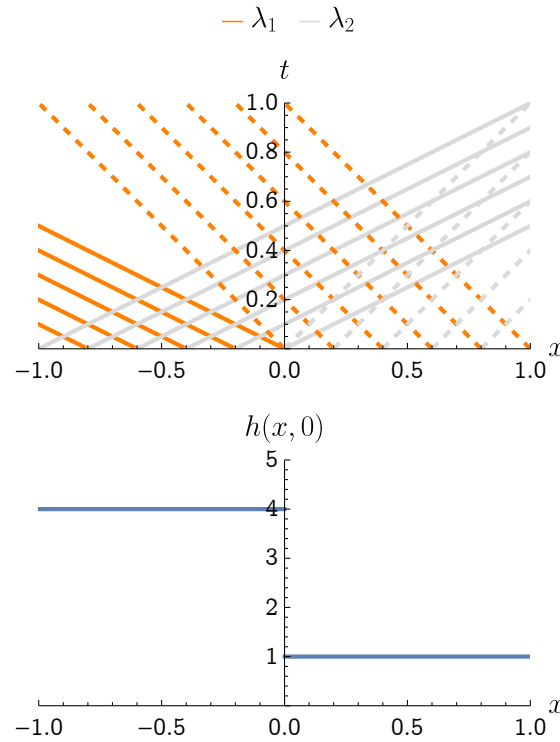


Figure 2.14 We consider the following Riemann problem $q_l = (4, 0)$ and $q_r = (1, 0)$. (a) Flow depth profile. (b) The solid lines refer to 1- and 2-characteristics related to the left state, while the dashed lines refer to the right state. We use the following colour code: orange for the characteristic curves $x = x_0 + \lambda_1 t$, and grey for the characteristic curves $x = x_0 + \lambda_2 t$. We also assumed that $g = 1 \text{ m/s}^2$.

right quadrant. From this inspection of the characteristic curves, we deduce that the solution involves a 1-rarefaction wave and 2-shock wave.

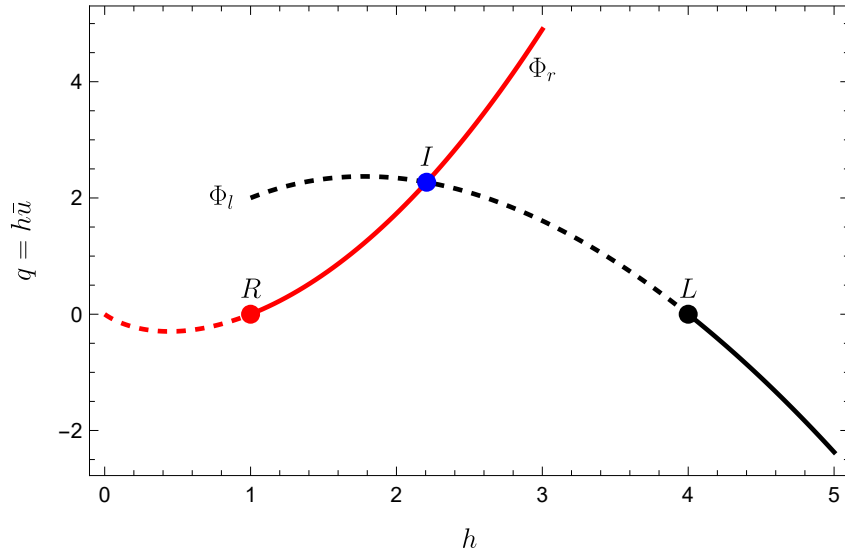


Figure 2.15 Example of solution with $q_l = (4, 0)$ and $q_r = (1, 0)$. The solid lines show shock waves, while the dashed lines refer to the rarefaction waves.

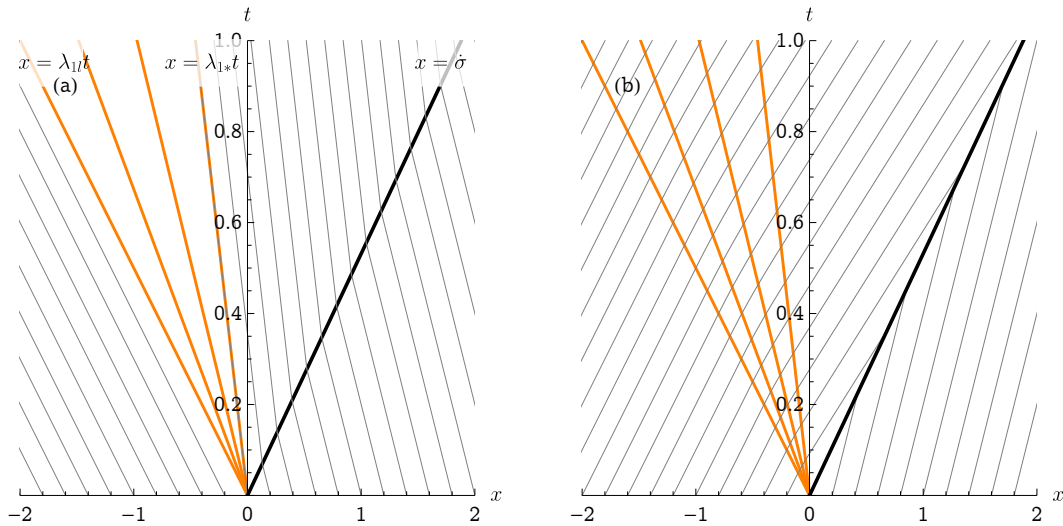


Figure 2.16 Solution to the dambreak problem $q_l = (4, 0)$ and $q_r = (1, 0)$. (a) 1-characteristic curves. (b) 2-characteristic curves. The orange lines show the rarefaction fan that separates the 1-characteristic curves issuing from the left and right states. The solid black line is the shock curve connecting the intermediate and right states.

We can determine the intermediate state $q_* = (h_*, h_* \bar{u}_*)$ that is connected to the left and right states. We apply the computational strategy outlined above to calculate this state. We find $h_* = 2.207$ m and $\bar{u}_* = 1.028$ m/s. Figure 2.15 confirms that the 1-wave connecting q_l to q_* is a rarefaction wave, whereas the 2-wave is a shock wave.

We can now plot the characteristic curves. Figure 2.16 shows the shock wave (black line):

$$x = \dot{\sigma} t \text{ with } \dot{\sigma} = \frac{h_r \bar{u}_r - h_* \bar{u}_*}{h_r - h_*} = 1.88 \text{ m/s,}$$

and the rarefaction fan (orange lines) delineated by the two characteristic curves:

$$x = \lambda_{1l}t \text{ and } x = \lambda_{1*}t,$$

where $\lambda_{1l} = \bar{u}_l - \sqrt{gh_l} = -\sqrt{gh_l}$ and $\lambda_{1*} = \bar{u}_* - \sqrt{gh_*}$. Figure 2.16(a) shows the 1-characteristic curves. On the left of the rarefaction fan, they take the form:

$$x = x_0 + \lambda_{1l}t,$$

where x_0 is any position on the x -axis. On the right, they undergo a slight direction change when crossing the shock curve. On the right of the shock curve, the 1-characteristic curves are:

$$x = x_0 + \lambda_{1r}t,$$

while on the shock curve's left, they have the form:

$$x = x_1 + \lambda_{1*}t,$$

where $x_1(x_0)$ is set by the continuity with the 1-characteristic curves arriving from the region downstream of the shock curve. Indeed, the 1-characteristic curve crosses the 2-shock curve at time:

$$\dot{\sigma}t = x_0 + \lambda_{1r}t \Rightarrow t = \frac{x_0}{\dot{\sigma} - \lambda_{1r}},$$

and thus $x_1(x_0) = \dot{\sigma}x_0/(\dot{\sigma} - \lambda_{1r})$.

Figure 2.16(a) shows the 2-characteristic curves. On the left of the rarefaction wave, these curves take the form:

$$x = x_0 + \lambda_{2l}t.$$

When these curves move through the rarefaction wave, they are no longer straight lines. Their governing equation is:

$$\frac{dx}{dt} = \bar{u} + \sqrt{gh}. \quad (2.86)$$

In the 1-rarefaction fan, the depth and velocity are determined to within a constant of integration. First, the Riemann invariant r_1 is constant in the 1-rarefaction fan, and the constant is fixed by the boundary condition on the left:

$$r_1 = \bar{u} + 2\sqrt{gh} = 2c_l, \quad (2.87)$$

where $c_l = \sqrt{gh_l}$. Second, the 1-characteristic curves are straight lines pivoting around the axis origin:

$$x = \lambda_1 t = (\bar{u} - \sqrt{gh})t. \quad (2.88)$$

Solving Eqs. (2.87)–(2.88) for $\bar{u}$ and h gives:

$$\bar{u} = \frac{2}{3} \left(c_l + \frac{x}{t} \right) \text{ and } h = \frac{1}{9g} \left(2c_l - \frac{x}{t} \right)^2 \quad (2.89)$$

Substituting Eq. (2.89) into Eq. (2.86) and integrating:

$$x = 2c_l t + bt^{1/3}, \quad (2.90)$$

where b is a constant that will be determined by imposing the continuity between 2-characteristic curves on both sides of the boundary $x = \lambda_{1l}t$. On the right of the shock wave, the 2-characteristic curves take the form:

$$x = x_0 + \lambda_{2r}t.$$

Figure 2.17 shows the flow-depth and velocity profiles at time $t = 0.5$ s. The rarefaction and shock waves are readily identifiable on the flow depth profile.

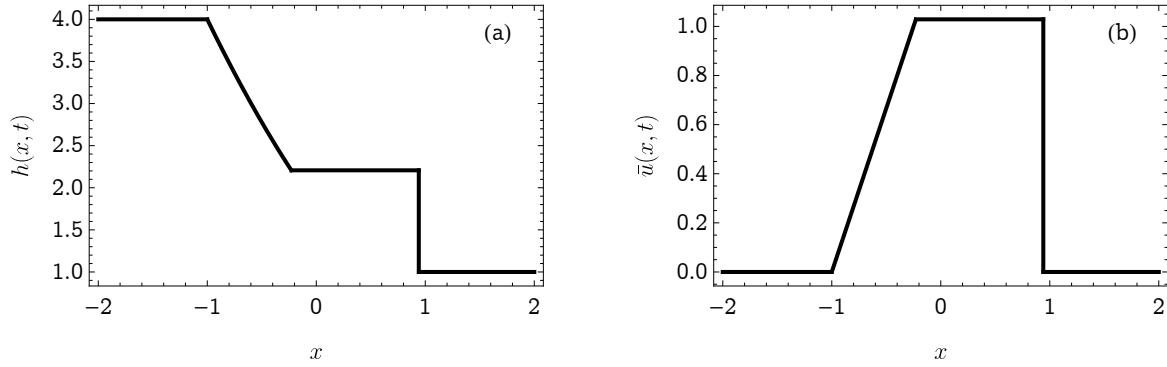


Figure 2.17 Solution to the dam break problem $q_l = (4, 0)$ and $q_r = (1, 0)$ at time $t = 0.5$ s (we set g to 1 m/s^2). (a) Flow depth profile. (b) Velocity profile.

2.6.2 Dry bed

We now consider a dam break on a smooth (frictionless) bottom, which is dry on the right and wet on the left:

$$\begin{aligned} \text{for } -\infty < x < \infty, \quad u(x, 0) &= 0, \\ \text{for } x < 0, \quad h(x, 0) &= h_0, \\ \text{for } x > 0, \quad h(x, 0) &= 0. \end{aligned} \quad (2.91)$$

In § 2.5.2, we have seen how we can calculate the depth and velocity of the rarefaction wave when one of the initial states is zero (see p. 36). Here we show that we find the same result without using the method of characteristics, but by seeking a similarity solution to the Saint-Venant equations subject to the initial conditions (2.91).

We seek a similarity solution in the form:

$$\bar{u} = t^{\beta/\alpha} U(\zeta) \text{ and } h = t^{\gamma/\alpha} H(\zeta),$$

where $\zeta = x/t^\alpha$ is the similarity variable, H and U two functions to be determined. Substituting $\bar{u}$ and h with the similarity forms into the governing equations (2.1)–(2.2), we find: $\beta + \alpha = 1$ and $\gamma + 2\alpha = 2$. For the similarity form to satisfy the initial and boundary conditions (2.91), we must set $\beta = \gamma = 0$, hence $\alpha = 1$. Equations (2.1)–(2.2) become:

$$\begin{aligned} H \frac{dU}{d\zeta} + (U - \zeta) \frac{dH}{d\zeta} &= 0, \\ (U - \zeta) \frac{dU}{d\zeta} + g \frac{dH}{d\zeta} &= 0. \end{aligned}$$

We obtain the following system of equations:

$$\begin{pmatrix} H & U - \zeta \\ U - \zeta & g \end{pmatrix} \cdot \begin{pmatrix} U' \\ H' \end{pmatrix} = 0,$$

where the prime refers to the ζ -derivative. For this system to have a nontrivial solution, its determinant must be zero, which leads to $gH = (U - \zeta)^2$. We substitute this relation into the system of equations above and obtain $U' = 2\zeta/3$, hence $U = 2(\zeta + c_0)/3$, where c_0 is a constant of integration, $H =$

$4(c_0 - \frac{1}{2}\zeta)^2/(9g)$. The constant c_0 is found by using the boundary conditions: $c_0 = \sqrt{gh_0}$. Returning to the original variables, we end up with the *Ritter solution* to the Saint-Venant equations:

$$\bar{u}(x, t) = \bar{u} = \frac{2}{3} \left(\frac{x}{t} + c_0 \right), \quad (2.92)$$

$$h(x, t) = \frac{1}{9g} \left(-\frac{x}{t} + 2c_0 \right)^2. \quad (2.93)$$

Figure 2.18 shows the flow-depth and velocity profiles at time $t = 0.5$ s.

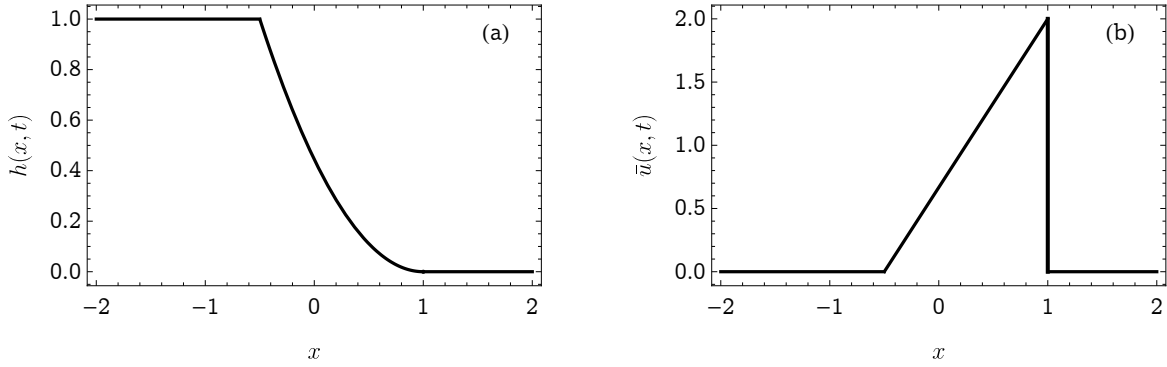


Figure 2.18 Solution to the dam break problem $q_l = (1, 0)$ and $q_r = (0, 0)$ at time $t = 0.5$ s (we set g to 1 m/s^2). (a) Flow depth profile. (b) Velocity profile.

Finite volume methods

3.1 General formulation

3.1.1 Finite-volume averaging

Let us consider a hyperbolic equation in one space dimension and in a conservative form

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = 0, \quad (3.1)$$

where q is a vector with m components representing the unknowns and f is the flux function. We consider a uniform grid over an interval $[a, b]$ divided into N identical cells of size $\Delta x = (b - a)/N$. The cell centers are denoted by $x_{i-1/2}$ (see Fig. 3.1):

$$x_i = a + \left(i - \frac{1}{2}\right) \Delta x.$$

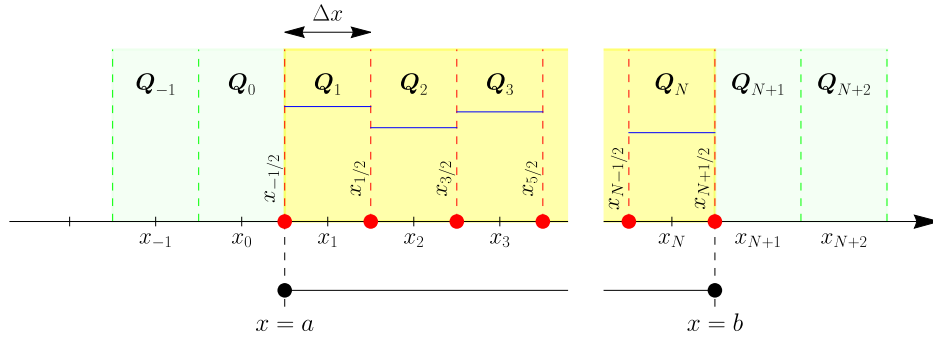
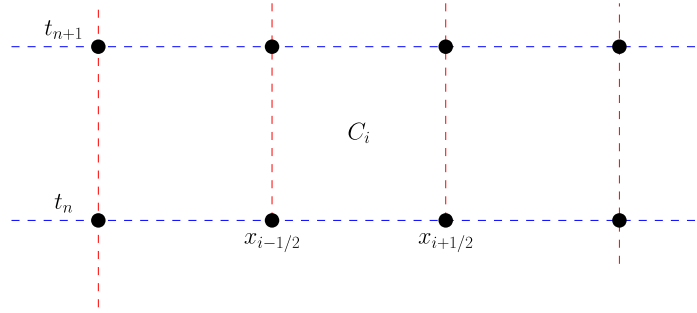


Figure 3.1 Computational grid. The yellow-coloured area represents the computational domain $[a, b]$, while the green-coloured area show the extended domain for handling the boundary conditions. The computational domain can be extended by including *ghost cells* (see § 3.10).

We define the cell $C_i = [x_{i-1/2}, x_{i+1/2})$, centred around the cell x_i and whose bounds (or interfaces) are $x_{i\pm 1/2}$ (see Fig. 3.2). Note that:

$$x_{-1/2} = a \text{ and } x_{N+1/2} = b.$$

Figure 3.2 Computational domain and grid in the $x - t$ plane.

The time step is denoted by $\Delta t = t_{n+1} - t_n$. The computational domain is $[a, b]$, but we will see that we will need to extend it to handle boundary conditions (see § 3.10).

We integrate Eq. (3.1) over the cell C_i :

$$\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial q}{\partial t} dx + [f(q)]_{x_{i-1/2}}^{x_{i+1/2}} = 0. \quad (3.2)$$

Integrating this equation again over $(t_n, t_{n+1}]$ gives:

$$\int_{x_{i-1/2}}^{x_{i+1/2}} (q(x, t_{n+1}) - q(x, t_n)) dx + \int_{t_n}^{t_{n+1}} [f(q)]_{x_{i-1/2}}^{x_{i+1/2}} dt = 0. \quad (3.3)$$

We define the cell-averaged value of q at time t_n :

$$Q_i^n = \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} q(x, t_n) dx, \quad (3.4)$$

and a time-averaged flux

$$F_{i\pm 1/2}^n = \frac{1}{\Delta t} \int_{t_n}^{t_{n+1}} f(q(x_{i\pm 1/2}, t)) dt. \quad (3.5)$$

We can develop an explicit time-marching algorithm by rearranging Eq. (3.3) and introducing the time- and grid-averaged variables

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (F_{i+1/2}^n - F_{i-1/2}^n). \quad (3.6)$$

3.1.2 Upwind method

Let us give an application example of (3.6) with the upwind method. The idea behind this method is that for hyperbolic problems, information propagates in different directions and at different speeds. If we know how these waves carrying information behave, then we can determine the flux $F_{i\pm 1/2}^n$. We start with the simplest case

$$\frac{\partial q}{\partial t} + \bar{u} \frac{\partial q}{\partial x} = 0, \quad (3.7)$$

where $\bar{u} > 0$ is a constant. The time-averaged flux (3.5) is thus

$$F_{i-1/2}^n = \frac{1}{\Delta t} \int_{t_n}^{t_{n+1}} f(q(x_{i-1/2}, t)) dt = \bar{u} Q_{i-1}^n,$$

since Q_{i-1}^n is constant along the characteristic $x_{i-1} + \bar{u}t$. As a consequence for the linear advection problem, the first-order upwind method built from (3.6) is

$$Q_i^{n+1} = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x} (Q_i^n - Q_{i-1}^n), \quad (3.8)$$

which looks like the finite-difference discretisation of Eq. (3.7) (one-sided first-order approximation).

This equation can be interpreted in (at least) two different ways, one focusing on space-averaging, the other on time-averaging. First, let us assume that we have a piecewise constant approximation of the solution $q(x, t)$ to (3.7) at time t_n as illustrated by Fig. 3.3. At time $t_{n+1} = t_n + \Delta t$, the curve has been shifted by $\bar{u}\Delta t$. We now calculate the value Q_i^{n+1} by averaging the shifted function over the cell C_i :

$$Q_i^{n+1} = \frac{1}{\Delta x} ((\Delta x - \bar{u}\Delta t)Q_i^n + \bar{u}\Delta t Q_{i-1}^n), \quad (3.9)$$

which imposes $\Delta x - \bar{u}\Delta t > 0$ for the averaging to make sense. This constraint on Δt

$$0 \leq \frac{\bar{u}\Delta t}{\Delta x} \leq 1$$

is called the *Courant-Friedrichs-Lewy condition* (CFL). It can be checked that after rearrangement, Eq. (3.9) leads to the upwind method (3.8).

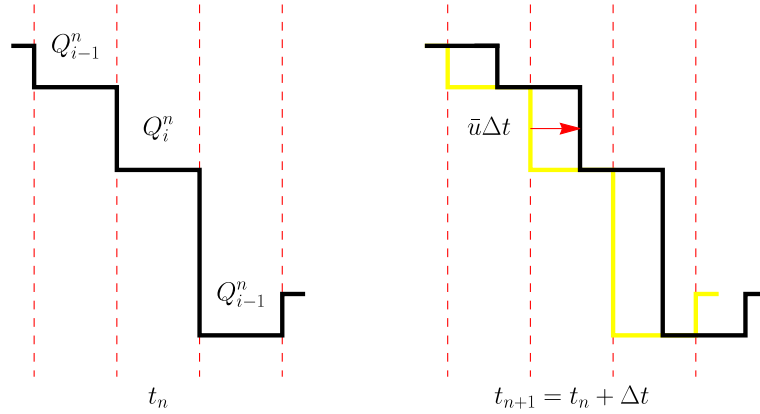


Figure 3.3 Advection of a piecewise continuous function. At time $t = t_n$, the function is piecewise constant, with value Q_i^n on cell C_i . At time t_{n+1} , this function has been shifted by $\bar{u}\Delta t$. The value Q_i^{n+1} is obtained by space-averaging the shifted function on cell C_i . The problem can be interpreted as the propagation of discontinuities originating from the cell boundaries $x_{i-1/2}$. These discontinuities form the wave $W_{i-1/2} = Q_i - Q_{i-1}$.

Another interpretation given by [LeVeque \(2002\)](#) is the wave-propagation viewpoint. If we define the jump

$$W_{i-1/2} = Q_i^n - Q_{i-1}^n,$$

then Eq. (3.8) can be rewritten

$$Q_i^{n+1} = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x} W_{i-1/2}. \quad (3.10)$$

The interpretation is the following: the wave $W_{i-1/2}$ moves at velocity $\bar{u}$. When it moves through C_i , it changes the value of Q_i^n by $-W_{i-1/2}$ at each point it passes (see Fig. (3.3)). Over the time step Δt , the portion $\bar{u}\Delta t/\Delta x$ has been affected by the wave, and thus the cell average Q_i^{n+1} is given by Eq. (3.10). This interpretation will be used in the wave-propagation form of Godunov's method below.

Note that the upwind method takes the form (3.8) when $\bar{u} > 0$. When $\bar{u} < 0$, the upwind method leads to

$$Q_i^{n+1} = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x} (Q_i^n - Q_{i-1}^n), \quad (3.11)$$

and the wave-propagation interpretation is

$$Q_i^{n+1} = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x} W_{i+1/2}, \quad (3.12)$$

where $W_{i+1/2} = Q_{i+1}^n - Q_i^n$. It is possible to generalise the upwind method by posing

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (\bar{u}^+ W_{i-1/2} + \bar{u}^- W_{i+1/2}), \quad (3.13)$$

where

$$\bar{u}^+ = \max(0, \bar{u}) \text{ and } \bar{u}^- = \min(0, \bar{u}).$$

The flux approximation is

$$F_{i-1/2}^n = \frac{\bar{u}\Delta t}{\Delta x} (\bar{u}^- Q_i^n + \bar{u}^+ Q_{i-1}^n).$$

3.2 Godunov's method for linear systems

Godunov's method has been a major achievement in the field of hyperbolic equations, which has opened up the way to modern finite-volume techniques (Godunov, 1959; Toro, 2001; LeVeque, 2002; Toro & Garcia-Navarro, 2007; Guinot, 2010). It consists of three steps: reconstructing, evolving, and averaging.

1. *Reconstruction.* We assume that we can approximate the solution $q(x, t)$ by a piecewise constant function $\tilde{q}_i^n(x, t_n) = Q_i^n$ for $x \in C_i = (x_{i-1/2}, x_{i+1/2})$. Note that to second order, we have

$$\begin{aligned} Q_i^n &= \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \tilde{q}(x, t_n) dx, \\ &= \frac{1}{\Delta x} \int_{x_i - \Delta x/2}^{x_i + \Delta x/2} \left(\tilde{q}(x_i, t_n) + (x - x_i) \frac{\partial}{\partial x} q(x_i, t_n) + \frac{(x - x_i)^2}{2} \frac{\partial^2}{\partial x^2} q(x_i, t_n) \right) dx, \\ &= q(x_i, t_n) + \frac{\Delta x^2}{24} \frac{\partial^2}{\partial x^2} q(x_i, t_n). \end{aligned}$$

Godunov's method is a first-order accurate scheme. We can use higher-order reconstructions of the approximate the function $q(x, t)$ (e.g., a piecewise linear function with a nonzero slope in each grid cell). See § 3.8.

2. *Evolution.* Using Eq. (3.6) or another method, we look at how the solution $\tilde{q}_i^{n+1}$ has evolved from the previous state at time t_n . This step amounts to solving Riemann problems at each cell boundary $x_{i\pm 1/2}$.
3. *Averaging.* We average this function over each grid cell

$$Q_i^{n+1} = \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \tilde{q}(x, t_{n+1}) dx. \quad (3.14)$$

We can piece together the Riemann solutions provided that the waves from two adjacent interfaces have not started to interact. This condition is usually met when the *Courant–Friedrichs–Lewy* (CFL) condition is satisfied (no wave passes through more than one grid cell within Δt):

$$\frac{c_{max}\Delta t}{\Delta x} \leq 1, \quad (3.15)$$

where c_{max} represents the largest wave speed over the computational domain.

Godunov's method was initially used to solve the Euler equations in gas dynamics (Godunov, 1959, 1962). The flux $F_{i+1/2}^n$ was determined from the exact solution to the Riemann problem for the Euler equations. Approximate Riemann solvers are used today because they are faster. Godunov's method is robust and stable when the CFL condition is met. When approximate solvers are used, this may not be the case, and thus special care has to be paid to robustness and stability. Moreover, Godunov's method tends to smear out solutions near discontinuities. By using limiters, approximate Riemann solvers deal more efficiently with discontinuities. They also build numerical solutions as linear combinations of travelling discontinuities—they do not use rarefaction waves, which are therefore approximated as discontinuities. Transonic waves¹ may need more care.

The crux in Godunov's method lies in the determination of the flux terms in Eq. (3.6). These fluxes are defined by Eq. (3.5):

$$F_{i\pm 1/2}^n = \frac{1}{\Delta t} \int_{t_n}^{t_{n+1}} f(q(x_{i\pm 1/2}, t)) dt.$$

When we solve a succession of Riemann problems, the integrand f is usually constant because either the solution to the Riemann problem involves a rarefaction wave (in this case, the line $x = x_{i-1/2}$ is a characteristic curve) or a shock wave (in that case, the shock wave separates two constant-state domains). Therefore, in both cases, the value q is a constant that can be expressed as a function denoted by $q^\downarrow$ by LeVeque (2002) of the left and right states:

$$q = q^\downarrow(Q_{i-1}^n, Q_i^n) \quad (3.16)$$

and thus we can define the value taken by q at the interface $x_{i-1/2}$ as

$$Q_{i-1/2}^n = q^\downarrow(Q_{i-1}^n, Q_i^n) \quad (3.17)$$

and the flux at the interface $x_{i-1/2}$:

$$F_{i-1/2}^n = f(q^\downarrow(Q_{i-1}^n, Q_i^n)) = \mathcal{F}(Q_{i-1}^n, Q_i^n) \quad (3.18)$$

To sum up, we can implement Godunov's method by following the three following steps:

1. Solve the Riemann problem at each interface $x_{i-1/2}$ and derive the value $q^\downarrow(Q_{i-1}^n, Q_i^n)$.
2. Compute the flux $F_{i-1/2}^n$ using Eq. (3.18).
3. Update the cell values Q_i^n by using the flux-differencing equation (3.6).

3.3 Wave decomposition for linear systems

3.3.1 Introductory example

In Clawpack, we will use a variant of Godunov's method based on the wave decomposition seen in Chapter 1 and close to the wave-propagation interpretation of the upwind method seen in § 3.1.2. There are other strategies such as *flux differencing* (Toro, 2001; LeVeque, 2002; Guinot, 2010). The advantage of wave decomposition over other approaches is that it can also be applied to non-conservative equations.

Let us illustrate how Clawpack proceeds with the flux estimation by considering a problem of dimension $m = 3$. Let us assume that we have three different eigenvalues such that $\lambda_1 < 0 < \lambda_2 < \lambda_3$. As shown by Fig. 3.4, from the node $x_{i-1/2}$ emerge three characteristics $x_{i-1/2} + \lambda_i t$, which will create three discontinuities in $\tilde{q}$ at time t_{n+1} . Similarly, three characteristics are issued from $x_{i+1/2} + \lambda_i t$. The

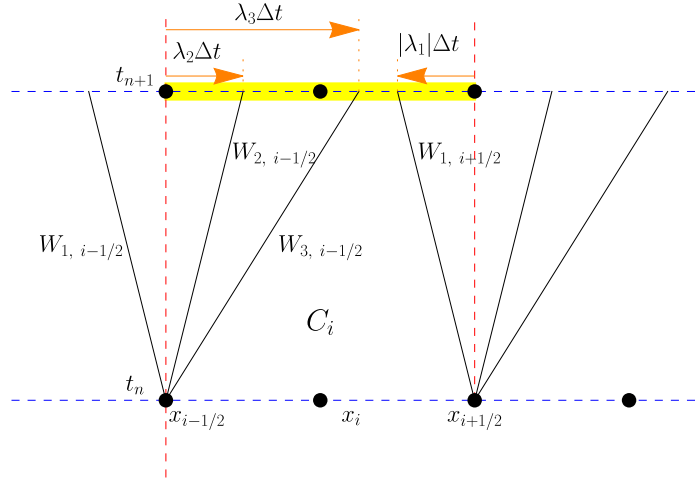


Figure 3.4 Wave structure for the nodes $x_{i-1/2}$ and $x_{i+1/2}$. At time t_n , we consider a series of Riemann problems with $\tilde{q}_i^n(x, t_n) = Q_i^n$. We can compute $\tilde{q}(x, t)$ the solution at later times by considering the waves W_i . The solution is given by Eq. (3.20). The next step is to average the value of $\tilde{q}(x, t_{n+1})$ over the interval $[x_{i-1/2}, x_{i+1/2}]$ (coloured in yellow).

coloured segment in Fig. 3.4 is crossed by three waves: $x_{i-1/2} + \lambda_2 t$, $x_{i-1/2} + \lambda_3 t$, and $x_{i+1/2} + \lambda_1 t$. The other waves will not cause any change to this segment.

Recall that in Chapter 1, we learned from Eq. (1.31) that the initial jump in the Riemann problem at $x_{i-1/2}$ can be decomposed into three waves:

$$Q_i - Q_{i-1} = \sum_{k=1}^m W_{k,i-1/2}, \quad (3.19)$$

where $W_{k,i-1/2}$ is collinear with the right eigenvectors: $W_{k,i-1/2} = \alpha_{k,i-1/2} w_{k,i-1/2}$ where

$$\alpha = L \cdot \Delta Q_{i-1/2}$$

and

$$\Delta Q_{i-1/2} = Q_i - Q_{i-1}.$$

As explained in § 1.1.6 for linear hyperbolic systems, the solution is given by Eqs. (1.35)–(1.36), which tells us that for the double Riemann problem considered here, we have

$$\tilde{q} = Q_i^n + W_{1,i+1/2} - W_{2,i-1/2} - W_{3,i-1/2}. \quad (3.20)$$

Let us examine how these waves modify the value of Q_i at time t_{n+1} . If we consider the first wave, we can see that it will modify the value of q over a fraction of the grid cell $|\lambda_1| \Delta t / \Delta x$ by the amount

$$|\lambda_1| \frac{\Delta t}{\Delta x} W_{1,i+1/2}$$

relative to the initial value Q_i (as $\lambda_1 < 0$, we have to take its absolute value). Similarly, for the 2- and 3-waves, the value of q is changed over a fraction of the grid cell $\lambda_2 \Delta t / \Delta x$ and $\lambda_3 \Delta t / \Delta x$, respectively, by the amount

$$-\lambda_2 \frac{\Delta t}{\Delta x} W_{2,i-1/2} - \lambda_3 \frac{\Delta t}{\Delta x} W_{3,i-1/2}.$$

¹see Fig. 3.6 for a quick definition.

If we sum the contributions, we obtain the averaged value of $\tilde{q}(x, t_{n+1})$ over the interval $[x_{i-1/2}, x_{i+1/2}]$:

$$Q_i^{n+1} = Q_i^n + |\lambda_1| \frac{\Delta t}{\Delta x} W_{1,i+1/2} - \lambda_2 \frac{\Delta t}{\Delta x} W_{2,i-1/2} - \lambda_3 \frac{\Delta t}{\Delta x} W_{3,i-1/2} \quad (3.21)$$

$$= Q_i^n - \frac{\Delta t}{\Delta x} (\lambda_2 W_{2,i-1/2} + \lambda_3 W_{3,i-1/2} + \lambda_1 W_{1,i+1/2}). \quad (3.22)$$

3.3.2 General formulation

The result obtained in § 3.3.1 can be readily generalised to arbitrary hyperbolic systems. Let us introduce the notation

$$\lambda^+ = \max(\lambda, 0) \text{ and } \lambda^- = \min(\lambda, 0). \quad (3.23)$$

The updated value Q_i^{n+1} is then

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} \left(\sum_{k=1}^m \lambda_k^+ W_{k,i-1/2} + \sum_{k=1}^m \lambda_k^- W_{k,i+1/2} \right), \quad (3.24)$$

The cell average depends on the right-going waves from $x_{i-1/2}$ and left-going waves from $x_{i+1/2}$.

LeVeque (2002) introduced a shorthand notation

$$\mathcal{A}^+ \cdot \Delta Q_{i-1/2} = \sum_{k=1}^m \lambda_k^+ W_{k,i-1/2}, \quad (3.25)$$

$$\mathcal{A}^- \cdot \Delta Q_{i+1/2} = \sum_{k=1}^m \lambda_k^- W_{k,i+1/2}, \quad (3.26)$$

which are interpreted as *fluctuations*: $\mathcal{A}^+ \cdot \Delta Q_{i-1/2}$ represents the effect of all right-going waves from $x_{i-1/2}$ (where there is a discontinuity $\Delta Q_{i-1/2} = Q_i - Q_{i-1}$) on the cell average at time t_{n+1} , whereas $\mathcal{A}^- \cdot \Delta Q_{i+1/2}$ represents the effect of all left-going waves from $x_{i+1/2}$. This formulation that holds for linear problems will be generalised to nonlinear problems.

For linear problems, we can relate the operators $\mathcal{A}^+$ and $\mathcal{A}^-$ to the matrix A . We use the spectral matrix Λ , namely the diagonal matrix whose entries are the eigenvalues of A . Making use of Eq. (1.8)

$$A = R \cdot \Lambda \cdot R^{-1} = R \cdot \Lambda \cdot L$$

where the right eigenvector matrix $R = [w_1, \dots, w_m]$ is a matrix whose columns are made of the right eigenvectors, and

$$\Lambda = \Lambda^- + \Lambda^+ = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & & 0 \\ \vdots & & \ddots & \vdots \\ 0 & \dots & 0 & \lambda_m \end{bmatrix} = \begin{bmatrix} \lambda_1^- & 0 & \dots & 0 \\ 0 & \lambda_2^- & & 0 \\ \vdots & & \ddots & \vdots \\ 0 & \dots & 0 & \lambda_m^- \end{bmatrix} + \begin{bmatrix} \lambda_1^+ & 0 & \dots & 0 \\ 0 & \lambda_2^+ & & 0 \\ \vdots & & \ddots & \vdots \\ 0 & \dots & 0 & \lambda_m^+ \end{bmatrix},$$

we can define the negative and positive parts of A :

$$A^- = R \cdot \Lambda^- \cdot L,$$

$$A^+ = R \cdot \Lambda^+ \cdot L.$$

Note that $A = A^- + A^+$. We can calculate $A^+ \cdot \Delta Q_{i-1/2}$

$$\begin{aligned} A^+ \cdot \Delta Q_{i-1/2} &= R \cdot \Lambda^+ \cdot L \cdot Q_{i-1/2}, \\ &= R \cdot \Lambda^+ \cdot \alpha_{i-1/2}, \\ &= \sum_{k=1}^m \alpha_{i-1/2}^k \lambda_k^+ w_k, \\ &= \sum_{k=1}^m \lambda_k^+ W_{i-1/2}^k, \\ &= \mathcal{A}^+ \cdot \Delta Q_{i-1/2}, \end{aligned}$$

where we have defined the wave

$$W_{i-1/2}^k = \alpha_{i-1/2}^k w_k.$$

Similarly we have

$$\mathcal{A}^- \cdot \Delta Q_{i+1/2} = A^- \cdot \Delta Q_{i+1/2} = \sum_{k=1}^m \lambda_k^- W_{i+1/2}^k.$$

For the linear advection problem, the fluctuations can be calculated by simply multiplying $A^\pm$ with the jump $\Delta Q_{i\pm 1/2}$. For the nonlinear case, the calculation is more complicated, but we will continue to use the operators $\mathcal{A}^\pm$ to represent the effects of right- and left-going waves on the value of Q_i^{n+1} .

3.3.3 Interface flux

Clawpack approach

Note that at the interface between two cells, the interface value can be written (see Eq. (1.38)):

$$Q_{i-1/2}^\downarrow = q^\downarrow(Q_{i-1}, Q_i) = Q_{i-1} + \sum_{\lambda_k < 0} W_{i-1/2}^k. \quad (3.27)$$

We then deduce that for a linear system, the flux at the interface is:

$$F_{i-1/2}^n = f(Q_{i-1/2}^\downarrow) = A \cdot Q_{i-1/2}^\downarrow = A \cdot Q_{i-1} + \sum_{\lambda_k < 0} A \cdot W_{i-1/2}^k. \quad (3.28)$$

As W_k is an eigenvector of A , we can rearrange the terms

$$F_{i-1/2}^n = A \cdot Q_{i-1} + \sum_{k=1}^m \lambda_k^- W_{i-1/2}^k = A \cdot Q_{i-1} + A^- \cdot \Delta Q_{i-1/2}. \quad (3.29)$$

Concurrently, we have

$$Q_{i-1/2}^\downarrow = Q_i - \sum_{\lambda_k > 0} W_{i-1/2}^k, \quad (3.30)$$

and consequently

$$F_{i-1/2}^n = A \cdot Q_i - \sum_{k=1}^m \lambda_k^+ W_{i-1/2}^k = A \cdot Q_i - A^+ \cdot \Delta Q_{i-1/2}. \quad (3.31)$$

Similarly, at the other interface, we have

$$\begin{aligned} F_{i+1/2}^n &= A \cdot Q_{i+1} - \sum_{k=1}^m \lambda_k^+ W_{i+1/2}^k = A \cdot Q_{i+1} - A^+ \cdot \Delta Q_{i+1/2}. \\ &= A \cdot Q_i + \sum_{k=1}^m \lambda_k^- W_{i+1/2}^k = A \cdot Q_i + A^- \cdot \Delta Q_{i+1/2}. \end{aligned}$$

This expression will be generalised to nonlinear systems (which, once linearised, involve only shock waves), for which we will assume that

$$F_{i-1/2}^n = f(Q_{i-1}) + \mathcal{A}^- \Delta Q_{i-1/2}. \quad (3.32)$$

or equivalently:

$$F_{i-1/2}^n = f(Q_i) - \mathcal{A}^+ \Delta Q_{i-1/2}. \quad (3.33)$$

Alternative: Harten–Lax–van Leer’s approach

Can we proceed differently for nonlinear systems? For a nonlinear problem, the theoretical expression of the flux is more complicated, but there are approximate methods, such as the Harten–Lax–van Leer (HLL) solver, that can be used to estimate the flux (Harten *et al.*, 1983; Toro, 2019). In the HLL approach, we focus on the lowest and highest wave velocities λ_1 and λ_m . If $\lambda_1 < \lambda_m < 0$, then $Q_{i-1/2} = Q_i$. If $0 < \lambda_1 < \lambda_m$, then $Q_{i-1/2} = Q_{i-1}$. The only unknown case is $\lambda_1 < 0$ and $\lambda_m > 0$. Let us define X_L such that $x_L = x_{i-1/2} + \lambda_1 \Delta t$ and x_R such that $x_R = x_{i-1/2} + \lambda_m \Delta t$ (see Fig. 3.5).

Integrating the hyperbolic equation (3.1) over $[x_L, x_{i-1/2}] \times [t_n, t_{n+1}]$ gives

$$\int_{x_L}^{x_{i-1/2}} q(x, t_{n+1}) dx = \int_{x_L}^{x_{i-1/2}} q(x, t_n) dx + \int_{t_n}^{t_{n+1}} f(q(x_L, t)) dt - \int_{t_n}^{t_{n+1}} f(q(x_{i-1/2}, t)) dt,$$

and if $\Delta x = x_{i-1/2} - x_{i-3/2}$ is chosen such that it lies in the domain controlled by the initial conditions Q_{i-1} or Q_i as imposed by the CLF condition (in other words, the characteristics originating from $x_{i-1/2}$ and $x_{i-3/2}$ do not cross in the integrating domain $[x_{i-3/2}, x_{i-1/2}] \times [t_n, t_{n+1}]$), then we can rearrange the terms

$$\int_{x_L}^{x_{i-1/2}} q(x, t_{n+1}) dx = (x_{i-1/2} - x_L) Q_{i-1} + \Delta t f(Q_{i-1}) - \Delta t F_{i-1/2}.$$

This gives us the relation:

$$F_{i-1/2} = f(Q_{i-1}) - \lambda_1 Q_{i-1} - \frac{1}{\Delta t} \int_{-\Delta x}^0 q(x, t_{n+1}) dx. \quad (3.34)$$

We now need to evaluate the last term implying $q(x, t_{n+1})$ that is unknown. For this purpose, we redo the calculation above, but by integrating over $[x_L, x_R]$

$$\int_{x_L}^{x_R} q(x, t_{n+1}) dx = \int_{x_L}^{x_{i-1/2}} q(x, t_n) dx + \int_{t_n}^{t_{n+1}} f(q(x_L, t)) dt - \int_{t_n}^{t_{n+1}} f(q(x_R, t)) dt.$$

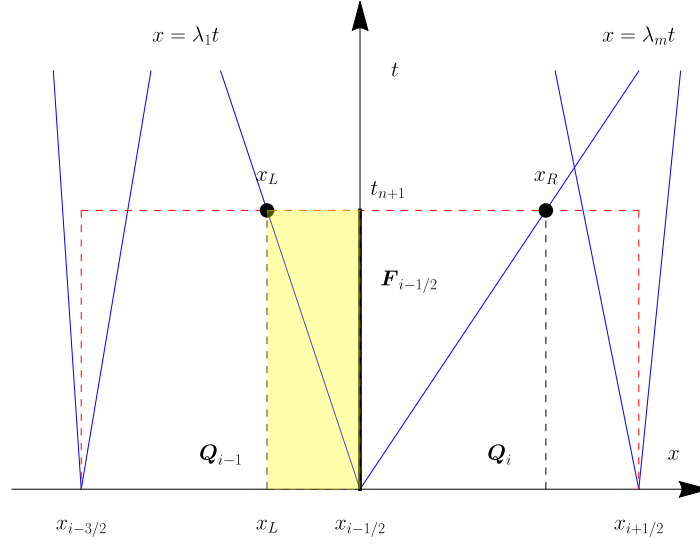


Figure 3.5 Calculating the interface flux $F_{i-1/2}$ in the HLL method. The yellow area is the integrating domain $[x_{i-3/2}, x_{i-1/2}] \times [t_n, t_{n+1}]$.

The contributions to the right-hand side are easy to calculate by interpreting the various terms in Fig. 3.5

$$\begin{aligned} \int_{x_L}^{x_{i-1/2}} q(x, t_n) dx &= -\lambda_1 \Delta t Q_{i-1} + \lambda_m \Delta t Q_i, \\ \int_{t_n}^{t_{n+1}} f(q(x_L, t)) dt &= f(Q_{i-1}) \Delta t, \\ \int_{t_n}^{t_{n+1}} f(q(x_R, t)) dt &= f(Q_i) \Delta t. \end{aligned}$$

We then deduce that

$$\int_{x_L}^{x_R} q(x, t_{n+1}) dx = \Delta t (\lambda_m Q_i - \lambda_1 Q_{i-1} + f(Q_{i-1}) - f(Q_i)),$$

and thus the mean value Q^{LR} of $q(x, t_{n+1})$ over the domain $[x_L, x_R]$ is

$$Q^{LR} = \frac{1}{x_R - x_L} \int_{x_L}^{x_R} q(x, t_{n+1}) dx = \frac{\lambda_m Q_i - \lambda_1 Q_{i-1} + f(Q_{i-1}) - f(Q_i)}{\lambda_m - \lambda_1}$$

We can use this expression to evaluate the integral on the right-hand side of Eq. (3.34), and we can deduce the flux at $x_{i-1/2}$

$$F_{i-1/2} = \frac{\lambda_m f(Q_{i-1}) - \lambda_1 f(Q_i) + \lambda_1 \lambda_m (Q_i - Q_{i-1})}{\lambda_m - \lambda_1}. \quad (3.35)$$

To sum up, the HLL solver determines the flux $F_{i-1/2}$ without trying to determine the value of $Q_{i-1/2}$ as in the Godunov method:

$$F_{i-1/2} = \begin{cases} f(Q_{i-1}) & \text{if } \lambda_1 > 0 \\ \frac{\lambda_m f(Q_{i-1}) - \lambda_1 f(Q_i) + \lambda_1 \lambda_m (Q_i - Q_{i-1})}{\lambda_m - \lambda_1} & \text{if } \lambda_1 < 0 < \lambda_m \\ f(Q_i) & \text{if } \lambda_m < 0 \end{cases} \quad (3.36)$$

3.4 Approximate Riemann solvers for nonlinear problems

Earlier in this chapter, we have seen that a general time-marching algorithm to solve the hyperbolic equation (3.1) is given by Eq. (3.6):

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (F_{i+1/2}^n - F_{i-1/2}^n).$$

At each interface $x_{i-1/2}$, the flux function is given by

$$F_{i-1/2}^n = f(Q_{i-1/2}^n),$$

where $Q_{i-1/2}^n$ is the value of Q obtained along the ray $x = x_{i-1/2}$. It depends on the values Q_i^n and Q_{i-1}^n of either side of the interface. In the absence of a source term, Q_i^n remains constant along this ray.

3.4.1 Scalar problems

Scalar Riemann problems are associated with five possible wave configurations (see Fig. 3.6):

- (a) Left-going shock wave: $Q_{i-1/2}^n = Q_i^n$.
- (b) Left-going rarefaction wave: $Q_{i-1/2}^n = Q_i^n$.
- (c) Transonic² rarefaction wave: $Q_{i-1/2}^n = q_s(Q_i^n, Q_{i-1}^n)$. This is the only case for which we cannot set the $Q_{i-1/2}^n$ value in a simple way. Further calculations are needed to evaluate the value q_s . The unknown value q_s satisfies

$$Q_{i-1}^n < q_s < Q_i^n,$$

and this is associated with the vertical ray, its characteristic speed is zero. Therefore, q_s is the solution to

$$f'(q_s) = 0. \quad (3.37)$$

- (d) Right-going rarefaction wave: $Q_{i-1/2}^n = Q_{i-1}^n$.
- (e) Right-going shock wave: $Q_{i-1/2}^n = Q_{i-1}^n$.

For a convex scalar flux, we can summarise all these possibilities

$$F_{i-1/2}^n = \begin{cases} f(Q_{i-1}^n) & \text{if } Q_{i-1}^n > q_s \text{ and } \dot{\sigma} > 0 \\ f(Q_i^n) & \text{if } Q_i^n < q_s \text{ and } \dot{\sigma} < 0 \\ f(q_s) & \text{if } Q_{i-1}^n < q_s < Q_i^n, \end{cases} \quad (3.38)$$

where the shock speed $\dot{\sigma}$ is given by:

$$\dot{\sigma} = \frac{f(Q_i^n) - f(Q_{i-1}^n)}{Q_i^n - Q_{i-1}^n}.$$

Proof. It can be seen that cases (a) and (b) correspond to left-going waves with the following possibilities:

²It is called transonic because it moves with velocity 0. In gas dynamics, this happens when one of the eigenvalues $u \pm c$ (c : sound speed) takes the value 0, thus when the fluid moves at the same speed as sound. In Fig. 3.6(c) the fluid is accelerated from a subsonic velocity to a supersonic one through a rarefaction wave.

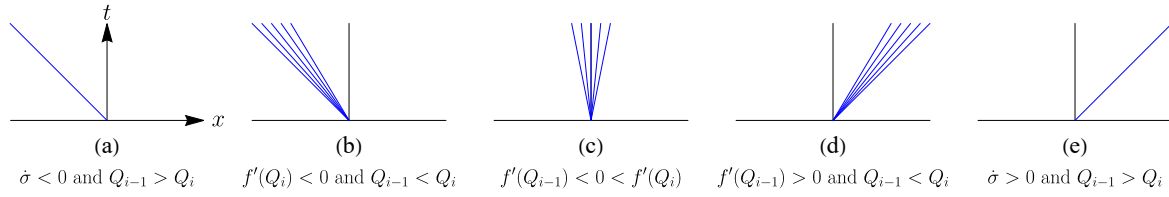


Figure 3.6 The five possible solutions to a scalar Riemann problem: (a) left-going shock wave; (b) left-going rarefaction wave; (c) transonic rarefaction wave; (d) right-going rarefaction wave; and (e) right-going shock wave.

- (a) if initially $Q_{i-1}^n > Q_i^n$, the wave is a shock propagating at velocity $\dot{\sigma} < 0$, and the characteristic velocity is negative: $f'(Q_i^n) < 0$. Since f is assumed to be convex (and thus $f'(q)$ is an increasing function of q) and q_s is defined as $f'(q_s) = 0$, then $Q_i^n < q_s$.
- (b) if initially $Q_{i-1}^n < Q_i^n$, then the wave is a rarefaction wave. The largest velocity must be negative: $f'(Q_i^n) < 0$. Since f is convex, then

$$f'(Q_{i-1}) < \frac{f(Q_i^n) - f(Q_{i-1}^n)}{Q_i^n - Q_{i-1}^n} < f'(Q_i),$$

and thus the ratio $\dot{\sigma}$ is also negative.

We repeat the reasoning for cases (c) to (e). $\square$

A more compact way used in Clawpack is given

$$F_{i-1/2}^n = \begin{cases} \min_{Q_{i-1}^n \leq q \leq Q_i^n} f(q) & \text{if } Q_{i-1}^n \leq Q_i^n, \\ \max_{Q_i^n \leq q \leq Q_{i-1}^n} f(q) & \text{if } Q_{i-1}^n \geq Q_i^n, \end{cases} \quad (3.39)$$

because q_s is the global minimum or maximum of f . This equation can be generalised to the non-convex case.

Remark. We have seen in § 1.2.2 that the Lax entropy condition is an extra condition imposed to shock waves for them to be physically admissible. A shock wave must dissipate energy, not create energy (or from a thermodynamical standpoint, entropy increases through a shock, and does not decrease). A shock wave satisfies the Lax entropy condition if its speed $\dot{\sigma}$ lies between bounds fixed by the initial data

$$f'(Q_{i-1}^n) > \dot{\sigma} > f'(Q_i^n). \quad (3.40)$$

For a scalar problem, the time-marching algorithm to solve the hyperbolic equation (3.1) is given by Eq. (3.6):

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (F_{i+1/2}^n - F_{i-1/2}^n),$$

or equivalently by using the notation $F_{i-1/2}^n = f(Q_{i-1/2}^\downarrow)$

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} [f(Q_{i+1/2}^\downarrow) - f(Q_i^n) - (f(Q_{i-1/2}^\downarrow) - f(Q_i^n))].$$

We can make an analogy with the formulation for linear equations, which emphasizes the role of fluctuations (see § 3.3). We put the equation above in a form consistent with LeVeque's notation:

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (\mathcal{A}^+ \Delta Q_{i-1/2} + \mathcal{A}^- \Delta Q_{i+1/2}),$$

where the operator $\mathcal{A}^\pm$ represent flux fluctuations, and are defined by

$$\mathcal{A}^+ \Delta Q_{i-1/2} = f(Q_i^n) - f(Q_{i-1/2}^\downarrow),$$

$$\mathcal{A}^- \Delta Q_{i+1/2} = f(Q_{i+1/2}^\downarrow) - f(Q_i^n).$$

High-resolution techniques involve defining the wave $W_{i-1/2}$ and speed $\dot{\sigma}_{i-1/2}$ associated with the Riemann problem:

$$W_{i-1/2} = Q_i - Q_{i-1},$$

$$\dot{\sigma}_{i-1/2} = \begin{cases} \frac{f(Q_i^n) - f(Q_{i-1}^n)}{Q_i^n - Q_{i-1}^n} & \text{if } Q_i \neq Q_{i-1}, \\ f'(Q_i^n) & \text{if } Q_i = Q_{i-1}. \end{cases}$$

When the Riemann solution is a shock wave, the speed chosen is the one given by the Rankine-Hugoniot equation. When it is a rarefaction wave, we have seen in § 2.3 that the Rankine-Hugoniot equation provides a correct estimate of the actual wave speed, and the wave behaviour can be approximated locally by a shock wave even if the latter would not satisfy the entropy condition (3.40). The big advantage is that we can treat all waves as (local) shock waves regardless of their actual nature. When the solution is not a transonic wave, we can also express the fluctuations as:

$$\mathcal{A}^+ \Delta Q_{i-1/2} = \dot{\sigma}_{i-1/2}^+ W_{i-1/2}, \quad (3.41)$$

$$\mathcal{A}^- \Delta Q_{i+1/2} = \dot{\sigma}_{i+1/2}^- W_{i+1/2}, \quad (3.42)$$

where $\dot{\sigma}^+ = \max(\dot{\sigma}, 0)$ and $\dot{\sigma}^- = \min(\dot{\sigma}, 0)$. Equation (3.41) is used in Clawpack for solving scalar problems.

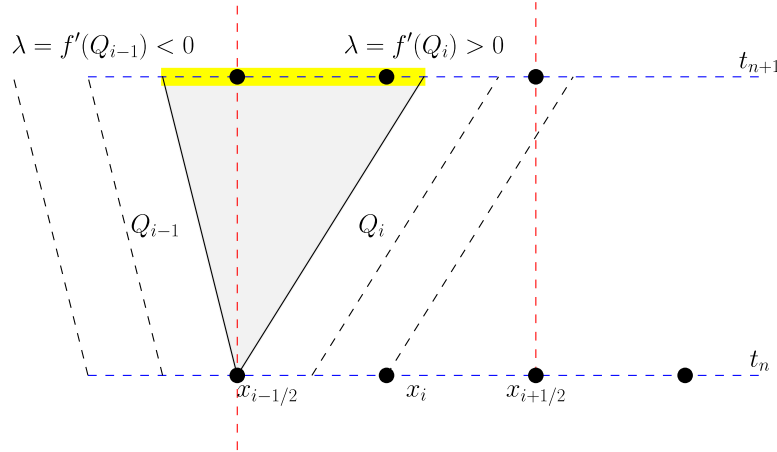


Figure 3.7 A transonic wave occurs whenever the velocity on the left and the right of the interface $x_{i-1/2}$ are not of the same sign. In this case, part of the interval $[x_{i-1/2}, x_{i+1/2}]$ at time t_{n+1} , highlighted in yellow in the figure, is not affected by the conditions at time t_n . There is thus a transonic wave that propagates information in the gray wedge around the interface $x_{i-1/2}$.

When the Riemann solution consists of a transonic rarefaction wave, the fluctuation terms $\mathcal{A}^\pm \Delta Q_{i\pm 1/2}$ need to be corrected using an *entropy fix*. In Clawpack, the wave W_i and speed $\dot{\sigma}_i$ are first computed, and from them, we determine the fluctuations using Eq. (3.41). If $f'(Q_{i-1}^n) < 0 < f'(Q_i^n)$

(see Fig. 3.7), then the fluctuations in Eq. (3.41) are replaced by one of the equations (for the interface from which the transonic wave originates):

$$\mathcal{A}^+ \Delta Q_{i-1/2} = f(Q_i^n) - f(q_s), \quad (3.43)$$

$$\mathcal{A}^- \Delta Q_{i+1/2} = f(q_s) - f(Q_i^n). \quad (3.44)$$

Although this approach based on an entropy fix is unnecessary for scalar problems, it is easy to generalise to nonlinear systems of hyperbolic equations, for which there is no easy way to determine the rarefaction wave structure exactly.

3.4.2 Systems of equations

The method used for scalar problems can be generalised to systems of hyperbolic equations. The crux lies in the determination of the interface value $Q_{i-1/2}^\downarrow$ (the value of q along the interface). This value is usually one of the intermediate states that connect the left and right states Q_i^n and Q_{i-1}^n through a series of shock and rarefaction waves, and thus it can be expressed as a function of these two states:

$$Q_{i-1/2}^\downarrow = Q_{i-1/2}^\downarrow(Q_{i-1}^n, Q_i^n).$$

When $Q_{i-1/2}^\downarrow$ is part of a transonic rarefaction wave, additional work is required to determine the wave structure.

The computational approach to solving the nonlinear Riemann problem is the same as the one taken for linear problems. When dealing with Godunov's equation (3.6), Clawpack still uses the wave-propagation form

$$\begin{aligned} Q_i^{n+1} &= Q_i^n - \frac{\Delta t}{\Delta x} (F_{i+1/2}^n - F_{i-1/2}^n), \\ &= Q_i^n - \frac{\Delta t}{\Delta x} (\mathcal{A}^+ \Delta Q_{i-1/2}^n + \mathcal{A}^- \Delta Q_{i+1/2}^n), \end{aligned} \quad (3.45)$$

where the operators $\mathcal{A}^\pm$ represent the fluctuations, and are defined by generalising the linear case (see § 3.3.3):

$$\mathcal{A}^- \Delta Q_{i+1/2}^n = f(Q_{i+1/2}^{\downarrow n}) - f(Q_i^n), \quad (3.46)$$

$$\mathcal{A}^+ \Delta Q_{i-1/2}^n = f(Q_i^n) - f(Q_{i-1/2}^{\downarrow n}), \quad (3.47)$$

These definitions are useful when the solution to the Riemann problem is a transonic wave.

When the solution is a shock or rarefaction wave, the fluctuations can also be approximated by considering that in the close vicinity of the initial state the solution behaves like a shock wave (see § 2.3), and like in the linear case, the fluctuations are given by:

$$\mathcal{A}^- \Delta Q_{i+1/2}^n = \sum_{k=1}^{M_w} \dot{\sigma}_{k,i+1/2}^- W_{k,i+1/2}^n, \quad (3.48)$$

$$\mathcal{A}^+ \Delta Q_{i-1/2}^n = \sum_{k=1}^{M_w} \dot{\sigma}_{k,i-1/2}^+ W_{k,i-1/2}^n, \quad (3.49)$$

where M_w is the number of waves (usually $M_w = m$), $\dot{\sigma}_{k,i+1/2}$ is the speed of the k th wave $W_{k,i+1/2}$ at $x_{i-1/2}$, $\dot{\sigma}^- = \min(0, \dot{\sigma})$ and $\dot{\sigma}^+ = \max(0, \dot{\sigma})$. When the solution involves only shocks, the k th wave speed s_k is the k eigenvalue ($\dot{\sigma}_k = \lambda_k$) and the Eqs. (3.48)–(3.49) give the same results as Eqs. (3.46)–(3.47) do, but in the general case, they are just an approximation of the interface fluxes. When

the solution involves rarefaction waves, their speed varies spatially, but a correct approximation is the following:

$$\dot{\sigma}_{k,i-1/2} = \frac{\lambda_{k,i-1} + \lambda_{k,i}}{2}.$$

The computational cost is high if we use exact Riemann solvers. A variety of approximate Riemann solvers have been proposed to reduce this cost (Toro, 2001; LeVeque, 2002). If a linearised Riemann solver (see below) is used, then the operators $\mathcal{A}^\pm$ are based on the positive and negative parts $\hat{A}^\pm$ of the linearised solver $\hat{A}$, the wave speeds $\dot{\sigma}_k$ match its eigenvalues $\hat{\lambda}_k$, and the waves W_k are right eigenvectors of $\hat{A}$. An entropy fix can be applied to modify the flux fluctuations.

Linearised solvers

Nonlinear equations

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = 0$$

can be linearised when the initial values q_l and q_r are sufficiently close to each other, and put in the linear form

$$\frac{\partial q}{\partial t} + \hat{A} \cdot \frac{\partial q}{\partial x} = 0,$$

for each interface $x_{i-1/2}$. The constant matrix $\hat{A}$ is an approximation of $f'(q)$ when Q_{i-1} and Q_i tend to q . The approximate solution involves m shock waves that are proportional to the right eigenvectors $q_{i-1/2}$ of A and move at speed $\dot{\sigma}_{i-1/2} = \lambda_{i-1/2}$ given by the eigenvalues of A . One possibility among other options is to set

$$A = f' \left(\frac{Q_i + Q_{i-1}}{2} \right).$$

An alternative choice is

$$\hat{A}_{i-1/2} = \frac{1}{2}(f'(Q_{i-1}) + f'(Q_i)).$$

This approach works fine when the solution is continuous, but may run into trouble when the solution involves discontinuities. Other constraints are needed to avoid the difficulties. The Roe function (studied later) is an example of linearised solvers (Roe, 1981).

When using the linearised Roe solver, the approximate solution involve only shock waves. As shock and rarefaction waves have the same behaviour in the close vicinity of a given state (see § 1.3.2), the approximate solution is expected to mimic the exact solution. There are, however, situations in which the agreement is no longer good: indeed, we have seen for the nonlinear case that when $\lambda(Q_{i-1}) < 0 < \lambda(Q_i)$, then the left and right states Q_i and Q_{i-1} are connected by a rarefaction wave (see § 3.4.1). For hyperbolic systems, the same problem will arise whenever one or more eigenvalues λ^k satisfy $\lambda_{i-1}^k < 0 < \lambda_i^k$. The Riemann solver must be modified to handle this case properly. Such a modification is called an *entropy fix*.

Two-wave solvers

Several approximate solvers are based on the idea that the Riemann solution can be approximated by picking up two of the m waves, W_1 and W_2 , and defining an intermediate state Q_* such that $W_1 = Q_* - Q_l$ and $W_2 = Q_r - Q_*$. The Rankine-Hugoniot implies that $f(Q_l) - f(Q_*) = \dot{\sigma}_1 W_1$ and $f(Q_r) - f(Q_*) = \dot{\sigma}_2 W_2$. By adding these two equations, we end up with a system of m equations

$$f(Q_r) - f(Q_l) = \dot{\sigma}_1 W_1 + \dot{\sigma}_2 W_2,$$

which gives

$$Q_* = \frac{f(Q_r) - f(Q_l) - \dot{\sigma}_2 Q_r + \dot{\sigma}_1 Q_l}{\dot{\sigma}_1 - \dot{\sigma}_2}.$$

The various solvers proposed so far differ by the choice of the speeds $\dot{\sigma}_1$ and $\dot{\sigma}_2$ along with the waves W_1 and W_2 . Lax-Friedrichs and Harten-Lax-van Leer (HLL) are classic solvers (see § 3.3.3 for a rationale).

The advantage of HLL solvers is that they usually do not need an entropy fix to compute transonic rarefaction waves. As they involve only two waves (thereby ignoring all other waves), they may lead to poorer resolutions for systems made of $m > 2$ equations (Toro, 2019).

3.5 Roe solver

The Roe solver linearises the governing equation (3.1):

$$\frac{\partial q}{\partial t} + \hat{A}_{i-1/2} \frac{\partial q}{\partial x} = 0.$$

The matrix $\hat{A}_{i-1/2}$ is constructed so that it approximates $f'(q)$ in the neighbourhood of Q_i and Q_{i-1} and satisfies the conditions

1. Continuity condition:

$$\hat{A}_{i-1/2} \rightarrow f'(q) \text{ when } Q_{i-1}, Q_i \rightarrow q.$$

2. Hyperbolicity: $\hat{A}_{i-1/2}$ is diagonalisable, with m right eigenvectors $\hat{w}_{k,i-1/2}$ associated with eigenvalues $\lambda_{k,i-1/2}$.
3. Roe linearisation. This third property states that if Q_{i-1} and Q_i are connected by a single wave $W_{i-1/2} = Q_i - Q_{i-1}$ in the original Riemann problem, then $W_{i-1/2}$ should also be an eigenvector of $\hat{A}_{i-1/2}$:

$$f(Q_i) - f(Q_{i-1}) = \hat{A}_{i-1/2} \cdot (Q_i - Q_{i-1}) = \dot{\sigma}(Q_i - Q_{i-1}), \quad (3.50)$$

where $\dot{\sigma}$ is the wave speed: $\dot{\sigma} = \lambda_{k,i-1/2}$ since $W_{i-1/2}$ is a right eigenvector of $\hat{A}_{i-1/2}$.

Formally, the matrix $\hat{A}_{i-1/2}$ can be determined by integrating the Jacobian over a straight-line path $q(\xi) = Q_{i-1} + \xi(Q_i - Q_{i-1})$:

$$\hat{A}_{i-1/2} = \int_0^1 \frac{df(q(\xi))}{dq} d\xi, \quad (3.51)$$

for $0 \leq \xi \leq 1$. Indeed, we have

$$\begin{aligned} f(Q) - f(Q_{i-1}) &= \int_0^1 \frac{df}{d\xi} d\xi \\ &= \int_0^1 \frac{df(q(\xi))}{dq} \cdot \frac{dq}{d\xi} d\xi \\ &= \left(\int_0^1 \frac{df(q(\xi))}{dq} d\xi \right) \cdot (Q_i - Q_{i-1}), \end{aligned}$$

since the ξ derivative of q is $q' = Q_i - Q_{i-1}$. By comparing the equation above with Eq. (3.50), we deduce that the linearised matrix is given by Eq. (3.51).

There is no guarantee that the resulting matrix is diagonalisable with m real eigenvalues and that it takes an analytical form. By making a change of variable, Roe (1981) showed that this difficulty can often be overcome (LeVeque, 2002, see pp. 317-323).

3.6 Two-wave solver: HLL solver

The idea underpinning the HLL solver's derivation is that the solution to the Riemann problem consists of two shock waves separating an intermediate state from the left and right initial states (see also § 3.3.3). The speeds $\dot{\sigma}_1$ and $\dot{\sigma}_2$ of these waves are given by the Rankine-Hugoniot equation

$$f(Q_{i-1}) - f(Q_*) = \dot{\sigma}_1(Q_{i-1} - Q_*), \quad (3.52)$$

$$f(Q_*) - f(Q_i) = \dot{\sigma}_2(Q_* - Q_i) \quad (3.53)$$

Solving Eqs. (3.52) and (3.53) for Q_* and $F_* = f(Q_*)$ gives

$$Q_* = \frac{\dot{\sigma}_2 Q_i - \dot{\sigma}_1 Q_{i-1}}{\dot{\sigma}_2 - \dot{\sigma}_1} + \frac{F_{i-1} - F_i}{\dot{\sigma}_2 - \dot{\sigma}_1} \quad (3.54)$$

$$F_* = \frac{\dot{\sigma}_2 F_{i-1} - \dot{\sigma}_1 F_i}{\dot{\sigma}_2 - \dot{\sigma}_1} - \dot{\sigma}_2 \dot{\sigma}_2 \frac{Q_{i-1} - Q_i}{\dot{\sigma}_2 - \dot{\sigma}_1}, \quad (3.55)$$

with $F_i = f(Q_i)$. For the Harten–Lax–van-Leer (HLL) solver, the speeds are defined as the lower and upper bounds of all characteristic speeds. [Einfeldt \(1988\)](#) suggested

$$\dot{\sigma}_{1, i-1/2} = \min_{1 \leq k \leq m} \left(\min(\lambda_{k, i}, \hat{\lambda}_{k, i-1/2}) \right), \quad (3.56)$$

$$\dot{\sigma}_{2, i-1/2} = \max_{1 \leq k \leq m} \left(\max(\lambda_{k, i-1}, \hat{\lambda}_{k, i-1/2}) \right), \quad (3.57)$$

where $\lambda_{k, i}$ is the k th eigenvalue of the Jacobian $f'(Q_i)$ and $\hat{\lambda}_{k, i-1/2}$ is k th eigenvalue of the Roe matrix (linearised Jacobian). The resulting scheme is called the HLLE solver.

3.7 Alternative: the f-wave method

An alternative approach to the wave decomposition is to first split the jump in f into f-waves:

$$f(Q_i) - f(Q_{i-1}) = \sum_{k=1}^{m_w} Z_{k, i-1/2}, \quad (3.58)$$

moving at speeds $\dot{\sigma}_{k, i-1/2}$, then express the fluctuations in terms of the f-waves. This method is useful to study the second-order accuracy of wave-propagation methods or in the context of spatially-varying flux functions $f(q, x)$ ([LeVeque, 2002](#), see § 15.5). It also guarantees that approximate Riemann solvers are conservative.

When dealing with a linear or linearised problems, we can decompose $f(Q_i) - f(Q_{i-1})$ as a linear combination of the right eigenvectors $\hat{w}_{k, i-1/2}$ of the linearised matrix $\hat{A}_{i-1/2}$:

$$f(Q_i) - f(Q_{i-1}) = \sum_{k=1}^{m_w} \beta_{i-1/2}^k \hat{w}_{i-1/2}^k, \quad (3.59)$$

where the coefficient vector $\beta_{i-1/2}$ is the solution to the linear system (3.59):

$$\beta_{i-1/2} = R^{-1} \cdot (f(Q_i) - f(Q_{i-1})) = L \cdot (f(Q_i) - f(Q_{i-1})). \quad (3.60)$$

The f-waves are then

$$Z_{i-1/2}^k = \beta_{i-1/2}^k \hat{w}_{i-1/2}^k. \quad (3.61)$$

These f-waves are related to the waves $W_{k,i-1/2}$ when the wave speeds are nonzero

$$W_{i-1/2}^k = \frac{Z_{i-1/2}^k}{\dot{\sigma}_{i-1/2}^k}, \quad (3.62)$$

and the fluctuations are

$$A_{i-1/2}^- \Delta Q_{i-1/2} = \sum_{k: \dot{\sigma}_k < 0} Z_{i-1/2}^k, \quad (3.63)$$

$$A_{i-1/2}^+ \Delta Q_{i-1/2} = \sum_{k: \dot{\sigma}_k > 0} Z_{i-1/2}^k. \quad (3.64)$$

Another advantage of the f-wave decomposition is the possibility of including the source term into this decomposition (Ketcheson *et al.*, 2013). The f-wave approach is highly recommended when the solution is close to a steady state, as it leads to a well-balanced system. Let us assume that we are concerned with the equation

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = S(q, x) \quad (3.65)$$

where $S(q, x)$ is the source term. We now express the flux difference at the interface $x_{i-1/2}$

$$f(Q_i) - f(Q_{i-1}) - \Delta x S(Q_i, Q_{i-1}, x_{i-1/2}) = \sum_{k=1}^{m_{\text{fw}}} Z_{i-1/2}^k = \beta_{i-1/2}^k \hat{w}_{i-1/2}^k, \quad (3.66)$$

where $S(Q_i, Q_{i-1}, x_{i-1/2})$ is a representative value of the source term S at the interface $x_{i-1/2}$, which depends on Q_i and Q_{i-1} .

3.8 High-resolutions methods

3.8.1 Scalar problems

Derivation of the time-marching algorithm

Let us consider the scalar advection equation

$$\frac{\partial q}{\partial t} + \bar{u} \frac{\partial q}{\partial x} = 0, \quad (3.67)$$

where $\bar{u} > 0$ denotes the advection velocity. We assume that from the cell average Q_i^n at time t_n , we can infer a piecewise-polynomial approximation order p

$$\tilde{q}(x, t_n) = \sum_{k=0}^p a_{k,i} (x - x_i)^k = Q_i^n + \sum_{k=1}^p a_{k,i} (x - x_i)^k \text{ over } x_{i-1/2} \leq x \leq x_{i+1/2}, \quad (3.68)$$

where the coefficients $a_{k,i}$ satisfy

$$a_{0,i} = Q_i^n \text{ and } \int_{x_{i-1/2}}^{x_{i+1/2}} \sum_{k=1}^p a_{k,i} (x - x_i)^k dx = \sum_{k=1}^p a_{k,i} \frac{((-1)^k + 1)}{k+1} \left(\frac{\Delta x}{2}\right)^{k+1} = 0$$

so that the cell average of $\tilde{q}$ is Q_i^n .

The solution to Eq. (3.67) subject to the initial condition (3.68) is

$$q(x, t_{n+1}) = \tilde{q}(x - \bar{u} \Delta t, t_n). \quad (3.69)$$

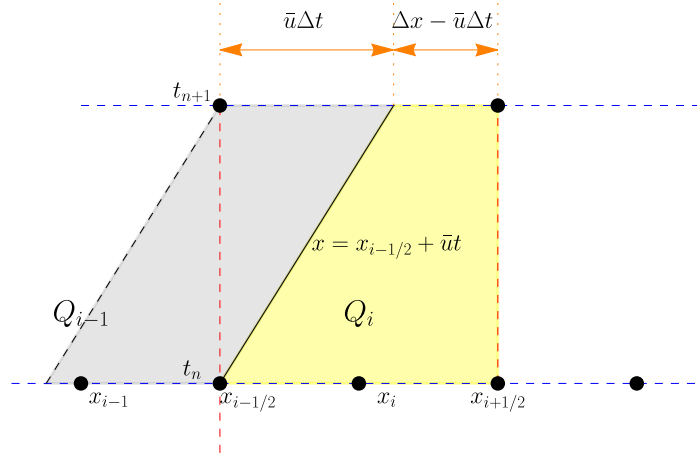


Figure 3.8 Domain of influence: the yellow area shows the time domain influenced by the condition $q^n(x, t_n)$ over $[x_{i-1/2}, x_{i+1/2}]$ at time t^{n+1} while the grey are shows the domain influenced by $q^n(x, t_n)$ over $[x_{i-3/2}, x_{i-1/2}]$.

Integrating Eq. (3.69) over $[x_{i-1/2}, x_{i+1/2}]$ gives

$$\begin{aligned} Q_i^{n+1} &= \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} q(x, t^{n+1}) dx \\ &= \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \tilde{q}(x - \bar{u}\Delta t, t^n) dx \\ &= \frac{1}{\Delta x} \int_{x_{i-1/2} - \bar{u}\Delta t}^{x_{i+1/2} - \bar{u}\Delta t} \tilde{q}(\xi, t^n) d\xi \end{aligned}$$

and taking into account the domain of influence (see Fig. 3.8) gives

$$\begin{aligned} Q_i^{n+1} &= Q_{i-1}^n \frac{\bar{u}\Delta t}{\Delta x} + Q_i^n \left(1 - \frac{\bar{u}\Delta t}{\Delta x}\right) + \frac{1}{\Delta x} \sum_{k=1}^p \int_{x_{i-1/2} - \bar{u}\Delta t}^{x_{i-1/2}} a_{k,i-1} (x - x_{i-1})^k dx \\ &\quad + \frac{1}{\Delta x} \sum_{k=1}^p \int_{x_{i-1/2}}^{x_{i+1/2} - \bar{u}\Delta t} a_{k,i} (x - x_i)^k dx. \end{aligned}$$

To first order in Δx and second order in Δt , this expression reads

$$Q_i^{n+1} = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x} (Q_i^n - Q_{i-1}^n) - \frac{1}{2} \frac{\bar{u}\Delta t}{\Delta x} (\Delta x - \bar{u}\Delta t) (a_{1,i} - a_{1,i-1}). \quad (3.70)$$

Alternative 1

This result can be derived differently (which will be useful when extending the method to higher dimensions). The second-order Taylor expansion of $q(x, t_{n+1} - \Delta t)$ is

$$\begin{aligned} q(x, t_n) &= q(x, t_{n+1} - \Delta t) = q(x, t_{n+1}) - \Delta t \frac{\partial}{\partial t} q(x, t_{n+1}) + \frac{\Delta t^2}{2} \frac{\partial^2}{\partial t^2} q(x, t_{n+1}), \\ &= q(x, t_{n+1}) + \bar{u}\Delta t \frac{\partial}{\partial x} q(x, t_{n+1}) + \frac{\bar{u}^2 \Delta t^2}{2} \frac{\partial^2}{\partial x^2} q(x, t_{n+1}), \end{aligned} \quad (3.71)$$

since

$$\begin{aligned}\frac{\partial q}{\partial t} &= -\bar{u} \frac{\partial q}{\partial x}, \\ \frac{\partial^2 q}{\partial t^2} &= -\bar{u} \frac{\partial^2 q}{\partial t \partial x} = -\bar{u} \frac{\partial}{\partial x} \left(\frac{\partial q}{\partial t} \right) = \bar{u}^2 \frac{\partial^2 q}{\partial x^2}.\end{aligned}$$

By integrating Eq. (3.71) over $[x_{i-1/2}, x_{i+1/2}]$ and dividing by Δx gives

$$Q_i^n = Q_i^{n+1} + \bar{u} \frac{\Delta t}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial}{\partial x} q(x, t_{n+1}) dx + \frac{\bar{u}^2 \Delta t^2}{2 \Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial^2}{\partial x^2} q(x, t_{n+1}) dx, \quad (3.72)$$

where the derivatives should be understood in the sense of distributions. Recall that for any piecewise $\mathcal{C}^1$ function q with a jump at x_j

$$q(x, t) = \begin{cases} Q_{i-1} + a_{i-1}(x - x_{i-1}) & \text{if } x < x_j, \\ Q_i + a_i(x - x_i) & \text{if } x > x_j \end{cases}$$

the derivative of its distribution is

$$T'[q] = \llbracket q \rrbracket \delta(x - x_j) + \begin{cases} a_{i-1} & \text{if } x < x_j, \\ a_i & \text{if } x > x_j \end{cases}$$

where δ is the Dirac function and the jump in q is

$$\llbracket q \rrbracket = \lim_{\substack{x \rightarrow x_j \\ x > x_j}} q(x, t) - \lim_{\substack{x \rightarrow x_j \\ x < x_j}} q(x, t) = Q_i + a_i(x_j - x_i) - Q_{i-1} - a_{i-1}(x_j - x_{i-1}).$$

The second derivative of the associate distribution is

$$T''[q] = \llbracket q' \rrbracket \delta(x - x_j),$$

where $\llbracket q' \rrbracket = a_i - a_{i-1}$.

Moreover Eq (3.69) tells us that $q(x, t_{n+1}) = \tilde{q}(x - \bar{u}\Delta t, t_n)$, and thus we can express the integral terms of Eq. (3.72) as

$$\begin{aligned}\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial}{\partial x} q(x, t_{n+1}) dx &= \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial}{\partial x} \tilde{q}(x - \bar{u}\Delta t, t_n) dx \\ &= \int_{x_{i-1/2} - \bar{u}\Delta t}^{x_{i+1/2} - \bar{u}\Delta t} \frac{\partial}{\partial x} \tilde{q}(x, t_n) dx.\end{aligned} \quad (3.73)$$

Given that the function $\tilde{q}$ is discontinuous at $x_j = x_{i-1/2} = x_i - \Delta x/2$, the jumps are

$$\llbracket \tilde{q} \rrbracket = Q_i^n - Q_{i-1}^n - \frac{\Delta x}{2}(a_{i-1} + a_i) \text{ and } \llbracket \tilde{q}' \rrbracket = a_i - a_{i-1}.$$

With these two definitions, we can decompose the integral term of Eq. (3.73):

$$\begin{aligned}\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial}{\partial x} \tilde{q}(x, t_{n+1}) dx &= a_{i-1} \bar{u} \Delta t + (\Delta x - \bar{u} \Delta t) a_i + \llbracket \tilde{q} \rrbracket, \\ &= Q_i^n - Q_{i-1}^n + (a_i - a_{i-1}) \frac{\Delta x}{2} - (a_i - a_{i-1}) \bar{u} \Delta t,\end{aligned} \quad (3.74)$$

and

$$\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial^2}{\partial x^2} \tilde{q}(x, t_{n+1}) dx = \llbracket \tilde{q}' \rrbracket = a_i - a_{i-1}. \quad (3.75)$$

By substituting Eqs. (3.74) and (3.75) into Eq. (3.72), we find the second-order (in time) approximation of Q_i^{n+1} given by Eq. (3.70):

$$Q_i^{n+1} = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x}(Q_i^n - Q_{i-1}^n) - \frac{1}{2} \frac{\bar{u}\Delta t}{\Delta x}(\Delta x - \bar{u}\Delta t)(a_i - a_{i-1}).$$

If we now assume that $\bar{u} < 0$, then we find:

$$\begin{aligned} \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial}{\partial x} \tilde{q}(x, t_{n+1}) dx &= -a_i \bar{u} \Delta t + (\Delta x + \bar{u} \Delta t) a_{i+1} + [\tilde{q}], \\ &= Q_{i+1}^n - Q_i^n + (a_i - a_{i+1}) \frac{\Delta x}{2} + (a_i - a_{i+1}) \bar{u} \Delta t, \end{aligned}$$

and

$$\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial^2}{\partial x^2} \tilde{q}(x, t_{n+1}) dx = [\tilde{q}'] = a_{i+1} - a_i.$$

We deduce that Q_i^{n+1} is given by

$$Q_i^{n+1} = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x}(Q_{i+1}^n - Q_i^n) + \frac{1}{2} \frac{\bar{u}\Delta t}{\Delta x}(\Delta x + \bar{u}\Delta t)(a_{i+1} - a_i). \quad (3.76)$$

We can merge Eqs. (3.70) and (3.76) into a single equation

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x}(\bar{u}^+ \Delta Q_{i-1/2} + \bar{u}^- \Delta Q_{i+1/2}) - \frac{|\bar{u}| \Delta t}{2} \left(1 - \frac{|\bar{u}| \Delta t}{\Delta x}\right) \Delta a, \quad (3.77)$$

where $\bar{u}^+ = \max(\bar{u}, 0)$, $\bar{u}^- = \min(\bar{u}, 0)$, $\Delta Q_{i-1/2} = Q_i - Q_{i-1}$, $\Delta Q_{i+1/2} = Q_{i+1} - Q_i$, and

$$\Delta a = \begin{cases} a_{i+1} - a_i & \text{if } \bar{u} < 0, \\ a_i - a_{i-1} & \text{if } \bar{u} > 0. \end{cases}$$

Alternative 2

Let us assume again that $\bar{u} > 0$. Equation (3.70) can also be obtained by applying the time-marching algorithm (3.6):

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (F_{i+1/2}^n - F_{i-1/2}^n), \quad (3.78)$$

where the interface flux is

$$\begin{aligned} F_{i\pm 1/2}^n &= \frac{1}{\Delta t} \int_{t_n}^{t_{n+1}} \bar{u} q(x_{i\pm 1/2}, t) dt, \\ &= \frac{\bar{u}}{\Delta t} \int_{t_n}^{t_{n+1}} \tilde{q}(x_{i\pm 1/2} - \bar{u}t) dt \\ &= \frac{1}{\Delta t} \int_{x_{i\pm 1/2} - \bar{u}\Delta t}^{x_{i\pm 1/2}} \tilde{q}(\xi) d\xi, \end{aligned}$$

from which we deduce the following expressions to first order

$$F_{i-1/2} = \bar{u} Q_{i-1} + \frac{1}{2} a_{i-1} \bar{u} (\Delta x - \bar{u} \Delta t), \quad (3.79)$$

and

$$F_{i+1/2} = \bar{u} Q_i + \frac{1}{2} a_i \bar{u} (\Delta x - \bar{u} \Delta t). \quad (3.80)$$

Using these expressions in Eq. (3.78), we obtain Eq. (3.70) again. We can interpret the latter by noting that

$$\bar{u}(Q_i^n - Q_{i-1}^n) = \bar{u}\Delta Q_{i-1/2} = \bar{u}^+\Delta Q_{i-1/2} + \bar{u}^-\Delta Q_{i+1/2},$$

where $\bar{u}^+ = \max(\bar{u}, 0)$, $\bar{u}^- = \min(\bar{u}, 0)$, and defining the flux corrections when $\bar{u} > 0$

$$\hat{F}_{i-1/2} = \frac{a_{i-1}}{2}\bar{u}(\Delta x - \bar{u}\Delta t) \text{ and } \hat{F}_{i+1/2} = \frac{a_i}{2}\bar{u}(\Delta x - \bar{u}\Delta t).$$

we can recast Eq. (3.70) in the following form

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (\bar{u}^+\Delta Q_{i-1/2} + \bar{u}^-\Delta Q_{i+1/2}) - \frac{\Delta t}{\Delta x} (F'_{i+1/2} - F'_{i-1/2}). \quad (3.81)$$

When $\bar{u} < 0$, then the second-order (in time) flux corrections are

$$\hat{F}_{i-1/2} = -\frac{a_i}{2}\bar{u}(\Delta x + \bar{u}\Delta t) \text{ and } \hat{F}_{i+1/2} = -\frac{a_{i+1}}{2}\bar{u}(\Delta x + \bar{u}\Delta t).$$

Alternative 3

It is possible to cast the time-marching algorithm (3.70) in a slightly different form. This new form will lead to a set of coupled ordinary differential equations (method of lines). To that end, we need new variables (see Fig. 3.9): at each interface $x_{i-1/2}$, we define the value of q on its right:

$$q_{i-1/2}^+ = Q_i^n - a_i \frac{\Delta x}{2}, \quad (3.82)$$

while the value of q on the left of $x_{i-1/2}$ is

$$q_{i-1/2}^- = Q_{i-1}^n + a_{i-1} \frac{\Delta x}{2}. \quad (3.83)$$

The jump of q at the interface is denoted by

$$\Delta q_{i-1/2} = q_{i-1/2}^+ - q_{i-1/2}^- = Q_i^n - Q_{i-1}^n - \frac{\Delta x}{2}(a_{i-1} + a_i) \quad (3.84)$$

while the q variation over $[x_{i-1/2}, x_{i+1/2}]$ is denoted by

$$\Delta q_i = q_{i+1/2}^- - q_{i-1/2}^+ = a_i \Delta x. \quad (3.85)$$

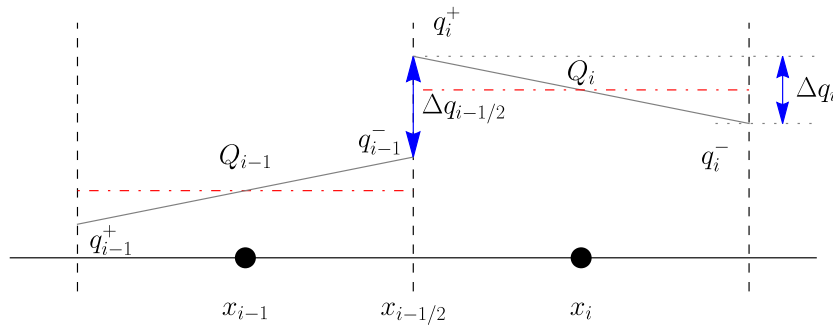


Figure 3.9 Notation.

Substituting Eqs. (3.82) and (3.83) into the time-marching equation (3.70) (for $\bar{u} > 0$) and keeping first-order terms in Δx and Δt leads to

$$Q_i^{n+1} = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x} (q_{i-1/2}^+ - q_{i-1/2}^- + a_i\Delta x) = Q_i^n - \frac{\bar{u}\Delta t}{\Delta x} (\Delta q_{i-1/2} + \Delta q_i).$$

If we now take the case $\bar{u} < 0$ into account and generalise the previous equation, we arrive at the following variant of the time-marching equation (3.70):

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (\bar{u}^+ \Delta q_{i-1/2} + \bar{u}^- \Delta q_{i+1/2} + \bar{u} \Delta q_i),$$

and if we take the limit $\Delta t \rightarrow 0$, then we obtain a set of coupled ordinary differential equations

$$\frac{\partial Q_i}{\partial t} = -\frac{1}{\Delta x} (\bar{u}^+ \Delta q_{i-1/2} + \bar{u}^- \Delta q_{i+1/2} + \bar{u} \Delta q_i) \text{ for } 1 \leq i \leq n. \quad (3.86)$$

This equation is used in SharpClaw, which implements *essentially non-oscillating* (ENO) and *weighted essentially non-oscillating* (WENO) algorithms to extend the accuracy of the numerical solutions of hyperbolic problems to arbitrarily high order accuracy by using (i) spatial reconstruction of the solution at each time step t^{n+1} and (ii) a high-order accurate ODE solver (often based a Runge–Kutta algorithm) (Ketcheson *et al.*, 2013). ENO and WENO algorithms are detailed elsewhere (Shu, 1998, 2020).

Choice of slopes

There are three obvious choices for slope a_i when reconstructing a piecewise-linear approximation of $q(x, t)$:

- Centred slope (Fromm)

$$a_i = \frac{Q_{i+1}^n - Q_{i-1}^n}{2\Delta x}.$$

- Upwind slope (beam-warming)

$$a_i = \frac{Q_i^n - Q_{i-1}^n}{\Delta x}.$$

- Downwind slope (Lax–Wendroff)

$$a_i = \frac{Q_{i+1}^n - Q_i^n}{\Delta x}.$$

High-resolution methods improve accuracy when the solution is smooth, but they also tend to smear discontinuities and create spurious fluctuations. A strategy to avoid this issue is to apply a second-order correction wherever the solution is smooth, but to use a first-order scheme near discontinuities. This can be achieved by limiting the slope a_i . Methods implementing this strategy are called *slope-limiter methods*. An example of these methods is the minmod slope:

$$a_i = \text{minmod} \left(\frac{Q_i^n - Q_{i-1}^n}{\Delta x}, \frac{Q_{i+1}^n - Q_i^n}{\Delta x} \right),$$

where the minmod function defined by:

$$\text{minmod}(a, b) = \begin{cases} a & \text{if } |a| < |b| \text{ and } ab > 0, \\ b & \text{if } |b| < |a| \text{ and } ab > 0, \\ 0 & \text{if } ab \leq 0, \end{cases}$$

returns the argument that is smaller in absolute value when the arguments are of the same sign, but zero when they are of different sign (which indicates a local minimum or maximum). A variant is the *superbee* limiter:

$$a_i = \maxmod \left(a_i^{(1)}, a_i^{(2)} \right),$$

where the maxmod function returns the largest argument in magnitude and

$$a_i^{(1)} = \minmod \left(\frac{Q_{i+1}^n - Q_i^n}{\Delta x}, 2 \frac{Q_i^n - Q_{i-1}^n}{\Delta x} \right),$$

$$a_i^{(2)} = \minmod \left(2 \frac{Q_{i+1}^n - Q_i^n}{\Delta x}, \frac{Q_i^n - Q_{i-1}^n}{\Delta x} \right).$$

Slope-limiter methods were initially developed by van Leer in an approach called *monotonic upstream-centered scheme for conservation laws* (MUSCL) (van Leer, 1979).

Alternative: flux-limiter methods

In the time-marching algorithm (3.79), the interface fluxes were given by Eqs. (3.79) and (3.80). Since the slope a_{i-1} is expressed as the difference $\Delta Q_{i-1/2}$ between the cell averages Q_i and Q_{i-1} when applying a slope-limited method, then this means that we actually express the flux (3.79) as

$$\begin{aligned} F_{i-1/2} &= \bar{u} Q_{i-1} + \frac{1}{2} a_{i-1} \bar{u} \Delta x \left(1 - \frac{\bar{u} \Delta t}{\Delta x} \right) a_{i-1}, \\ &= \bar{u}^- Q_i + \bar{u}^+ Q_{i-1} + \frac{1}{2} a_{i-1} |\bar{u}| \left(1 - \left| \frac{\bar{u} \Delta t}{\Delta x} \right| \right) a_{i-1} \Delta x, \\ &= \bar{u}^- Q_i + \bar{u}^+ Q_{i-1} + \frac{1}{2} a_{i-1} |\bar{u}| \left(1 - \left| \frac{\bar{u} \Delta t}{\Delta x} \right| \right) \delta(\Delta Q_{i-1/2}), \end{aligned}$$

where δ is a function that modulates the difference $\Delta Q_{i-1/2}$. By doing so, we can turn a slope-limiter method into a flux-limiter method:

$$\delta(\Delta Q_{i-1/2}) = \phi(\theta_{i-1/2}) \delta(\Delta Q_{i-1/2}), \quad (3.87)$$

where the function ϕ measures the degree of smoothness near $x_{i-1/2}$:

$$\phi(\Delta Q_{i-1/2}) = \frac{\delta(\Delta Q_{j-1/2})}{\delta(\Delta Q_{i-1/2})} \text{ with } j = \begin{cases} i-1 & \text{if } |\bar{u}| > 0, \\ i+1 & \text{if } |\bar{u}| < 0. \end{cases} \quad (3.88)$$

Typically, when the function is smooth at $x_{i-1/2}$, then ϕ should be close to unity, whereas it takes much higher or lower values if there is a discontinuity. For linear methods, we can encode the previous choices of a_i by setting

- Upwind scheme: $\phi(\theta) = 0$.
- Downwind slope (Lax-Wendroff): $\phi(\theta) = 1$.
- Upwind slope (beam-warming): $\phi(\theta) = \theta$.
- Centred slope (Fromm): $\phi(\theta) = \frac{1}{2}(1 + \theta)$.

For high-resolution methods, limiters of common use are:

- minmod: $\phi(\theta) = \minmod(1, \theta)$.

- superbee: $\phi(\theta) = \max(0, \min(1, 2\theta), \min(2, \theta))$.
- monotonised centred (MC): $\phi(\theta) = \max(0, \min((1 + \theta)/2, 2, 2\theta))$.
- van Leer:

$$\phi(\theta) = \frac{\theta + |\theta|}{1 + |\theta|}.$$

All these methods are *total-variation diminishing* (TVD):

$$\text{TV}(Q^{n+1}) \leq \text{TV}(Q^n), \quad (3.89)$$

where

$$\text{TV}(Q^{n+1}) = \sum_i |Q_i - Q_{i+1}|.$$

The total variation (3.89) is the discretisation of the total variation for continuous functions (or distributions)

$$\text{TV}(q) = \int |f'(q)| dx.$$

The exact solution to the linear advection equation does not change shape over time, and thus its total variation must be constant in time. For the numerical solution to the linear advection equation, we expect that the total variation does not increase if no spurious oscillations are created by the numerical scheme.

A theorem due to [Harten \(1983\)](#) guarantees that if ϕ varies within the range

$$0 \leq \phi(\theta) \leq \minmod(1, \theta),$$

then the method is TVD. [Sweby \(1984\)](#) found that second-order TVD methods must lie in the region bounded by the Lax–Wendroff and beam-warming curves (see Fig. 3.10).

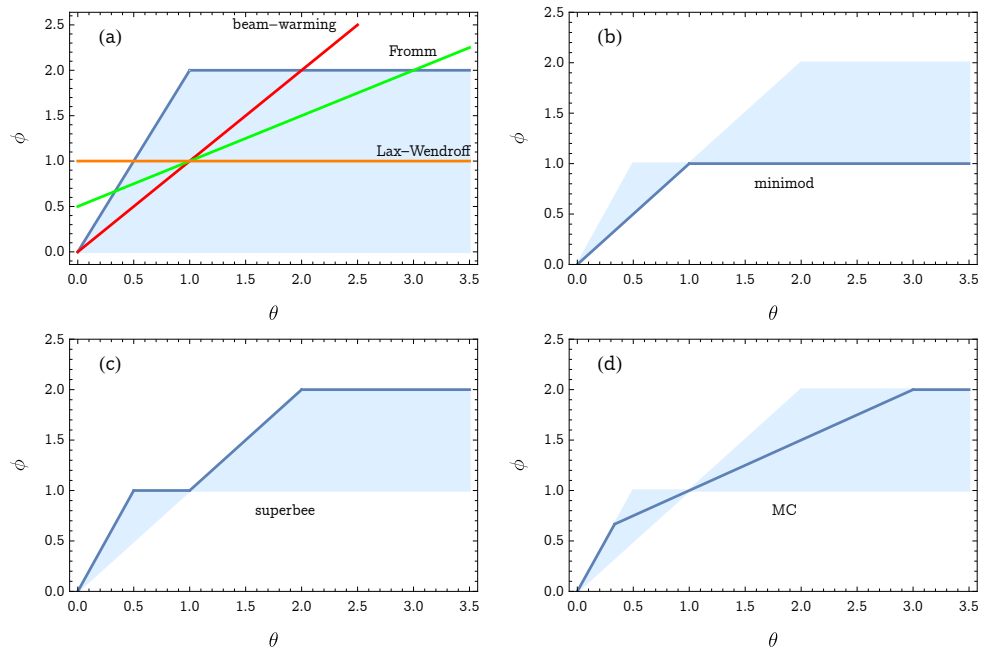


Figure 3.10 (a) Comparison of the beam-warming, Fromm, and Lax-Wendroff limiters. The coloured area shows where the function values must lie for the method to be TVD. (b) Minmod limiter. The coloured area shows where the function values must lie for the method to be second-order TVD. (c) Superbee limiter. (d) MC limiter.

3.8.2 Linear systems

The Lax–Wendroff method

Let us consider the following linear hyperbolic problem

$$\frac{\partial q}{\partial t} + A \cdot \frac{\partial q}{\partial x} = 0. \quad (3.90)$$

where $q = (q_k)_{1 \leq k \leq n}$ is the unknown vector, $A = R \cdot \Lambda \cdot L$ is a $m \times m$ matrix with right eigenvectors w_k and eigenvalues λ_k ($1 \leq k \leq m$). L is the left-eigenvector matrix ($L = R^{-1}$) whose rows are made of the left eigenvectors v_k , and $\Lambda = \text{diag}(\lambda_{1 \leq k \leq n})$ is the spectral matrix.

The second-order Taylor expansion of $q(x, t_n + \Delta t)$ is

$$q(x, t_n + \Delta t) = q(x, t_n) + \Delta t \frac{\partial q}{\partial t}(x, t_n) + \frac{1}{2} \Delta t^2 \frac{\partial^2 q}{\partial t^2}(x, t_n). \quad (3.91)$$

From Eq. (3.90), we deduce that

$$\frac{\partial q}{\partial t} = -A \cdot \frac{\partial q}{\partial x},$$

and after differentiating it with respect to time gives

$$\begin{aligned} \frac{\partial^2 q}{\partial t^2} &= -A \cdot \frac{\partial}{\partial x} \left(-A \cdot \frac{\partial q}{\partial x} \right), \\ &= A^2 \cdot \frac{\partial^2 q}{\partial x^2}. \end{aligned}$$

Equation (3.90) can be recast

$$q(x, t_n + \Delta t) = q(x, t_n) - \Delta t A \cdot \frac{\partial q}{\partial x}(x, t_n) + \frac{1}{2} \Delta t^2 A^2 \cdot \frac{\partial^2 q}{\partial x^2}(x, t_n). \quad (3.92)$$

We discretise the spatial derivatives of Eq. (3.92) using central finite differences:

$$\frac{\partial q}{\partial x} = \frac{Q_{i+1}^n - Q_{i-1}^n}{2\Delta x} \quad \text{and} \quad \frac{\partial^2 q}{\partial x^2} = \frac{Q_{i+1}^n + Q_{i-1}^n - 2Q_i^n}{\Delta x^2}.$$

We arrive at the Lax–Wendroff method, which is second-order accurate:

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{2\Delta x} A \cdot (Q_{i+1}^n - Q_{i-1}^n) + \frac{\Delta t^2}{2\Delta x^2} A^2 \cdot (Q_{i+1}^n + Q_{i-1}^n - 2Q_i^n), \quad (3.93)$$

which leads to the following flux function

$$F_{i-1/2}^n = \frac{1}{2} A \cdot (Q_i^n + Q_{i-1}^n) - \frac{\Delta t}{2\Delta x} A^2 \cdot (Q_i^n - Q_{i-1}^n). \quad (3.94)$$

Introducing the negative and positive parts of A along with its absolute part (see § 3.3.2):

$$A = A^+ + A^- \quad \text{and} \quad |A| = A^+ - A^-,$$

we can transform the sum $A \cdot (Q_i^n + Q_{i-1}^n)$ into

$$\begin{aligned} A \cdot (Q_i^n + Q_{i-1}^n) &= A^+ \cdot Q_{i-1}^n + A^- \cdot Q_i^n + A^- \cdot Q_{i-1}^n + A^+ \cdot Q_i^n, \\ &= A^+ \cdot Q_{i-1}^n + A^- \cdot Q_i^n + (A^+ - |A|) \cdot Q_{i-1}^n + (A^+ + |A|) \cdot Q_i^n, \\ &= 2A^+ \cdot Q_{i-1}^n + 2A^- \cdot Q_i^n + |A| \cdot (Q_i^n - Q_{i-1}^n). \end{aligned}$$

We can then express the flux function (3.94) in the more traditional form, which makes the first-order flux correction apparent:

$$F_{i-1/2}^n = A^+ \cdot Q_{i-1}^n + A^- \cdot Q_i^n + \frac{1}{2}|A| \cdot \left(1 - \frac{\Delta t}{\Delta x}|A|\right) \cdot (Q_i^n - Q_{i-1}^n). \quad (3.95)$$

Like for the scalar case, we would like to limit the flux correction wherever it is needed. This limitation cannot be applied abruptly to the vector $\Delta Q_{i-1/2} = Q_i^n - Q_{i-1}^n$ itself, but to its components along the wave propagation direction Δw_k . To that end, we decompose $\Delta Q_{i-1/2}$ in the base $(w_k)_{1 \leq k \leq n}$

$$\Delta Q_{i-1/2} = \sum_{k=1}^m \alpha_{i-1/2}^k w_k,$$

and we then apply the limiter function ϕ (which can be any function used in the scalar case) to each component

$$\hat{\alpha}_{i-1/2}^k = \phi(\theta_{i-1/2}^k),$$

where the function θ examines the degree of smoothness

$$\theta_{i-1/2}^k = \frac{\theta_{I-1/2}^k}{\theta_{i-1/2}^k} \text{ with } I = \begin{cases} i-1 & \text{if } \lambda_k > 0, \\ i & \text{if } \lambda_k < 0 \end{cases}.$$

To sum up, the Lax–Wendroff method leads to a flux function of the form

$$F_{i-1/2}^n = A^+ \cdot Q_{i-1}^n + A^- \cdot Q_i^n + \hat{F}_{i-1/2}^n, \quad (3.96)$$

where the correction term is based on a flux-limiter function

$$\hat{F}_{i-1/2}^n = \frac{1}{2}|A| \cdot \left(1 - \frac{\Delta t}{\Delta x}|A|\right) \cdot \sum_{k=1}^m \hat{\alpha}_{i-1/2}^k w_k = \frac{1}{2} \sum_{k=1}^m |\lambda_k| \left(1 - \frac{\Delta t}{\Delta x}|\lambda_k|\right) \hat{\alpha}_{i-1/2}^k w_k. \quad (3.97)$$

Alternative 1

Let us first assume that the time step Δt satisfies the CFL condition, or in other words, the waves from adjacent interfaces $x_{i\pm 1/2}$ cross the line t_{n+1} inside the interval $[x_{i-3/2}, x_{i+3/2}]$. Let us introduce the variable change (see § 1.1.4)

$$s = L \cdot q$$

We use the same reasoning as in § 3.8.1. Let us make a second-order expansion of $s(x, t + \Delta t)$ and make use of Eq. (3.90) to link time and space derivatives:

$$\begin{aligned} s(x, t_{n+1} - \Delta t) &= s(x, t_{n+1}) - \Delta t \frac{\partial s}{\partial t}(x, t_{n+1}) + \frac{1}{2} \Delta t^2 \frac{\partial^2 s}{\partial t^2}(x, t_{n+1}) + O(\Delta t^2) \\ &= s(x, t_{n+1}) - \Delta t \Lambda \cdot \frac{\partial s}{\partial x}(x, t_{n+1}) - \frac{1}{2} \Delta t^2 \Lambda^2 \cdot \frac{\partial^2 s}{\partial x^2}(x, t_{n+1}) \end{aligned}$$

Integration of this equation over $[x_{i-1/2}, x_{i+1/2}]$ leads to

$$L \cdot (Q_i^{n+1} - Q_i^n) = \frac{\Delta t}{\Delta x} \Lambda \cdot \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial s}{\partial x}(x, t_{n+1}) dx + \frac{1}{2} \frac{\Delta t^2}{\Delta x} \Lambda^2 \cdot \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial^2 s}{\partial x^2}(x, t_{n+1}) dx. \quad (3.98)$$

We can express $s(x, t_{n+1})$ as a function of what happened at time t_n :

$$s(x, t_{n+1}) = L \cdot \tilde{q}(x - \lambda_k \Delta t, t) = \sum_{k=1}^m q_k(x - \lambda_k \Delta t, t) v_k.$$

We can express the integral terms of Eq. (3.98) as

$$\begin{aligned} \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial}{\partial x} s(x, t_{n+1}) dx &= \sum_{k=1}^m v_k \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial}{\partial x} \tilde{q}_k(x - \lambda_k \Delta t, t_n) dx, \\ &= \sum_{k=1}^m v_k \int_{x_{i-1/2} - \lambda_k \Delta t}^{x_{i+1/2} - \lambda_k \Delta t} \frac{\partial}{\partial x} \tilde{q}_k(x, t_n) dx. \end{aligned} \quad (3.99)$$

Given that the function $\tilde{q}_k$ is discontinuous, the jumps are

$$[\![\tilde{q}_k]\!] = Q_{i,k}^n - Q_{i-1,k}^n - \frac{\Delta x}{2}(a_{i-1,k} + a_{i,k}) \text{ and } [\![\tilde{q}'_k]\!] = a_{i,k} - a_{i-1,k},$$

for $\lambda_k > 0$, whereas for $\lambda_k < 0$ we have:

$$[\![\tilde{q}_k]\!] = Q_{i+1,k}^n - Q_{i,k}^n - \frac{\Delta x}{2}(a_{i+1,k} + a_{i,k}) \text{ and } [\![\tilde{q}'_k]\!] = a_{i+1,k} - a_{i,k},$$

We deduce that for $\lambda_k > 0$, we have

$$\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial}{\partial x} s(x, t_{n+1}) dx = \sum_{k=1}^m \left(Q_{i,k}^n - Q_{i-1,k}^n + (a_{i,k} - a_{i-1,k}) \frac{\Delta x}{2} - (a_{i,k} - a_{i-1,k}) \lambda_k \Delta t \right) v_k, \quad (3.100)$$

whereas for $\lambda_k < 0$, we have

$$\sum_{k=1}^m \left(Q_{i+1,k}^n - Q_{i,k}^n + (a_{i,k} - a_{i+1,k}) \frac{\Delta x}{2} + (a_{i,k} - a_{i+1,k}) \lambda_k \Delta t \right) v_k. \quad (3.101)$$

For the second derivative, we have

$$\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{\partial^2}{\partial x^2} s(x, t_{n+1}) dx = \sum_{k=1}^m (a_{i,k} - a_{i-1,k}) v_k \text{ for } \lambda_k > 0, \quad (3.102)$$

$$= \sum_{k=1}^m (a_{i+1,k} - a_{i,k}) v_k \text{ for } \lambda_k < 0, \quad (3.103)$$

By substituting Eqs. (3.100)–(3.102) into Eq. (3.98), we find that for $\lambda_k > 0$, the second-order (in time) approximation of Q_i^{n+1} is given by:

$$Q_{i,k}^{n+1} = Q_{i,k}^n - \frac{\lambda_k \Delta t}{\Delta x} (Q_{i,k}^n - Q_{i-1,k}^n) - \frac{1}{2} \frac{\lambda_k \Delta t}{\Delta x} (\Delta x - \lambda_k \Delta t) (a_{i,k} - a_{i-1,k}),$$

while for $\lambda_k < 0$, we have

$$Q_{i,k}^{n+1} = Q_{i,k}^n - \frac{\lambda_k \Delta t}{\Delta x} (Q_{i+1,k}^n - Q_{i,k}^n) + \frac{1}{2} \frac{\lambda_k \Delta t}{\Delta x} (\Delta x + \lambda_k \Delta t) (a_{i+1,k} - a_{i,k}).$$

We can provide a vector form

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (F_{i+1/2}^n - F_{i-1/2}^n), \quad (3.104)$$

where the flux is

$$F_{i-1/2}^n = A^+ \cdot Q_{i-1}^n + A^- \cdot Q_i^n + \hat{F}_{i-1/2}^n \quad (3.105)$$

and the flux correction $\hat{F}_{i-1/2}^n$ is

$$F_{i-1/2}^n = \frac{1}{2} |A^+| \cdot \left(1 - |A| \frac{\Delta t}{\Delta x} \right) \cdot (\Delta x a_{i-1/2}) + \frac{1}{2} |A^-| \cdot \left(1 - |A| \frac{\Delta t}{\Delta x} \right) \cdot (\Delta x a_{i+1/2}). \quad (3.106)$$

If we select the upwind slope $\Delta x a_{i-1/2} = \Delta Q_{i-1/2}$, then the flux function takes the form

$$F_{i-1/2}^n = \frac{1}{2} |A^+| \cdot \left(1 - |A| \frac{\Delta t}{\Delta x} \right) \cdot \Delta Q_{i-1/2} + \frac{1}{2} |A^-| \cdot \left(1 - |A| \frac{\Delta t}{\Delta x} \right) \cdot \Delta Q_{i+1/2} \quad (3.107)$$

This form shows differences with the Lax-Wendroff expression (3.95) in that it depends on the propagation direction.

Alternative 2

In § 3.8.1, we presented Alternative 3, where the Sharpclaw method was outlined. Generalising Eq. (3.86) leads to a system of coupled ordinary differential equations:

$$\frac{\partial Q_i}{\partial t} = -\frac{1}{\Delta x} \left(A^+ \cdot \Delta q_{i-1/2} + A^- \cdot \Delta q_{i+1/2} + A \cdot \Delta q_i \right) \text{ for } 1 \leq i \leq n. \quad (3.108)$$

where $\Delta q_{i-1/2} = q_{i-1/2}^+ - \Delta q_{i-1/2}^l$ and $\Delta q_i = q_{i+1/2}^- - q_{i+1/2}^+$ (see Fig. 3.9 for the notation).

Note that this equation can be obtained heuristically in a quite general way. The time-marching algorithm is

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} \left(F_{i+1/2}^n - F_{i-1/2}^n \right), \quad (3.109)$$

and in the limit of $\Delta t \rightarrow 0$, we have:

$$\frac{\partial Q_i}{\partial t} = -\frac{1}{\Delta x} \left(F_{i+1/2} - F_{i-1/2} \right), \quad (3.110)$$

where the flux are given by

$$F_{i-1/2} = A \cdot q_{i-1/2}^\downarrow.$$

where $q_{i-1/2}^\downarrow$ is the value of q along the interface $x = x_{i-1/2}$. At short times $\delta t \ll 1$, this interface value is

$$q_{i-1/2}^\downarrow(\delta t) = q_{i-1/2}^0 + \delta t \partial_t q_{i-1/2}^\downarrow(0) + O(\delta t^2),$$

where the leading-order term $q_{i-1/2}^0$ is the solution to the Riemann problem where the left and right states are $q_{i-1/2}^-$ and $q_{i-1/2}^+$ (see Eq. (1.35)):

$$q_{i-1/2}^0 = q_{i-1/2}^+ - \sum_{k: \lambda_k > 0} W_{i-1/2}^k.$$

Similarly, at the interface $x = x_{i+1/2}$, we have (see Eq. (1.36)):

$$q_{i+1/2}^0 = q_{i+1/2}^- + \sum_{k: \lambda_k < 0} W_{i+1/2}^k.$$

To leading order in time, the interfaces fluxes are thus:

$$\begin{aligned} F_{i-1/2} &= A \cdot \left(q_{i-1/2}^+ - \sum_{k: \lambda_k > 0} W_{i-1/2}^k \right) = A \cdot q_{i-1/2}^+ - A^+ \cdot \Delta q_{i-1/2}, \\ F_{i+1/2} &= A \cdot \left(q_{i+1/2}^- + \sum_{k: \lambda_k < 0} W_{i+1/2}^k \right) = A \cdot q_{i+1/2}^- + A^- \cdot \Delta q_{i+1/2}. \end{aligned}$$

Taking the flux difference leads to the system of equations (3.108).

3.8.3 Extension to nonlinear equations

High-resolutions methods developed for linear systems can be extended to nonlinear systems. This extension does cause trouble when the solutions involve shock waves, but may be more tricky when they involve rarefaction waves, in particular transonic waves. They take the form

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} \left(\mathcal{A}^- \Delta Q_{i+1/2}^n + \mathcal{A}^+ \Delta Q_{i-1/2}^n \right) - \frac{\Delta t}{\Delta x} \left(\hat{F}_{i+1/2}^n - \hat{F}_{i-1/2}^n \right), \quad (3.111)$$

where the operators $\mathcal{A}^\pm$ are defined by Eqs. (3.46) and (3.46). When the solution does not involve a transonic wave, it can be computed using Eqs. (3.48) and (3.49)

$$\mathcal{A}^- \Delta Q_{i+1/2}^n = \sum_{k=1}^{M_w} s_{k,i+1/2}^- W_{k,i+1/2}^n, \quad (3.112)$$

$$\mathcal{A}^+ \Delta Q_{i-1/2}^n = \sum_{k=1}^{M_w} s_{k,i-1/2}^+ W_{k,i-1/2}^n, \quad (3.113)$$

where M_w is the number of waves (in most cases $M_w = m$) and $s_{i-1/2}^k$ is speed of the k th wave at $x_{i-1/2}$. An entropy fix is needed to handle transonic waves properly.

Equation (3.111) also involves the flux correction $\hat{F}_{i+1/2}^n$:

$$\hat{F}_{i+1/2}^n = \frac{1}{2} \sum_{k=1}^{M_w} |s_{i-1/2}^k| \left(1 - \frac{\Delta t}{\Delta x} |s_{i-1/2}^k| \right) \tilde{W}_{i-1/2}^k \quad (3.114)$$

where $\tilde{W}_{i-1/2}^k$ is the limited version of the k th wave $W_{i-1/2}^k$ obtained by comparing this wave with the jump $W_{I-1/2}^k$ in the upwind direction ($I = i - 1$ is $s_{i-1/2}^k > 0$ and $I = i + 1$ is $s_{i-1/2}^k < 0$) (LeVeque, 2002, see Chaps. 6 and 15). Here arises a problem specific to nonlinear systems. Generally, the vectors $W_{I-1/2}^k$ and $W_{i-1/2}^k$ are not collinear, which makes it difficult to compare their components in the limiter function (LeVeque, 2002, see Sec. 9.13). A strategy used in Clawpack to solve this issue is to project the vector $W_{I-1/2}^k$ onto $W_{i-1/2}^k$ and compare their components.

3.9 Source term

We are concerned with the following hyperbolic equation with a source term:

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = S, \quad (3.115)$$

where q is a vector with m components representing the unknowns, f is the flux function, and S is the source term, possibly a function of q , its spatial derivatives, and position x .

3.9.1 Fractional-step method

One strategy to solve Eq. (3.115) is the fractional-step method, which involves splitting the problem into subproblems:

- Solving the homogenous equation $\partial_t q + \partial_x f(q) = 0$.
- Updating the solution by solving the ordinary differential equation

$$\frac{dq}{dt} = S(q).$$

This method is first-order accurate (in time) regardless of the accuracy of each method used for solving the subproblems. For instance, if we want to solve the linear advection equation with a source term

$$\frac{\partial}{\partial t} q + \bar{u} \frac{\partial}{\partial x} q = S(q), \quad (3.116)$$

then combining the upwind method for the first subproblem and the forward Euler method for the second one leads to consider the following set of equations involving an intermediate state Q_i^* :

$$\begin{aligned} Q_i^* &= Q_i^n - \frac{\bar{u}\Delta t}{\Delta x}(Q_i^n - Q_{i-1}^n) \\ Q_i^{n+1} &= Q_i^* + \Delta t S(Q_i^*). \end{aligned}$$

3.9.2 Strang splitting method

This method used an intermediate time step $\Delta t/2$, we start by solving the first subproblem with this time step to obtain the intermediate state Q^* . We then solve the second subproblem over the full time step Δt , with Q^* as the initial value and obtain Q^{**} , and we eventually return to the first subproblem and solve it again over the time step $\Delta t/2$ with Q^{**} as the initial value. This method is second-order accurate (in time).

If we take the example of linear advection (3.116) again, these three steps leads to the following scheme:

$$\begin{aligned} Q_i^* &= Q_i^n - \frac{\bar{u}\Delta t}{2\Delta x}(Q_i^n - Q_{i-1}^n) \\ Q_i^{**} &= Q_i^* + \Delta t S(Q_i^*), \\ Q_i^{n+1} &= Q_i^n - \frac{\bar{u}\Delta t}{2\Delta x}(Q_i^* - Q_{i-1}^*). \end{aligned}$$

3.9.3 Stiff source terms and implicit methods

When the second subproblem involves a *stiff*³ differential equation, it may be necessary to use an implicit method for solving the second subproblem. For simple source terms, the trapezoidal method is usually efficient:

$$Q_i^{n+1} = Q_i^* + \frac{\Delta t}{2}(S(Q_i^*) + S(Q_i^{n+1})). \quad (3.117)$$

This method usually does not generate insurmountable problems since there is no spatial coupling, and thus Eq. (3.117) can be solved in one time step. The Bank–Coughran–Fichtner method (also called *TR-BDF2 method*) offers better performances. It is based on a two-stage Runge–Kutta approach:

$$\begin{aligned} Q_i^{**} &= Q_i^* + \frac{\Delta t}{2}(S(Q_i^*) + S(Q_i^{**})) \\ Q_i^{n+1} &= \frac{1}{3}(4Q_i^{**} - Q_{i-1}^* + \Delta t S(Q_i^{n+1})). \end{aligned}$$

When the source term involves high-order spatial derivatives of q (e.g., diffusive terms), the implicit method usually requires more work. For instance, if the source term involves diffusion

$$\frac{\partial}{\partial t}q + \bar{u}\frac{\partial}{\partial x}q = \mu\frac{\partial^2}{\partial x^2}q, \quad (3.118)$$

with μ a constant, then the spatial discretisation of the second-order derivative involves the neighbouring cells

$$\frac{\partial^2}{\partial x^2}q \approx \frac{Q_{i-1} - 2Q_i + Q_{i+1}}{\Delta x^2},$$

³Loosely speaking, stiff equations may have solutions that exhibit significant variations over short distances. Explicit methods are usually not efficient for handling these equations. Indeed, explicit methods usually requires to select time steps $\Delta t = O(\Delta x^2)$ to be stable.

and therefore the trapezoidal equation (3.117) turns into the Crank-Nicolson method

$$Q_i^{n+1} = Q_i^* + \mu \frac{\Delta t}{2} \left(\frac{Q_{i-1}^{n+1} - 2Q_i^{n+1} + Q_{i+1}^{n+1}}{\Delta x^2} + \frac{Q_{i-1}^* - 2Q_i^* + Q_{i+1}^*}{\Delta x^2} \right). \quad (3.119)$$

This method can be still improved by using the TR-BDF2 method rather than the trapezoidal equation.

3.9.4 Well-balanced algorithms

The hyperbolic equation (3.118) with a nonzero source term admits a steady-state solution q_{ss} , which is the solution to the ordinary differential equation

$$\frac{d}{dx} f(q_{ss}) = S(q_{ss}). \quad (3.120)$$

Time-marching algorithms may find the right steady-state solution, but as the advection and sources are handled separately in two distinct subproblems, time-marching algorithms may not converge, but oscillate in time around the steady-state solution.

This issue has led to the development of *well-balanced algorithms*. An algorithm is said to be well-balanced if it is able to provide not only the time-dependent solution to a boundary initial value problem, but also the steady-state solution.

3.10 Boundary conditions

There is a large literature on boundary conditions for hyperbolic equations: existence and uniqueness of solutions, typology of boundary conditions, numerical implementation of boundary conditions, etc. We will not deal with all these issues here, but we will assume that the (physical) boundary conditions of the domain are known and that we are trying to write them in a numerical form consistent with the finite-volume method. In Clawpack, the strategy adopted is to extend the computational domain by using ghost cells (see 3.11). In the general case, we need $m_{bc} = 2$ ghost cells for each boundary so that we can use the high-resolution methods seen in § 3.8.

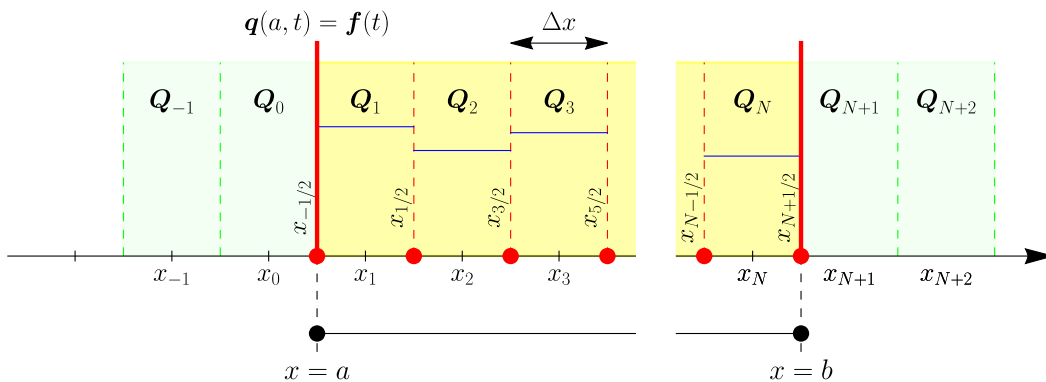


Figure 3.11 Computational grid and its extension to include ghost cells. The yellow-coloured area represents the computational domain $[a, b]$, while the green-coloured areas show the extended domain for handling the boundary conditions.

3.10.1 Periodic conditions

Periodic conditions imply that what goes out of the interval at $x = b$ must enter from the left into $x = a$. When $m_{bc} = 2$ ghost cells are used, this condition on the left of the computational domain can be written as:

$$Q_{-1} = Q_{N-1} \text{ and } Q_0 = Q_N,$$

while on its right, it writes

$$Q_{N+1} = Q_1 \text{ and } Q_{N+2} = Q_2.$$

3.10.2 Extrapolation

In many (if not most) problems, the computational domain includes only a portion of the physical space in which the physical process studied takes place. Its extension is thus often arbitrary, and the boundaries conditions required mathematically and numerically to solve the problem do are not physical, but artificial. We refer to these artificial boundaries as *transmissive*, *absorbing* or *non-reflecting boundary conditions* (LeVeque, 2002; Toro, 2009). We would like the waves to cross these boundaries, when passing out of the computational domain, without generating disturbances or spurious reflections. Problems may arise whenever the characteristic speeds at the boundary take positive and negative values (like in subcritical flow problems), implying that there are incoming waves whose characteristics are fixed by what happens outside the computational domain. The simplest method to set appropriate boundary conditions is to assume that the solution behaves smoothly near boundaries, and thus its values can be obtained by extrapolating those determined in the computational domain.

We could determine the value Q_{N+1} by assuming that the solution can be continued by mere extrapolation:

$$Q_{N+1} = Q_N + \frac{Q_N - Q_{N-1}}{\Delta x} \Delta x = 2Q_N - Q_{N-1},$$

which is the discretisation of the first-order expansion $q(x + dx) = q(x) + q'(x)dx$. First-order extrapolation can lead to stability problems and is generally not recommended (LeVeque, 2002). Instead, a zero-order extrapolation is a simpler technique:

$$Q_{N+1} = Q_N \text{ and } Q_{N+2} = Q_N$$

This strategy is often effective, but sometimes it may fail to produce the right result.

3.10.3 Solid Wall

One-dimensional equations

Let us consider that at the left boundary $x = a$, there is a solid wall and q vanishes at this point:

$$q(a, t) = 0.$$

For one-dimensional problem associated with a positive eigenvalue at $x = a$ (incoming wave with velocity $\lambda(a, t) > 0$), then we just have to set

$$Q_0 = 0.$$

In the opposite case (outgoing wave with velocity $\lambda(a, t) < 0$), then there is no inflow, and we can use a zero-order extrapolation:

$$Q_0 = Q_1.$$

Systems of linear equations

For multidimensional problems, more work is required. We will outline a technique for implementing boundary conditions representing the wall condition, which is effective for the Euler and Saint-Venant equations as well as a number of systems of equations in fluid mechanics. Let us assume that one of the unknowns q^2 is velocity u while the other q^1 is pressure p (or depth h). LeVeque (2002) noted that when extending the computational domain to the left by setting

$$u(x, t) = -u(-x, t) \text{ and } p(x, t) = p(-x, t) \text{ for } x \leq a$$

then we have

$$u(a, t) = -u(-a, t)$$

and thus

$$u(a, t) = 0.$$

In that case, the ghost cells are defined as follows:

$$Q_0^1 = Q_1^1 \text{ and } Q_0^2 = -Q_1^2, \quad (3.121)$$

and

$$Q_{-1}^1 = Q_2^1 \text{ and } Q_{-1}^2 = -Q_2^2. \quad (3.122)$$

The result can be generalized to more complicated boundary conditions when the governing equations involve a (linearised) Jacobian matrix in the form

$$A = \begin{bmatrix} u_a & p \\ q & u \end{bmatrix} \quad (3.123)$$

associated with eigenvalues $\lambda_1 = u_a - \sqrt{pq}$ and $\lambda_2 = u_a + \sqrt{pq}$, left and right eigenvectors

$$v_1 = \frac{1}{2} \begin{bmatrix} -\sqrt{q/p} \\ 1 \end{bmatrix}, v_2 = \frac{1}{2} \begin{bmatrix} \sqrt{q/p} \\ 1 \end{bmatrix}, w_1 = \begin{bmatrix} -\sqrt{p/q} \\ 1 \end{bmatrix} \text{ and } w_2 = \begin{bmatrix} \sqrt{p/q} \\ 1 \end{bmatrix},$$

where u_a , p , and q are constants. Typical examples include the shallow water and the acoustic equations. We would like to solve the Riemann problem at $x = a$ (see Fig. 3.12) and determine the intermediate state Q_* .

From Eq. (1.36), we can connect the intermediate $Q_* = (Q_*^1, Q_*^2)$ to the right state:

$$Q_* = Q_1 - \alpha_2 w_2,$$

where α_2 is the coefficient satisfying:

$$\alpha = (\alpha_1, \alpha_2) = L \cdot \Delta Q \text{ where } \Delta Q = Q_1 - Q_0$$

Let us assume that there is no variation in the first component of Q_* (zero-order extrapolation):

$$Q_0^1 = Q_1^1 \Rightarrow \Delta Q^1 = 0.$$

Let us now focus on the second component of Q_* (carrying information on the velocity u):

$$Q_*^2 = Q_1^2 - \alpha_2 w_2^2 = Q_1^2 - \frac{1}{2}(Q_1^2 - Q_0^2). \quad (3.124)$$

If we want to impose the zero velocity at $x = a$, then we must set $Q_*^2 = 0$ and Eq. (3.124) tells us that

$$Q_1^2 - \frac{1}{2}(Q_1^2 - Q_0^2) = 0 \Rightarrow Q_0^2 = -Q_1^2$$

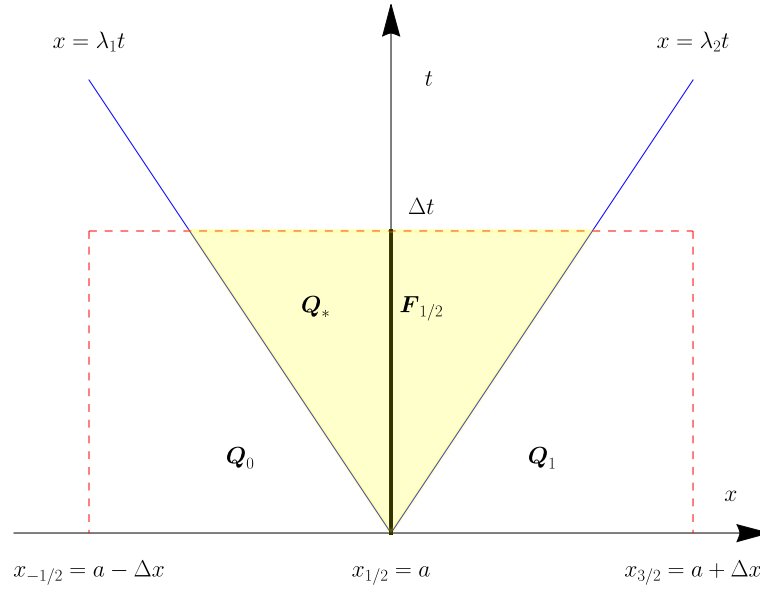


Figure 3.12 Riemann problem at the boundary $x = a$.

consistently with what is given by Eq. (3.121). If we now impose an oscillating wall as the left boundary condition

$$u(a, t) = U(t),$$

with U the wall velocity, then we must set $Q_*^2 = U$ and Eq. (3.124) tells us that

$$Q_1^2 - \frac{1}{2}(Q_1^2 - Q_0^2) = U \Rightarrow Q_0^2 = 2U - Q_1^2$$

Systems of nonlinear equations

We can establish a similar result when knowing the rarefaction wave's structure. Let us take the example of the Saint-Venant equations (see Fig. 3.13). We assume that the intermediate state $Q_* = (h_*, h_* \bar{u}_*)$ is connected to the right state $Q_1 = (h_1, h_1 \bar{u}_1)$ by a 2-rarefaction wave (2.63):

$$\bar{u}_1 - 2(\sqrt{gh_1} - \sqrt{gh_*}) = \bar{u}_*,$$

and is connected to left right state $Q_0 = (h_0, h_0 \bar{u}_0)$ by a 1-rarefaction wave (2.63):

$$\bar{u}_0 + 2(\sqrt{gh_0} - \sqrt{gh_*}) = \bar{u}_*.$$

If we add these two equations, then we obtain

$$\bar{u}_1 + \bar{u}_0 + 2\sqrt{gh_0} - 2\sqrt{gh_1} = 2\bar{u}_*.$$

If we set $h_0 = h_1$, then we must define the boundary condition for $\bar{u}_0$

$$\bar{u}_0 = 2\bar{u}_* - \bar{u}_1.$$

If the boundary involves a solid wall moving at velocity $U(t)$, then $\bar{u}_* = U$, and thus we find again Eq. (3.121)

$$\bar{u}_0 = 2U - \bar{u}_1.$$

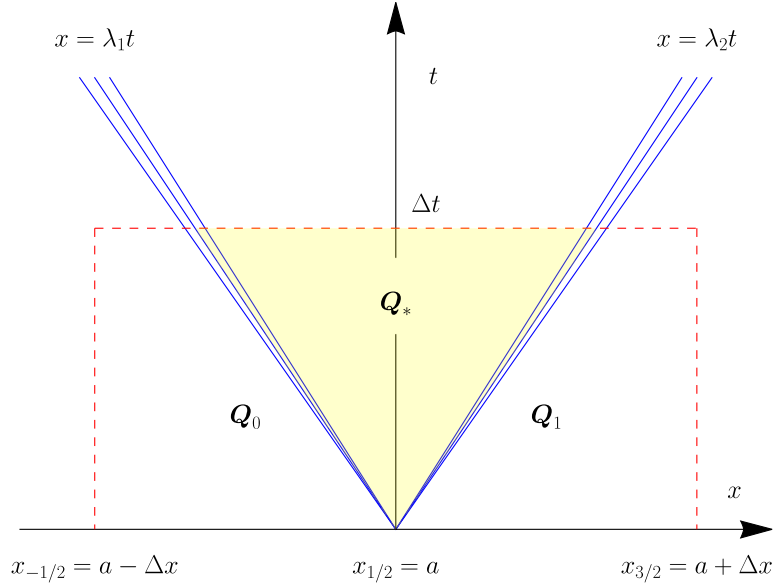


Figure 3.13 Riemann problem at the boundary $x = a$ with an intermediate state separated from the left and right states by rarefaction waves.

3.10.4 Inflow boundary conditions

One-dimensional equations

Let us consider that we impose a Dirichlet boundary condition at the left boundary (see Fig. 3.14):

$$q(a, t) = g(t). \quad (3.125)$$

We assume that the problem is linear or linearised so that information is propagated at constant velocity λ and the characteristic curves are straight lines $x = x_0 + \lambda t$. Setting the boundary value Q_0 amounts to determining the integral

$$Q_0^n = \frac{1}{\Delta x} \int_{a-\Delta x}^a q(x, t_n) dx. \quad (3.126)$$

For any characteristic $x = x_0 + \lambda(t - t_n)$ with $a - \Delta x < x_0 < a$, the characteristic curve crosses the line $x = a$ at time

$$a = x_0 + \lambda(t - t_n) \Rightarrow t = t_n + \frac{a - x_0}{\lambda}.$$

The value of q is constant and given by $g(t)$. We can put the integral (3.126) in the following form:

$$Q_0^n = \frac{1}{\Delta x} \int_{a-\Delta x}^a g\left(t_n + \frac{a - x_0}{\lambda}\right) dx_0.$$

We make the following change of variable

$$\tau = t_n + \frac{a - x_0}{\lambda} \Rightarrow d\tau = \frac{dx_0}{\lambda}.$$

This shows that the value of Q_0^n is

$$Q_0^n = \frac{\lambda}{\Delta x} \int_{t_n}^{t_n + \Delta x/\lambda} g(\tau) d\tau.$$

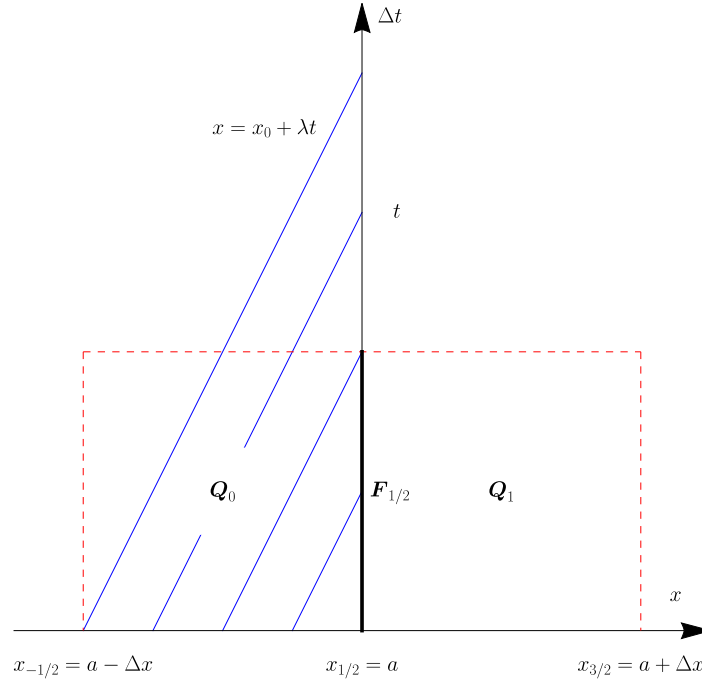


Figure 3.14 Dirichlet condition $q(a, t) = g(t)$ imposed at $x = a$.

LeVeque (2002) suggests approximating this integral by using the mean value theorem (evaluated at the midpoint $t_n + \Delta x/(2\lambda)$):

$$Q_0^n = g\left(t_n + \frac{\Delta x}{2\lambda}\right). \quad (3.127)$$

Similarly we have:

$$Q_{-1}^n = g\left(t_n + \frac{3\Delta x}{2\lambda}\right). \quad (3.128)$$

3.11 Implementation in Clawpack

Clawpack is a Fortran-based library developed to solve hyperbolic partial differential equations in the form:

$$\kappa \frac{\partial}{\partial t} q + \nabla \cdot f(q) = S, \quad (3.129)$$

where κ is the capacity function (or constant), q the unknown, f the flux function, and S the source term.

3.11.1 Clawpack installation

Prerequisites and installation

Linux is best suited to run the Clawpack library⁴. Installation requires a few additional libraries (see www.clawpack.org/prereqs.html)

⁴Installing Clawpack on a Windows machine seems possible, see the thread on <https://groups.google.com/g/claw-user> using Windows Subsystem for Linux.

- Compiler: gfortran (available from most linux distributions) or ifort (which needs a license, free for academic activities).
- Python: version 3, scipy, numpy, pip, meson-python and git
- It can be useful to install [anaconda](#). This environment makes it possible to manage the python packages and offers several functionalities like jupyter (a system of Python-based notebooks that can be read by a web browser), spyder (a scientific environment written for Python), R, and julia. Jupyter notebooks available from github can be read locally on the computer or via a web interface such as [nbviewer.jupyter.org/](#).

Following the procedure based on pip (see www.clawpack.org/installing.html) is usually the easy way to install Clawpack and Pyclaw:

```
81 pip install --user clawpack==v5.10.0
```

If one wants to install all packages (including Geoclaw), then one can use the following command:

```
82 pip install --src=$HOME/clawpack_src --user --no-build-isolation -e \
83 git+https://github.com/clawpack/clawpack.git@v5.10.0#egg=clawpack
```

Development version. See § 3.11.5 for further information on the Clawpack development version.

Finally, it is necessary to edit the `.bashrc` file by providing the required environment variables

```
84 export CLAW=$HOME/clawpack-v5.9.2
85 export FC=gfortran
```

Other environment variables may be required, e.g.:

```
86 BOUSSLIB=$CLAW/geoclaw/src/1d_classic/bouss
```

Build process

As of version 5.9.2, Clawpack used Meson-Python for the build process of the python modules. When modifying the clawpack working directory (including updates or use of earlier versions of Clawpack), the paths should be set properly by using the command (see [/www.clawpack.org/installing_pip.html](http://www.clawpack.org/installing_pip.html))

```
87 cd $CLAW
88 pip install --user --no-build-isolation -e ./
```

Makefile

Using the `openmp` library and specifying the number of threads, it is possible to make the Clawpack computations much faster (see www.clawpack.org/openmp.html). This can be implemented by setting the compiler flags in the prompt command:

```
89 export FFLAGS='-O2 -fopenmp' # or hardwire FFLAGS in the Makefile
90 make new
91 export OMP_NUM_THREADS=2
```

or directly in the Makefile:

```
92 export FFLAGS='-O2 -fopenmp' # or hardwire FFLAGS in the Makefile
93 make new
94 export OMP_NUM_THREADS=2
```

Similarly when using the Lapack or Blas library (linear algebra), it is possible to specify the appropriate compiler flag in the Makefile:

```
95 FFLAGS += -O2 -fopenmp
96 OMP_NUM_THREADS=4
```

The following commands are implemented (the complete list is also available by executing the command `make help`):

- `make new` makes a new compilation of everything.
- `make clobber` deletes all the intermediary files (*.o, *.mo, output).
- `make .output` runs the code. The results are by default placed in the subdirectory named `_output`.
- `make .plots` runs the code and plots the results using the python script `setplot`. The graphic files are placed in the subdirectory named `_plots`.
- `make .data` provides the data files using the python script `setrun`.

3.11.2 Legacy Clawpack

In its original form developed by Randall LeVeque, Clawpack has been based on a set of Fortran 77 routines (LeVeque, 2002).

The main programme was originally located in the file `driver.f`. This file allocated storage for the arrays used by Clawpack. This is now done automatically, and the user does not need to fill this file. This programme then calls `claw1ez`, which reads the file `claw.data` created by the python script `setrun.py` (it can be created by typing `make .data`).

The initial condition is contained in the file `qinit.f`. We should define the cell average value Q_i over the entire domain, but for a continuous function q , this average value is the value taken by q at x_i (cell midpoint): $Q_i = q(x_i)$.

The initial conditions are processed in the file `bc1.f`. The type of boundary conditions is prescribed in the file `claw.data`.

The Riemann solver is contained in the file `rp1.f`. The idea is to decompose any discontinuity into a set of waves W_k moving at velocity s^k (see § 3.3):

$$Q_i - Q_{i-1} = \sum_{k=1}^m W_k$$

avec m is the wave number (which is usually equal to the dimension of the system). For Godunov's method, the value Q_i is updated as follows:

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (A^+ \cdot \Delta Q_{i-1/2} + A^- \cdot \Delta Q_{i+1/2}),$$

where we distinguish between the left-going wave (coming from the right endpoint $x_{i+1/2}$):

$$A^+ \cdot \Delta Q_{i+1/2} = \sum_k \min(s_{i+1/2}^k, 0) W_{i+1/2}^k,$$

and the right-going wave

$$A^- \cdot \Delta Q_{i+1/2} = \sum_k \max(s_{i-1/2}^k, 0) W_{i-1/2}^k,$$

The left-going wave is zero if $s_{i+1/2}^k > 0$ (because the wave moves only from right to left) and the right-going wave is zero if $s_{i-1/2}^k < 0$.

The Riemann solver needs two input data: the two arrays `ql` and `qr` related to the values of q on the left and right of every cell. High-resolution methods require further information. Note that for the Riemann solver at the interface $x_{i-1/2}$, we use the following notation for referring to cells $i-1$ and i : $qr(i-1, :) = q_{i-1/2}^r$ and $ql(i-1, :) = q_{i-1/2}^l$, and in this notation, left and right refer to the left and right of the cell i or $i-1$, and not what happens relative to the interface.

The solver provides:

- the functions `amdq` (literally “a minus delta q”, which is the vector $A^- \cdot \Delta Q_{i+1/2}$) and `apdq` (vecteur $A^+ \cdot \Delta Q_{i+1/2}$),
- `wave` (the wave $W_{i-1/2}^k$), and
- `s` (the speed $s_{i-1/2}^k$).

Caveat. Note⁵ that the Riemann problem at the interface $x_{i-1/2}$ between cells $i-1$ and i has the following input data (see Fig. 3.15):

- Left state: $q_{i-1}^R = qr(:, i-1)$.
- Right state: $q_{i-1}^L = ql(:, i)$.

This notation is ambiguous since until now, q_l has been used to denote the left state of a Riemann problem, while q_r denotes its right state.

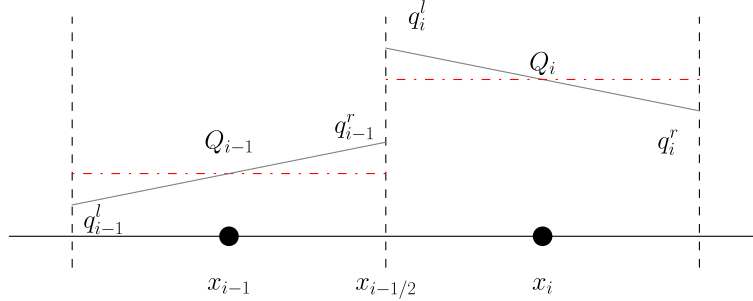


Figure 3.15 Notation for the Riemann problem in Clawpack.

There are many other routines, which are not always required. They are called by the main driver by default, but do not return anything. Among the most important:

- `setprob.f`: the routine `claw1ez` calls `setprob.f` before each execution, which makes it possible to initialise some parameters.
- `setaux.f`: the routine `claw1ez` calls the routine `setaux.f` before each execution to initialise the auxiliary variables (for instance, bed topography).
- `b4step1.f`: the routine `claw1` calls the routine `bc4step1.f` before each step to perform additional tasks.
- `src1.f`: if the equation involves a source term, this file is used to correct the solution to the homogenous equation.

⁵See www.clawpack.org/riemann.html.

3.11.3 *AMRClaw*

AMRClaw is the Clawpack version that provides Adaptive Mesh Refinement (AMR) capabilities in 1, 2 or 3 space dimensions (Berger & LeVeque, 1998). Depending on the mesh refinement criteria (which may be automatic or user-selected), the mesh may be locally refined to improve numerical accuracy. Numerical solutions are recorded as a series of nested grids. Computation is done using a coarse mesh when the solution behaves smoothly, and is done with finer resolution when this is locally required. See [the AMRClaw webpage](#) for further information.

GeoClaw, based on the Clawpack library and devoted to large-scale free-surface flows over topography, uses the AMR toolbox (George, 2006, 2008, 2011).

3.11.4 *Pyclaw*

Pyclaw is a python package that offers a convenient framework for pre- and post-processing information, interfacing and running Clawpack or Sharpclaw (Ketcheson *et al.*, 2012; Mandli *et al.*, 2016). It can call Fortran or Python routines. Interfaced with PyWENO and PETSc, Pyclaw provides extended functionality in terms of parallel computing (Ketcheson *et al.*, 2012). In particular, PyClaw can be run in parallel using the PETSc library.

PyClaw includes two types of solvers:

- Classic solvers are those originally developed in Clawpack 4 (algorithms for solving one- to three-dimensional problems).
- SharpClaw solvers are higher-order wave propagation routines using WENO reconstruction and Runge–Kutta integration, but reserved for one- and two-dimensional problems for the moment.

Solver initialisation takes one argument (the Riemann solver’s name). For instance, if one selects `riemann.acoustics_1D` from the Riemann repository to solve one-dimensional acoustic problems, then one can initialise the solver as follows

```
1 from clawpack import pyclaw
2 from clawpack import riemann
3 solver = pyclaw.ClawSolver1D(riemann.acoustics_1D)
```

and if one prefers the Sharpclaw solver

```
3 solver = pyclaw.SharpClawSolver1D(riemann.acoustics_1D)
```

If the solver is defined in a script or in a function, then one must initialise the `ClawSolver` object and specify the number of equations and waves:

```
1 solver = pyclaw.ClawSolver1D(solver_name)
2 solver.num_waves = 1
3 solver.num_eqn = 1
4 solver.kernel_language = 'Python'
```

When using a Clawpack solver from the Riemann repository, the code automatically determines the numbers of waves and equations. Fortran solvers can be used in python using the [numpy command f2py](#). See www.clawpack.org/pyclaw/problem.html and [/www.clawpack.org/riemann.html](http://www.clawpack.org/riemann.html) for further information to set up your own problem and your own Riemann solver.

It is also possible to call solvers written in fortran. To that end, the fortran file should first be compiled using the `numpy` command called `f2py` to wrap fortran files to python. This command compiles all sources and builds an extension module containing the wrappers. In a command prompt, this achieved using the command

```
1 f2py -c my_riemann_solver.f90 -m solver_name
```

or it can be done in python

```
1 python -m numpy.f2py my_riemann_solver.f90 -m solver_name
```

If successful, the command generates a file like `solver_name.cpython-39-x86_64-linux-gnu.so`. This file can then be imported in Pyclaw:

```
1 import solver_name
2 ...
3 solver.kernel_language = 'Fortran'
4 solver = pyclaw.ClawSolver1D(solver_name)
```

The classic and SharpClaw solvers have two important differences:

- The classic solver involves the integral of the source term over a step (implemented using `solver.step_source`), whereas SharpClaw needs the instantaneous value of the source term (implemented using `solver.dq_src`).
- The list of options and their possible values differ from one variant to the other.

See www.clawpack.org/pyclaw/solvers.html for further information.

A PyClaw simulation provides a set of frames, each one being the solution at a given time (depending on the selected number of outputs and final time). By default, these frames are written in a directory named `_outdir` unless the option `claw.output_format` is set to `None`. If `claw.keep_copy` is set to `True`, then output frames are also saved in memory in the list `claw.frames`. The solution is available at any time from `claw.frames[i].q` where i the frame number ($i = 0$: initial condition, $i = -1$ final state). See www.clawpack.org/pyclaw/output.html for further information.

Pyclaw simulations involve Solution objects, which evolve as a result of the action of a Solver object depending on the Domain and State objects. A Pyclaw Solution comprises one or several Domain and State objects (see Fig. 3.16). Each State object inhabits a Grid, composed of Dimension objects that define the extents of the Domain. See www.clawpack.org/pyclaw/classes.html.

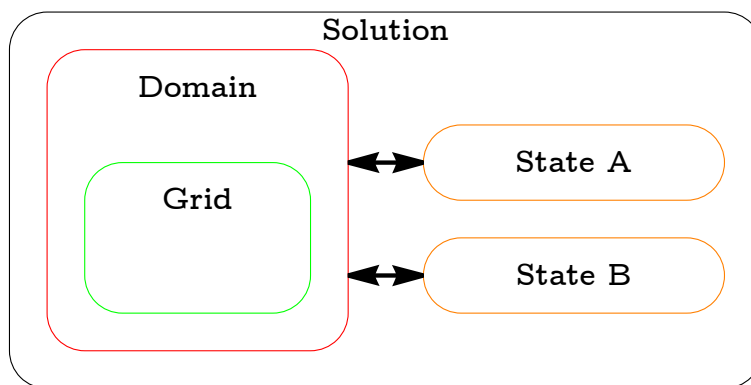


Figure 3.16 Pyclaw solution structure.

3.11.5 Geoclaw

3.11.6 *DClaw*

3.11.7 *Development version*

A read-only development version of Clawpack can be created (see www.clawpack.org/dev/developers.html) for those interested in testing the applications under development.

To clone clawpack from github, use the following where you to create your development folder

```
1 git clone https://github.com/clawpack/clawpack.git
2 cd clawpack
3 git submodule init
4 git submodule update
```

You can then update clawpack to the most recent version

```
1 source pull_all.sh
```

For instance, if you want to install dclaw, then write

```
1 git clone https://github.com/geoflows/dclaw
```

As of version 5.10, it is recommended to install all the required packages listed in `requirements-dev.txt`:

```
1 pip install -r requirements-dev.txt
```

To finish the installation, you need to run the command

```
1 pip install --no-build-isolation -e ./
```

The instructions above leads to a top-level CLAW directory that is cloned to the tip of the main branch, and each subdirectory is cloned to a particular commit as specified in the main branch. To clone each subdirectory to the main branch as well, one needs to run `$CLAW/pull_all.sh`:

```
1 source pull_all.sh
```

The command clones the main branch to each directory and then do a git pull to make sure it is up to date.

3.11.8 *Implementation of boundary conditions in Clawpack*

Classic Clawpack

In the classic version of Clawpack, the boundary conditions are handled in the file `bc1.f` (one-dimensional problems) or `bc2.f` (two-dimensional problems). A number of classic boundary conditions are proposed:

- extrapolation,
- periodic, or
- wall.

It is possible to include a custom boundary conditions. The type of boundary condition is specified in the `setrun.py` file. For instance, we want to use custom boundary conditions, we write:

```

1 clawdata.bc_lower[0] = 'user'    # at xlower
2 clawdata.bc_upper[0] = 'user'    # at xupper

```

In fortran, it is possible to use negative indices in arrays, and as a consequence, the notation is very close to the mathematical one: Q_{-1} is denoted by $q(i, -1)_{1 \leq i \leq m}$. For instance, the period boundary condition is written as:

```

1 dimension q(meqn,1-mbc:mx+mbc)
2 do ibc=1,mbc
3     do m=1,meqn
4         q(m,1-ibc) = q(m,mx+1-ibc)
5     end do
6 end do

```

Pyclaw

Pyclaw uses an extended array of q called qbc . If we assume that we use $m_{bc} = 2$ ghost cells at each boundary, then

- $qbc[0]$ and $qbc[1]$ represent the left boundary conditions.
- We have $qbc[2]=q[0]$ and so on.
- $qbc[-1]$ and $qbc[-2]$ represent the right boundary conditions.

The boundary conditions are called in the setup routine:

```

1 solver.bc_lower[0] = pyclaw.BC.custom
2 solver.user_bc_lower = inlet_bc
3 solver.bc_upper[0] = pyclaw.BC.custom
4 solver.user_bc_upper = outlet_bc

```


Linear advection

4.1 Linear advection equation

4.1.1 Governing equation and solution

We are studying the linear advection equation

$$\frac{\partial q}{\partial t} + a \frac{\partial q}{\partial x} = 0, \quad (4.1)$$

where $a \neq 0$ is a constant called the advection velocity. The solution to the Riemann problem

$$q(x, 0) = \begin{cases} q_l & \text{if } x < 0 \\ q_r & \text{if } x > 0 \end{cases}$$

is straightforward

$$q(x, t) = \begin{cases} q_l & \text{if } x < at \\ q_r & \text{if } x > at \end{cases}$$

Equation (4.1) can be written in characteristic form

$$\frac{1}{dt} = \frac{a}{dx} = \frac{0}{dq},$$

and thus the solution is any function in the form

$$q(x, t) = F(x - at).$$

In particular, when the initial condition is $q(x, 0) = q_0(x)$, then

$$q(x, t) = q_0(x - at). \quad (4.2)$$

We can implement the method detailed in § 3.4.1 by defining the wave and fluctuations

$$W_{i-1/2} = Q_i - Q_{i-1}, \quad (4.3)$$

$$A^+ \Delta Q_{i-1/2} = \dot{\sigma}_{i-1/2}^+ W_{i-1/2}, \quad (4.4)$$

$$A^- \Delta Q_{i+1/2} = \dot{\sigma}_{i+1/2}^- W_{i+1/2}, \quad (4.5)$$

where $\dot{\sigma}^+ = \max(a, 0)$ and $\dot{\sigma}^- = \min(a, 0)$.

4.1.2 Implementation in Clawpack

In the recent versions of Clawpack, any application needs a number of files (see § 3.11.2):

- The `Makefile` for linking the executables and running the code.
- Two python files: `setrun.py` generate the data needed by clawpack, while `setplot.y` specifies how to plot the numerical results. `setrun.py` generates two data files: `setprob.data`, which contains model parameters (here the advection velocity a) and `claw.data`, which includes all the essential information needed by clawpack.
- The following fortran files:
 - `qinit.f90` provides the initial values
 - `setprob.f90` reads the model parameters from the file `setprob.data`.

Note that the Riemann solver uses the arrays `ql` and `qr`: `ql(i)` is the value of q on the left of the cell, whereas `qr(i)` denotes the value of q on its right (see Fig. 3.15). The Riemann problem at the interface $x_{i-1/2}$ between cells i and $i-1$ thus involves the difference between the left state `ql(i-1,:)` and right state `ql(i,:)`. This is why the wave is defined as $wave(1,1,i) = ql(1,i) - qr(1,i-1)$ in the following. This notation is potentially confusing because throughout this document, q_r and q_l refer to the right and left states, respectively. This notation is consistent with the notation used in Fig. 3.9 (see also the section devoted to high-resolution methods in § 3.8).

In classic Clawpack, the algorithm for the solver is quite simple.

```

1  ! =====
2  subroutine rp1(maxm,meqn,mwaves,maux,mbc,mx,ql,qr,auxl,auxr,wave,s,amdq,
   apdq)
3  ! =====
4
5      implicit double precision (a-h,o-z)
6
7      dimension wave(meqn, mwaves, 1-mbc:maxm+mbc)
8      dimension s(mwaves,1-mbc:maxm+mbc)
9      dimension ql(meqn, 1-mbc:maxm+mbc)
10     dimension qr(meqn, 1-mbc:maxm+mbc)
11     dimension apdq(meqn, 1-mbc:maxm+mbc)
12     dimension amdq(meqn, 1-mbc:maxm+mbc)
13
14     !      # advection velocity:
15     !      # (should be set in setprob.f)
16     common /cparam/ a
17
18     !      # define the wave:
19     do 20 i = 2-mbc, mx+mbc
20
21     !      # Compute the waves.
22
23         wave(1,1,i) = ql(1,i) - qr(1,i-1)
24         s(1,i) = a
25     20 END DO
26
27
28     !      # compute the leftgoing and rightgoing flux differences:
29     !      # Note s(1,i) < 0 and s(2,i) > 0.
30

```

```

31 do 220 m=1, meqn
32   do 220 i = 2-mbc, mx+mbc
33     amdq(m,i) = dmin1(0.d0,s(1,i))*wave(m,1,i)
34     apdq(m,i) = dmax1(0.d0,s(1,i))*wave(m,1,i)
35   220 END DO
36
37   return
38 end subroutine rp1

```

4.1.3 Example

Let us consider Eq. (4.1) over the $[0, 1]$ interval with $a = 0.5 \text{ m}\cdot\text{s}^{-1}$. The boundary conditions are cyclic:

$$q(1, t) = q(0, t).$$

The initial condition is the superposition of a Gaussian and a unit box

$$q_0 = \exp(-\beta(x - x_0)^2) + U(x)$$

with $\beta = 200 \text{ m}^{-2}$ and $x_0 = 0.6 \text{ m}$, and

$$U(x) = \begin{cases} 0 & \text{if } x < 0.1 \text{ or } x > 0.4 \\ 1 & \text{if } 0.1 \leq x \leq 0.4 \end{cases}$$

Figure 4.1 shows the comparison between the exact and numerical solutions.

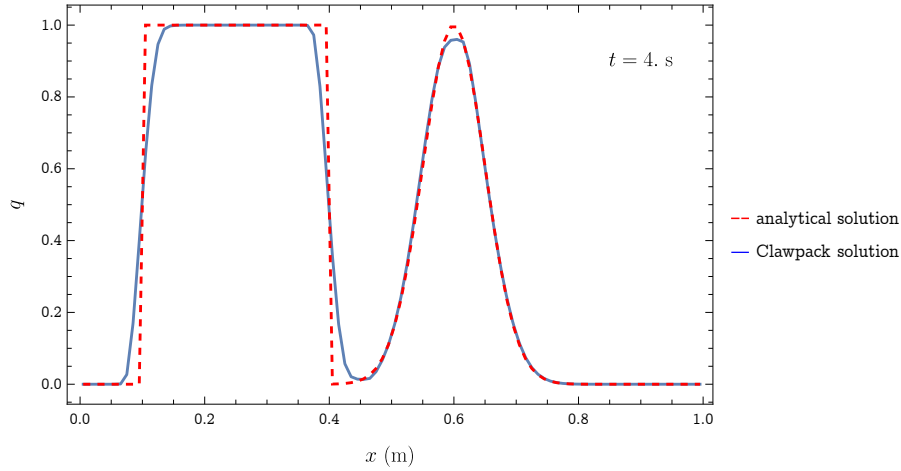


Figure 4.1 Comparison of the numerical and analytical solutions at time $t = 4 \text{ s}$. Computation with $n = 100$ cells. We used a second-order scheme (Law–Wendrow–LeVeque).

Figure 4.1 compares the effects of three flux limiters on the numerical solution. In this test, the MC and superbee limiters perform better than the mindmod limiter, and naturally far better than the simple Godunov scheme.

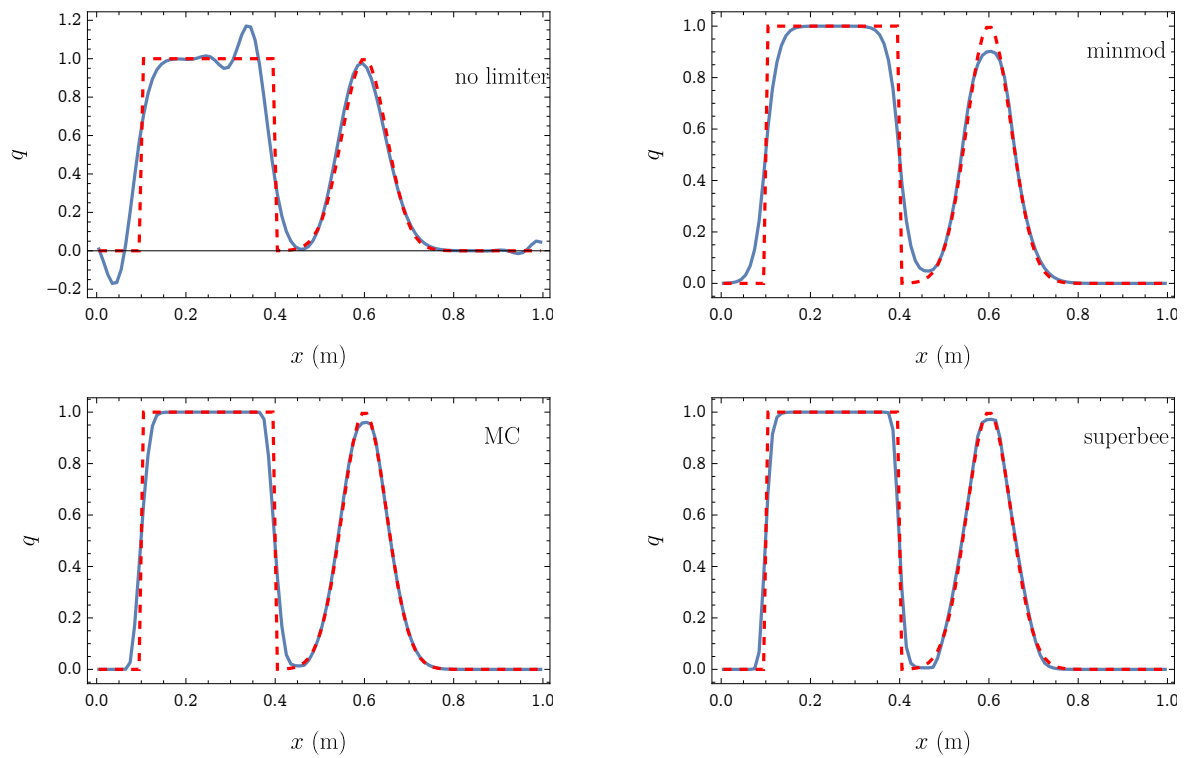


Figure 4.2 Comparison of the numerical and analytical solutions at time $t = 4$ s. Computation with $n = 100$ cells. We tested a first-order Godunov scheme (no limiter) and three second-order schemes (limiters: MC, minmod, and superbee).

4.1.4 *Implementation in AMRClaw*

AMRClaw allows us to compute high-accuracy solutions by refining the mesh locally. Using AMRClaw requires a few changes in the setup files:

- Makefile: the variables `CLAW_PKG` and `EXE` must be set to `xamr` and `amrclaw\verb`, respectively.
- Setup.py: a number of variables related to mesh refinement must be defined in the `setrun.py` file. See [the AMRClaw webpage](#) devoted to this file.

See also [the AMRClaw webpage](#) for general information.

Here we show an example of numerical solution obtained using AMRClaw with three grid levels. This is the same example as the one seen above. Figure 4.3 shows that AMRClaw performs better than the classic Clawpack version (see Figure 4.2).

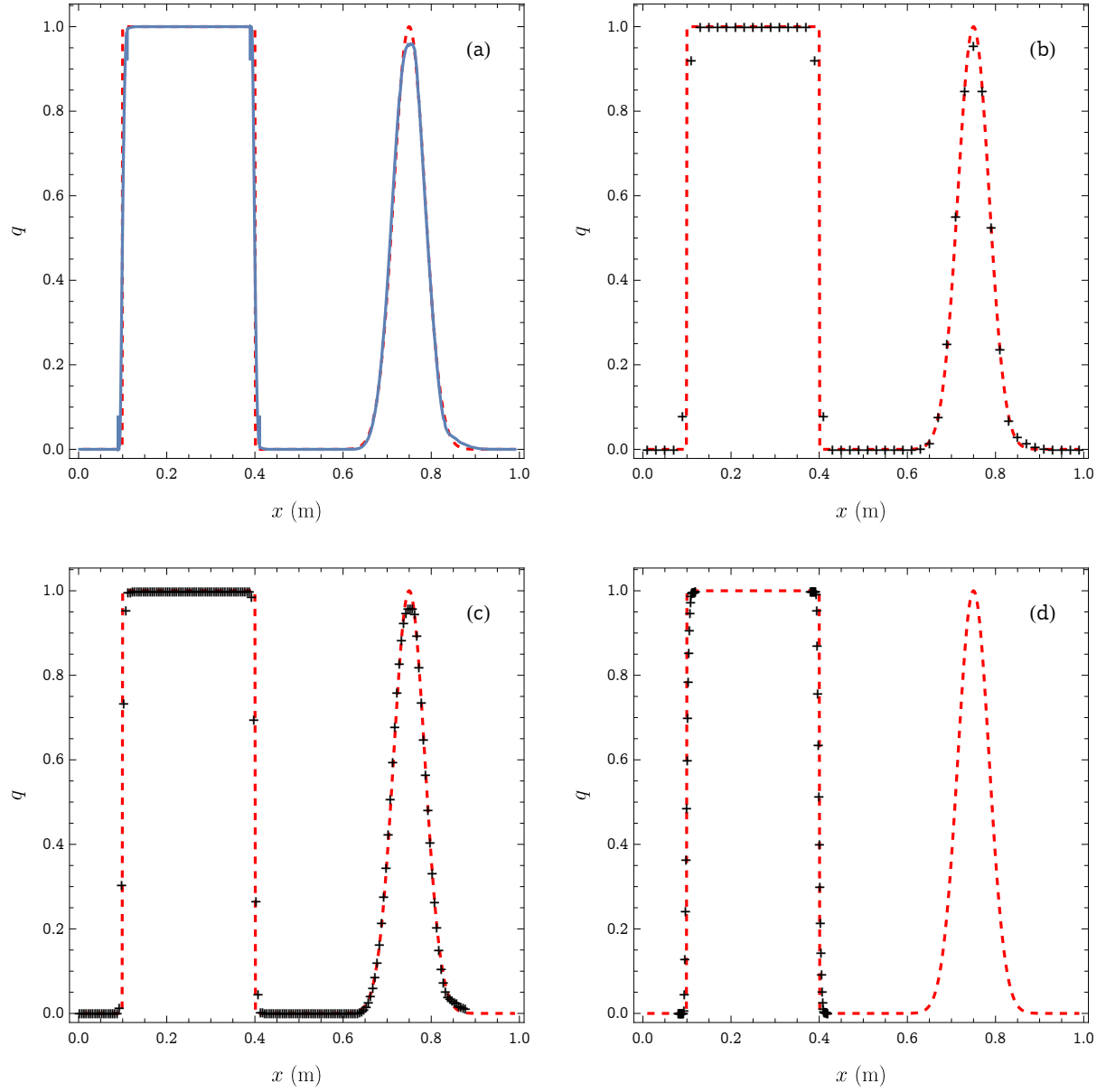


Figure 4.3 Comparison of the numerical and analytical solutions at time $t = 4$ s. Computation with $n = 100$ cells. We used AMRClaw with three grids. (a) Comparison of the numerical solution with the exact solution. (b) Coarse mesh (grid level 1) with $\Delta x = 0.02$ m. (c) Medium mesh (grid level 2) with $\Delta x = 0.005$ m. (d) Fine mesh (grid level 3) with $\Delta x = 0.00125$ m.

4.1.5 Implementation in Pyclaw

Classic solvers

To solve a problem using Pyclaw, we will define the following objects:

- The controller handles the running, output, and can be used for plotting the solution (we can run Pyclaw without using a controller, but it makes life easier).
- The time interval over which the solution is computed.
- The solver for the governing equation to be solved. In Pyclaw, a number of solvers are available. They are defined by the class `ClawSolverxD` or `SharpClawSolverxD`, where $x = 1, 2, 3$ is the problem's spatial dimension. SharpClaw algorithms involve on WENO reconstruction and Runge-Kutta time stepping (Ketcheson *et al.*, 2013). User-defined solvers can also be used.
- The computational domain is the domain over which the problem is solved.
- The solution is an object that initially contains the initial data, and then after iterations, the solutions at specific times.

Contrary to the classic Clawpack notation, Pyclaw considers that the variables `q_l` and `q_r` refer to the states left and right states at each interface $x_{i-1/2}$. This is a source of confusion when working with both classic Clawpack and Pyclaw.

```

1 %matplotlib inline
2
3 from numpy import sqrt, exp, cos, logical_and, where
4 from clawpack import riemann
5 from clawpack import pyclaw
6
7 def advection(q_l,q_r,aux_l,aux_r,problem_data):
8     r"""
9     1d linear advection riemann solver
10    """
11    import numpy as np
12    num_eqn = 1
13    num_waves = 1
14
15    # Convenience
16    num_rp = q_l.shape[1]
17
18    # Return values
19    wave = np.empty( (num_eqn, num_waves, num_rp) )
20    s = np.empty( (num_waves, num_rp) )
21    amdq = np.empty( (num_eqn, num_rp) )
22    apdq = np.empty( (num_eqn, num_rp) )
23
24    # Local values
25    delta = np.empty(np.shape(q_l))
26
27    delta = q_r - q_l
28    a = problem_data['a']
29
30    # Compute the wave
31    # 1-Wave
32    wave[0,0,:] = delta

```

```

33     s[0,:] = a
34
35     # Compute the left going and right going fluctuations
36     for m in range(num_eqn):
37         amdq[m,:] = min(a,0) * wave[m,0,:]
38         apdq[m,:] = max(a,0) * wave[m,0,:]
39
40     return wave, s, amdq, apdq
41
42
43 def setup(outdir='./_output', output_style=1):
44
45     solver = pyclaw.ClawSolver1D(advection)
46     solver.num_waves = 1
47     solver.num_eqn = 1
48     solver.kernel_language = 'Python'
49     solver.limiters = pyclaw.limiters.tvd.MC
50     solver.bc_lower[0] = pyclaw.BC.periodic
51     solver.bc_upper[0] = pyclaw.BC.periodic
52     solver.order = 1 #1: Godunov, 2: Lax-Wendrow-LeVeque
53
54     x = pyclaw.Dimension(0.0, 1.0, 100, name='x')
55     domain = pyclaw.Domain(x)
56     num_eqn = 1
57
58     state = pyclaw.State(domain, num_eqn)
59
60     a = 0.5 # advection velocity
61     state.problem_data['a'] = a
62
63     xc = domain.grid.x.centers
64     beta = 100
65     gamma = 0
66     x0 = 0.75
67     state.q[0, :] = exp(-beta * (xc-x0)**2) + where(logical_and(xc > 0.1,
68     xc < 0.4), 1, 0)
69
70     claw = pyclaw.Controller()
71     claw.solution = pyclaw.Solution(state, domain)
72     claw.solver = solver
73     claw.outdir = outdir
74     claw.output_style = output_style
75     claw.tfinal = 4.0
76     claw.num_output_times = 10
77     claw.keep_copy = True
78     #claw.setplot = setplot
79
80     return claw
81
82 def setplot(plotdata):
83     """
84     Plot solution using VisClaw.
85     """
86     plotdata.clearfigures() # clear any old figures,axes,items data
87
88     plotfigure = plotdata.new_plotfigure(name='q', figno=1)
89

```

```

90 # Set up for axes in this figure:
91 plotaxes = plotfigure.new_plotaxes()
92 plotaxes.ylimits = [-.2,1.0]
93 plotaxes.title = 'q'
94
95 # Set up for item on these axes:
96 plotitem = plotaxes.new_plotitem(plot_type='1d_plot')
97 plotitem.plot_var = 0
98 plotitem.plotstyle = '-o'
99 plotitem.color = 'b'
100 plotitem.kwargs = {'linewidth':2,'markersize':5}
101
102 return plotdata
103
104
105 def qtrue(x,t):
106     """
107     The true solution, for comparison.
108     """
109     from numpy import mod, exp, where, logical_and
110     beta = 100
111     gamma = 0
112     x0 = 0.75
113     u = claw.solution.state.problem_data['a']
114     xm = x - u*t
115     xm = mod(xm, 1.) # because of periodic boundary conditions
116     q = exp(-beta * (xm-x0)**2) + where(logical_and(xm > 0.1, xm < 0.4),
117     1, 0)
117     return q

```

We run the code

```

1 claw = setup()
2 claw.run()
3
4 from clawpack.visclaw import data
5 from clawpack.visclaw import frametools
6 plotdata = data.ClawPlotData()
7 plotdata.setplot = setplot
8 claw.plotdata = frametools.call_setplot(setplot,plotdata)
9
10 frame = claw.load_frame(10)
11 f=claw.plot_frame(frame)

```

Next, we plot the solution for time $t = 2$ s. The figure compares the numerical and exact solutions at time $t = 2$ s. As we used a first-order scheme (Godunov), the numerical viscosity smooths out sharp variations in q .

```

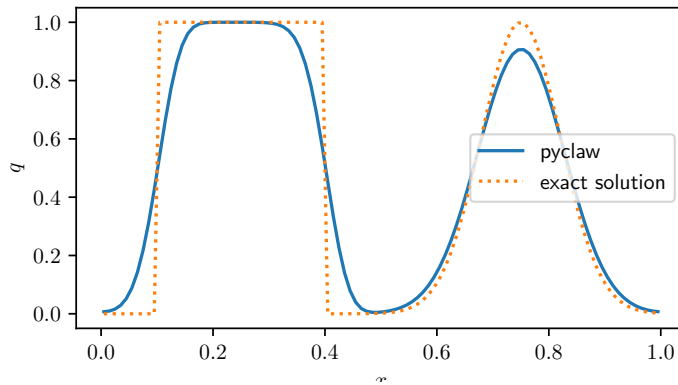
1 import numpy as np
2 import matplotlib.pyplot as plt
3 plt.rcParams['text.usetex'] = True
4 frame = claw.frames[5]
5 dt = claw.tfinal/claw.num_output_times
6 t = dt*5
7 true = qtrue(x,t)
8 fig, ax = plt.subplots(figsize=(5, 2.7))
9 w = frame.q[0,:]
10 x = frame.state.grid.c_centers
11 x = x[0]

```

```

12 ax.plot(x, w, label='pyclaw')
13 ax.plot(x, true, ':', label='exact solution')
14 ax.legend(loc='right')
15 ax.set_xlabel(r'$x$')
16 ax.set_ylabel(r'$q$')
17 plt.savefig('frameAdvection5.pdf')

```



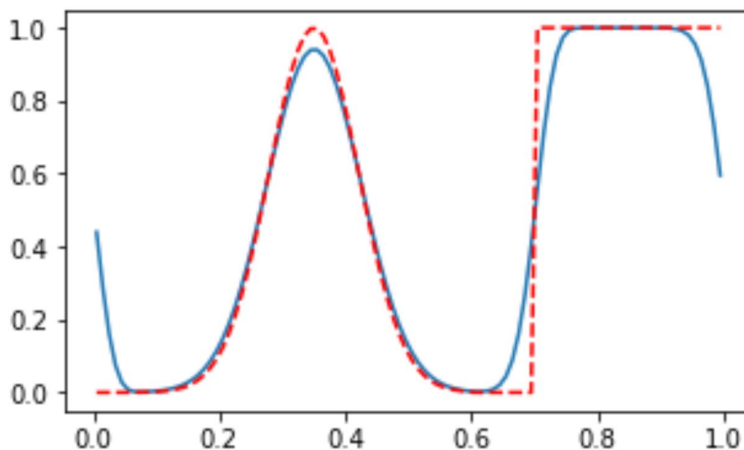
We can also plot an animation:

```

1 nsimul=np.size(claw.frames)
2 figs = []
3 for i in range(nsimul):
4     fig = plt.figure(figsize=(5,3))
5     frame = claw.frames[i]
6     w = frame.q[0,:]
7     x = frame.state.grid.c_centers
8     x = x[0]
9     dt = claw.tfinal/claw.num_output_times
10    t = dt*i
11    true = qtrue(x,t)
12    plt.plot(x, w)
13    plt.plot(x, true, '--',color = 'r')
14    figs.append(fig)
15    plt.close(fig)
16 from clawpack.visclaw import animation_tools
17 animation_tools.interact_animate_figs(figs)

```

frameno



Sharpclaw solvers

We can also use Sharpclaw solvers by initiating the solver `solver = pyclaw.SharpClawSolver1D()` and specifying its type (e.g., here the custom solver `advection`). A number of additional variables must be set, in particular the WENO order (defined by `weno_order`) and the time integrator (defined by `time_integrator`). See [the Clawpack documentation](#) for further information on these variables. Here we provide an example with the third-order strong stability-preserving method SSP33 proposed by Shu and Osher ([Ketcheson *et al.*, 2011](#)).

```

1 %matplotlib inline
2
3 from numpy import sqrt, exp, cos, logical_and, where
4 from clawpack import riemann
5 from clawpack import pyclaw
6
7 def advection(q_l,q_r,aux_l,aux_r,problem_data):
8     """
9     1d linear advection riemann solver
10    """
11    import numpy as np
12    num_eqn = 1
13    num_waves = 1
14
15    # Convenience
16    num_rp = q_l.shape[1]
17
18    # Return values
19    wave = np.empty( (num_eqn, num_waves, num_rp) )
20    s = np.empty( (num_waves, num_rp) )
21    amdq = np.empty( (num_eqn, num_rp) )
22    apdq = np.empty( (num_eqn, num_rp) )
23
24    # Local values
25    delta = np.empty(np.shape(q_l))
26
27    delta = q_r - q_l
28    a = problem_data['u']
29
30    # Compute the wave
31    # 1-Wave
32    wave[0,0,:] = delta
33    s[0,:] = a
34
35    # Compute the left going and right going fluctuations
36    for m in range(num_eqn):
37        amdq[m,:] = min(a,0) * wave[m,0,:]
38        apdq[m,:] = max(a,0) * wave[m,0,:]
39
40    return wave, s, amdq, apdq
41
42 def setup(outdir='./_output', output_style=1):
43
44     solver = pyclaw.SharpClawSolver1D()
45     solver.rp = advection
46     solver.weno_order = 5
47     solver.lim_type = 2
48     solver.time_integrator = 'SSP33'

```

```

49 solver.cfl_max = 0.5
50
51 solver.num_waves = 1
52 solver.num_eqn = 1
53 solver.kernel_language = 'Python'
54 solver.limiters = pyclaw.limiters.tvd.superbee
55 solver.bc_lower[0] = pyclaw.BC.periodic
56 solver.bc_upper[0] = pyclaw.BC.periodic
57
58 x = pyclaw.Dimension(0.0, 1.0, 100, name='x')
59 domain = pyclaw.Domain(x)
60 num_eqn = 1
61
62 state = pyclaw.State(domain, num_eqn)
63
64 a = 0.5 # advection velocity
65 state.problem_data['u'] = a
66
67 xc = domain.grid.x.centers
68 beta = 100
69 gamma = 0
70 x0 = 0.75
71 state.q[0, :] = exp(-beta * (xc-x0)**2) + where(logical_and(xc > 0.1,
xc < 0.4), 1, 0)
72
73 claw = pyclaw.Controller()
74 claw.solution = pyclaw.Solution(state, domain)
75 claw.solver = solver
76 claw.outdir = outdir
77 claw.output_style = output_style
78 claw.tfinal = 4.0
79 claw.num_output_times = 10
80 claw.keep_copy = True
81 #claw.setplot = setplot
82
83 return claw
84
85
86 def setplot(plotdata):
87     """
88     Plot solution using VisClaw.
89     """
90     plotdata.clearfigures() # clear any old figures,axes,items data
91
92     plotfigure = plotdata.new_plotfigure(name='q', figno=1)
93
94     # Set up for axes in this figure:
95     plotaxes = plotfigure.new_plotaxes()
96     plotaxes.ylimits = [-.2,1.0]
97     plotaxes.title = 'q'
98
99     # Set up for item on these axes:
100     plotitem = plotaxes.new_plotitem(plot_type='1d_plot')
101     plotitem.plot_var = 0
102     plotitem.plotstyle = '-o'
103     plotitem.color = 'b'
104     plotitem.kwargs = {'linewidth':2,'markersize':5}
105

```

```

106     return plotdata
107

```

We run the code.

```

1 claw = setup()
2 claw.run()
3
4 from clawpack.visclaw import data
5 from clawpack.visclaw import frametools
6 plotdata = data.ClawPlotData()
7 plotdata.setplot = setplot
8 claw.plotdata = frametools.call_setplot(setplot, plotdata)
9
10 frame = claw.load_frame(10)
11 f=claw.plot_frame(frame)

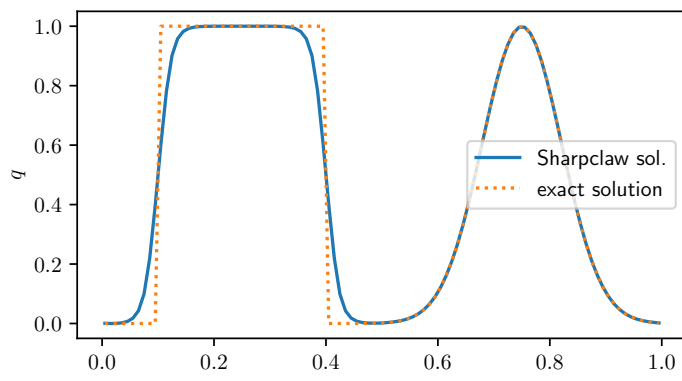
```

We can eventually plot the solution at time $t = 4$ s and compare with the exact solution.

```

1 def qtrue(x,t):
2     """
3     The true solution, for comparison.
4     """
5     from numpy import mod, exp, where, logical_and
6     beta = 100
7     gamma = 0
8     x0 = 0.75
9     u = claw.solution.state.problem_data['u']
10    xm = x - u*t
11    xm = mod(xm, 1.) # because of periodic boundary conditions
12    q = exp(-beta * (xm-x0)**2) + where(logical_and(xm > 0.1, xm < 0.4),
13    1, 0)
14    return q
15
16 %matplotlib inline
17 import numpy as np
18 import matplotlib.pyplot as plt
19 plt.rcParams['text.usetex'] = True
20 frame = claw.frames[5]
21 dt = claw.tfinal/claw.num_output_times
22 t = dt*5
23 x = frame.state.grid.c_centers
24 x = x[0]
25 true = qtrue(x,t)
26 fig, ax = plt.subplots(figsize=(5, 2.7))
27 w = frame.q[0,:]
28 ax.plot(x, w, label='Sharpclaw sol.')
29 ax.plot(x, true, ':', label='exact solution')
30 ax.legend(loc='right')
31 ax.set_xlabel(r'$x$')
32 ax.set_ylabel(r'$q$')
33 plt.savefig("frameAdvectionSharpClaw.pdf")
34

```

4.2 Acoustic waves

4.2.1 Governing equations

When linearised, the acoustic wave equations take the form:

$$\begin{aligned}\frac{\partial p}{\partial t} + K \frac{\partial u}{\partial x} &= 0, \\ \frac{\partial u}{\partial t} + \frac{1}{\rho} \frac{\partial p}{\partial x} &= 0,\end{aligned}$$

where K is the bulk modulus, ρ the density, $u(x, t)$ and $p(x, t)$ the velocity and pressure. We define the speed of sound as $c = \sqrt{K/\rho}$ and impedance $Z = \sqrt{K\rho}$. In tensorial form, the acoustic wave equation is:

$$\frac{\partial q}{\partial t} + A \cdot \frac{\partial q}{\partial x}, \text{ with } q = \begin{pmatrix} p \\ u \end{pmatrix} \text{ and } A = \begin{pmatrix} 0 & K \\ \rho^{-1} & 0 \end{pmatrix}.$$

We define the right and left eigenvector matrices R and L :

$$R = \begin{pmatrix} -Z & Z \\ 1 & 1 \end{pmatrix} \text{ et } L = \frac{1}{2} \begin{pmatrix} Z^{-1} & 1 \\ Z & 1 \end{pmatrix}$$

and the eigenvalue matrix Λ :

$$\Lambda = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} \text{ avec } \lambda_1 = -c \text{ et } \lambda_2 = +c.$$

We diagonalise the matrix A :

$$A = R \cdot \Lambda \cdot L.$$

By introducing the Riemann variables:

$$r = L \cdot q$$

we want to solve:

$$\frac{\partial r}{\partial t} + \Lambda \cdot \frac{\partial r}{\partial x} = 0,$$

subject to the initial conditions:

$$r_i = \begin{cases} r_{i,l} & \text{if } x < 0, \\ r_{i,r} & \text{if } x > 0. \end{cases}$$

In a Riemann problem, the left and right states can be connected using the right eigenvectors:

$$q_r - q_l = \alpha_1 w_1 + \alpha_2 w_2 = R \cdot \alpha,$$

thus

$$\alpha = R^{-1} \cdot (q_r - q_l) = L \cdot (q_r - q_l),$$

which leads to:

$$\begin{aligned}\alpha_1 &= \frac{1}{2} \left(-\frac{p_r - p_l}{Z} + u_r - u_l \right), \\ \alpha_2 &= \frac{1}{2} \left(\frac{p_r - p_l}{Z} + u_r - u_l \right).\end{aligned}$$

The jump from q_l to q_* is $W_1 = \alpha_1 r_1$ while the jump from q_* to q_r is $W_2 = \alpha_2 r_2$.

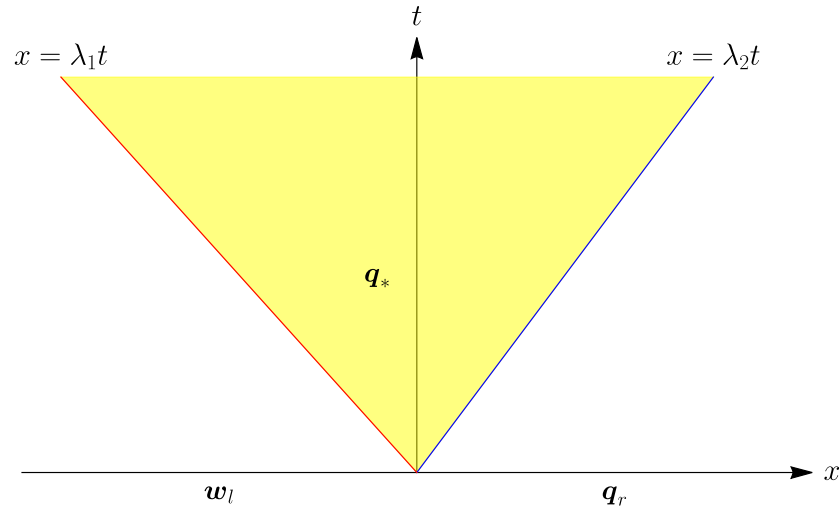


Figure 4.4 Solution to the Riemann problem.

4.2.2 Implementation

In classic Clawpack, the algorithm for the solver is quite simple.

```

1  ! =====
2  subroutine rp1(maxm,meqn,mwaves,maux,mbc,mx,ql,qr,auxl,auxr,wave,s,amdq,
3  ! =====
4
5      implicit double precision (a-h,o-z)
6
7      dimension wave(meqn, mwaves, 1-mbc:maxm+mbc)
8      dimension s(mwaves,1-mbc:maxm+mbc)
9      dimension ql(meqn, 1-mbc:maxm+mbc)
10     dimension qr(meqn, 1-mbc:maxm+mbc)
11     dimension apdq(meqn, 1-mbc:maxm+mbc)
12     dimension amdq(meqn, 1-mbc:maxm+mbc)
13
14     ! local arrays
15     ! -----
16     dimension delta(2)
17
18     ! # density, bulk modulus, and sound speed, and impedance of medium:
19     ! # (should be set in setprob.f)
20     common /cparam/ rho,bulk,cc,zz
21
22     ! # find a1 and a2, the coefficients of the 2 eigenvectors:
23     do 20 i = 2-mbc, mx+mbc
24         delta(1) = ql(1,i) - qr(1,i-1)
25         delta(2) = ql(2,i) - qr(2,i-1)
26         a1 = (-delta(1) + zz*delta(2)) / (2.d0*zz)
27         a2 = (delta(1) + zz*delta(2)) / (2.d0*zz)
28
29     ! # Compute the waves.
30
31     wave(1,1,i) = -a1*zz
32     wave(2,1,i) = a1

```

```

33      s(1,i) = -cc
34
35      wave(1,2,i) = a2*zz
36      wave(2,2,i) = a2
37      s(2,i) = cc
38
39  20 END DO
40
41
42  !      # compute the leftgoing and rightgoing flux differences:
43  !      # Note s(1,i) < 0    and    s(2,i) > 0.
44
45  do 220 m=1,meqn
46      do 220 i = 2-mbc, mx+mbc
47          amdq(m,i) = s(1,i)*wave(m,1,i)
48          apdq(m,i) = s(2,i)*wave(m,2,i)
49  220 END DO
50
51  return
52  end subroutine rp1

```

4.2.3 Implementation in Pyclaw

Here we give an example of Pyclaw script. We start with the initialisation of the controller.

```

1 claw = pyclaw.Controller()  # Creation of the controller
2 claw.tfinal = 1.0          # Final time
3 claw.keep_copy = True      # Keep solution data in memory for plotting
4 claw.output_format = None  # Don't write solution data to file
5 claw.num_output_times = 50 # Write 50 output frames
6
7 # Riemann solver
8 # Here we use the one provided in Clawpack
9 # We could have also used SharpClawSolver1D
10 riemann_solver = riemann.acoustics_1D
11 claw.solver = pyclaw.ClawSolver1D(riemann_solver)

```

For the boundary conditions, we can specify all boundaries at once by selecting periodic (periodic), wall (wall), user-defined (custom), or extrapolation (extrap):

```

12 # Boundary conditions
13 claw.solver.all_bcs = pyclaw.BC.periodic

```

We can also define the boundary conditions separately

```

14 claw.solver.bc.lower[0] = pyclaw.BC.wall
15 claw.solver.bc.upper[0] = pyclaw.BC.wall

```

We will solve the governing equation over the unit line $[0, 1]$ using 100 grid cells. The arguments of the Domain object are tuples:

```

16 domain = pyclaw.Domain( (0.,), (1.,), (100,))

```

Next we create a solution object whose dimension is given by `claw.solver.num_eqn` (set automatically when selecting the solver) and which belongs to the controller and extends over the domain specified above:

```

17 claw.solution = pyclaw.Solution(claw.solver.num_eqn,domain)

```

The initial data is specified in the array `q`: the pressure is contained in `q[0,:]` and the velocity in `q[1,:]`. We impose a wavepacket to the initial pressure and zero velocity.

```
18 x=domain.grid.x.centers
19 beta=100; gamma=5; x0=0.75
20 claw.solution.q[0,:] = np.exp(-beta * (x-x0)**2) * np.cos(gamma * (x - x0))
21 claw.solution.q[1,:] = 0.
```

The Riemann solver requires some physical parameters to be specified:

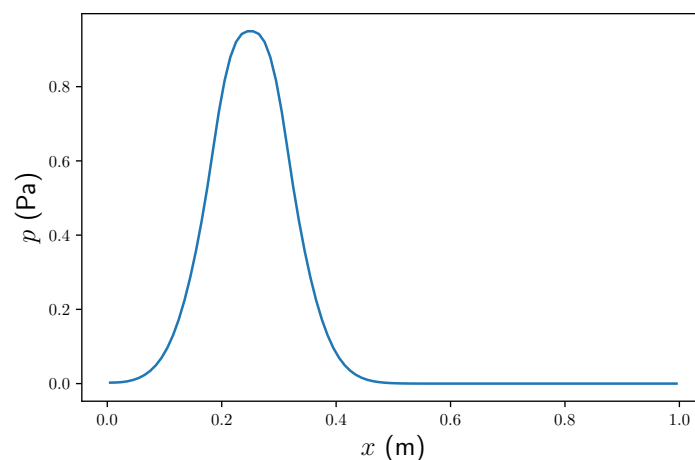
```
22 import numpy as np
23
24 density = 1.0
25 bulk_modulus = 1.0
26 impedance = np.sqrt(density*bulk_modulus)
27 sound_speed = np.sqrt(density/bulk_modulus)
28
29 claw.solution.state.problem_data = {
30     'rho' : density,
31     'bulk' : bulk_modulus,
32     'zz' : np.sqrt(density*bulk_modulus),
33     'cc' : np.sqrt(bulk_modulus/density)
34 }
```

We can now run Pyclaw:

```
1 claw.solver.dt_initial = 1.e-99
2 status = claw.run()
```

The results are contained in `claw.frames[:]`. We can plot a single frame using matplotlib:

```
1 pressure = claw.frames[50].q[0,:]
2 plt.rcParams['text.usetex'] = True
3 #plt.plot(x,pressure,'-')
4 fig, ax = plt.subplots(figsize=(6, 4), tight_layout=True)
5 ax.plot(x, pressure)
6
7 ax.set_xlabel(r'$x$ (m)', fontsize=16)
8 ax.set_ylabel(r'$p$ (Pa)', fontsize=16)
9 plt.savefig('solution50.pdf')
```

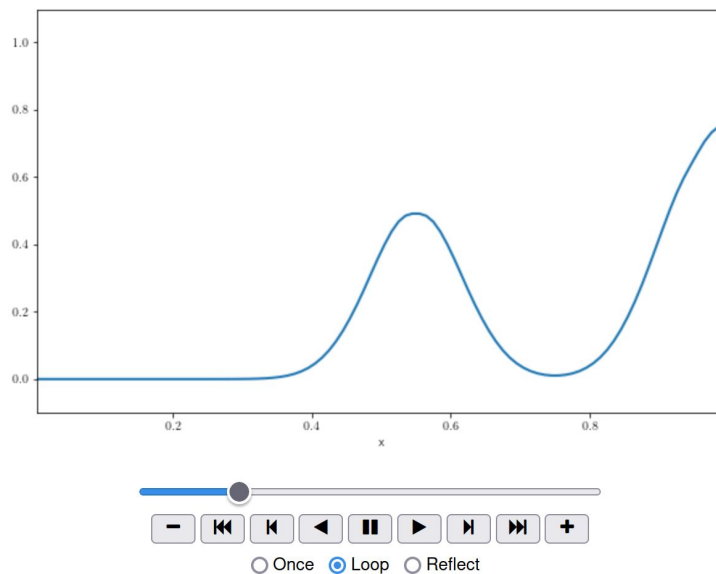


We can also see animations importing `ianimate` from the `visclaw` library (Clawpack visualisation tools):

```

35 from clawpack.visclaw import ianimate
36 ianimate.ianimate(claw)

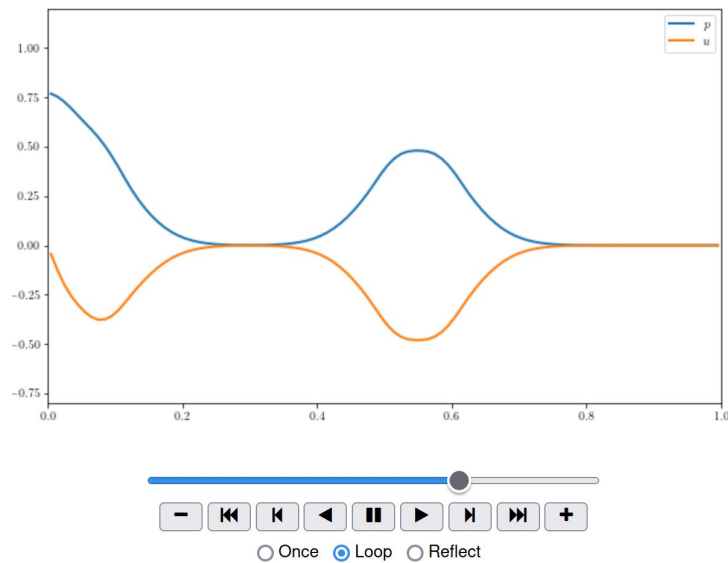
```



```

37 from matplotlib import animation
38 import matplotlib.pyplot as plt
39 from IPython.display import HTML
40 import numpy as np
41
42 fig = plt.figure(figsize=(10,6))
43 ax = plt.axes(xlim=(0, 1), ylim=(-0.8, 1.2))
44
45 frame = claw.frames[0]
46 pressure = frame.q[0,:]
47 line1, = ax.plot([], [], lw=2)
48 line2, = ax.plot([], [], lw=2)
49 plt.legend([r'$p$', r'$u$'])
50
51 def fplot(frame_number):
52     frame = claw.frames[frame_number]
53     pressure = frame.q[0,:]
54     velocity = frame.q[1,:]
55     line1.set_data(x,pressure)
56     line2.set_data(x,velocity)
57     return line1,
58
59 anim = animation.FuncAnimation(fig, fplot, frames=len(claw.frames),
60                               interval=30, repeat=False)
61 plt.close()
62 HTML(anim.to_jshtml())

```



Here is how Pyclaw has encoded the solver of the Riemann problem.

```

1 def acoustics_1D(q_l,q_r,aux_l,aux_r,problem_data):
2     r"""
3     Basic 1d acoustics riemann solver, with interleaved arrays
4
5     *problem_data* is expected to contain -
6     - *zz* - (float) Impedance
7     - *cc* - (float) Speed of sound
8
9     See :ref:'pyclaw_rp' for more details.
10
11     :Version: 1.0 (2009-02-03)
12     """
13     import numpy as np
14
15     # Convenience
16     num_rp = np.size(q_l,1)
17     num_eqn = 2
18     num_waves = 2
19
20     # Return values
21     wave = np.empty( (num_eqn, num_waves, num_rp) )
22     s = np.empty( (num_waves, num_rp) )
23     amdq = np.empty( (num_eqn, num_rp) )
24     apdq = np.empty( (num_eqn, num_rp) )
25
26     # Local values
27     delta = np.empty(np.shape(q_l))
28
29     delta = q_r - q_l
30     a1 = (-delta[0,:] + problem_data['zz']*delta[1,:]) / (2.0 *
31     problem_data['zz'])
32     a2 = (delta[0,:] + problem_data['zz']*delta[1,:]) / (2.0 * problem_data
33     ['zz'])
34
35     # Compute the waves

```

```

34 # 1-Wave
35 wave[0,0,:] = -a1 * problem_data['zz']
36 wave[1,0,:] = a1
37 s[0,:] = -problem_data['cc']
38
39 # 2-Wave
40 wave[0,1,:] = a2 * problem_data['zz']
41 wave[1,1,:] = a2
42 s[1,:] = problem_data['cc']
43
44 # Compute the left going and right going fluctuations
45 for m in range(num_eqn):
46     amdq[m,:] = s[0,:] * wave[m,0,:]
47     apdq[m,:] = s[1,:] * wave[m,1,:]
48
49 return wave, s, amdq, apdq

```

Here is another example of notebook setting up a solver for the acoustic wave equations, where a user-defined solver is employed.

```

1 %matplotlib inline
2
3 from numpy import sqrt, exp, cos
4 from clawpack import riemann
5 from clawpack import pyclaw
6 def acoustics(q_l,q_r,aux_l,aux_r,problem_data):
7     """
8     Basic 1d acoustics riemann solver, with interleaved arrays
9
10    *problem_data* is expected to contain -
11    - *zz* - (float) Impedance
12    - *cc* - (float) Speed of sound
13
14    See :ref:'pyclaw_rp' for more details.
15
16    :Version: 1.0 (2009-02-03)
17    """
18    import numpy as np
19    num_eqn = 2
20    num_waves = 2
21
22    # Convenience
23    num_rp = np.size(q_l,1)
24
25    # Return values
26    wave = np.empty( (num_eqn, num_waves, num_rp) )
27    s = np.empty( (num_waves, num_rp) )
28    amdq = np.empty( (num_eqn, num_rp) )
29    apdq = np.empty( (num_eqn, num_rp) )
30
31    # Local values
32    delta = np.empty(np.shape(q_l))
33
34    delta = q_r - q_l
35    a1 = (-delta[0,:] + problem_data['zz']*delta[1,:]) / (2.0 *
36    problem_data['zz'])
37    a2 = (delta[0,:] + problem_data['zz']*delta[1,:]) / (2.0 * problem_data
38    ['zz'])

```



```

38 # Compute the waves
39 # 1-Wave
40 wave[0,0,:] = -a1 * problem_data['zz']
41 wave[1,0,:] = a1
42 s[0,:] = -problem_data['cc']
43
44 # 2-Wave
45 wave[0,1,:] = a2 * problem_data['zz']
46 wave[1,1,:] = a2
47 s[1,:] = problem_data['cc']
48
49 # Compute the left going and right going fluctuations
50 for m in range(num_eqn):
51     amdq[m,:] = s[0,:] * wave[m,0,:]
52     apdq[m,:] = s[1,:] * wave[m,1,:]
53
54     return wave, s, amdq, apdq
55
56 def setup(outdir='._output', output_style=1):
57
58     riemann_solver = acoustics
59     solver = pyclaw.ClawSolver1D(riemann_solver)
60     solver.limiters = pyclaw.limiters.tvd.MC
61     solver.kernel_language = 'Python'
62     solver.num_waves = 2
63     solver.num_eqn = 2
64     x = pyclaw.Dimension(0.0, 1.0, 100, name='x')
65     domain = pyclaw.Domain(x)
66     num_eqn = 2
67     state = pyclaw.State(domain, num_eqn)
68
69     solver.bc_lower[0] = pyclaw.BC.periodic
70     solver.bc_upper[0] = pyclaw.BC.periodic
71
72     rho = 1.0 # Material density
73     bulk = 1.0 # Material bulk modulus
74
75     state.problem_data['rho'] = rho
76     state.problem_data['bulk'] = bulk
77     state.problem_data['zz'] = sqrt(rho*bulk) # Impedance
78     state.problem_data['cc'] = sqrt(bulk/rho) # Sound speed
79
80     xc = domain.grid.x.centers
81     beta = 100
82     gamma = 0
83     x0 = 0.75
84     state.q[0, :] = exp(-beta * (xc-x0)**2) * cos(gamma * (xc - x0))
85     state.q[1, :] = 0.0
86
87     solver.dt_initial = domain.grid.delta[0] / state.problem_data['cc'] *
0.1
88
89     claw = pyclaw.Controller()
90     claw.solution = pyclaw.Solution(state, domain)
91     claw.solver = solver
92     claw.outdir = outdir
93     claw.output_style = output_style
94     output_style = 1

```

```

95     claw.tfinal = 1.0
96     claw.num_output_times = 10
97     claw.keep_copy = True
98
99     return claw

```

To run the script and plot one result here (frame 10), the following can be done:

```

1  claw = setup()
2  claw.run()
3
4  from clawpack.visclaw import data
5  from clawpack.visclaw import frametools
6  plotdata = data.ClawPlotData()
7  plotdata.setplot = setplot
8  claw.plotdata = frametools.call_setplot(setplot, plotdata)
9
10 frame = claw.load_frame(10)
11 f=claw.plot_frame(frame)

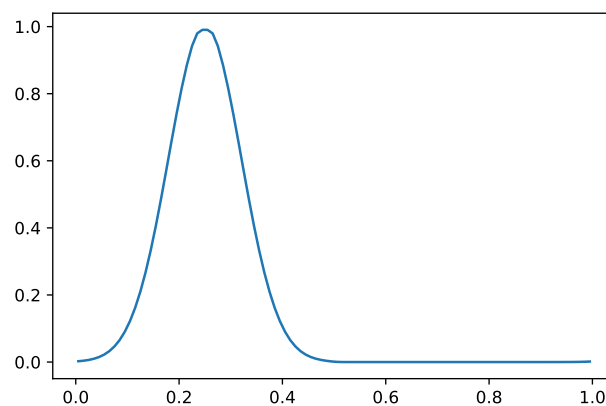
```

We can also plot a frame directly:

```

1  %matplotlib inline
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  frame = claw.frames[5]
6  w = frame.q[0,:]
7  x = frame.state.grid.c_centers
8  x = x[0]
9
10 plt.plot(x, w)

```



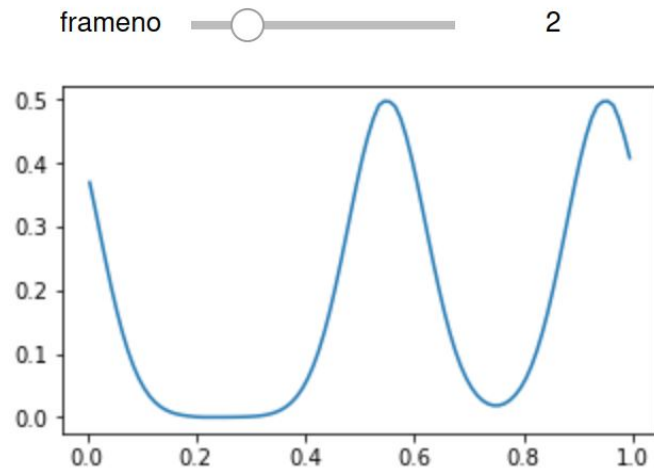
We can also plot an animation:

```

1  nsimul=np.size(claw.frames)
2  figs = []
3  for i in range(nsimul):
4      fig = plt.figure(figsize=(5,3))
5      frame = claw.frames[i]
6      w = frame.q[0,:]
7      x = frame.state.grid.c_centers

```

```
8     x = x[0]
9     plt.plot(x, w)
10    figs.append(fig)
11    plt.close(fig)
12    #plt.plot(x, w)
13    from clawpack.visclaw import animation_tools
14    animation_tools.interact_animate_figs(figs)
```



4.3 Spatially-varying advection equation

4.3.1 Theory

Let us consider the variable-coefficient linear advection equation:

$$\frac{\partial}{\partial t} q + A(x) \cdot \frac{\partial}{\partial x} q = 0, \quad (4.6)$$

where $A(x)$ is a non-constant diagonalisable matrix with real and distinct eigenvalues at any position x of a given domain. In this form, Eq. (4.6) is not in conservative form. Of course, we could make it consistent with the conservative form:

$$\frac{\partial}{\partial t} A + \frac{\partial}{\partial x} (A \cdot q) = (A' \cdot q), \quad (4.7)$$

but when doing so, we transform a homogeneous equation into a non-homogeneous equation, and from the numerical standpoint, handling the resulting source term may be problematic. It can be shown that the overall approach implemented in Clawpack can be applied to equations like Eq. (4.6) (see Chap. 9.12 [LeVeque, 2002](#)):

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (\mathcal{A}^- \Delta Q_{i+1/2}^n + \mathcal{A}^+ \Delta Q_{i-1/2}^n) - \frac{\Delta t}{\Delta x} (\hat{F}_{i+1/2}^n - \hat{F}_{i-1/2}^n), \quad (4.8)$$

where the operators $\mathcal{A}^\pm$ are defined by Eqs. (3.46) and (3.46), and the flux correction $\hat{F}_{i+1/2}^n$ is:

$$\hat{F}_{i+1/2}^n = \frac{1}{2} \sum_{k=1}^{M_w} |s_{i-1/2}^k| \left(1 - \frac{\Delta t}{\Delta x} |s_{i-1/2}^k| \right) \tilde{W}_{i-1/2}^k \quad (4.9)$$

The limiters $\tilde{W}_{i-1/2}^k$ require more work (see Chap. 9.13 [LeVeque, 2002](#)).

The Clawpack approach can be extended to spatially-varying advection equations in a simple way. Let us consider the one-dimensional advection equation

$$\frac{\partial}{\partial t} q + u(x) \frac{\partial}{\partial x} q = 0, \quad (4.10)$$

where $u(x)$ is a function of x and the initial condition is

$$q(x, 0) = q_0(x). \quad (4.11)$$

Contrary to the linear case, we cannot express the cell average Q_i^{n+1} as a function of the flux difference $F_{i+1/2} - F_{i-1/2}$ since Eq. (4.10) is not in a conservative form, but we can still express the change in Q_i^{n+1} from the wave

$$W_{i-1/2} = Q_i - Q_{i-1}$$

and speed

$$s_{i-1/2} = \begin{cases} u_i & \text{if } u_i > 0, \\ u_{i-1} & \text{if } u_i < 0. \end{cases}$$

. Note that in this discretisation, the velocity u is evaluated at the cell center, but we could have defined at the cell interface (which would make more sense in multidimensional problems). We define the fluctuations

$$\begin{aligned} \mathcal{A}^+ \Delta Q_{i-1/2} &= s_{i-1/2}^+ W_{i-1/2}, \\ \mathcal{A}^- \Delta Q_{i-1/2} &= s_{i-1/2}^- W_{i-1/2}. \end{aligned}$$

4.3.2 Implementation in Pyclaw

The solver is a simple adaptation of the solver for linear advection.

```

1 def SpatialAdvection(q_l,q_r,aux_l,aux_r,problem_data):
2     r"""Basic 1d advection riemann solver
3     *aux(i)* should contain -
4     - *u(x_i)* - (float) advection speed
5     """
6     num_eqn = 1
7     num_waves = 1
8     # Number of Riemann problems we are solving
9     num_rp = q_l.shape[1]
10
11     # Return values
12     wave = np.empty( (num_eqn,num_waves,num_rp) )
13     s = np.empty( (num_waves,num_rp) )
14     amdq = np.zeros( (num_eqn,num_rp) )
15     apdq = np.zeros( (num_eqn,num_rp) )
16
17     wave[0,0,:] = q_r[0,:] - q_l[0,:]
18
19     s[0,:] = aux_l[0,:]
20
21     apdq[0,:] = (aux_l[0,]>0)*s[0,:] * wave[0,0,:]
22     amdq[0,:] = (aux_l[0,]<0)*s[0,:] * wave[0,0,:]
23
24     return wave, s, amdq, apdq

```

4.3.3 Application

Let us consider the case $a = 2\sqrt{x}$. In this case, Eq. (4.10) can be cast in the characteristic form

$$\frac{dq}{dt} = 0 \text{ along } \frac{dx}{dt} = 2\sqrt{x}.$$

This shows that the characteristics are curves

$$\frac{dx}{dt} = 2\sqrt{x} \Rightarrow \frac{dx}{2\sqrt{x}} = dt \Rightarrow \sqrt{x} - \sqrt{x_0} = t.$$

The characteristic form implies that

$$q(x,t) = q_0(x_0) = q_0((\sqrt{x} - t)^2).$$

As initial condition, we take the same condition as in § 4.1.3. We need define the auxiliary variable `aux` in the `auxinit` function to define the advection velocity.

```

1 %matplotlib inline
2
3 import numpy as np
4 from clawpack import riemann
5 from clawpack import pyclaw
6 import matplotlib.pyplot as plt
7
8 def qtrue(x,t):
9     """

```

```

10     The true solution, for comparison.
11     """
12     from numpy import mod, exp, where, logical_and
13     beta = 100
14     gamma = 0
15     x0 = 0.75
16
17     xm = (np.sqrt(x)-t)**2
18     q = exp(-beta * (xm-x0)**2) + where(logical_and(xm > 0.1, xm < 0.4),
19     1, 0)
20     return q
21
22 # Advection speed
23 def auxinit(state):
24     # Initialize aux
25     xc = state.grid.x.centers
26     state.aux[0,:] = 2*np.sqrt(xc)
27
28 def setup(outdir='./_output'):
29     from clawpack import riemann
30
31     solver = pyclaw.ClawSolver1D(SpatialAdvection)
32     solver.num_waves = 1
33     solver.num_eqn = 1
34     solver.kernel_language = 'Python'
35     solver.limiters = pyclaw.limiters.tvd.MC
36     solver.bc_lower[0] = pyclaw.BC.extrap
37     solver.bc_upper[0] = pyclaw.BC.extrap
38     solver.aux_bc_lower[0] = pyclaw.BC.extrap
39     solver.aux_bc_upper[0] = pyclaw.BC.extrap
40
41     xlower=0.0; xupper=2.0; mx=200
42     x = pyclaw.Dimension(xlower,xupper,mx,name='x')
43     domain = pyclaw.Domain(x)
44     num_aux, num_eqn = 1, 1
45     state = pyclaw.State(domain,num_eqn,num_aux)
46
47     xc = domain.grid.x.centers
48     state.q[0,:] = qtrue(xc,0)
49     auxinit(state)
50
51     claw = pyclaw.Controller()
52     claw.outdir = outdir
53     claw.solution = pyclaw.Solution(state,domain)
54     claw.solver = solver
55
56     claw.tfinal = .3
57     claw.num_output_times = 10
58     claw.keep_copy = True
59
60     return claw

```

We run the code and plot the solution for $t = 0.3$ s.

```

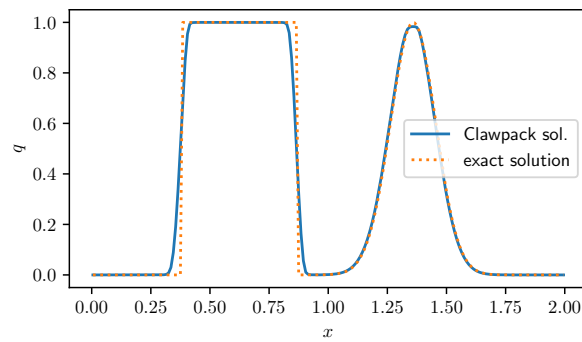
1 claw = setup()
2 claw.run()
3 plt.rcParams['text.usetex'] = True
4 index = 10

```

```

5 frame = claw.frames[index]
6 dt = claw.tfinal/claw.num_output_times
7 t = dt*index
8 x = frame.state.grid.c_centers
9 x = x[0]
10
11 true = qtrue(x,t)
12 fig, ax = plt.subplots(figsize=(5, 2.7))
13 w = frame.q[0,:]
14
15 ax.plot(x, w, label='Clawpack sol.')
16 ax.plot(x, true, ':', label='exact solution')
17 ax.legend(loc='right')
18 ax.set_xlabel(r'$x$')
19 ax.set_ylabel(r'$q$')
20 plt.savefig("SpatiallyVaryingAdvection.pdf",bbox_inches='tight')

```



4.4 Conservative spatially-varying advection equation

4.4.1 Theory

Let us consider the one-dimensional advection equation, which has a conservative form but also exhibits a space-dependent flux function

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} (u(x)q) = 0, \quad (4.12)$$

where $u(x) > 0$ is a function of x and the initial condition is

$$q(x, 0) = q_0(x). \quad (4.13)$$

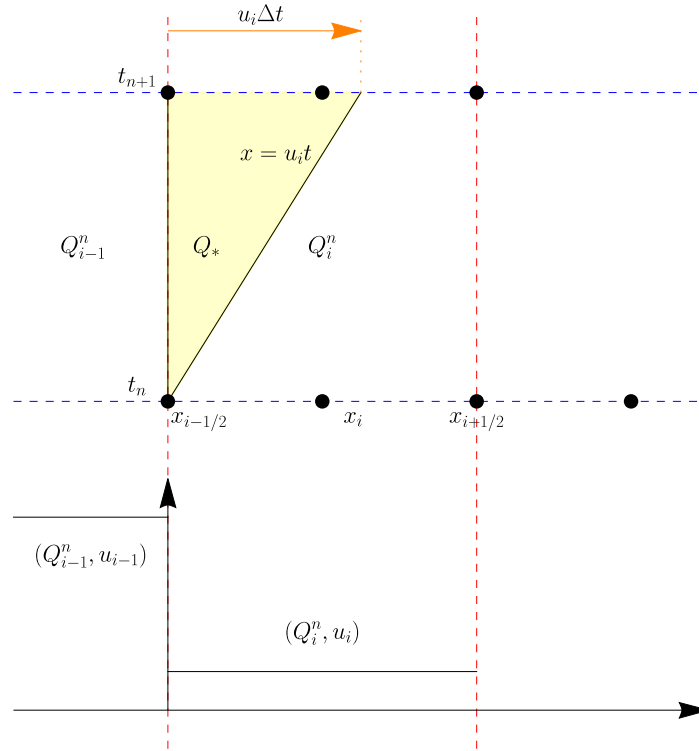


Figure 4.5 Notation for the Riemann problem in Clawpack.

When discretising the equation and solving the associated Riemann problem, there are in fact two Riemann subproblems (see Fig. 4.5):

$$\begin{cases} \partial_t q + u_{i-1} \partial_x q = 0 & \text{subject to } q(x, t_n) = Q_{i-1}^n & \text{if } x < x_{i-1/2}, \\ \partial_t q + u_i \partial_x q = 0 & \text{subject to } q(x, t_n) = Q_i^n & \text{if } x > x_{i-1/2}. \end{cases} \quad (4.14)$$

These Riemann subproblems are the following:

- At the interface $x_{i-1/2}$, there is a jump from u_{i-1} to u_i . As this jump occurs at the interface $x_{i-1/2}$, its velocity is zero. This jump is associated with a change in q since the flux $f(x, q) = u(x)q$ remains constant. We assume that q jumps from Q_{i-1}^n to an intermediate state Q_* .

- In the cell C_i , the second Riemann involves the left state Q_* and the right state Q_i^n . The shock moves with the velocity $s = u_i$.

The intermediate state is computed by considering the flux conservation

$$u_{i-1}Q_{i-1}^n = u_i Q_* \Rightarrow Q_* = \frac{u_{i-1}Q_{i-1}^n}{u_i}.$$

The first discontinuity has the strength

$$W_{i-1/2}^1 = Q_* - Q_{i-1}$$

and is stationary ($s_{i-1/2}^1 = 0$). The second wave has the strength and velocity

$$W_{i-1/2}^2 = Q_i^n - Q_* \text{ and } s_{i-1/2}^2 = u_i$$

We define the fluctuations

$$\begin{aligned} \mathcal{A}^+ \Delta Q_{i-1/2} &= \sum_{k=1,2} s_{i-1/2}^k W_{i-1/2}^k = u_i \left(Q_i^n - \frac{u_{i-1}Q_{i-1}^n}{u_i} \right) = u_i Q_i^n - u_{i-1} Q_{i-1}^n, \\ \mathcal{A}^- \Delta Q_{i-1/2} &= 0. \end{aligned}$$

Similarly, when $u(x) < 0$, we have

$$\begin{aligned} Q_* &= \frac{u_i Q_i^n}{u_{i-1}}, \\ W_{i-1/2}^2 &= Q_* - Q_{i-1}^n, \\ s_{i-1/2}^2 &= u_{i-1}, \\ \mathcal{A}^+ \Delta Q_{i-1/2} &= 0, \\ \mathcal{A}^- \Delta Q_{i-1/2} &= \sum_{k=1,2} s_{i-1/2}^k W_{i-1/2}^k = u_{i-1} \left(\frac{u_i Q_i^n}{u_{i-1}} - Q_{i-1}^n \right) = u_i Q_i^n - u_{i-1} Q_{i-1}^n. \end{aligned}$$

The general case is thus

$$\begin{aligned} Q_* &= \begin{cases} \frac{u_{i-1}Q_{i-1}^n}{u_i} & \text{if } u_i > 0, \\ \frac{u_i Q_i^n}{u_{i-1}} & \text{if } u_i < 0 \end{cases}, \\ W_{i-1/2}^2 &= \begin{cases} Q_i^n - Q_* & \text{if } u_i > 0, \\ Q_* - Q_{i-1}^n & \text{if } u_i < 0 \end{cases}, \\ s_{i-1/2}^2 &= \begin{cases} u_i & \text{if } u_i > 0, \\ u_{i-1} & \text{if } u_i < 0 \end{cases}, \\ \mathcal{A}^+ \Delta Q_{i-1/2} &= s_{i-1/2}^- W_{i-1/2}^2 = u_i Q_i^n - u_{i-1} Q_{i-1}^n, \\ \mathcal{A}^- \Delta Q_{i-1/2} &= s_{i-1/2}^+ W_{i-1/2}^2 = u_i Q_i^n - u_{i-1} Q_{i-1}^n. \end{aligned}$$

Although the overall problem involves two waves, only one affects the evolution of Q_i^{n+1} since the other wave has a zero velocity, and thus its contribution to $\mathcal{A}^\pm \Delta Q_{i-1/2}$ is zero.

4.4.2 Implementation in Clawpack

The Clawpack solver encodes the general case.

```

1  ! =====
2  subroutine rp1(maxmx,meqn,mwaves,maux,mbc,mx,ql,qr,auxl,auxr,wave,s,amdq,
   apdq)
3  ! =====
4
5      implicit double precision (a-h,o-z)
6      dimension  ql(meqn,1-mbc:maxmx+mbc)
7      dimension  qr(meqn,1-mbc:maxmx+mbc)
8      dimension  qs(meqn,1-mbc:maxmx+mbc)
9      dimension  auxl(maux,1-mbc:maxmx+mbc)
10     dimension  auxr(maux,1-mbc:maxmx+mbc)
11     dimension  s(mwaves,1-mbc:maxmx+mbc)
12     dimension  wave(meqn, mwaves,1-mbc:maxmx+mbc)
13     dimension  amdq(meqn,1-mbc:maxmx+mbc)
14     dimension  apdq(meqn,1-mbc:maxmx+mbc)
15     common /comrp/ u, p
16
17     do 30 i=1-mbc,mx+mbc
18
19         u = auxl(1,i)
20         p = auxr(1,i-1)
21
22         if (u > 0.d0) then
23             qs(1,i) = qr(1,i-1) * p / u
24             wave(1,1,i) = ql(1,i) - qs(1,i)
25             s(1,i) = u
26             amdq(1,i) = 0.d0
27             apdq(1,i) = u * wave(1,1,i)
28         endif
29         if (u < 0.d0) then
30             qs(1,i) = qr(1,i) * u / p
31             wave(1,1,i) = qs(1,i) - ql(1,i-1)
32             s(1,i) = p
33             amdq(1,i) = p * wave(1,1,i)
34             apdq(1,i) = 0.d0
35         endif
36         if (u == 0.d0) then
37             wave(1,1,i) = 0.d0
38             s(1,i) = 0.d0
39             amdq(1,i) = 0.d0
40             apdq(1,i) = 0.d0
41         endif
42
43     30 end do
44
45     return
46 end subroutine rp1

```

4.4.3 Implementation in Pyclaw

Here is the Pyclaw version of the solver. In the function `setup()`, we give the choice between the Pyclaw and Fortran versions of the solver. The function `u` is defined by the function. `auxinit(state)`

```

1 def SpatialAdvection(q_l,q_r,aux_l,aux_r,problem_data):
2     r"""Basic 1d advection riemann solver
3     *aux(i)* should contain -
4     - *u(x_i)* - (float) advection speed
5     """
6     num_eqn = 1
7     num_waves = 1
8     # Number of Riemann problems we are solving
9     num_rp = q_l.shape[1]
10
11     # Return values
12     wave = np.empty( (num_eqn,num_waves,num_rp) )
13     s = np.empty( (num_waves,num_rp) )
14     amdq = np.zeros( (num_eqn,num_rp) )
15     apdq = np.zeros( (num_eqn,num_rp) )
16     q_sp = np.zeros( (num_eqn,num_rp) )
17     q_sm = np.zeros( (num_eqn,num_rp) )
18     eps = 1.e-12
19
20     q_sp[0,:] = q_l[0,:]*aux_l[0,:]/ (aux_r[0,:]+eps)
21     q_sm[0,:] = q_r[0,:]*aux_r[0,:]/ (aux_l[0,:]+eps)
22     wave[0,0,:] = (aux_l[0,:]<0)*(q_sm[0,:] - q_l[0,:])+(aux_l[0,:]>0)*(q_r
23     [0,:] - q_sp[0,:])
24
25     s[0,:] = (aux_l[0,:]>0)* aux_r[0,:] +(aux_l[0,:]<0)* aux_l[0,:]
26
27     apdq[0,:] = (aux_l[0,:]>0)*s[0,:] * wave[0,0,:]
28     amdq[0,:] = (aux_l[0,:]<0)*s[0,:] * wave[0,0,:]
29
30     return wave, s, amdq, apdq
31
32 # Advection speed
33 def auxinit(state):
34     xc = state.grid.x.centers
35     state.aux[0,:] = 2*np.sqrt(xc)
36
37 def setup(typeSolver=1,outdir='./_output'):
38     from clawpack import riemann
39
40     if typeSolver == 2:
41         solver = pyclaw.ClawSolver1D(SpatialAdvection)
42         solver.kernel_language = 'Python'
43     elif typeSolver == 1:
44         solver = pyclaw.ClawSolver1D(advectionVariableBis)
45         solver.kernel_language = 'Fortran'
46     solver.num_waves = 1
47     solver.num_eqn = 1
48     solver.limiters = pyclaw.limiters.tvd.superbee
49     solver.order = 2 #1: Godunov, 2: Lax-Wendroff-LeVeque
50
51     solver.limiters = pyclaw.limiters.tvd.MC
52     solver.bc_lower[0] = pyclaw.BC.extrap
53     solver.bc_upper[0] = pyclaw.BC.extrap
54     solver.aux_bc_lower[0] = pyclaw.BC.extrap
55     solver.aux_bc_upper[0] = pyclaw.BC.extrap
56
57     xlower = 0.00; xupper = 2.0; mx = 200
58     x = pyclaw.Dimension(xlower,xupper,mx,name='x')

```

```

58 domain = pyclaw.Domain(x)
59 num_aux, num_eqn = 1, 1
60 state = pyclaw.State(domain, num_eqn, num_aux)
61
62 xc = domain.grid.x.centers
63 state.q[0,:] = qinit(xc)
64 auxinit(state)
65
66 claw = pyclaw.Controller()
67 claw.outdir = outdir
68 claw.solution = pyclaw.Solution(state, domain)
69 claw.solver = solver
70
71 claw.tfinal = 1
72 claw.num_output_times = 20
73 claw.keep_copy = True
74
75 return claw

```

4.4.4 Application

Let us consider Eq. (4.12) with

$$u(x) = 2\sqrt{x}.$$

Eq. (4.12) can be recast

$$\frac{\partial}{\partial t}q + u(x)\frac{\partial}{\partial x}(q) = -q(x,t)\frac{d}{dx}(u),$$

and in characteristic form:

$$\frac{d}{dt}q = -\frac{q}{\sqrt{x}} \text{ along } \frac{dx}{dt} = 2\sqrt{x}.$$

The characteristic curve's equation is thus

$$\sqrt{x} - \sqrt{x_0} = t.$$

The q time derivative becomes

$$\frac{dq}{dt} = -\frac{q}{\sqrt{x}} = -\frac{q}{t + \sqrt{x_0}} \Rightarrow \frac{dq}{q} = -\frac{dt}{t + \sqrt{x_0}}.$$

The solution is obtained by taking the initial condition (4.13)

$$\ln \frac{q}{q_0} = -\ln \frac{t + \sqrt{x_0}}{\sqrt{x_0}}.$$

We eventually find

$$\begin{aligned} q(x, t) &= q_0(x_0) \frac{\sqrt{x_0}}{t + \sqrt{x_0}}, \\ &= q_0 ((\sqrt{x} - t)^2) \frac{|\sqrt{x} - t|}{\sqrt{x}}. \end{aligned}$$

We encode the exact solution.

```

1 def qtrue(x,t):
2     """
3     The true solution, for comparison.
4     """
5     from numpy import mod, exp, where, logical_and
6     beta = 100
7     gamma = 0
8     x0 = 0.75
9     if isinstance(x,(np.ndarray,list)):
10         m = np.size(x)
11         q = np.empty((m))
12         for i in range(m):
13             xm = (np.sqrt(x[i])-t)**2
14             if x[i] != 0.:
15                 q[i] = qinit(xm)/np.sqrt(x[i])*np.sqrt(xm)
16             else:
17                 q[i] = 0.
18         if isinstance(x,(float,int)):
19             if x != 0:
20                 xm = (np.sqrt(x)-t)**2
21                 q = qinit(xm)/np.sqrt(x)*np.sqrt(xm)
22             else:
23                 q = 0
24         return q
25
26 def qinit(x):
27     """
28     The true solution, for comparison.
29     """
30     from numpy import mod, exp, where, logical_and
31     beta = 100
32     gamma = 0
33     x0 = 0.75
34     q = exp(-beta * (x-x0)**2) + where(logical_and(x > 0.1, x < 0.4), 1,
35     0)
36     return q

```

We can use the matplotlib animation toolbox to plot the solution at different times.

```

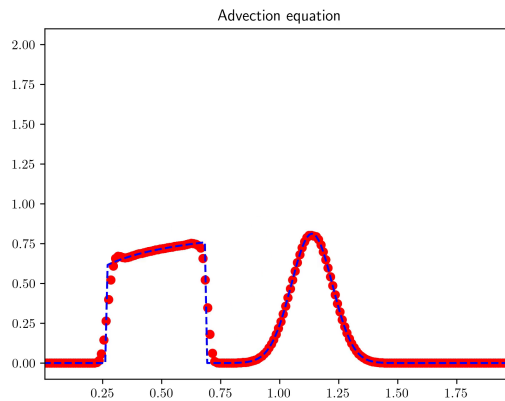
1 def advection_animation():
2     import matplotlib.animation
3     import numpy
4     # compute the solution with the method defined above:
5     claw = setup(typeSolver=2)
6     claw.keep_copy = True
7     claw.run()
8     x = claw.frames[0].grid.dimensions[0].centers
9     x_true = numpy.linspace(0.00, 2.0, 200)
10
11     fig = plt.figure()
12     axes = plt.subplot(1, 1, 1)
13     axes.set_xlim((x[0], x[-1]))
14     axes.set_ylim((-0.1, 2))
15     axes.set_title("Advection equation")
16
17     def init():
18         axes.set_xlim((x[0], x[-1]))
19         axes.set_ylim((-0.1,2.1))
20         computed_line, = axes.plot(x[0], claw.frames[0].q[0, :][0], 'ro')

```

```

21     exact_line, = axes.plot(x_true, qtrue(x_true,0.0), 'b--')
22     return (computed_line, exact_line)
23
24     computed_line, exact_line = init()
25
26     def fplot(n):
27         computed_line.set_data([x,], [claw.frames[n].q[0, :]])
28         exact_line.set_data([x_true], [qtrue(x_true,claw.frames[n].t)])
29         return (computed_line, exact_line)
30
31     frames_to_plot = range(0, len(claw.frames))
32     plt.close(fig)
33     return matplotlib.animation.FuncAnimation(fig, fplot, frames=
34     frames_to_plot, interval=100,
35                                           blit=True, init_func=None, repeat=False)
36
37 from IPython.display import HTML
38 anim = advection_animation()
39 HTML(anim.to_jshtml())

```



We can also plot an animation using the Clawpack animation tools.

```

1 nsimul=np.size(claw.frames)
2 figs = []
3 for i in range(nsimul):
4     fig, ax = plt.subplots(figsize=(5, 3))
5
6     frame = claw.frames[i]
7     w = frame.q[0,:]
8     x = frame.state.grid.c_centers
9     x = x[0]
10    dt = claw.tfinal/claw.num_output_times
11    t = dt*i
12    true = qtrue(x,t)
13    ax.set_xlabel(r'$x$')
14    ax.set_ylabel(r'$q$')
15    plt.plot(x, w)
16    plt.plot(x, true, '--',color = 'r')
17    figs.append(fig)
18    plt.close(fig)
19
20 from clawpack.visclaw import animation_tools

```

```
21 animation_tools.interact_animate_figs(figs)
```

4.4.5 Alternative

We can also handle Eq. (4.12) as a non-homogeneous advection equation

$$\frac{\partial q}{\partial t} + u(x) \frac{\partial q}{\partial x} = -q \frac{du}{dx}.$$

```
1 %matplotlib inline
2
3 import numpy as np
4 from clawpack import riemann
5 from clawpack import pyclaw
6
7 def qtrue(x,t):
8     """
9     The true solution, for comparison.
10    """
11    from numpy import mod, exp, where, logical_and
12    beta = 100
13    gamma = 0
14    x0 = 0.75
15    if isinstance(x,(np.ndarray,list)):
16        m = np.size(x)
17        q = np.empty((m))
18        for i in range(m):
19            xm = (np.sqrt(x[i])-t)**2
20            if x[i] != 0.:
21                q[i] = qinit(xm)/np.sqrt(x[i])*np.sqrt(xm)
22            else:
23                q[i] = 0.
24    if isinstance(x,(float,int)):
25        if x != 0:
26            xm = (np.sqrt(x)-t)**2
27            q = qinit(xm)/np.sqrt(x)*np.sqrt(xm)
28        else:
29            q = 0
30    return q
31
32 def qinit(x):
33     """
34     The true solution, for comparison.
35    """
36    from numpy import mod, exp, where, logical_and
37    beta = 100
38    gamma = 0
39    x0 = 0.75
40    q = exp(-beta * (x-x0)**2) + where(logical_and(x > 0.1, x < 0.4), 1,
41    0)
42    q = where(logical_and(x > 0.1, x < 0.4), 1, 0)
43    return q
44
45 def advection(q_l,q_r,aux_l,aux_r,problem_data):
46     r"""
```

```

47 1d linear advection riemann solver
48 """
49 import numpy as np
50 num_eqn = 1
51 num_waves = 1
52
53 # Number of Riemann problems
54 num_rp = q_l.shape[1]
55
56 # Return values
57 wave = np.empty( (num_eqn, num_waves, num_rp) )
58 s = np.empty( (num_waves, num_rp) )
59 amdq = np.empty( (num_eqn, num_rp) )
60 apdq = np.empty( (num_eqn, num_rp) )
61
62 # Local values
63 delta = np.empty(np.shape(q_l))
64 delta = q_r - q_l
65 # Compute the wave
66 # 1-Wave
67 wave[0,0,:] = delta
68 s[0,:] = (aux_l[0,:]+aux_r[0,:])/2.
69 s_index = np.zeros((2,num_rp))
70 s_index[0,:] = s[0,:]
71 amdq[0,:] = np.min(s_index,axis=0) * wave[0,0,:]
72 apdq[0,:] = np.max(s_index,axis=0) * wave[0,0,:]
73
74 return wave, s, amdq, apdq
75
76 def source_term(solver, state, dt):
77     from scipy.linalg import solve_banded
78     import numpy as np
79     qs = state.q[0,:]
80     xc = state.grid.c_centers[0]
81     dq = -qs/np.sqrt(xc) *dt
82     state.q[0,:] = qs+dq
83
84 # Advection speed
85 def auxinit(state):
86     xc = state.grid.x.centers
87     state.aux[0,:] = 2*np.sqrt(xc)
88
89 def setup(outdir='./_output'):
90     from clawpack import riemann
91
92     solver = pyclaw.ClawSolver1D(advection)
93     solver.kernel_language = 'Python'
94     solver.num_waves = 1
95     solver.num_eqn = 1
96
97     solver.limiters = pyclaw.limiters.tvd.MC
98     solver.bc_lower[0] = pyclaw.BC.extrap
99     solver.bc_upper[0] = pyclaw.BC.extrap
100     solver.aux_bc_lower[0] = pyclaw.BC.extrap
101     solver.aux_bc_upper[0] = pyclaw.BC.extrap
102     solver.step_source = source_term
103
104     xlower = 0.0; xupper = 2.0; mx = 200

```



```

105 x      = pyclaw.Dimension(xlower,xupper,mx,name='x')
106 domain = pyclaw.Domain(x)
107 num_aux, num_eqn = 1, 1
108 state  = pyclaw.State(domain,num_eqn,num_aux)
109
110 xc = domain.grid.x.centers
111 state.q[0,:] = qinit(xc)
112 auxinit(state)
113
114 claw = pyclaw.Controller()
115 claw.outdir = outdir
116 claw.solution = pyclaw.Solution(state,domain)
117 claw.solver = solver
118
119 claw.tfinal = 1
120 claw.num_output_times = 20
121 claw.keep_copy = True
122
123 return claw
124

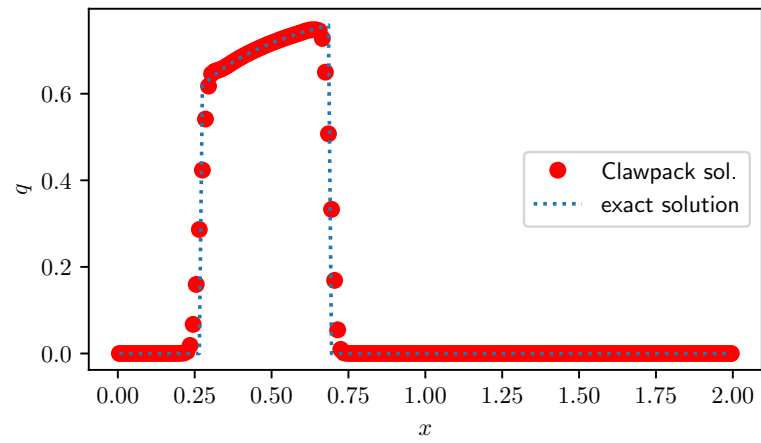
```

We run the code and plot the solution at time $t = 0.2$ s.

```

1 import matplotlib.pyplot as plt
2 claw = setup()
3 claw.run()
4 plt.rcParams['text.usetex'] = True
5 index = 4
6 frame = claw.frames[index]
7 dt = claw.tfinal/claw.num_output_times
8 t = dt*index
9 x = frame.state.grid.c_centers
10 x = x[0]
11
12 true = qtrue(x,t)
13 fig, ax = plt.subplots(figsize=(5, 2.7))
14 w = frame.q[0,:]
15
16 ax.plot(x, w, label='Clawpack sol.')
17 ax.plot(x, true, ':', label='exact solution')
18 ax.legend(loc='right')
19 ax.set_xlabel(r'$x$')
20 ax.set_ylabel(r'$q$')
21 plt.savefig("SpatiallyVaryingAdvectionWithSource.pdf",bbox_inches='tight')

```



Burgers' equations

5.1 Theory

Let us consider Burgers' equation, which is a nonlinear advection equation:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0, \quad (5.1)$$

or in conservative form

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0 \text{ where } f(u) = \frac{u^2}{2}.$$

The solution to the Riemann problem has two types of solution:

- *Rarefaction wave*: $u = U(\xi)$ with $\xi = x/t$. Substituting this form into Eq. (5.1) gives

$$\frac{dU}{d\xi} \left(-\xi + \frac{df}{dU}(U) \right) = 0 \text{ where } f'(U) = U$$

whose solution is

$$U(\xi) = \xi.$$

- *Shock wave*: The Rankine-Hugoniot equation tells us that the shock moves at speed:

$$\dot{\sigma} = \frac{[[f(u)]]}{[[u]]} = \frac{u^+ + u^-}{2},$$

where u^+ and u^- are the u value on the shock wave's right and left sides.

The solution to the Riemann problem

$$u(x, 0) = \begin{cases} u_l & \text{if } x < 0, \\ u_r & \text{if } x > 0. \end{cases} \quad (5.2)$$

depends on the sign of $u_r - u_l$, where u_r and u_l are the right and left initial states:

- If $u_r > u_l$, we have a rarefaction wave separating the two initial states. The characteristic curves separating $U(\xi)$ from u_l and u_r are, respectively, $x = u_l t$ and $x = u_r t$.
- If $u_r < u_l$, we have a shock wave moving at the speed:

$$\dot{\sigma} = \frac{1}{2}(u_r + u_l).$$

5.2 Approximate solvers

We have seen in § 3.4.1 Clawpack, the Riemann solver can be expressed in terms of fluctuations:

$$Q_i^{n+1} = Q_i^n - \frac{\Delta t}{\Delta x} (\mathcal{A}^+ \Delta Q_{i-1/2} + \mathcal{A}^- \Delta Q_{i+1/2}),$$

where the fluctuations are

$$\begin{aligned}\mathcal{A}^+ \Delta Q_{i-1/2} &= f(Q_i) - f(Q_{i-1/2}^\downarrow), \\ \mathcal{A}^- \Delta Q_{i+1/2} &= f(Q_{i+1/2}^\downarrow) - f(Q_i),\end{aligned}$$

where $Q_{i\pm 1/2}$ represents the value advected along the characteristic coming from the $x_{i\pm 1/2}$ interface and $Q_{i+1/2}^\downarrow = q^\downarrow(Q_i, Q_{i-1})$ is the q value along the ray $x = x_{i-1/2}$.

There are five possible wave configurations (see Fig. 3.6): left- and right-going shock waves, left- and right-going rarefaction waves, and transonic rarefaction wave. When the solution to the Riemann problem is not a transonic wave, the idea is to approximate this solution as a shock wave even though it is a rarefaction wave. The shock $\mathcal{W}$ propagates at a speed s :

$$\begin{aligned}\mathcal{W}_{i-1/2} &= Q_i - Q_{i-1}, \\ s_{i-1/2} &= \frac{f(Q_i) - f(Q_{i-1})}{Q_i - Q_{i-1}},\end{aligned}$$

for $Q_i \neq Q_{i-1}$. We then deduce

$$\begin{aligned}\mathcal{A}^+ \Delta Q_{i-1/2} &= s_{i-1/2} \mathcal{W}_{i-1/2}, \\ \mathcal{A}^- \Delta Q_{i-1/2} &= s_{i-1/2} \mathcal{W}_{i-1/2},\end{aligned}$$

When the solution to the Riemann problem is a transonic wave, we use the definition of the fluctuations

$$\begin{aligned}\mathcal{A}^+ \Delta Q_{i-1/2} &= f(Q_i) - f(q_s), \\ \mathcal{A}^- \Delta Q_{i-1/2} &= f(q_s) - f(Q_{i-1}),\end{aligned}$$

where q_s is the value such as $f'(q_s) = 0$ (vertical characteristic corresponding to $x - x_{i-1/2} = 0 \cdot t$). For the Burgers equation we have $q_s = 0$.

To summarize, we express the functions `amdp`, `apdp`, `s`, and `W` in Clawpack:

$$\mathcal{A}^+ \Delta U_{i-1/2} = s_{i-1/2} \mathcal{W}_{i-1/2}, \quad (5.3)$$

$$\mathcal{A}^- \Delta U_{i-1/2} = s_{i-1/2} \mathcal{W}_{i-1/2}, \quad (5.4)$$

with

$$\mathcal{W}_{i-1/2} = U_i - U_{i-1}, \quad (5.5)$$

$$s_{i-1/2} = \frac{1}{2}(U_i + U_{i-1}), \quad (5.6)$$

but if $U_{i-1} < 0$ and $U_i > 0$ then

$$\mathcal{A}^+ \Delta U_{i-1/2} = \frac{1}{2} U_i^2, \quad (5.7)$$

$$\mathcal{A}^- \Delta U_{i-1/2} = -\frac{1}{2} U_{i-1}^2, \quad (5.8)$$

In Clawpack, the treatment of the transonic wave is called an *entropy fix*, and its use in the Riemann solver is indicated through the Boolean variable `efix`. Note that in the present case, the Roe and HLL solvers provide the same approximate solution as above. Indeed, linearising the Burgers equation (5.1) yields:

$$\frac{\partial q}{\partial t} + \hat{q} \frac{\partial q}{\partial x} = 0, \quad (5.9)$$

where the advection velocity $\hat{q}$ is given by

$$\hat{q} = \frac{1}{2}(q_r + q_l)$$

so that the linearised Burgers solution provides the exact solution when the initial states u_r and u_l lie on the Hugoniot locus. For the HLL solver, we need to determine two waves, but the solution to one-dimensional equations such as the Burgers equation involves a single wave. We thus assume that one of the waves vanishes, while the other moves at speed s given by

$$f(q_r) - f(q_l) = s(q_r - q_l) \Rightarrow s = \frac{1}{2}(q_r + q_l).$$

For the same reason, the f-wave solver will provide the same result.

5.2.1 Implementation in Clawpack

The Riemann solver is just the transcript of Eqs. (5.3) to (5.8).

```

1 C
2 C
3     efix = .true.    !# Compute correct flux for transonic rarefactions
4 C
5     do 30 i=2-mbc,mx+mbc
6 C
7         # Compute the wave and speed
8 C
9         wave(i,1,1) = ql(i,1) - qr(i-1,1)
10        s(i,1) = 0.5d0 * (qr(i-1,1) + ql(i,1))
11 C
12 C
13        # compute left-going and right-going flux differences:
14 C
15 C
16        amdq(i,1) = dmin1(s(i,1), 0.d0) * wave(i,1,1)
17        apdq(i,1) = dmax1(s(i,1), 0.d0) * wave(i,1,1)
18 C
19        if (efix) then
20 C
21            # entropy fix for transonic rarefactions:
22            if (qr(i-1,1).lt.0.d0 .and. ql(i,1).gt.0.d0) then
23                amdq(i,1) = - 0.5d0 * qr(i-1,1)**2
24                apdq(i,1) =  0.5d0 * ql(i,1)**2
25            endif
26        endif
30    continue

```

5.2.2 Implementation in Pyclaw

The Riemann solver is just the transcript of Eqs. (5.3) to (5.8).

```

1 def burgers_1D(q_l,q_r,aux_l,aux_r,problem_data):
2     r"""
3     Riemann solver for Burgers equation in 1d
4     *problem_data* should contain -
5     - *efix* - (bool) Whether a entropy fix should be used, if not present
6     ,
7     false is assumed
8     """
9
10    num_rp = q_l.shape[1]
11    # Output arrays
12    wave = np.empty( (num_eqn, num_waves, num_rp) )
13    s = np.empty( (num_waves, num_rp) )
14    amdq = np.empty( (num_eqn, num_rp) )
15    apdq = np.empty( (num_eqn, num_rp) )
16
17    # Basic solve
18    wave[0,:,:] = q_r - q_l
19    s[0,:] = 0.5 * (q_r[0,:] + q_l[0,:])
20
21    s_index = np.zeros((2,num_rp))
22    s_index[0,:] = s[0,:]
23    amdq[0,:] = np.min(s_index,axis=0) * wave[0,0,:]
24    apdq[0,:] = np.max(s_index,axis=0) * wave[0,0,:]
25
26    # Compute entropy fix
27    if problem_data['efix']:
28        transonic = (q_l[0,:] < 0.0) * (q_r[0,:] > 0.0)
29        amdq[0,transonic] = -0.5 * q_l[0,transonic]**2
30        apdq[0,transonic] = 0.5 * q_r[0,transonic]**2
31
32    return wave, s, amdq, apdq

```

In this routine, `q_l` is the left initial condition. It is a $m \times N$ array ($m = 1$ the problem dimension, and N the number of cells). So, `num_rp = q_l.shape[1]` gives N . First, the `amdq`, `apdq`, `s`, and `W` are initialised, then `s` and `W` are defined. Finally, the fluctuations are defined using the numpy function `numpy.min`, which provides the minimum value: absolute, for each column (with the `axis=0` option), or for each row (with the `axis=1` option). For instance, the lines

```

32 import numpy as np
33 x=np.array( [[1,4], [2,5], [3,-2]] )
34 np.min(x,axis=None)
35 np.min(x,axis=0)
36 np.min(x,axis=1)

```

provide the values: -2, `array([ 1, -2])` and `array([ 1, 2, -2])`, respectively.

5.3 Examples

5.3.1 Solution to the Riemann problem

Here we solve the Riemann problem and compare the numerical and exact solutions (the exact solution is outlined in § 5.1). The code is initialised as follows:

```

1 %matplotlib inline

```

```

2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 from numpy import sqrt, log
6 from clawpack import riemann
7 from clawpack import pyclaw
8
9 def qsol(x,t,ql,qr):
10     """
11     The initial and true solution.
12     """
13     import numpy as np
14     dim = x.shape[0]
15     q = np.empty(dim)
16
17     if qr < ql:
18         s = (qr+ql)/2
19         for i in range(dim):
20             if x[i]>=s*t:
21                 q[i] = qr
22             else:
23                 q[i] = ql
24     else:
25         if t > 0:
26             xr = qr*t
27             xl = ql*t
28             for i in range(dim):
29                 if x[i]>=xl and x[i]<=xr:
30                     q[i] = x[i]/t
31                 elif x[i]>xr:
32                     q[i] = qr
33                 elif x[i]<xl:
34                     q[i] = ql
35             else:
36                 for i in range(dim):
37                     if x[i]>=0:
38                         q[i] = qr
39                     else:
40                         q[i] = ql
41     return q
42
43 def setup(ql,qr):
44
45     solver = pyclaw.ClawSolver1D()
46     solver.rp = riemann.burgers_1D_py.burgers_1D
47     solver.num_waves = 1
48     solver.num_eqn = 1
49     solver.kernel_language = 'Python'
50     solver.limiters = pyclaw.limiters.tvd.superbee
51     solver.bc_lower[0] = pyclaw.BC.extrap
52     solver.bc_upper[0] = pyclaw.BC.extrap
53     solver.order = 2 #1: Godunov, 2: Lax-Wendroff-LeVeque
54
55     x = pyclaw.Dimension(-1, 1., 200, name='x')
56     domain = pyclaw.Domain(x)
57     num_eqn = 1
58
59     state = pyclaw.State(domain, num_eqn)

```



```

60 state.problem_data['efix'] = True
61
62 xc = domain.grid.x.centers
63
64 state.q[0, :] = qsol(xc, 0, ql, qr)
65
66 claw = pyclaw.Controller()
67 claw.solution = pyclaw.Solution(state, domain)
68 claw.solver = solver
69 claw.outdir = './_output'
70 claw.output_style = 1
71 claw.tfinal = 2.0
72 claw.num_output_times = 20
73 claw.keep_copy = True
74 return claw

```

We run the code for different initial conditions.

```

1 claw = setup(2, 0)
2 claw.run()
3
4 plt.rcParams['text.usetex'] = True
5 index = 3
6 frame = claw.frames[index]
7 dt = claw.tfinal/claw.num_output_times
8 t = dt*index
9 x = frame.state.grid.c_centers
10 x = x[0]
11 true = qsol(x, t, 2, 0)
12 fig, ax = plt.subplots(figsize=(3, 3))
13 w = frame.q[0, :]
14
15 ax.plot(x, w, label='Clawpack sol.')
16 ax.plot(x, true, ':', label='exact solution')
17 ax.legend(loc='lower left')
18 ax.set_xlabel(r'$x$')
19 ax.set_ylabel(r'$q$')
20 plt.text(-1, 1.5, '(a)')
21 plt.savefig("ShockBurgerClaw.pdf", bbox_inches='tight')

```

Figure 5.1 shows the numerical and solutions at time $t = 0.3$ s. We considered the case: (a) shock wave with $(u_l = 2, u_r = 0)$; (a) rarefaction wave with $(u_l = 0, u_r = 2)$; (a) transonic wave with $(u_l = -1, u_r = 1)$.

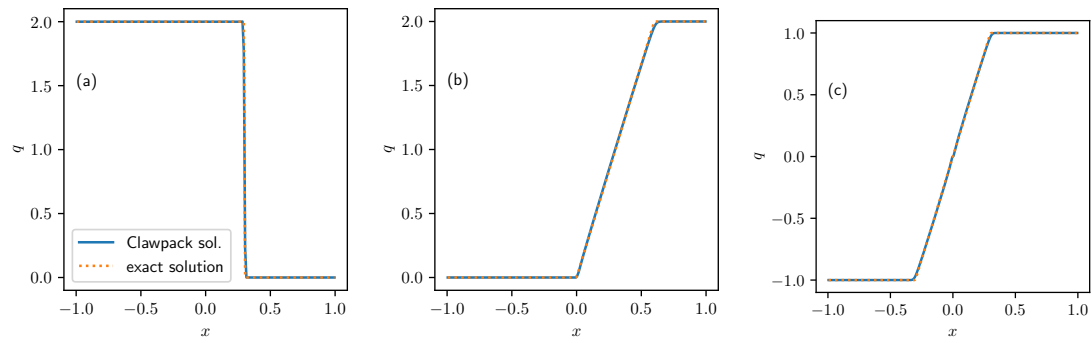


Figure 5.1 Solutions to the Riemann problem: (a) shock wave; rarefaction wave (b) ; (c) transonic rarefaction wave. Lax–Wendroff–LeVeque solver with $\Delta x = 10^{-2}$ m.

5.3.2 Solution to an initial value problem

Let us now consider a more complicated initial-value problem:

$$u(x, 0) = u_0(x) = \begin{cases} 0 & \text{if } x < 0 \text{ or } x > b, \\ ax & \text{if } 0 \leq x \leq b, \end{cases}$$

where a and b are two constants. Using the method of characteristics, we can cast Eq. (5.1) in the form:

$$\frac{du}{dt} = 0 \text{ along } \frac{dx}{dt} = u, \quad (5.10)$$

which shows that the characteristics are straight lines in the form

$$x = x_0 + ut,$$

and since $u(x, t)$ is constant along this curve and equal to $u(x, 0) = u_0(x_0)$, we deduce that $u(x, t)$ is the solution to

$$u_0(x_0) = a(x - ut) = u \Rightarrow u = \frac{ax}{at + 1}.$$

The initial condition has a finite support with an initial discontinuity at $x = b$. Since the right state is lower than the left state, this initial discontinuity will propagate as a shock wave $x = s(t)$, whose velocity is given by the Rankine–Hugoniot equation:

$$\dot{s} = \frac{[f(u)]}{[u]} = \frac{1}{2} \frac{u^2(s(t), t)}{u(s(t), t)} = \frac{1}{2} u(s(t), t),$$

subject to $s(0) = b$. The solution is

$$s(t) = b\sqrt{1 + at}.$$

The exact solution to Eq. (5.1) is thus

$$u(x, t) = \begin{cases} 0 & \text{if } x < 0 \text{ or } x > s(t), \\ \frac{ax}{at + 1} & \text{if } 0 \leq x \leq s(t) \end{cases} \quad (5.11)$$

We initialise the code with $a = 1$ and $b = 1/2$.

```

1 %matplotlib inline
2
3 from numpy import sqrt, log
4 from clawpack import riemann
5 from clawpack import pyclaw
6
7 def xfr(t): return sqrt(1+t)/2
8
9 def qtrue(x,t):
10     """
11     The true solution, for comparison.
12     """
13     import numpy as np
14     dim = x.shape[0]
15     q = np.empty(dim)
16
17     xf = xfr(t)
18     for i in range(dim):
19         if x[i]>=0.0 and x[i]<=xf:
20             q[i] = (x[i])/(t+1)
21         else:

```

```

22         q[i] = 0
23     return q
24
25 def burgers(q_l,q_r,aux_l,aux_r,problem_data):
26     r"""
27     1d burgers riemann solver
28     """
29     import numpy as np
30     num_eqn = 1
31     num_waves = 1
32
33     # Convenience
34     num_rp = q_l.shape[1]
35
36     # Return values
37     wave = np.empty( (num_eqn, num_waves, num_rp) )
38     s = np.empty( (num_waves, num_rp) )
39     amdq = np.empty( (num_eqn, num_rp) )
40     apdq = np.empty( (num_eqn, num_rp) )
41
42     # Local values
43     delta = np.empty(np.shape(q_l))
44     delta = q_r - q_l
45
46     # Compute the wave
47     # 1-Wave
48     wave[0,0,:] = delta
49     s[0,:] = 0.5 * (q_r[0,:] + q_l[0,:])
50
51     # Compute the left going and right going fluctuations
52     s_index = np.zeros((2,num_rp))
53     s_index[0,:] = s[0,:]
54     amdq[0,:] = np.min(s_index,axis=0) * wave[0,0,:]
55     apdq[0,:] = np.max(s_index,axis=0) * wave[0,0,:]
56
57     # Compute entropy fix
58     if problem_data['efix']:
59         transonic = (q_l[0,:] < 0.0) * (q_r[0,:] > 0.0)
60         amdq[0,transonic] = -0.5 * q_l[0,transonic]**2
61         apdq[0,transonic] = 0.5 * q_r[0,transonic]**2
62
63     return wave, s, amdq, apdq
64
65
66 def setup(outdir='./_output', output_style=1):
67
68     solver = pyclaw.ClawSolver1D()
69     solver.rp = burgers
70
71     solver.num_waves = 1
72     solver.num_eqn = 1
73     solver.kernel_language = 'Python'
74     solver.limiters = pyclaw.limiters.tvd.superbee
75     solver.bc_lower[0] = pyclaw.BC.extrap
76     solver.bc_upper[0] = pyclaw.BC.extrap
77     solver.order = 2 #1: Godunov, 2: Lax-Wendroff-LeVeque
78
79     x = pyclaw.Dimension(-0.10, 1.5, 320, name='x')

```

```

80 domain = pyclaw.Domain(x)
81 xc = domain.grid.x.centers
82 num_eqn = 1
83
84 state = pyclaw.State(domain, num_eqn)
85 state.problem_data['efix'] = True
86 state.q[0, :] = qtrue(xc, 0)
87
88 claw = pyclaw.Controller()
89 claw.solution = pyclaw.Solution(state, domain)
90 claw.solver = solver
91 claw.outdir = outdir
92 claw.output_style = output_style
93 claw.tfinal = 2.0
94 claw.num_output_times = 20
95 claw.keep_copy = True
96
97 return claw

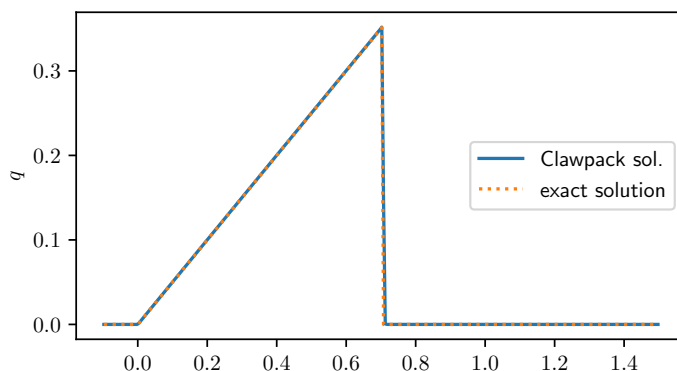
```

We plot the numerical solution at time $t = 1$ s.

```

1 claw = setup()
2 claw.run()
3 %matplotlib inline
4 import numpy as np
5 import matplotlib.pyplot as plt
6 plt.rcParams['text.usetex'] = True
7 index = 10
8 frame = claw.frames[index]
9 dt = claw.tfinal/claw.num_output_times
10 t = dt*index
11 x = frame.state.grid.c_centers
12 x = x[0]
13 true = qtrue(x,t)
14 fig, ax = plt.subplots(figsize=(5, 2.7))
15 w = frame.q[0,:]
16
17 ax.plot(x, w, label='Clawpack sol.')
18 ax.plot(x, true, ':', label='exact solution')
19 ax.legend(loc='right')
20 ax.set_xlabel(r'$x$')
21 ax.set_ylabel(r'$q$')
22 plt.savefig("InitialValueBurgerClaw.pdf")

```



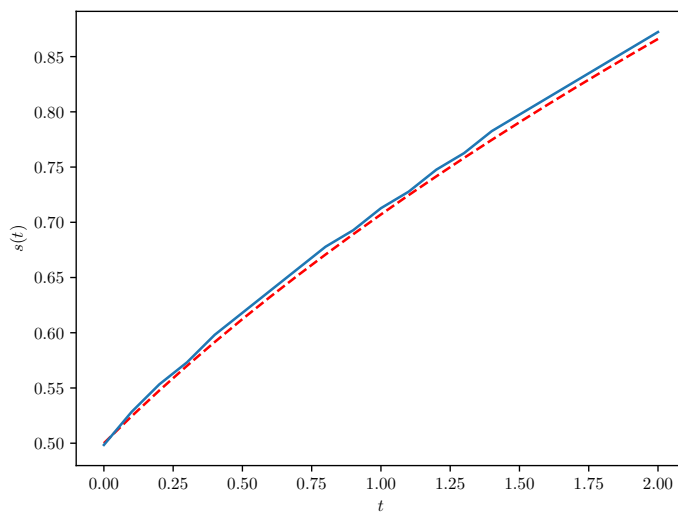
We can also determine the front position $x = s(t)$ in the numerical solution and compare it with

the exact solution. We compute the shock position for a set of discrete times for which the numerical solution was recorded. A more accurate representation of $s(t)$ could have been obtained by using the function `b4step` (see the example of § 5.5).

```

1 nsimul=np.size(claw.frames)
2 positionFront = []
3 for i in range(nsimul):
4     frame = claw.frames[i]
5     w = frame.q[0,:]
6     xc=frame.state.grid.c_centers[0]
7     nx=xc.size
8     dx=(xc[-1]-xc[0])/nx
9     pos= dx*w[w>1.e-5].size
10    positionFront.append(pos)
11
12 t = np.linspace(0, claw.tfinal, nsimul)
13 xfront = xfr(t)
14
15 fig, ax = plt.subplots()
16 ax.set_xlabel(r'$t$')
17 ax.set_ylabel(r'$s(t)$')
18
19 ax.plot(t, xfront, '--',color = 'r')
20 ax.plot(t, positionFront)
21 plt.savefig("frontBurgerClaw.pdf")

```



5.4 Nonlinear advection equation with a diffusive term

5.4.1 Theoretical considerations

Let us consider the Burgers equation with a diffusive term representing viscous effect:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad (5.12)$$

where ν is the dynamic viscosity. This equation is subject to an initial condition:

$$u(x, 0) = u_0(x), \quad (5.13)$$

and the following boundary conditions:

$$\lim_{x \rightarrow \pm\infty} \partial_x u(x, t) = 0. \quad (5.14)$$

General solutions

Exact solutions can be worked out using the Cole–Hopf transformation. To that end, let us first define the primitive of $u(x, t)$ with respect to x and denoted by $U(x, t)$. We have $u = \partial_x U$. Substituting this new variable into Eq. (5.12), integrating it and taking the boundary conditions (5.14) leads to

$$\frac{\partial U}{\partial t} + \frac{1}{2} \left(\frac{\partial U}{\partial x} \right)^2 = \nu \frac{\partial^2 U}{\partial x^2}, \quad (5.15)$$

subject to $U(x, 0) = U_0(x)$ where U_0 is the primitive of U_0 . We now use the new variable ϕ in the Cole–Hopf transformation:

$$U(x, t) = -2\nu \ln \phi(x, t)$$

Substituting this form into Eq. (5.15) gives the simple linear diffusion equation

$$\frac{\partial \phi}{\partial t} = \nu \frac{\partial^2 \phi}{\partial x^2}, \quad (5.16)$$

subject to

$$\phi(x, 0) = \exp \left(-\frac{1}{2\nu} U_0(x) \right).$$

The general solution of Eq. (5.16) is the convolution of the initial value $\phi(x, 0)$ with the kernel function $K(x, t)$

$$\phi(x, t) = \int_{-\infty}^{+\infty} K(x - y, t) \phi(y, 0) dy \text{ where } K(y, t) = \frac{1}{\sqrt{4\pi\nu t}} \exp \left(-\frac{y^2}{4\nu t} \right) \quad (5.17)$$

The solution to Eq. (5.15) is thus

$$U(x, t) = -2\nu \ln \int_{-\infty}^{+\infty} \frac{1}{\sqrt{4\pi\nu t}} \exp \left(-\frac{(x - y)^2}{4\nu t} - \frac{1}{2\nu} U_0(y) \right) dy. \quad (5.18)$$

By differentiating this equation with respect to x , we obtain the general solution $u(x, t)$

$$u(x, t) = \frac{\partial U}{\partial x} = \frac{\int_{-\infty}^{+\infty} \frac{x - y}{t} \exp \left(-\frac{(x - y)^2}{4\nu t} - \frac{1}{2\nu} U_0(y) \right) dy}{\int_{-\infty}^{+\infty} \exp \left(-\frac{(x - y)^2}{4\nu t} - \frac{1}{2\nu} U_0(y) \right) dy}. \quad (5.19)$$

It is difficult to go further with this equation, but it can be evaluated numerically and provide an approximation to the exact solution.

Travelling-wave solutions

It is possible to work out exact solutions to the viscous Burgers equation (5.12) by noting that this equation is invariant to the translation group $x \rightarrow x + s\lambda$ and $t \rightarrow t + \lambda$, which indicates that there must be a travelling-wave solution in the form:

$$u(x, t) = U(\xi) \text{ where } \xi = x - st.$$

Substituting this form into the governing equation (5.12) leads to the principal differential equation

$$-s \frac{dU}{d\xi} + U \frac{dU}{d\xi} = \nu \frac{d^2U}{d\xi^2}, \quad (5.20)$$

which can be integrated to give

$$-sU + \frac{1}{2}U^2 = \nu \frac{dU}{d\xi} + c, \quad (5.21)$$

where c is constant of integration. We assume that the governing equation (5.12) is supplemented by the boundary conditions

$$\lim_{x \rightarrow -\infty} u(x, t) = u_l \text{ and } \lim_{x \rightarrow +\infty} u(x, t) = u_r.$$

Using these boundary conditions shows that the constant of integration satisfies the equations

$$c - su_r + \frac{1}{2}u_r^2 = c - su_l + \frac{1}{2}u_l^2,$$

which is possible only if

$$s = \frac{u_l + u_r}{2},$$

and thus

$$c = -\frac{u_l u_r}{2}.$$

We can recast Eq. (5.21)

$$\begin{aligned} \frac{1}{2\nu} &= \frac{dU}{U^2 - 2sU - c}, \\ &= \frac{dU}{(U - s)^2 - b^2} \text{ where } b^2 = s^2 + c = \left(\frac{u_l - u_r}{2}\right)^2, \end{aligned}$$

whose integral is

$$\operatorname{arctanh} \frac{s - u}{b} = |b| \frac{\xi}{2\nu} \Rightarrow u = s - |b| \tanh \left(b \frac{\xi}{2\nu} \right) + a,$$

where a is a constant of integration found to be zero. The travelling-wave solution is therefore:

$$u = \frac{u_l + u_r}{2} - \left| \frac{u_l - u_r}{2} \right| \tanh \left(\frac{u_l - u_r}{4\nu} \left(x - \frac{u_l + u_r}{2} t \right) \right). \quad (5.22)$$

5.4.2 Numerical implementation in Clawpack

The scripts for solving the Burgers equation (5.12) are the same as those for solving the inviscid case in § 5.2.1 except for the source term, where we implement the Crank–Nicolson method (see § 3.9.3):

$$Q_i^{n+1} = Q_i^* + \nu \frac{\Delta t}{2} \left(\frac{Q_{i-1}^{n+1} - 2Q_i^{n+1} + Q_{i+1}^{n+1}}{\Delta x^2} + \frac{Q_{i-1}^* - 2Q_i^* + Q_{i+1}^*}{\Delta x^2} \right). \quad (5.23)$$

We assume a no-flux condition at the domain boundaries, and thus $Q_1^{n+1} = Q_1^n$ and $Q_m^{n+1} = Q_m^n$. Let us define the ratio r

$$r = \nu \frac{\Delta t}{2\Delta x^2}.$$

We have to solve the linear system

$$\begin{bmatrix} 1+r & -r & 0 & \dots & 0 \\ -r & \ddots & & & \\ \dots & -r & 1+2r & -r & \dots \\ & & & \ddots & \\ 0 & \dots & 0 & -r & 1+r \end{bmatrix} \cdot \begin{bmatrix} Q_1^{n+1} \\ Q_2^{n+1} \\ \vdots \\ Q_i^{n+1} \\ \vdots \\ Q_m^{n+1} \end{bmatrix} = \begin{bmatrix} (1-r)Q_1^n + rQ_2^n \\ Q_1^n + (1-2r)Q_2^n + rQ_3^n \\ \vdots \\ Q_{i-1}^n + (1-2r)Q_i^n + rQ_{i+1}^n \\ \vdots \\ (1-r)Q_m^n + rQ_{m-1}^n \end{bmatrix}$$

The file `src1.f` solves this system. It calls the LAPACK routine `dgtsv` for inverting tridiagonal matrices (included in the file `tridiag.f`). Makefile must be updated by adding `src1.f` and `tridiag.f`.

```

1 C =====
2       subroutine src1(meqn,mbc,mx,xlower,dx,q,maux,aux,t,dt)
3 C =====
4       implicit double precision (a-h,o-z)
5       dimension q(meqn,1-mbc:mx+mbc)
6
7 C       # solve the diffusion equation q_t = q_{xx} using Crank-Nicolson
8 C       # The LAPACK tridiagonal solver dgtsv is used, which is in tridiag.f
9 C       # local storage:
10      parameter (maxldb = 5000)
11      dimension b(maxldb,1), d(maxldb), dl(maxldb), du(maxldb)
12      common /comsrc/ dcoef
13
14      if (mx .gt. maxldb) then
15          write(6,*) 'ERROR: increase maxldb in src1.f'
16      endif
17      ldb = maxldb
18      nrhs = 1
19      dtdx2 = dcoef * dt / (2.d0*dx*dx)
20 C       # set coefficients in tridiagonal matrix and RHS:
21      do i=1,mx
22          dl(i) = -dtdx2
23          d(i) = 1.d0 + 2.d0*dtdx2
24          du(i) = -dtdx2
25          b(i,1) = q(1,i) + dtdx2 * (q(1,i-1) - 2.d0*q(1,i) + q(1,i+1))
26      enddo
27 C       # no-flux boundary conditions for diffusion step:
28 C       # Adjust matrix entries to use q(1,0)=q(1,1) and q(1,mx+1)=q(1,mx)
29 C       # at end of time step:
30      d(1) = d(1) - dtdx2
31      d(mx) = d(mx) - dtdx2
32 C       # solve the tridiagonal system:
33      call dgtsv(mx,nrhs,dl,d,du,b,ldb,info)
34
35      if (info .ne. 0) then
36          write(6,*) 'ERROR in src1 from call to dgtsv... info = ',info
37          stop
38      endif
39
40      do i=1,mx

```

```

41      q(1,i) = b(i,1)
42  enddo
43  return
44  end

```

We first test the code to the unit box case. The initial condition is

$$u(x,0) = \begin{cases} 0 & \text{if } x < -\frac{1}{2}, \\ 1 & \text{if } -\frac{1}{2} \leq x \leq \frac{1}{2}, \\ 0 & \text{if } x > \frac{1}{2}. \end{cases}$$

The file `qinit.f` encodes this initial condition.

```

1 C =====
2      subroutine qinit(meqn,mbc,mx,xlower,dx,q,maux,aux)
3 C =====
4 C      # Set initial conditions for q.
5 C
6      implicit double precision (a-h,o-z)
7      dimension q(meqn,1-mbc:mx+mbc)
8      dimension aux(maux,1-mbc:mx+mbc)
9      common /comsrc/ dcoef
10 C
11      do 150 i=1,mx
12      xcell = xlower + (i-0.5d0)*dx
13 C      # unit box
14      q(1,i) = 1.d0
15      if (xcell .lt. -0.5d0) q(1,i) = 0.d0
16      if (xcell .gt. 0.5d0) q(1,i) = 0.d0
17 150 continue
18      return
19      end

```

Figure 5.2 compares the numerical solution with the exact solution computed using Eq. (5.19) (evaluated numerically).

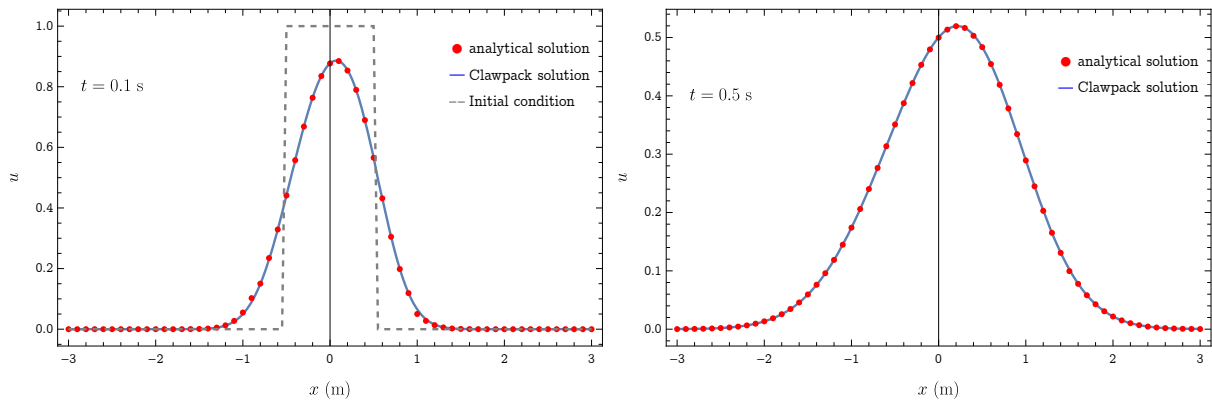


Figure 5.2 Snapshots of the solution at time $t = 0.1$ s and $t = 0.5$. Parameter $\nu = 0.5 \text{ m}^2/\text{s}$.

We now consider the travelling-wave case. The initial state is calculated using Eq. (5.22) evaluated at $t = 0$:

$$u(x,0) = \frac{u_l + u_r}{2} - \left| \frac{u_l - u_r}{2} \right| \tanh \left(\frac{u_l - u_r}{4\nu} x \right).$$

The file `qinit.f` initiates the travelling wave solution by using (5.22) at time $t = 0$.

```

1 C =====
2   subroutine qinit(meqn,mbc,mx,xlower,dx,q,maux,aux)
3 C =====
4 C   # Set initial conditions for q.
5 C
6   implicit double precision (a-h,o-z)
7   dimension q(meqn,1-mbc:mx+mbc)
8   dimension aux(maux,1-mbc:mx+mbc)
9   common /comsrc/ dcoef
10
11   do 150 i=1,mx
12     xcell = xlower + (i-0.5d0)*dx
13     ul = 0.d0
14     ur = 2.d0
15     q(1,i) = (ur+ul)/2-dabs((ul-ur))/2.d0*tanh(xcell*(ul-ur)/
16 &               4.d0/dcoef)
17 150 continue
18 C
19   return
20 end

```

Figure 5.3 compares the numerical solution with the exact solution computed using Eq. (5.22).

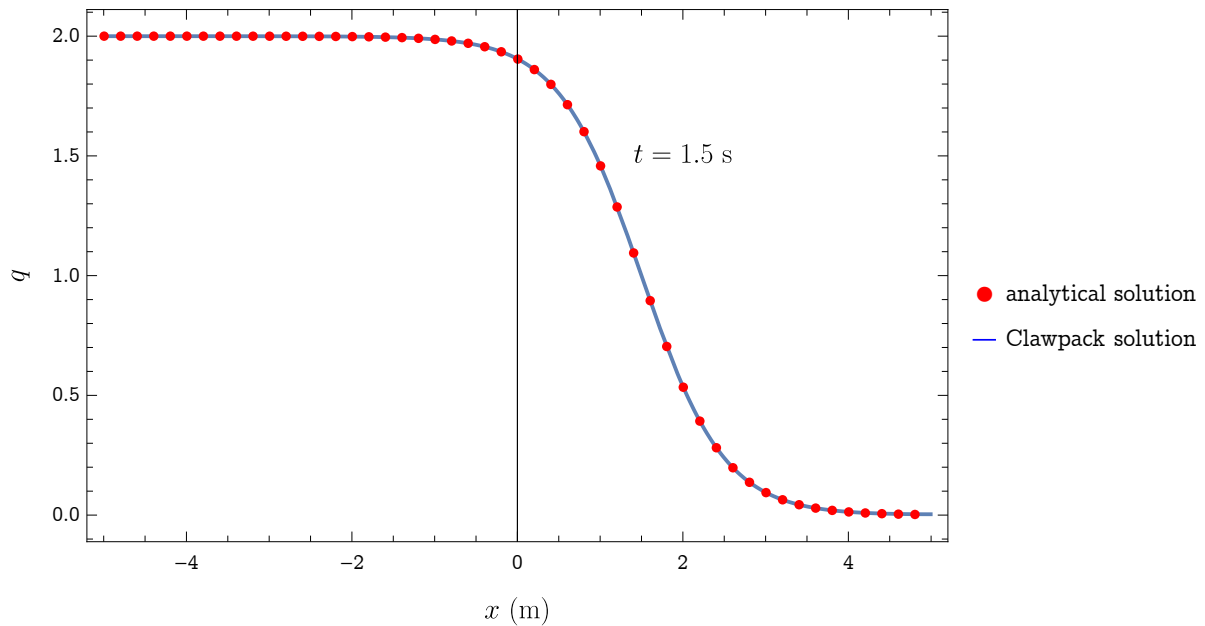


Figure 5.3 Snapshots of the solution at time $t = 0.1$ s and $t = 0.5$. Parameters $\nu = 0.5 \text{ m}^2/\text{s}$, $u_l = 2$, and $u_r = 0$. Lax–Wendroff scheme (limiter: superbee) with $\Delta x = 10^{-2} \text{ m}$.

5.4.3 Numerical implementation in Pyclaw

We implemented the Crank–Nicolson method in the previous Pyclaw script used for solving the inviscid case. For the application, we used the same parameters as above: $\nu = 0.5 \text{ m}^2/\text{s}$, $u_l = 2$, and $u_r = 0$. We use the built-in Scipy function `solve_banded`, which provides the numerical solution to $B \cdot q^{n+1} = F(q^n)$, where B is a band matrix.

```

1 %matplotlib inline
2
3 from numpy import sqrt, log, tanh
4 from clawpack import riemann
5 from clawpack import pyclaw
6
7 # Travelling wave solution
8 def travel(x,t,ul,ur,nu): return (ur+ul)/2-abs((ul-ur))/2*tanh((x-(ur+ul)
    /2*t)*(ul-ur)/4/nu )
9
10 def qtrue(x,t,ul,ur,nu):
11     """
12     The true solution, for comparison.
13     """
14     import numpy as np
15     dim = x.shape[0]
16     q = np.empty(dim)
17
18     for i in range(dim):
19         q[i] = travel(x[i],t,ul,ur,nu)
20     return q
21
22
23
24 def source_term(solver, state, dt):
25     from scipy.linalg import solve_banded
26     import numpy as np
27     qs = state.q[0,:]
28     xc = state.grid.c_centers[0]
29     nx = xc.size
30     dx = (xc[-1]-xc[0])/nx
31     nu = state.problem_data['nu']
32     r = nu*dt/(dx**2*2.)
33     m = qs.shape[0]
34     b = np.empty(m)
35     for i in range(m):
36         if i == 0:
37             b[0] = (1-r)*qs[0]+r*qs[1]
38         elif i == m-1:
39             b[m-1] = (1-r)*qs[m-1]+ r*qs[m-2]
40         else:
41             b[i] = r*qs[i-1]+(1-2*r)*qs[i]+r*qs[i+1]
42
43     ab = np.empty((3,m))
44     ab[0,1:] = [-r]*(m-1)
45     ab[1,:] = [1+2*r]*m
46     ab[2,-1] = [-r]*(m-1)
47     ab[1,0] = 1+r
48     ab[1,-1]=1+r
49     state.q[0,:] = solve_banded((1,1),ab,b)
50
51
52 def setup(outdir='./_output', output_style=1):
53
54     solver = pyclaw.ClawSolver1D()
55     solver.rp = riemann.burgers_1D_py.burgers_1D
56
57     solver.num_waves = 1

```

```

58 solver.num_eqn = 1
59 solver.kernel_language = 'Python'
60 solver.limiters = pyclaw.limiters.tvd.superbee
61 solver.bc_lower[0] = pyclaw.BC.extrap
62 solver.bc_upper[0] = pyclaw.BC.extrap
63 solver.order = 2 #1: Godunov, 2: Lax-Wendroff-LeVeque
64
65 solver.step_source = source_term # inclusion of source term
66
67 mx = 600
68 x = pyclaw.Dimension(-3, 3, mx, name='x')
69 domain = pyclaw.Domain(x)
70 num_eqn = 1
71
72 state = pyclaw.State(domain, num_eqn)
73
74 # Parameters
75 nu = 0.5
76 ul = 2
77 ur = 0
78 state.problem_data['efix'] = True
79 state.problem_data['nu'] = nu #diffusivity
80 state.problem_data['ul'] = ul #left state
81 state.problem_data['ur'] = ur #right state
82
83 xc = domain.grid.x.centers
84
85 #for i in range(mx):
86     #state.q[0,i] = travel(xc[i],0,ul,ur,nu) #initial condition
87
88 state.q[0,:] = travel(xc,0,ul,ur,nu)
89
90 claw = pyclaw.Controller()
91 claw.solution = pyclaw.Solution(state, domain)
92 claw.solver = solver
93 claw.outdir = outdir
94 claw.output_style = output_style
95 claw.tfinal = 2.0
96 claw.num_output_times = 20
97 claw.keep_copy = True
98
99 return claw

```

We ran the code and plotted the solution for time $t = 1$ s.

```

1 claw = setup()
2 claw.run()
3 %matplotlib inline
4 import numpy as np
5 import matplotlib.pyplot as plt
6 plt.rcParams['text.usetex'] = True
7 index = 10
8 frame = claw.frames[index]
9 dt = claw.tfinal/claw.num_output_times
10 t = dt*index
11 x = frame.state.grid.c_centers
12 x = x[0]
13
14 ul = frame.state.problem_data['ul']

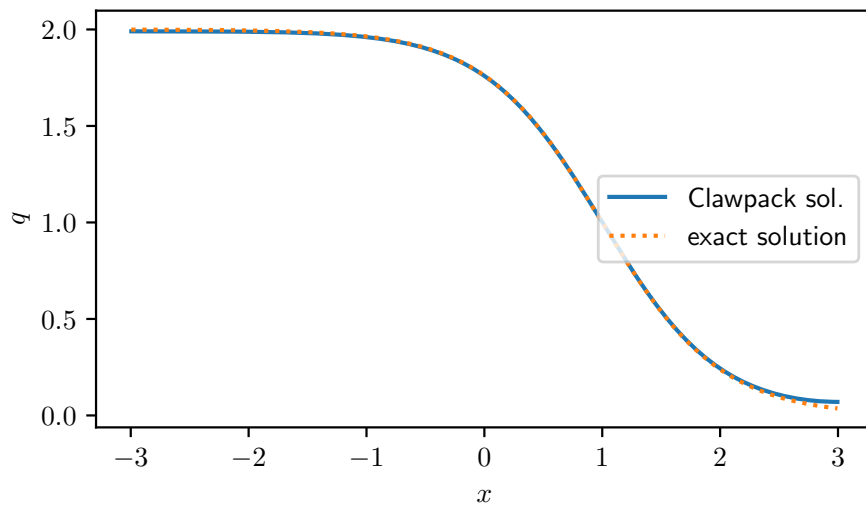
```

```

15 ur = frame.state.problem_data['ur']
16 nu = frame.state.problem_data['nu']
17 true = qtrue(x,t,ul,ur,nu)
18 fig, ax = plt.subplots(figsize=(5, 2.7))
19 w = frame.q[0,:]
20
21 ax.plot(x, w, label='Clawpack sol.')
22 ax.plot(x, true, ':', label='exact solution')
23 ax.legend(loc='right')
24 ax.set_xlabel(r'$x$')
25 ax.set_ylabel(r'$q$')
26 plt.savefig("ViscousBurgerClaw.pdf")

```

Figure 5.3 compares the numerical solution with the exact solution computed using Eq. (5.22) at time $t = 1$ s.



Here is an animation:

```

1 nsimul=np.size(claw.frames)
2 figs = []
3 for i in range(nsimul):
4     fig, ax = plt.subplots(figsize=(5, 3))
5
6     frame = claw.frames[i]
7     w = frame.q[0,:]
8     x = frame.state.grid.c_centers
9     x = x[0]
10    dt = claw.tfinal/claw.num_output_times
11    t = dt*i
12    true = qtrue(x,t,ul,ur,nu)
13    ax.set_xlabel(r'$x$')
14    ax.set_ylabel(r'$q$')
15    plt.plot(x, w)
16    plt.plot(x, true, '--', color = 'r')
17    figs.append(fig)
18    plt.close(fig)
19
20
21 from clawpack.visclaw import animation_tools
22 animation_tools.interact_animate_figs(figs)

```

We can export an animation of the solution at different times and compare with the exact solution using `FuncAnimation` from the library `matplotlib.animation`.

```

1  def q_true(t,ul,ur,nu):
2      import numpy
3      # True Solution
4      x_true = numpy.linspace(-3.0, 3.0, 1000)
5      q_true = travel(x_true,t,ul,ur,nu)
6      return q_true
7
8  def burgers_animation(ul=2, ur=0, nu=0.5):
9      import matplotlib.animation
10     import numpy
11     # compute the solution with the method define above:
12     claw = setup()
13     claw.keep_copy = True
14     claw.run()
15     x = claw.frames[0].grid.dimensions[0].centers
16     x_true = numpy.linspace(-3.0, 3.0, 1000)
17
18     fig = plt.figure()
19     axes = plt.subplot(1, 1, 1)
20     axes.set_xlim((x[0], x[-1]))
21     axes.set_ylim((-0.1, 2))
22     axes.set_title("Viscous burgers equation")
23
24     def init():
25         axes.set_xlim((x[0], x[-1]))
26         axes.set_ylim((-0.1,2.1))
27         computed_line, = axes.plot(x[0], claw.frames[0].q[0, :][0], 'ro')
28         exact_line, = axes.plot(x_true[0], q_true(0.0,ul,ur,nu)[0], 'k')
29         return (computed_line, exact_line)
30
31     computed_line, exact_line = init()
32
33     def fplot(n):
34         computed_line.set_data([x,], [claw.frames[n].q[0, :]])
35         exact_line.set_data([x_true], [q_true(claw.frames[n].t,ul,ur,nu)])
36         return (computed_line, exact_line)
37
38     frames_to_plot = range(0, len(claw.frames))
39     plt.close(fig)
40     return matplotlib.animation.FuncAnimation(fig, fplot, frames=
frames_to_plot, interval=100,
41                                             blit=True, init_func=init, repeat=False)
42
43 from IPython.display import HTML
44 anim = burgers_animation()
45 HTML(anim.to_jshtml())

```



We can export the animation using `save` as a video using `save`:

```
1 anim.save('ViscousBurgers.mp4', fps=5, writer="ffmpeg", dpi=300)
```

It may be useful to export a series of images instead of a video. This can be achieved by different means. A possibility involves the `imagemagick` library:

```
1 anim.save('ViscousBurgers.png', writer="imagemagick", dpi=300)
```

In the command prompt, we can ask `imagemagick` to decompose the animation into a series of images

```
1 convert -coalesce ViscousBurgers.mp4 ViscousBurgers_.png
```

It may be more convenient to export directly the figures as pdf or png files:

```
1 for i in range(len(figs)):
2     figs[i].savefig('ViscousBurgersFrame'+str(i)+'.png', bbox_inches='tight',
    ' ', dpi=300)
```

This series of images can be inserted in a pdf file and produce animations, for instance by using the `animate` and `tikz latex` libraries.

```
1 \animategraphics[autoplay, loop, width=8cm, controls={play, step, stop,
    speed}, buttons size=0.3cm]{5}{ViscousBurgersFrame}{1}{20}
```


5.5 Nonlinear advection equation with a source term and boundary conditions

5.5.1 Theoretical considerations

Let us consider a rainfall of intensity I over a sloping bed inclined at α (see Fig. 5.4). There are two possible runoff mechanisms: superficial or hyporheic flow. For both cases, we assume that the flow depth is $h(x, t)$ and velocity $u(x, t)$ related to h : $u = ah^b$, where a and b are two coefficients: $a = C\sqrt{\alpha}$ et $b = 1/2$ if one considers runoff with a Chézy friction C .

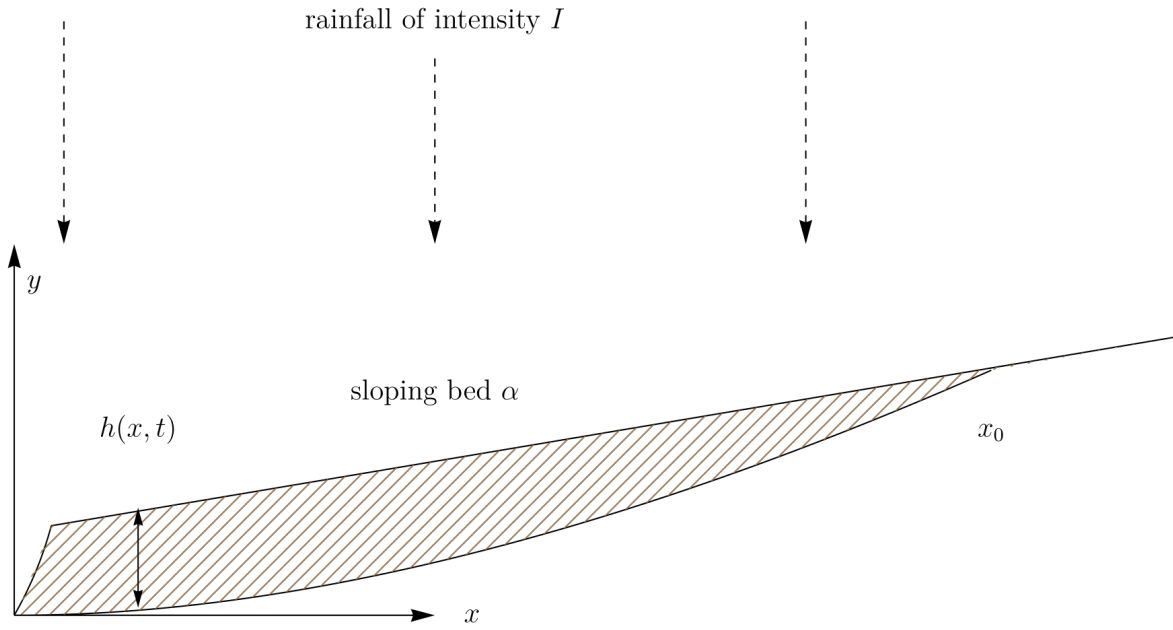


Figure 5.4 Flow generated by a rainfall.

The governing equation is given by mass conservation:

$$\frac{\partial h}{\partial t} + \frac{\partial hu}{\partial x} = I. \quad (5.24)$$

As we have $u = ah^b$, we obtain:

$$\frac{\partial h}{\partial t} + c(h) \frac{\partial h}{\partial x} = I \text{ with } c(h) = a(b+1)h^b.$$

or in a characteristic form:

$$\frac{dh}{dt} = I \text{ along } \frac{dx}{dt} = c(h) = a(b+1)h^b,$$

and we assume that initially the flow depth is zero (dry bed: $h(x, 0) = 0$) and no water comes from upstream of x_0 ($h(x_0, t) = 0$). The solution to the characteristic equation is $h = It$ along the characteristic curve:

$$x = \int a(b+1)h^b dt + x_1 = aI^b t^{1+b} + x_1, \quad (5.25)$$

where x_1 is constante of integration (such that at $x = x_1$, we have $h = 0$). This implies for any x ($0 \leq x \leq x_0$, with the frame used in Fig. 5.4, $x_0 = L_0$), we have:

- a linear growth $h(x, t) = It$ until time t_∞ such that $aI^bt_\infty^{1+b} = x_0 - x$;
- a stationary state for:

$$h(x, t) = h_\infty(x) = \left(\frac{I(x_0 - x)}{a} \right)^{1/(1+b)}.$$

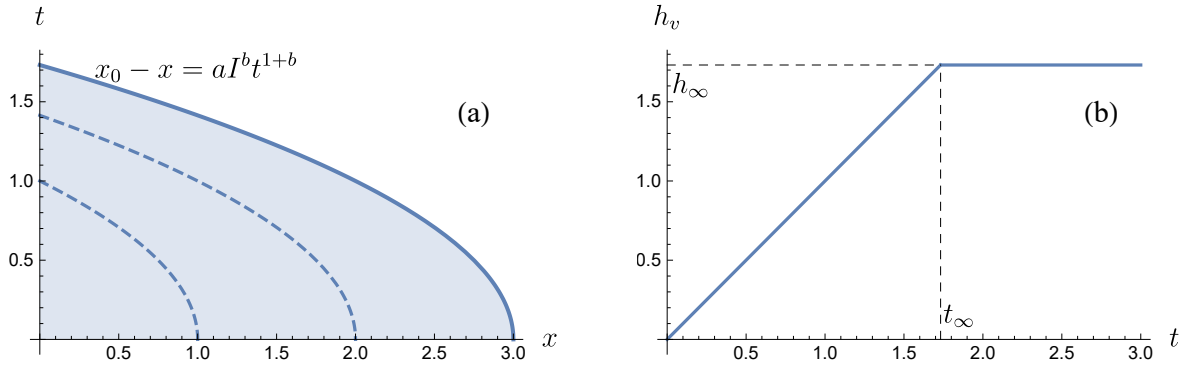


Figure 5.5 (a) characteristic curves (5.25). The thick line represents $x = aI^b t^{1+b}$, the path of a fluid parcel emitted from x_0 . The coloured area represents the domain controlled by the initial condition $h = 0$ for which we observe a linear growth $h(x, t) = It$. Above the curve $x = aI^b t^{1+b}$, the depth is constant and equal to $h_\infty(x)$. (b) Flow depth variation at $x = 0$. Computation for arbitrary values $a = 1$ 1/s $b = 1$, $I = 1$ m/s, and $x_0 = 3$ m.

5.5.2 Numerical implementation

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import os
4 from clawpack import riemann
5 plt.ioff()
6
7 #!/usr/bin/env python
8 # encoding: utf-8
9
10 r"""
11 Burgers' equation
12 """
13 def source_term(solver, state, dt):
14     i = state.problem_data['i']
15     h = state.q[0, :]
16     # Update to momentum
17     state.q[0, :] += dt * i
18
19 def inlet_bc(state, dim, t, qbc, auxbc, num_ghost):
20     "inlet boundary conditions"
21     qbc[0, :num_ghost] = 0.
22
23 def b4step(solver, state):
24     h = state.q[0, :]
25     t = state.t

```

```

26     hf = h[-1]
27     front.append([t,hf])
28
29
30 def setup(use_petsc=0,kernel_language='Fortran',outdir='./_output',
31          solver_type='classic'):
32     if use_petsc:
33         import clawpack.petclaw as pyclaw
34     else:
35         from clawpack import pyclaw
36
37     if kernel_language == 'Python':
38         riemann_solver = riemann.burgers_1D_py.burgers_1D
39     elif kernel_language == 'Fortran':
40         riemann_solver = riemann.burgers_1D
41
42     if solver_type=='sharpclaw':
43         solver = pyclaw.SharpClawSolver1D(riemann_solver)
44     else:
45         solver = pyclaw.ClawSolver1D(riemann_solver)
46         solver.limiters = pyclaw.limiters.tvd.vanleer
47     solver.kernel_language = kernel_language
48
49     solver.bc_lower[0] = pyclaw.BC.custom
50     solver.user_bc_lower = inlet_bc
51     solver.bc_upper[0] = pyclaw.BC.extrap
52     solver.step_source = source_term
53     solver.before_step = b4step
54
55     x = pyclaw.Dimension(0.0,10.0,1000,name='x')
56     domain = pyclaw.Domain(x)
57     num_eqn = 1
58     state = pyclaw.State(domain,num_eqn)
59     xc = state.grid.x.centers
60     state.q[0,:] = 0.
61     state.problem_data['efix']=True
62     state.problem_data['i'] = 1
63
64     claw = pyclaw.Controller()
65     claw.tfinal = 10
66     claw.num_output_times = 20
67     claw.solution = pyclaw.Solution(state,domain)
68     claw.solver = solver
69     claw.outdir = outdir
70     claw.setplot = setplot
71     claw.keep_copy = True
72
73     return claw
74
75 def setplot(plotdata):
76     """
77     Plot solution using VisClaw.
78     """
79     plotdata.clearfigures() # clear any old figures,axes,items data
80     # Figure for q[0]
81     plotfigure = plotdata.new_plotfigure(name='q[0]', figno=0)
82

```

```

83     # Set up for axes in this figure:
84     plotaxes = plotfigure.new_plotaxes()
85     plotaxes.xlimits = 'auto'
86     plotaxes.ylimits = [-1., 2.]
87     plotaxes.title = 'q[0]'
88     # Set up for item on these axes:
89     plotitem = plotaxes.new_plotitem(plot_type='1d')
90     plotitem.plot_var = 0
91     plotitem.plotstyle = '-o'
92     plotitem.color = 'b'
93
94     return plotdata

```

```

95 front = []
96 claw = setup()
97 claw.run()
98
99 ind=5
100 ind2=20
101 delta_t=claw.tfinal/claw.num_output_times
102
103 fig = plt.figure(figsize=(8,4))
104 left, bottom, width, height = 0.2, 0.2, 0.8, 0.8
105 ax = fig.add_axes((left ,bottom, width, height ))
106 ax.ylimits = [0,0.1]
107 frame = claw.frames[ind]
108 h = frame.q[0,:]
109 frame = claw.frames[ind2]
110 h2 = frame.q[0,:]
111
112 x = frame.state.grid.x.centers
113 ax.plot(x,h,label='t = {:.2f}'.format(ind*delta_t))
114 ax.plot(x,h2, 'k-.',label='t {:.2f}'.format(ind2*delta_t))
115
116 ax.set_xlabel(r'$x$ (m)')
117 ax.set_ylabel(r'$y$ (m)')
118 ax.legend()
119 plt.show()

```

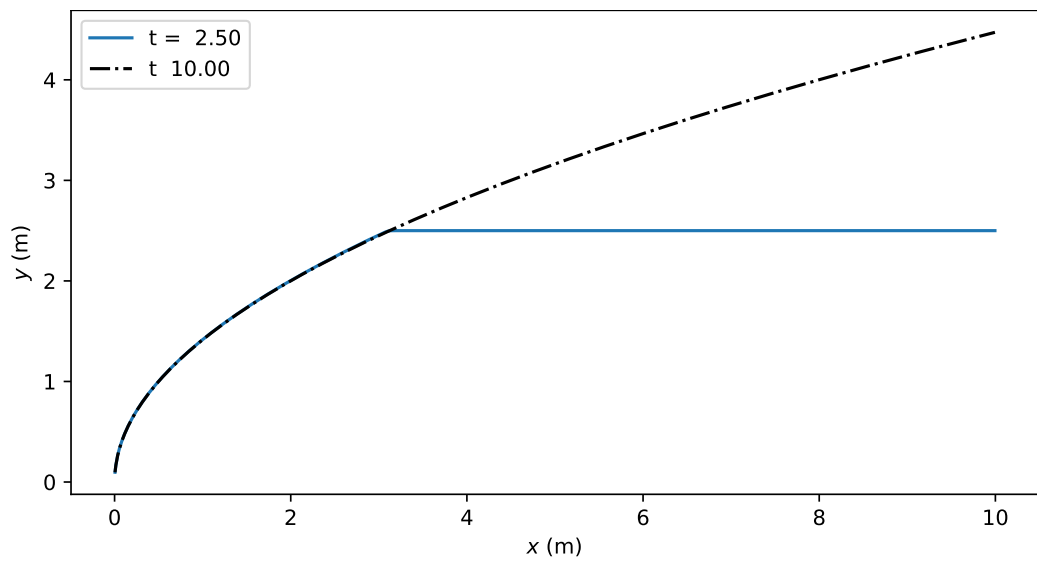


Figure 5.6 Numerical solutions at time $t = 2.5$ s and $t = 10$ s.

5.6 Nonlinear advection equation with a linear source term

5.6.1 Example

Let us consider the Burgers equation with a linear source term:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = -\mu u, \quad (5.26)$$

where $\mu > 0$ is a viscosity-like parameter. We further assume that the equation is subject to the initial condition:

$$u(x, 0) = ax \text{ for } 0 \leq x \leq b, \quad (5.27)$$

where a and b are two positive constants. We assume that the boundary $x = 0$ behaves like a wall.

We can work an analytical solution using the method of characteristics. If we differentiate $u(x, t)$ with respect to a new variable s :

$$\frac{du}{ds} = \frac{\partial u}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial u}{\partial t} \frac{\partial t}{\partial s}.$$

If we define:

$$\frac{\partial x}{\partial s} = u \text{ and } \frac{\partial t}{\partial s} = 1, \quad (5.28)$$

then we obtain:

$$\frac{du}{ds} = -\mu u. \quad (5.29)$$

The initial condition (5.27) can be put in the parametric form:

$$x(s = 0) = c, t(s = 0) = 0, \text{ and } u(s = 0) = ac, \quad (5.30)$$

where $c \geq 0$ is a dummy variable. Integrating Eq. (5.26) subject to the initial condition (5.30) gives:

$$u(s) = ace^{-\mu s}.$$

We can now integrate Eq. (5.28)

$$t = s \text{ and } x = c \left(1 + \frac{a}{\mu} (1 - e^{-\mu s}) \right).$$

Finally, we end up with:

$$u(x, t) = \frac{ax}{1 + \frac{a}{\mu} (1 - e^{-\mu t})} e^{-\mu t}. \quad (5.31)$$

This solution holds for $0 \leq x \leq x_f(t)$, where x_f denotes the front position. As initially there is a shock at $x = b$, then this shocks propagates at a velocity set by the Rankine-Hugoniot equation:

$$\frac{dx_f}{dt} = \frac{1}{2} \frac{[u^2]}{[u]} = \frac{u_f}{2}, \quad (5.32)$$

where $u_f(t) = u(x_f(t), t)$. We can solve the first-order differential equation (5.32)

$$x_f = \frac{b}{\sqrt{\mu}} \sqrt{a + \mu - ae^{-\mu t}}. \quad (5.33)$$

5.6.2 Numerical implementation

To solve numerically Eq. (5.26) subject to the initial condition (5.27), we use the approach outlined in § 3.9.1, which involves first solving the homogenous equation, and then integrating the source term using a fractional-step technique. Among other techniques, we can use the backward Euler method for the latter step:

$$u_i^{n+1} = u_i^n - \mu u_i^{n+1} dt \Rightarrow u_i^{n+1} = \frac{u_i^n}{1 + \mu dt}, \quad (5.34)$$

which is fully implicit. Another technique is the trapezoidal rule (one of the simplest version of the multistep Adams–Moulton method)

$$u_i^{n+1} = u_i^n - \mu \frac{u_i^{n+1} + u_i^n}{2} dt \Rightarrow u_i^{n+1} = u_i^n \frac{1 - \mu dt/2}{1 + \mu dt/2}, \quad (5.35)$$

The algorithm below shows the file `src1.f90` handling the source term.

```

1  ! =====
2  subroutine src1(meqn,mbc,mx,xlower,dx,q,maux,aux,t,dt)
3  ! =====
4      implicit none
5      ! Input parameters
6      integer, intent(in)      :: meqn,mbc,mx,maux
7      double precision, intent(in) :: xlower,dx,t,dt
8
9      !local
10     integer :: i, method
11
12
13     ! Output
14     double precision, intent(inout) :: q(meqn,1-mbc:mx+mbc)
15     double precision, intent(inout) :: aux(maux,1-mbc:mx+mbc)
16
17     double precision :: mu, xlim, a
18     common /comsrc/ mu, xlim, a
19
20
21     method = 1
22     select case(order)
23     case(1)
24         do i=1,mx
25             q(1,i) = q(1,i)/(1.d0+mu*dt)
26         end do
27     case(2)
28         do i=1,mx
29             q(1,i) = q(1,i)*(1.d0-mu*dt/2.d0)/(1.d0+mu*dt/2.d0)
30         end do
31     end select
32
33
34     return
35 end

```

5.6.3 Application

Figure 5.7 shows an example of resolution based on the backward Euler method. A similarly good agreement is obtained when using the trapezoidal rule.

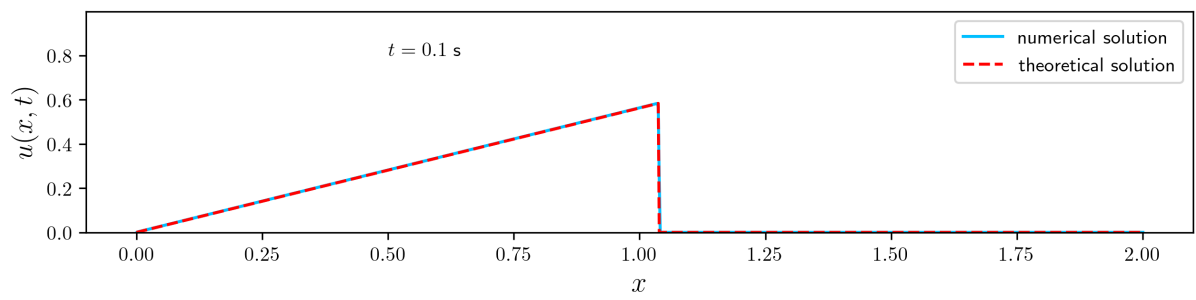


Figure 5.7 Numerical solutions at time $t = 0.1$. Parameters: $a = b = 1$ and $\mu = 5$.

5.7 Nonlinear advection equation with a nonlinear source term

5.7.1 Example

Let us consider the Burgers equation with a nonlinear source term:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = -\mu u^2, \quad (5.36)$$

where $\mu > 0$ is a Chézy-like parameter. We further assume that the equation is subject to the initial condition

$$u(x, 0) = ax \text{ for } 0 \leq x \leq b, \quad (5.37)$$

where a and b are two positive constants. We assume that the boundary $x = 0$ behaves like a wall.

We can work an analytical solution using the method of characteristics. If we differentiate $u(x, t)$ with respect to a new variable s

$$\frac{du}{ds} = \frac{\partial u}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial u}{\partial t} \frac{\partial t}{\partial s}.$$

If we define

$$\frac{\partial x}{\partial s} = u \text{ and } \frac{\partial t}{\partial s} = 1, \quad (5.38)$$

then we obtain

$$\frac{du}{ds} = -\mu u^2. \quad (5.39)$$

The initial condition (5.37) can be put in the parametric form

$$x(s = 0) = c, t(s = 0) = 0, \text{ and } u(s = 0) = ac, \quad (5.40)$$

where $c \geq 0$ is a dummy variable. Integrating Eq. (5.36) subject to the initial condition (5.40) gives

$$u(s) = \frac{1}{\mu s + (ac)^{-1}} = \frac{ac}{\mu sac + 1}.$$

We can now integrate Eq. (5.38)

$$t = s \text{ and } x = c + \frac{1}{\mu} \ln(1 + sac)$$

Using the Lambert function W , we can provide an analytical form to $x = c + \ln(1 + \mu sac)/\mu$:

$$c(x, t) = -\frac{1}{\mu at} + \frac{1}{\mu} W\left(\frac{1}{at} \exp\left(\mu x + \frac{1}{at}\right)\right)$$

Note that this function behaves like:

$$c = \frac{x}{1 + at} + O(x^2) \text{ when } t \rightarrow 0,$$

an approximation that can be useful when solving the Rankine–Hugoniot equation numerically. Finally, we end up with:

$$u(x, t) = \frac{ac(x, t)}{\mu ac(x, t)t + 1}. \quad (5.41)$$

This solution holds for $0 \leq x \leq x_f(t)$, where x_f denotes the front position. As initially there is a shock at $x = b$, then this shocks propagates at a velocity set by the Rankine-Hugoniot equation:

$$\frac{dx_f}{dt} = \frac{1}{2} \frac{[u^2]}{[u]} = \frac{u_f}{2}, \quad (5.42)$$

where $u_f(t) = u(x_f(t), t)$. To our knowledge, there is no solution to the ordinary differential equation (5.32), but we can integrate it numerically.

5.7.2 Numerical implementation

To solve numerically Eq. (5.36) subject to the initial condition (5.37), we use the approach outlined in § 3.9.1, which involves first solving the homogenous equation, and then integrating the source term using a fractional-step technique. Among other techniques, we can use an approximate backward Euler method for the latter step:

$$u_i^{n+1} = u_i^n - \mu u_i^{n+1} u_i^n dt \Rightarrow u_i^{n+1} = \frac{u_i^n}{1 + \mu dt u_i^n}, \quad (5.43)$$

which is fully explicit. If we want to use the backward Euler method, we have to solve the implicit equation:

$$u_i^{n+1} = u_i^n - \mu (u_i^{n+1})^2. \quad (5.44)$$

Here, this equation can be solved exactly since it is a second-order polynomial:

$$u_i^{n+1} = \frac{-1 + \sqrt{1 + 4dt\mu u_i^n}}{2dt\mu}. \quad (5.45)$$

For more complicated source terms, we can use an external solver to derive u_i^{n+1} from u_i^n . We provide an example below with the `fsolve.f90` routine proposed by [John Burkardt](#) based on Brent's method.

The algorithm below shows the file `src1.f90` handling the source term. By specifying the method (`method = 1, 2 or 3`), we can use the approximate backward Euler (5.43), the exact backward Euler (5.45), or the `fsolve.f90` routine.

```

1  ! =====
2  subroutine src1(meqn,mbc,mx,xlower,dx,q,maux,aux,t,dt)
3  ! =====
4      use exchange
5      implicit none
6      ! Input parameters
7      integer, intent(in)          :: meqn,mbc,mx,maux
8      double precision, intent(in) :: xlower,dx,t,dt
9
10     !local
11     integer :: i
12
13
14     ! Output
15     double precision, intent(inout) :: q(meqn,1-mbc:mx+mbc)
16     double precision, intent(inout) :: aux(maux,1-mbc:mx+mbc)
17
18     double precision :: mu, xlim, a
19     integer :: method
20     common /comsrc/ mu, xlim, a, method
21     double precision :: x, f, fx, tol
22     integer, parameter :: n = 1
23     integer :: info
24
25
26
27     select case(method)
28     case(1)
29         do i=1,mx
30             q(1,i) = q(1,i)/(1.d0+mu*dt*q(1,i))
31         end do
32     case(2)

```

```

33     do i=1,mx
34         q(1,i) = (-1.d0+sqrt(1.d0+4.d0*dt*q(1,i)*mu))/2.d0/dt/mu
35     end do
36     case(3)
37         do i=1,mx
38             call export_params(params,(/q(1,i), mu, dt/))
39             x = q(1,i)
40             tol = 1.d-8
41             call fsolve ( fsource, n, x, fx, tol, info )
42             q(1,i) = x
43         end do
44     end select
45
46
47     return
48 end

```

5.7.3 Application

Figure 5.8 shows the solution profile at $t = 1$ obtained using the approximate backward Euler method. Figure 5.9 shows that this approximate method is able to capture the time variation in front position.

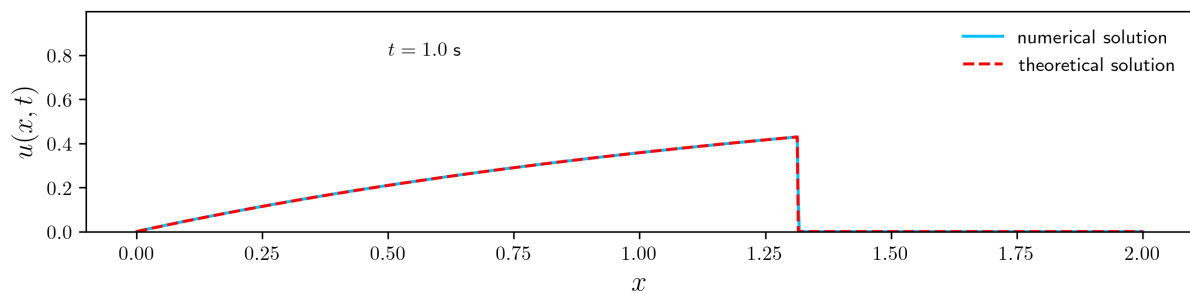


Figure 5.8 Numerical solution at time $t = 1$. Parameters: $a = b = 1$ and $\mu = 1$.

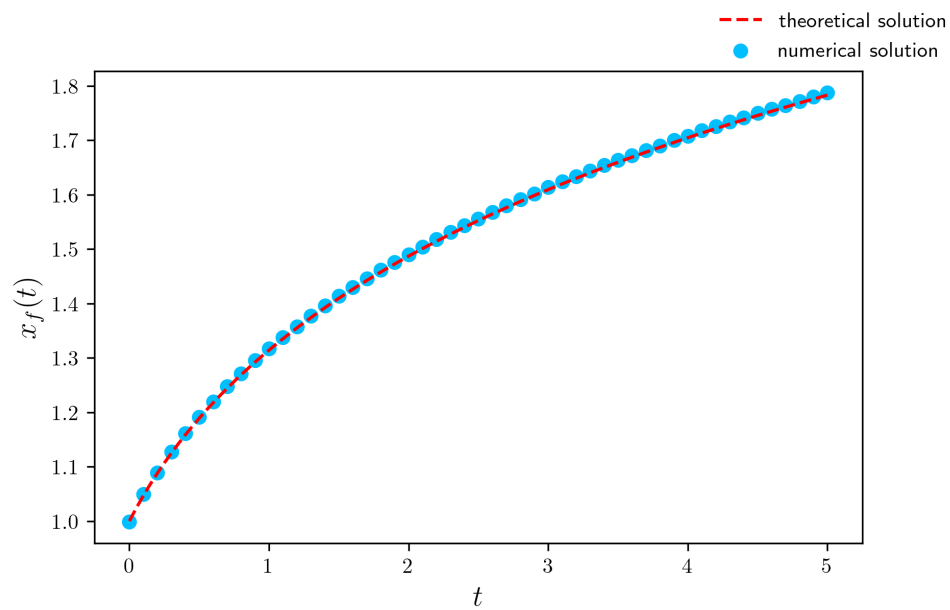


Figure 5.9 Front position as a function of time. Parameters: $a = b = 1$ and $\mu = 1$; method: approximate backward Euler.

Solving the Saint-Venant equations

6.1 Governing equations

6.1.1 Horizontal frictionless bed

The *Saint-Venant equations* (also called the shallow water equations) consist of the depth-averaged mass and momentum balance equations for a water flow along a sloping bed ([Saint-Venant, 1871](#)). In this chapter, we consider the simplest case, in which the bottom is horizontal and exerts no resistance, and the flow is one-directional (in the $x > 0$ direction). In this case, the conservative form of the governing equations comprises the mass balance equation:

$$\frac{\partial h}{\partial t} + \frac{\partial q}{\partial x} = 0, \quad (6.1)$$

the momentum balance equation:

$$\frac{\partial q}{\partial t} + \frac{\partial hu^2}{\partial x} + gh \frac{\partial h}{\partial x} = 0, \quad (6.2)$$

where g is gravitational acceleration, h denotes the flow depth, $q = hu$ is the flow rate, and u the depth-averaged velocity. The unknowns are q and h . In a matrix form, Eqs. (6.1)-(6.2) take the form:

$$\frac{\partial}{\partial t} q + \frac{\partial}{\partial x} f(q) = 0, \quad (6.3)$$

where the flux function is:

$$f = \begin{pmatrix} q \\ q^2/h + gh^2/2 \end{pmatrix} \text{ and } q = \begin{pmatrix} h \\ q \end{pmatrix}. \quad (6.4)$$

The Jacobian is:

$$f' = \begin{pmatrix} 0 & 1 \\ -q^2/h^2 + gh & 2q/h \end{pmatrix}, \quad (6.5)$$

whose eigenvalues are:

$$\lambda_1 = u - \sqrt{gh} \text{ and } \lambda_2 = u + \sqrt{gh}, \quad (6.6)$$

associated with the right eigenvectors:

$$w_1 = \begin{pmatrix} 1 \\ u - \sqrt{gh} \end{pmatrix} \text{ and } w_2 = \begin{pmatrix} 1 \\ u + \sqrt{gh} \end{pmatrix}. \quad (6.7)$$

6.2 Exact solver for the Riemann problem

6.2.1 Initial-value problem

We consider the Saint-Venant equations (6.3) with the following initial value condition:

$$q(x, 0) = \begin{cases} q_l & \text{for } x < 0, \\ q_r & \text{for } x > 0, \end{cases} \quad (6.8)$$

where q_l and q_r are the left (L) and right (R) states, respectively. The Saint-Venant equations can be solved exactly for this initial-value problem. The general solution has been outlined in § 2.5. The solution depends on the particular values taken by q_l and q_r . We will show that these solutions include:

- Dry initial states: if one state is zero, the bed is initially dry on one side, and wet on the other side. The solution will be a 1-rarefaction wave if $q_r = 0$, and a 2-rarefaction wave if $q_l = 0$.
- Initially wet bed: when the flow depth is initially nonzero everywhere, the bed is wet. There are usually three possibilities:
 - if $u_r = \phi_l(h_r|h_l, u_l)$ where the function ϕ_l is defined by Eq. (2.62), then the solution will be a 1-rarefaction ($h_r < h_l$) or a 1-shock wave ($h_r > h_l$).
 - if $u_l = \phi_r(h_l|h_r, u_r)$ where the function ϕ_r is defined by Eq. (2.63), then the solution will be a 2-rarefaction ($h_l < h_r$) or a 2-shock wave ($h_l > h_r$).
 - if the two conditions above are not met, then the solution comprises rarefaction and/or shock waves connecting an intermediate state $I q_* = (h_*, h_* u_*)$ to the left and right states. The nature of these waves depends on the position of h_* relative the min and max values: $h_0 = \min(h_l, h_r)$ and $h_1 = \max(h_l, h_r)$, and the positiveness of h_* . If the initial velocity difference $\Delta u = u_r - u_l$ does not exceed the critical value (2.69): $(\Delta u)_{cr} = 2\sqrt{gh_l} + 2\sqrt{gh_r}$, then we can find a unique strictly positive root to the sixth-order polynomial $f(h_*) = 0$ where f is given by Eq. (2.68). This is the *depth positivity condition*. In this case, the wave structure is the following:
 - * $0 < h_* < h_0$, the solution comprises two rarefaction waves.
 - * $h_0 < h_* < h_1$, the solution consists of one rarefaction wave and one shock wave.
 - * $h_1 < h_*$, the solution comprises two shock waves.

Otherwise, the intermediate state is a dry cavity ($h_* = 0$) separating the left and right states by two rarefaction waves.

The python script for the exact solver is provided on [github](#).

6.2.2 Examples

Right dry state

The solution is detailed on p. 36. The solution consists on a 1-rarefaction wave, as shown by Fig. 6.1 state. The 1-characteristic curves form a fan:

$$\frac{x}{t} = \lambda_1 = u - \sqrt{gh}, \quad (6.9)$$

with $u_l - \sqrt{gh_l} \leq \lambda_1 \leq u + 2\sqrt{gh_l}$, and within this fan, the 1-Riemann variable $r_1 = u + 2\sqrt{gh}$ is constant:

$$r_1 = u + 2\sqrt{gh} = u_l + 2\sqrt{gh_l}. \quad (6.10)$$

Solving these two equations for u and h provides

$$u(x, t) = \frac{r_1 + 2\xi}{3}, \quad (6.11)$$

$$h(x, t) = \frac{(\bar{u} - \xi)^2}{g} = \frac{(r_1 - \xi)^2}{9g}. \quad (6.12)$$

Once the solution has been worked out, we can determine the behaviour of the 2-characteristics:

$$\frac{dx}{dt} = \lambda_2 = u + \sqrt{gh} = \frac{2r_1 + \xi}{3}. \quad (6.13)$$

The general solution is

$$x = at^{1/3} + r_1. \quad (6.14)$$

The constant of integration a is set by imposing the continuity of the 2-characteristics across the 1-characteristic curve $x = \lambda_1^l t = (u_l - \sqrt{gh_l})t$. At short times, the 2-characteristic curve starting from x_0 is:

$$x = x_0 + \lambda_2^l t \text{ with } \lambda_2^l = u_l + \sqrt{gh_l}. \quad (6.15)$$

At time $t = x_0/(\lambda_1^l - \lambda_2^l)$, the 2-characteristic crosses the 1-characteristic $x = \lambda_1^l t$. For $t \geq t_0$, the 2-characteristic curve is given by Eq. (6.11) with

$$a = \frac{\lambda_1^l t_0 - r_1 t_0}{t_0^{1/3}}. \quad (6.16)$$

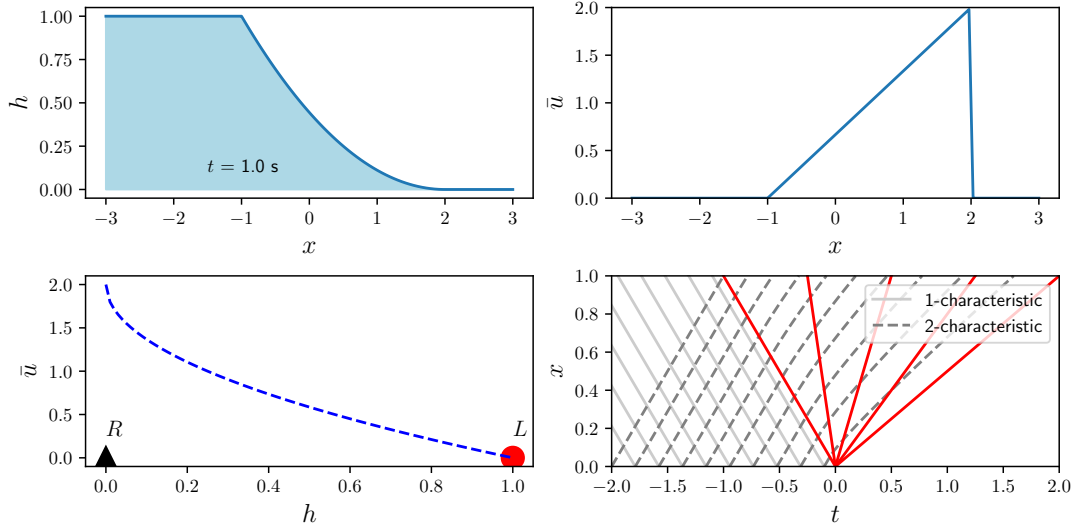


Figure 6.1 Riemann problem with an initially dry state on the right: $h_l = 1$ m, $h_r = 0$ m, $u_l = u_r = 0$ m/s. (a) Flow depth profile at time $t = 1$ s. (b) Velocity profile at time $t = 1$ s. (c) Integral curve related to the 1-rarefaction wave connecting L to R in the (h, hu) plane. (d) Characteristic curves in the (x, t) plane.

As shown by Fig. 6.1(c), the left and right states do not seem to be connected by a 1-rarefaction wave. It should be recalled that the phase plane should be plotted using the physical variables (h, hu) , and so by using the variable (h, u) instead, we produce this apparently spurious result that L and R are not connected.

Left dry state

The solution is detailed on p. 36. The solution consists on a 2-rarefaction wave, as shown by Fig. 6.1 state.

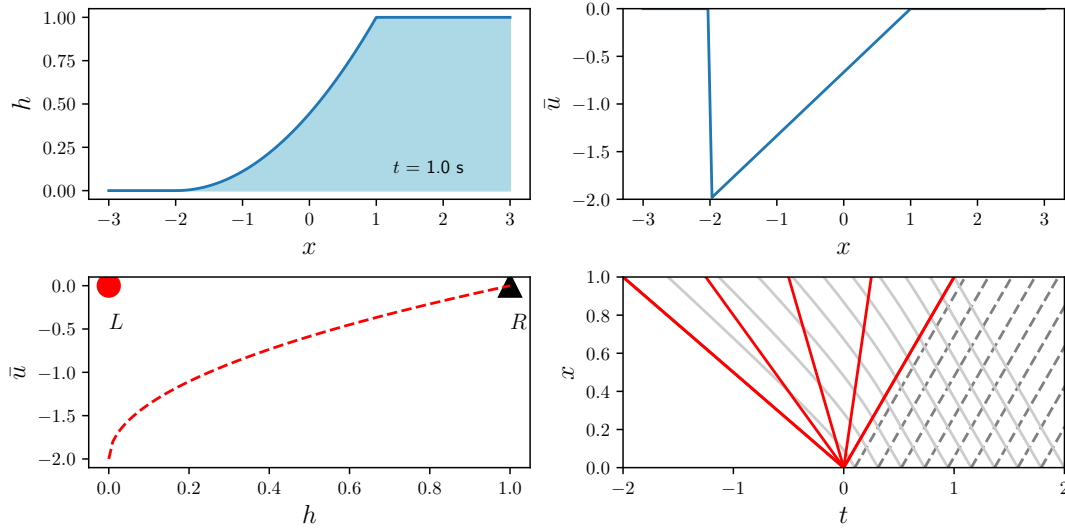


Figure 6.2 Riemann problem with an initially dry state on the left: $h_l = 0$ m, $h_r = 1$ m, $u_l = u_r = 0$ m/s. (a) Flow depth profile at time $t = 1$ s. (b) Velocity profile at time $t = 1$ s. (c) Integral curve related to the 2-rarefaction wave connecting L to R in the (h, hu) plane. (d) Characteristic curves in the (x, t) plane.

Wet bed: solution with an intermediate state

When the bed is initially wet and the left and right states do not lie on one integral curve or Hugoniot locus, the solution involves an intermediate state between the left (L) and right (R) states. This intermediate state can be connected to L and R by a 1- and a 2-rarefaction waves [see Fig. 6.3], by a 1-shock and a 2-rarefaction waves [see Fig. 6.4], by a 1-rarefaction and a 2-shock waves [see Fig. 6.5], and by a 2-shock and a 2-shock waves [see Fig. 6.6] depending on the specific values taken by q_l and q_r , provided that the depth positivity condition is satisfied.

When the depth positivity condition is not met, then a dry cavity forms (see Fig. 6.7).

Wet bed: solution with no intermediate state

If the left and right states lie on the same curve (Hugoniot locus or integral curve), then they are connected by a 1- (or 2-) shock or rarefaction wave. Figures 6.8 to 6.9 show two examples.

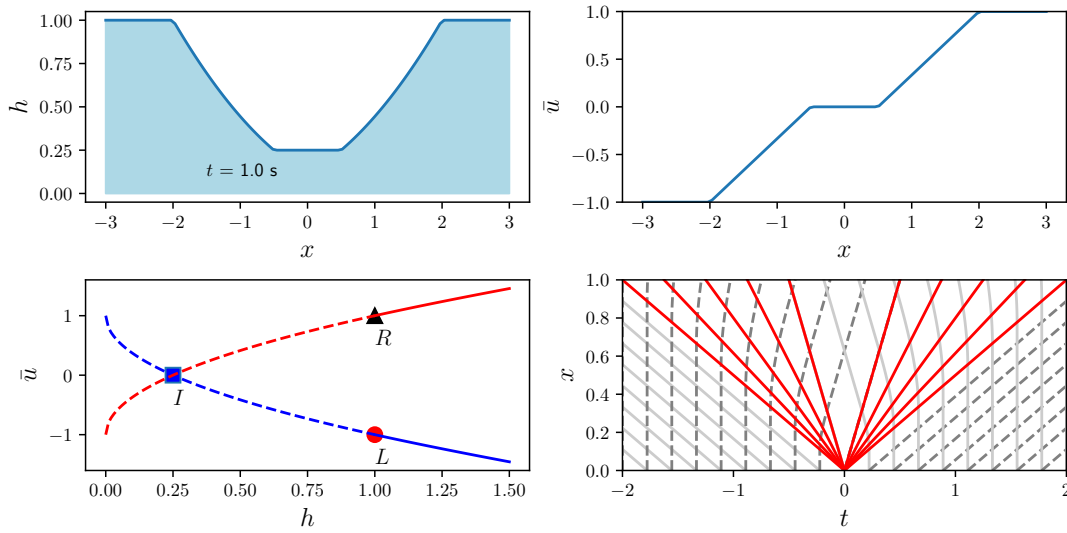


Figure 6.3 Riemann problem with $h_l = 1$ m, $h_r = 1$ m, $u_l = -1$ m/s and $u_r = 1$ m/s. (a) Flow depth profile at time $t = 1$ s. (b) Velocity profile at time $t = 1$ s. (c) Integral curve related to the 1-rarefaction and 2-rarefaction waves connecting L to R via I in the $(h, h\bar{u})$ plane. (d) Characteristic curves in the (x, t) plane.

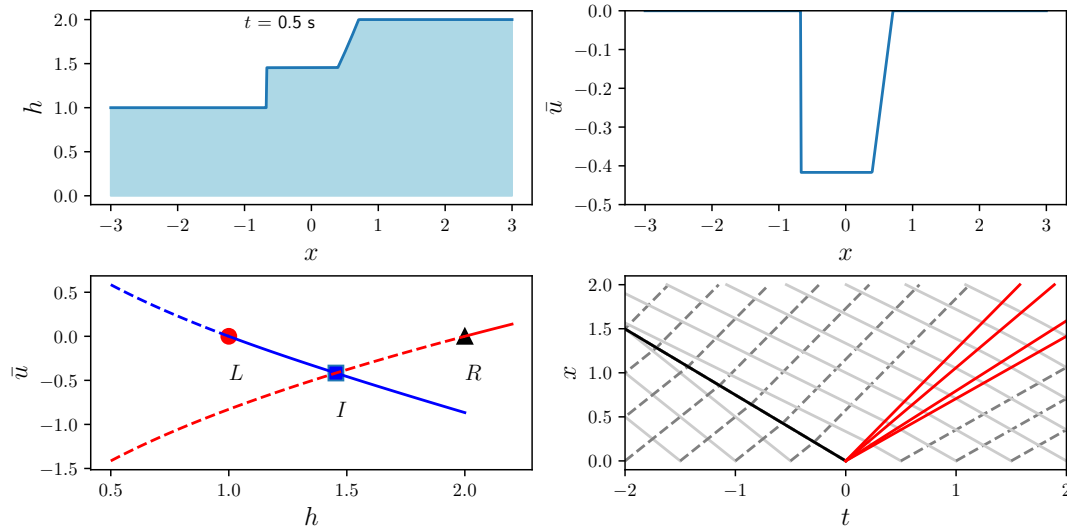


Figure 6.4 Riemann problem with $h_l = 1$ m, $h_r = 2$ m, $u_l = 0$ m/s and $u_r = 0$ m/s. (a) Flow depth profile at time $t = 0.5$ s. (b) Velocity profile at time $t = 0.5$ s. (c) Integral curve related to the 1-shock wave and the 2-rarefaction wave connecting L to R via I in the $(h, h\bar{u})$ plane. (d) Characteristic curves in the (x, t) plane.

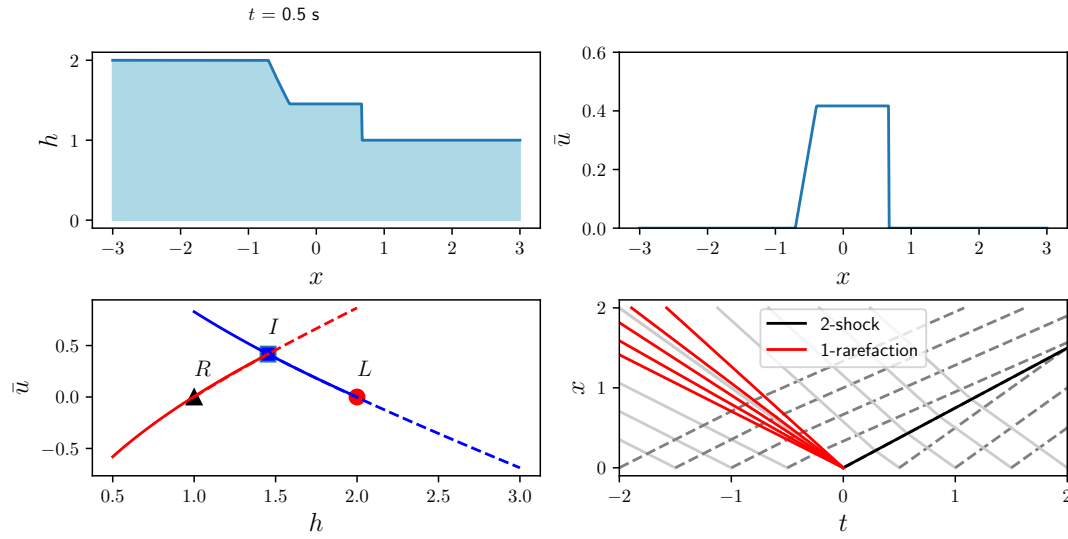


Figure 6.5 Riemann problem with $h_l = 2$ m, $h_r = 1$ m, $u_l = 0$ m/s and $u_r = 0$ m/s. (a) Flow depth profile at time $t = 0.5$ s. (b) Velocity profile at time $t = 0.5$ s. (c) Integral curve related to the 1-rarefaction wave and the 2-shock wave connecting L to R via I in the $(h, h\bar{u})$ plane. (d) Characteristic curves in the (x, t) plane.

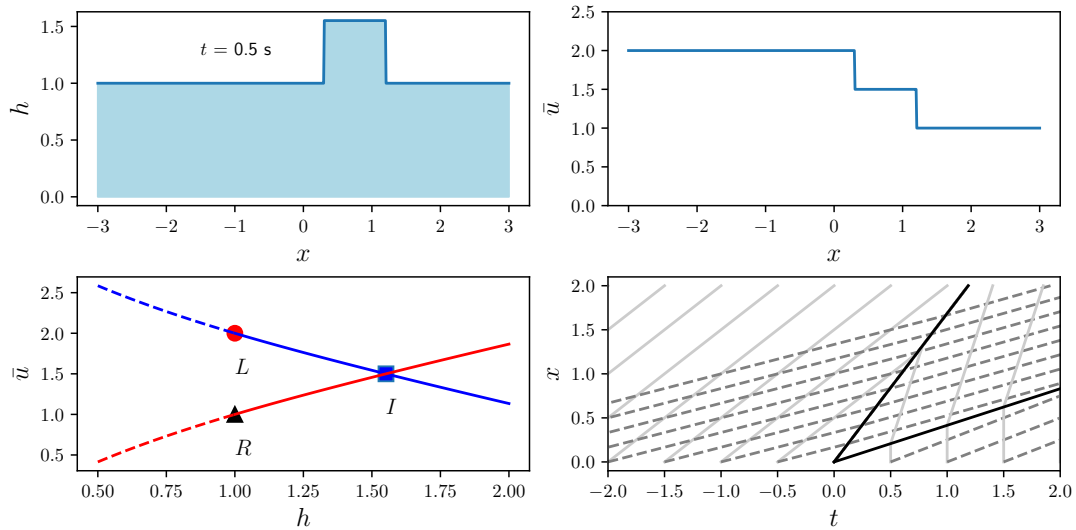


Figure 6.6 Riemann problem with $h_l = 1$ m, $h_r = 1$ m, $u_l = 2$ m/s and $u_r = 1$ m/s. (a) Flow depth profile at time $t = 0.5$ s. (b) Velocity profile at time $t = 0.5$ s. (c) Integral curve related to the 1- and 2-shock waves connecting L to R via I in the $(h, h\bar{u})$ plane. (d) Characteristic curves in the (x, t) plane.

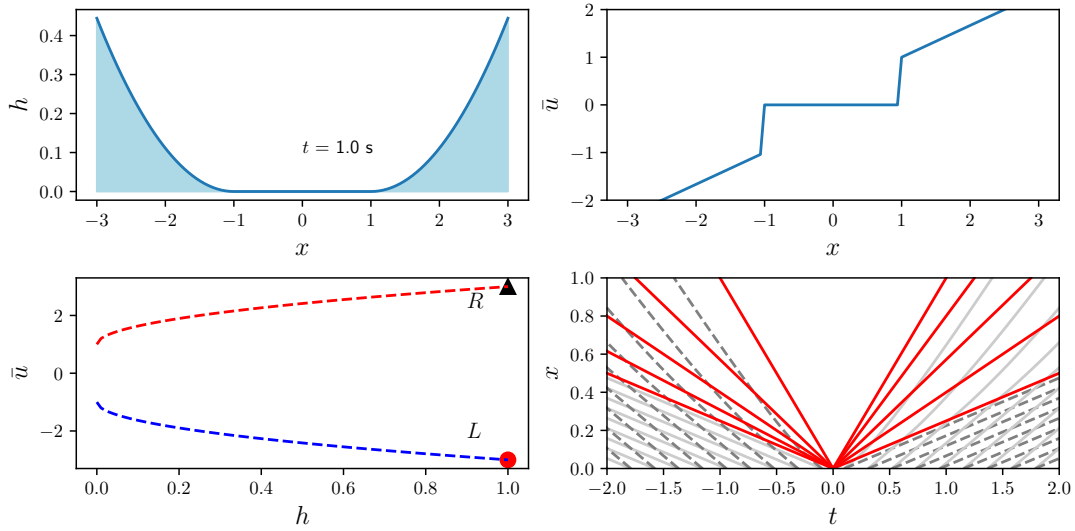


Figure 6.7 Riemann problem with a dry cavity formed after inception: $h_l = 1$ m, $h_r = 1$ m, $u_l = -3$ m/s and $u_r = 3$ m/s. (a) Flow depth profile at time $t = 1$ s. (b) Velocity profile at time $t = 1$ s. (c) Integral curve related to the 1- and 2-rarefaction wave connecting L to R via a dry state in the $(h, h\bar{u})$ plane. (d) Characteristic curves in the (x, t) plane.

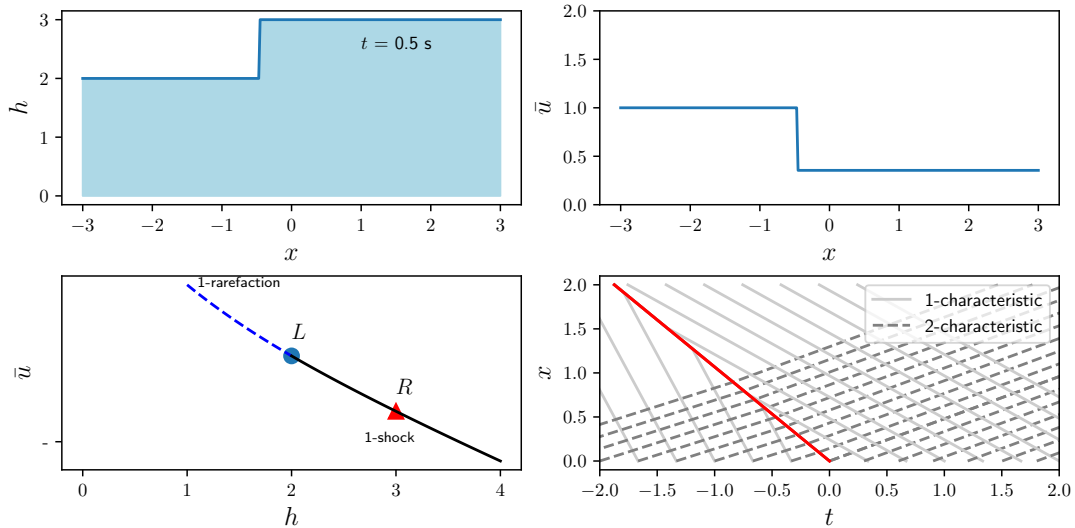


Figure 6.8 Riemann problem with $h_l = 2$ m, $h_r = 3$ m, $u_l = 1$ m/s and $u_r = 0.354$ m/s. (a) Flow depth profile at time $t = 0.5$ s. (b) Velocity profile at time $t = 0.5$ s. (c) Integral curve related to the 1-shock wave connecting L to R in the $(h, h\bar{u})$ plane. (d) Characteristic curves in the (x, t) plane.

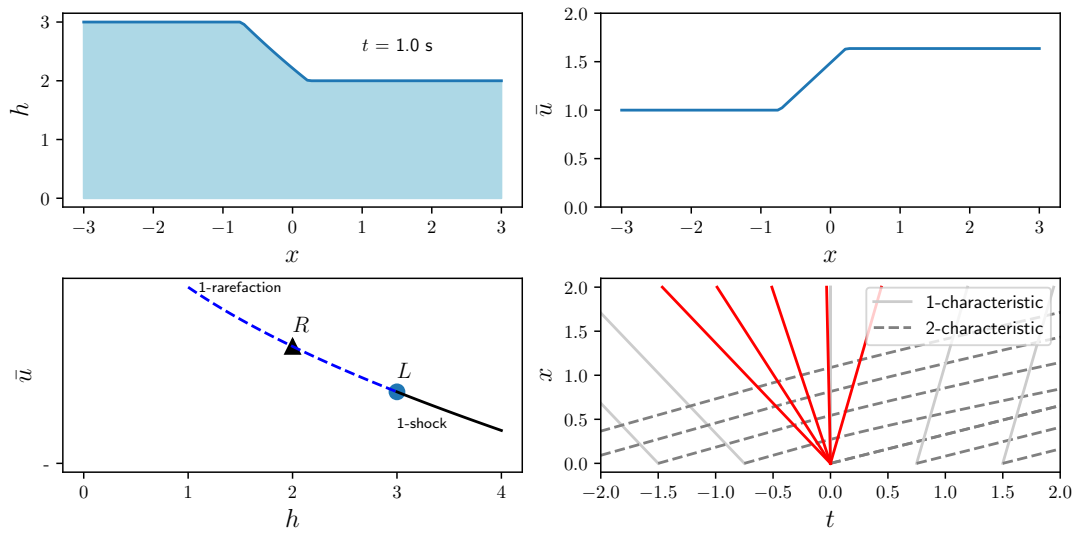


Figure 6.9 Riemann problem with $h_l = 3$ m, $h_r = 2$ m, $u_l = 1$ m/s and $u_r = 1.635$ m/s. (a) Flow depth profile at time $t = 1$ s. (b) Velocity profile at time $t = 1$ s. (c) Integral curve related to the 1-shock wave connecting L to R in the $(h, h\bar{u})$ plane. (d) Characteristic curves in the (x, t) plane.

6.3 Approximate solver: the Roe solver

6.3.1 Derivation

The Roe solver involves linearising the Jacobian (see § 3.5):

$$\hat{A}_{i-1/2} = \int_0^1 \frac{df(q(\xi))}{dq} d\xi, \quad (6.17)$$

where the Jacobian ∇f has been defined by Eq. (6.5) and the integral path is the straight line:

$$q = Q_{i-1} + \xi(Q_i - Q_{i-1}).$$

It has been shown in § 3.5 that the matrix $\hat{A}_{i-1/2}$ satisfies Eq. (3.50)

$$f(Q_i) - f(Q_{i-1}) = \hat{A}_{i-1/2} \cdot (Q_i - Q_{i-1}),$$

when Q_{i-1} and Q_i are connected by the same wave (shock or rarefaction). This property is essential to ensure that the method is conservative.

The problem is that the integration of Eq. (6.17) is complicated by the coupling between variables q and h in the entries of ∇f . Roe (1981) found a change of variable $z = z(q)$, which allowed him to solve this issue for the Euler equations. A similar trick can be applied to the shallow water equations. We make the following change of variable:

$$\begin{aligned} z &= \begin{pmatrix} z_1 \\ z_2 \end{pmatrix}, \\ &= \frac{q}{\sqrt{h}} = \begin{pmatrix} \sqrt{h} \\ u\sqrt{h} \end{pmatrix}. \end{aligned} \quad (6.18)$$

The inverse mapping is:

$$q = \begin{pmatrix} z_1^2 \\ z_1 z_2 \end{pmatrix} \Rightarrow \frac{dq}{dz} = \begin{pmatrix} 2z_1 & 0 \\ z_2 & z_1 \end{pmatrix}. \quad (6.19)$$

whose determinant is $2z_1^2 > 0$ wherever the flow depth is nonzero (as a consequence, $z(q)$ is invertible everywhere). With the new variables, the flux function and its Jacobian become:

$$f = \begin{pmatrix} z_1 z_2 \\ z_2^2 + \frac{1}{2} g z_1^4 \end{pmatrix} \Rightarrow \frac{df}{dz} = \begin{pmatrix} z_2 & z_1 \\ 2g z_1^3 & 2z_2 \end{pmatrix}.$$

Using the change of variable, we now integrate:

$$f(Q_i) - f(Q_{i-1}) = \int_0^1 \frac{df}{dz}(z) dz$$

along the straight line:

$$z(\xi) = Z_{i-1} + (Z_i - Z_{i-1})\xi.$$

As the ξ derivative of z gives $z' = Z_i - Z_{i-1}$, we obtain:

$$\begin{aligned} f(Q_i) - f(Q_{i-1}) &= \int_0^1 \frac{df}{dz}(z) \cdot \frac{dz}{d\xi} d\xi, \\ &= \int_0^1 \frac{df}{dz}(z) d\xi \cdot (Z_i - Z_{i-1}), \\ &= \begin{pmatrix} \bar{Z}^2 & \bar{Z}^1 \\ 2g\bar{Z}^1\bar{h} & 2\bar{Z}^2 \end{pmatrix} \cdot (Z_i - Z_{i-1}) \end{aligned} \quad (6.20)$$

where the k th average component $\bar{Z}^k$ is defined as:

$$\bar{Z}^k = \frac{1}{2}(Z_{i-1}^k + Z_i^k) \text{ and } \bar{h} = \frac{1}{2}(h_{i-1} + h_i).$$

Proof. Integration z^k (with $k = 1$ or 2) does not pose problems:

$$\begin{aligned} \int_0^1 z^k d\xi &= \int_0^1 (Z_{i-1}^k + \xi(Z_i^k - Z_{i-1}^k)) d\xi, \\ &= \left[\xi Z_{i-1}^k + \frac{\xi^2}{2}(Z_i^k - Z_{i-1}^k) \right]_0^1, \\ &= \frac{\xi}{2}(Z_i^k + Z_{i-1}^k), \\ &= \bar{Z}^k. \end{aligned} \tag{6.21}$$

The cubic term $(z^1)^3$ requires more work:

$$\begin{aligned} \int_0^1 (z^1)^3 d\xi &= \int_0^1 (Z_{i-1}^k + \xi(Z_i^k - Z_{i-1}^k))^3 d\xi, \\ &= \frac{1}{Z_i^k - Z_{i-1}^k} \int_{Z_{i-1}^k}^{Z_i^k} \eta^3 d\eta, \\ &= \frac{1}{4(Z_i^k - Z_{i-1}^k)} [\eta^4]_{Z_{i-1}^k}^{Z_i^k}, \\ &= \frac{1}{4} \frac{(Z_i^k)^4 - (Z_{i-1}^k)^4}{Z_i^k - Z_{i-1}^k}, \\ &= \frac{(Z_i^k)^2 + (Z_{i-1}^k)^2}{2} \frac{Z_i^k + Z_{i-1}^k}{2}, \\ &= \bar{Z}^1 \bar{h}, \end{aligned}$$

since $(Z_i^k)^2 + (Z_{i-1}^k)^2 = h_i + h_{i-1}$. $\square$

We now have Eq. (6.20), which relates the flux difference $f(Q_i) - f(Q_{i-1})$ to $Z_i - Z_{i-1}$. We need to express $Q_i - Q_{i-1}$ as a function of $Z_i - Z_{i-1}$ so that we can directly relate the flux difference to the difference $Q_i - Q_{i-1}$. To that end, we will use the following integral path $q = Q_{i-1} + \xi(Q_i - Q_{i-1})$ and integrate dq along it:

$$\begin{aligned} Q_i - Q_{i-1} &= \int_0^1 \frac{dq(\xi)}{d\xi} d\xi = \int_0^1 \frac{dq}{dz} \cdot \frac{dz}{d\xi} d\xi \\ &= \begin{pmatrix} 2\bar{Z}_1 & 0 \\ \bar{Z}_2 & \bar{Z}_1 \end{pmatrix} \cdot (Z_i - Z_{i-1}), \end{aligned}$$

where we made use of the definition (6.19) of the Jacobian $\nabla q(z)$ together with the relation (6.21). We eventually find:

$$\hat{A}_{i-1/2} = \begin{pmatrix} \bar{Z}_2 & \bar{Z}_1 \\ 2g\bar{Z}_1\bar{h} & 2\bar{Z}_2 \end{pmatrix} \cdot \begin{pmatrix} 2\bar{Z}_1 & 0 \\ \bar{Z}_2 & \bar{Z}_1 \end{pmatrix}^{-1} = \begin{pmatrix} 0 & 1 \\ g\bar{h} - (\bar{Z}_2/\bar{Z}_1)^2 & 2\bar{Z}_2/\bar{Z}_1 \end{pmatrix},$$

and after returning to the original variables, we obtain:

$$\hat{A}_{i-1/2} = \begin{pmatrix} 1 & 1 \\ -\hat{u} + g\bar{h} & 2\hat{u} \end{pmatrix} \text{ where } \hat{u} = \frac{\sqrt{h_{i-1}}u_{i-1} + \sqrt{h_i}u_i}{\sqrt{h_{i-1}} + \sqrt{h_i}}. \tag{6.22}$$

The Roe matrix is thus the Jacobian matrix $f'(q)$ evaluated at the intermediate state $q = (\bar{h}, \bar{h}\hat{u})$. It has the eigenvalues

$$\lambda_1 = \hat{u} - \sqrt{g\bar{h}} \text{ and } \lambda_2 = \hat{u} + \sqrt{g\bar{h}},$$

associated with the right eigenvectors:

$$w_1 = \begin{pmatrix} 1 \\ \hat{u} - \sqrt{g\bar{h}} \end{pmatrix} \text{ and } w_2 = \begin{pmatrix} 1 \\ \hat{u} + \sqrt{g\bar{h}} \end{pmatrix}. \quad (6.23)$$

We can decompose the initial jump $Q_i - Q_{i-1}$ in the right eigenvector basis $(w_k)_{k=1,2}$ as

$$\Delta Q = Q_i - Q_{i-1} = R \cdot \alpha, \quad (6.24)$$

where $R = [w_1, w_2]$ is the right-eigenvector matrix. We then deduce the α coefficients by inverting the matrix R

$$\begin{aligned} \alpha &= R^{-1} \cdot \Delta Q, \\ &= \frac{1}{2\hat{c}} \begin{pmatrix} \hat{u} + \hat{c} & -1 \\ \hat{c} - \hat{u} & 1 \end{pmatrix} \cdot \Delta Q, \end{aligned} \quad (6.25)$$

$$= \frac{1}{2\hat{c}} \begin{pmatrix} (\hat{u} + \hat{c})\Delta Q_1 - \Delta Q_2 \\ (-\hat{u} + \hat{c})\Delta Q_1 + \Delta Q_2 \end{pmatrix}, \quad (6.26)$$

where $\hat{c} = \sqrt{g\bar{h}}$ and $\Delta Q = (\Delta Q_1, \Delta Q_2)$.

6.3.2 Wave form

To summarize the results, we need the following equations to write the Roe solver's algorithm:

- The velocities associated with the intermediate state

$$\hat{u} = \frac{q_{i-1}/\sqrt{h_{i-1}} + q_i/\sqrt{h_i}}{\sqrt{h_{i-1}} + \sqrt{h_i}} \text{ and } \hat{c} = \sqrt{g\bar{h}} = \sqrt{\frac{1}{2}(\sqrt{h_{i-1}} + \sqrt{h_i})}. \quad (6.27)$$

- The waves W_k :

$$W_k = \alpha_k w_k, \quad k = 1, 2 \quad (6.28)$$

where α_k are the components of the α vector given by Eq. (6.25) and w_k are the right eigenvectors of the Roe matrix given by Eq. (6.23).

- The characteristic speeds

$$s_1 = \hat{u} - \hat{c} \text{ and } s_2 = \hat{u} + \hat{c}. \quad (6.29)$$

- The fluctuations are

$$\begin{aligned} A^- \cdot \Delta Q_{i-1/2} &= \sum_{k=1}^2 \min(\lambda_{i-1/2}^k, 0) W_{k,i-1/2}, \\ A^+ \cdot \Delta Q_{i+1/2} &= \sum_{k=1}^2 \max(\lambda_{i+1/2}^k, 0) W_{k,i+1/2}, \end{aligned}$$

which gives in the present context:

- if $s_k < 0$, then `amdq(m,i)=s*wave.`
- if $s_k > 0$, then `apdq(m,i)=s*wave.`

6.3.3 Sonic entropy fix

We have seen in § 3.4.2 that when the solution to the Riemann problem is a transonic wave, the Roe approximate solution may be incorrect (see also LeVeque, 2002, § 15.3.5). In that case, there is usually an intermediate state Q_* between the left and right states Q_{i-1} and Q_i , and the associated speeds are the following one:

- For the first eigenvalue, we have:

$$\lambda_{i-1}^* = u_{i-1} - \sqrt{gh_{i-1}}, \quad \lambda_*^1 = u_* - \sqrt{g\hat{h}_*}$$

- For the second eigenvalue, we have:

$$\lambda_*^2 = u_* + \sqrt{g\hat{h}_*}, \quad \lambda_i^2 = u_i + \sqrt{g\hat{h}_i}.$$

When $\lambda_{i-1}^1 < 0 < \lambda_*^1$ (resp. $\lambda_*^2 < 0 < \lambda_i^2$), we should consider that the 1-wave (resp. the 2-wave) is a transonic rarefaction wave rather than a 1-shock wave (see Fig. 6.10).

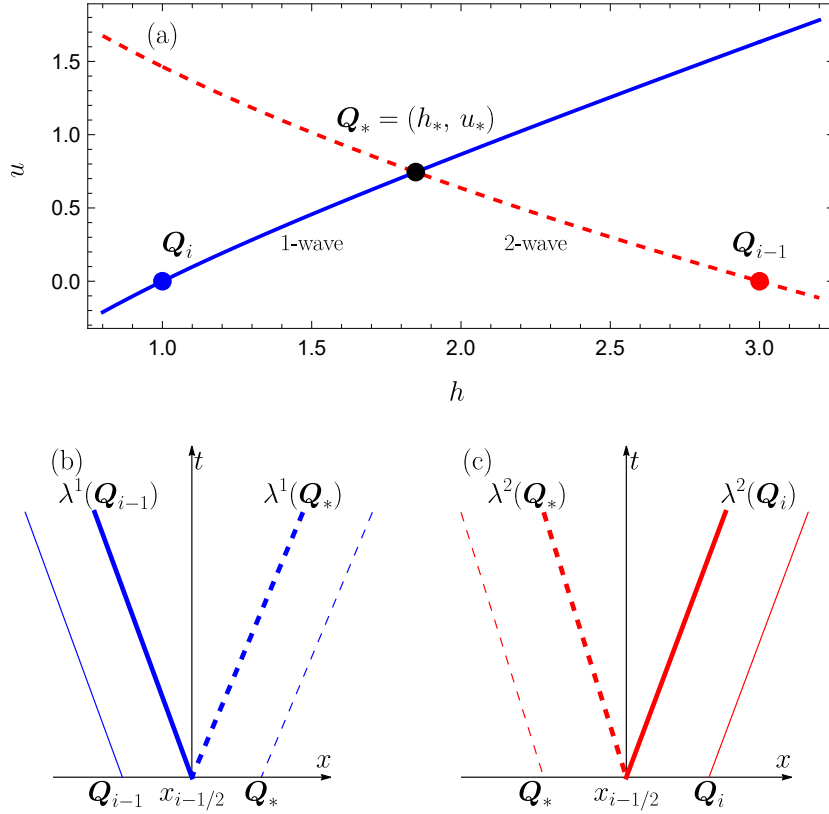


Figure 6.10 (a) In the general case, there is an intermediate state Q_* connecting the left and right states Q_{i-1} and Q_i by 1- and 2-waves of respective velocity λ^1 and λ^2 . (b) If $\lambda_{i-1}^1 < 0 < \lambda_*^1$, then the 1-wave is a transonic wave. (c) If $\lambda_*^2 < 0 < \lambda_i^2$, then the 2-wave is a transonic wave.

By using the analytical expression (2.51) for a centred rarefaction wave (evaluated at $\xi = 0$), we can deduce the interface value:

$$h_{i-1/2} = \frac{1}{9g} \left(u_{i-1} + 2\sqrt{gh_{i-1}} \right)^2, \quad (6.30)$$

while Eq. (2.50) yields:

$$u_{i-1/2} = u_{i-1} + 2 \left(\sqrt{gh_{i-1}} - \sqrt{gh_{i-1/2}} \right) \quad (6.31)$$

The flux fluctuations are computed using Eqs. (3.48) and (3.49).

6.3.4 Implementation in Clawpack

Here is how the Roe solver is implemented in Clawpack (without and without the entropy fix to compute transonic wave).

```

51 subroutine rp1(maxmx,num_eqn,num_waves,num_aux,num_ghost,num_cells, &
52               ql,q,auxl,auxr,wave,s,amdq,apdq)
53
54 ! waves: 2
55 ! equations: 2
56
57 implicit none
58 integer, intent(in) :: maxmx, num_eqn, num_waves, num_aux, num_ghost, &
59                        num_cells
60 real(kind=8), intent(in), dimension(num_eqn,1-num_ghost:maxmx+num_ghost)
61 :: ql, q
62 real(kind=8), intent(in), dimension(num_aux,1-num_ghost:maxmx+num_ghost)
63 :: auxl, auxr
64 real(kind=8), intent(out) :: s(num_waves, 1-num_ghost:maxmx+num_ghost)
65 real(kind=8), intent(out) :: wave(num_eqn, num_waves, 1-num_ghost:maxmx+
66 num_ghost)
67 real(kind=8), intent(out), dimension(num_eqn,1-num_ghost:maxmx+
68 num_ghost) :: amdq,apdq
69
70 ! local variables:
71 real(kind=8) :: a1,a2,ubar,cbar,s0,s1,s2,s3,hr1,uhr1,hl2,uhl2,sfract,df
72 real(kind=8) :: delta(2)
73 integer :: i,m,mw
74
75 logical :: efix
76
77 data efix /.true./      !# Use entropy fix for transonic rarefactions
78
79 ! Gravity constant set in setprob.f or the shallow1D.py file
80 real(kind=8) :: grav
81 common /cparam/ grav
82
83 ! Main loop of the Riemann solver.
84 do 30 i=2-num_ghost,num_cells+num_ghost
85
86     ! compute Roe-averaged quantities:
87     ubar = (q(2,i-1)/dsqrt(q(1,i-1)) + ql(2,i)/dsqrt(ql(1,i)))/ &
88           ( dsqrt(q(1,i-1)) + dsqrt(ql(1,i)) )
89     cbar=dsqrt(0.5d0*grav*(q(1,i-1) + ql(1,i)))
90
91     ! delta(1)=h(i)-h(i-1) and delta(2)=hu(i)-hu(i-1)
92     delta(1) = ql(1,i) - q(1,i-1)
93     delta(2) = ql(2,i) - q(2,i-1)
94
95     ! Compute coeffs in the evector expansion of delta(1),delta(2)
96     a1 = 0.5d0*(-delta(2) + (ubar + cbar) * delta(1))/cbar

```

```

94      a2 = 0.5d0*( delta(2) - (ubar - cbar) * delta(1))/cbar
95
96      ! Finally, compute the waves.
97      wave(1,1,i) = a1
98      wave(2,1,i) = a1*(ubar - cbar)
99      s(1,i) = ubar - cbar
100
101      wave(1,2,i) = a2
102      wave(2,2,i) = a2*(ubar + cbar)
103      s(2,i) = ubar + cbar
104
105  30 enddo
106
107  ! Compute fluctuations amdq and apdq
108  ! -----
109
110  if (efix) go to 110
111
112  ! No entropy fix
113  ! -----
114  ! amdq = SUM s*wave over left-going waves
115  ! apdq = SUM s*wave over right-going waves
116
117  do m=1,2
118      do i=2-num_ghost, num_cells+num_ghost
119          amdq(m,i) = 0.d0
120          apdq(m,i) = 0.d0
121          do mw=1,num_waves
122              if (s(mw,i) < 0.d0) then
123                  amdq(m,i) = amdq(m,i) + s(mw,i)*wave(m,mw,i)
124              else
125                  apdq(m,i) = apdq(m,i) + s(mw,i)*wave(m,mw,i)
126              endif
127          enddo
128      enddo
129  enddo
130
131  ! with no entropy fix we are done...
132  return
133
134  ! -----
135
136  110 continue
137
138  ! With entropy fix
139  ! -----
140
141  ! compute flux differences amdq and apdq.
142  ! First compute amdq as sum of s*wave for left going waves.
143  ! Incorporate entropy fix by adding a modified fraction of wave
144  ! if s should change sign.
145
146  do 200 i=2-num_ghost,num_cells+num_ghost
147
148      ! -----
149      ! check 1-wave:
150      ! -----
151

```

```

152      ! u-c in left state (cell i-1)
153      s0 = qr(2,i-1)/qr(1,i-1) - dsqrt(grav*qr(1,i-1))
154
155
156      ! check for fully supersonic case:
157      if (s0 >= 0.d0 .and. s(1,i) > 0.d0) then
158          ! everything is right-going
159          do m=1,2
160              amdq(m,i) = 0.d0
161          enddo
162          go to 200
163      endif
164
165      ! u-c to right of 1-wave
166      hr1 = qr(1,i-1) + wave(1,1,i)
167      uhr1 = qr(2,i-1) + wave(2,1,i)
168      s1 = uhr1/hr1 - dsqrt(grav*hr1)
169
170      if (s0 < 0.d0 .and. s1 > 0.d0) then
171          ! transonic rarefaction in the 1-wave
172          sfract = s0 * (s1-s(1,i)) / (s1-s0)
173      else if (s(1,i) < 0.d0) then
174          ! 1-wave is leftgoing
175          sfract = s(1,i)
176      else
177          ! 1-wave is rightgoing
178          sfract = 0.d0    !# this shouldn't happen since s0 < 0
179      endif
180
181      do m=1,2
182          amdq(m,i) = sfract*wave(m,1,i)
183      enddo
184
185      ! -----
186      ! check 2-wave:
187      ! -----
188      ! u+c in right state (cell i)
189      s3 = ql(2,i)/ql(1,i) + dsqrt(grav*ql(1,i))
190
191      ! u+c to left of 2-wave
192      hl2 = ql(1,i) - wave(1,2,i)
193      uhl2 = ql(2,i) - wave(2,2,i)
194      s2 = uhl2/hl2 + dsqrt(grav*hl2)
195
196      if (s2 < 0.d0 .and. s3 > 0.d0) then
197          ! transonic rarefaction in the 2-wave
198          sfract = s2 * (s3-s(2,i)) / (s3-s2)
199      else if (s(2,i) < 0.d0) then
200          ! 2-wave is leftgoing
201          sfract = s(2,i)
202      else
203          ! 2-wave is rightgoing
204          go to 200
205      endif
206
207      do m=1,2
208          amdq(m,i) = amdq(m,i) + sfract*wave(m,2,i)
209      enddo

```

```

210
211 200 enddo
212
213 ! compute the rightgoing flux differences:
214 ! df = SUM s*wave    is the total flux difference and apdq = df - amdq
215
216 do m=1,2
217     do i = 2-num_ghost, num_cells+num_ghost
218         df = 0.d0
219         do mw=1,num_waves
220             df = df + s(mw,i)*wave(m,mw,i)
221         enddo
222         apdq(m,i) = df - amdq(m,i)
223     enddo
224 enddo
225
226 return
227
228 end subroutine rp1

```

6.3.5 Implementation in Pyclaw

Here is how the Roe solver is implemented in Pyclaw (with no sonic entropy fix).

```

229 def shallow_roe_1D(q_l, q_r, aux_l, aux_r, problem_data):
230     r"""
231     Roe shallow water solver in 1d::
232     """
233     # Array shapes
234     num_rp = q_l.shape[1]
235
236     # Output arrays
237     wave = np.empty( (num_eqn, num_waves, num_rp) )
238     s = np.zeros( (num_waves, num_rp) )
239     amdq = np.zeros( (num_eqn, num_rp) )
240     apdq = np.zeros( (num_eqn, num_rp) )
241
242     # Compute roe-averaged quantities
243     ubar = ( (q_l[1,:]/np.sqrt(q_l[0,:]) + q_r[1,:]/np.sqrt(q_r[0,:])) /
244             (np.sqrt(q_l[0,:]) + np.sqrt(q_r[0,:])) )
245     cbar = np.sqrt(0.5 * problem_data['grav'] * (q_l[0,:] + q_r[0,:]))
246
247     # Compute Flux structure
248     delta = q_r - q_l
249     a1 = 0.5 * (-delta[1,:] + (ubar + cbar) * delta[0,:]) / cbar
250     a2 = 0.5 * ( delta[1,:] - (ubar - cbar) * delta[0,:]) / cbar
251
252     # Compute each family of waves
253     wave[0,0,:] = a1
254     wave[1,0,:] = a1 * (ubar - cbar)
255     s[0,:] = ubar - cbar
256
257     wave[0,1,:] = a2
258     wave[1,1,:] = a2 * (ubar + cbar)
259     s[1,:] = ubar + cbar
260
261     s_index = np.zeros((2,num_rp))

```

```

262     for m in range(num_eqn):
263         for mw in range(num_waves):
264             s_index[0,:] = s[mw,:]
265             amdq[m,:] += np.min(s_index,axis=0) * wave[m,mw,:]
266             apdq[m,:] += np.max(s_index,axis=0) * wave[m,mw,:]
267
268     return wave, s, amdq, apdq

```

6.4 HLL solver

6.4.1 Principle

The HLL method is a two-wave solver that considers that the solution to the Riemann problem consists of two shock waves separating the intermediate state Q_* from the left and right initial states Q_{i-1} and Q_i . This part is entirely similar to the Harten–Lax–van-Leer (HLL) method seen in § 3.6. We compute the wave speeds using Einfeldt’s method rather than the original definition in the HLL method (Einfeldt, 1988). For this reason, the algorithm is called HLLC.

In § 3.6, we have seen that this intermediate state and the associated flux can be determined by solving the Rankine–Hugoniot equations for the discontinuities across the two shock waves. Here we provide another proof based on volume integrals.

Integrating the shallow water equations (6.3) over the domain $[x_1, x_2] \times [0, \Delta t]$ in the $x - t$ plane (see Fig. 3.5) gives:

$$\int_{x_1}^{x_2} q(x, \Delta t) dx = \int_{x_1}^{x_2} q(x, 0) dx + \int_0^{\Delta t} f(q(x_1, t)) dt - \int_0^{\Delta t} f(q(x_2, t)) dt,$$

where $x_1 = s_1 \Delta t$ and $x_2 = s_2 \Delta t$. Since $q(x, 0)$ is fixed by the initial conditions, we deduce:

$$Q_* = \frac{1}{x_2 - x_1} \int_{x_1}^{x_2} q(x, \Delta t) dx = \frac{Q_i x_2 - Q_{i-1} x_1}{x_2 - x_1} - \frac{\Delta t}{x_2 - x_1} (F_i - F_{i-1}),$$

where

$$F_{i-1} = \frac{1}{\Delta t} \int_0^{\Delta t} f(q(x_1, t)) dt \text{ and } F_i = \frac{1}{\Delta t} \int_0^{\Delta t} f(Q(x_2, t)) dt.$$

We find that the intermediate state is:

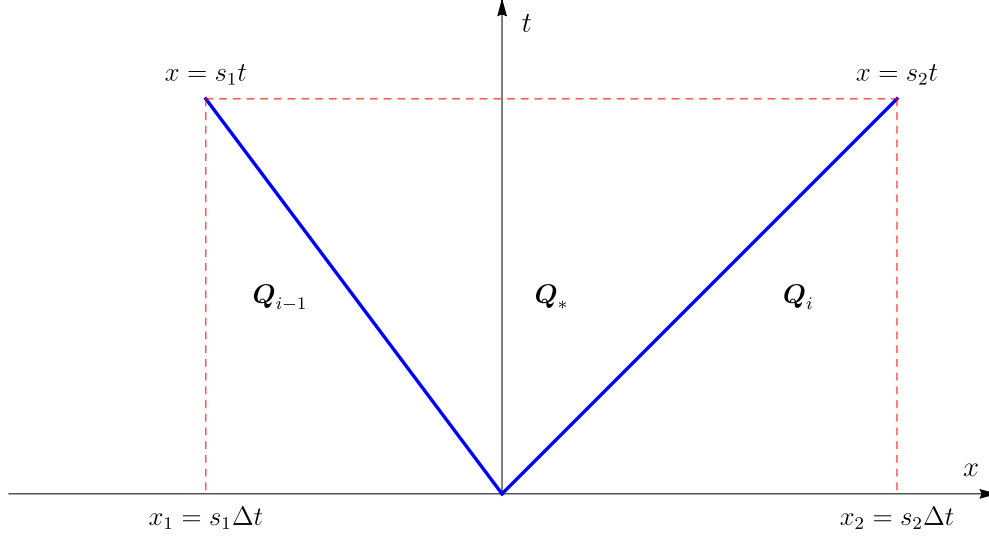
$$Q_* = \frac{Q_i s_2 - Q_{i-1} s_1}{s_2 - s_1} - \frac{F_i - F_{i-1}}{s_2 - s_1}. \quad (6.32)$$

In § 3.3.3, we derived the general expression (3.34) for computing the interface flux from the left and right flux

$$F_{i-1/2} = f(Q_{i-1}) + \frac{\delta x}{\delta t} Q_{i-1} - \frac{1}{\delta t} \int_{-\delta x}^0 Q(x, \delta t) dx.$$

which gives us when we take $\delta x = -x_2 = -s_2 \Delta t$:

$$F_* = \frac{s_2 F_{i-1} - s_1 F_i}{s_2 - s_1} - s_2 s_1 \frac{Q_{i-1} - Q_i}{s_2 - s_1}. \quad (6.33)$$



This expression of the flux holds when the two shock waves fan out on either side of $x = 0$. In that case, the interface flux is defined as $F_{i-1/2} = F_*$. If both shock waves go to the right (i.e., if $s_1 > 0$) then $F_{i-1/2} = F_{i-1}$. In the opposite case, then $F_{i-1/2} = F_i$:

$$F_{i-1/2} = \begin{cases} F_{i-1} & \text{if } s_1 > 0, \\ F_* & \text{if } s_1 \geq 0 \geq s_2, \\ F_i & \text{if } s_2 < 0, \end{cases} \quad (6.34)$$

The last problem to be settled is the determination of the shock speed s_1 and s_2 . We use Einfeldt's suggestion for computing these velocity. Let us first consider the 1-wave. If this wave is a rarefaction wave, its speeds ranges from $\lambda_1(Q_{i-1})$ to $\lambda_1(Q_i) = u_{i-1} - \sqrt{gh_{i-1}}$; we select the minimum value $\lambda_1(Q_{i-1})$. If it is a shock, its speed can be estimated using the Roe matrix (6.22): $s_1 = \hat{u} - \hat{c}$. As we do not know whether the 1-wave is a shock or rarefaction wave, we take the lower bound:

$$s_1 = \min(u_{i-1} - \sqrt{gh_{i-1}}, \hat{u} - \hat{c}). \quad (6.35)$$

The same applies for the 2-wave. We define s_2 as the upper bound:

$$s_2 = \max(u_i + \sqrt{gh_i}, \hat{u} + \hat{c}). \quad (6.36)$$

In short, we compute the Roe averages, and deduce the shock speeds (6.35) and (6.36). The intermediate state is given by Eq (6.32). The waves are:

$$W_1 = Q_* - Q_{i-1} \text{ and } W_2 = Q_i - Q_*. \quad (6.37)$$

The fluctuations are then:

$$A^- \cdot \Delta Q_{i-1/2} = s_1 W_1,$$

$$A^+ \cdot \Delta Q_{i-1/2} = s_1 W_2.$$

6.4.2 Implementation in Pyclaw

```

269 def shallow_hll_1D(q_l,q_r,aux_l,aux_r,problem_data):
270     r"""
271     HLL shallow water solver ::
272
273
274     W_1 = Q_hat - Q_l      s_1 = min(u_l-c_l,u_l+c_l,lambda_roe_1,
lambda_roe_2)
275     W_2 = Q_r - Q_hat      s_2 = max(u_r-c_r,u_r+c_r,lambda_roe_1,
lambda_roe_2)
276
277     Q_hat = ( f(q_r) - f(q_l) - s_2 * q_r + s_1 * q_l ) / (s_1 - s_2)
278
279     *problem_data* should contain:
280     - *g* - (float) Gravitational constant
281
282     :Version: 1.0 (2009-02-05)
283     """
284     # Array shapes
285     num_rp = q_l.shape[1]
286     num_eqn = 2
287     num_waves = 2
288
289     # Output arrays
290     wave = np.empty( (num_eqn, num_waves, num_rp) )
291     s = np.empty( (num_waves, num_rp) )
292     amdq = np.zeros( (num_eqn, num_rp) )
293     apdq = np.zeros( (num_eqn, num_rp) )
294
295     # Compute Roe and right and left speeds
296     ubar = ( (q_l[1,:]/np.sqrt(q_l[0,:]) + q_r[1,:]/np.sqrt(q_r[0,:])) /
297             (np.sqrt(q_l[0,:]) + np.sqrt(q_r[0,:])) )
298     cbar = np.sqrt(0.5 * problem_data['grav'] * (q_l[0,:] + q_r[0,:]))
299     u_r = q_r[1,:] / q_r[0,:]
300     c_r = np.sqrt(problem_data['grav'] * q_r[0,:])
301     u_l = q_l[1,:] / q_l[0,:]
302     c_l = np.sqrt(problem_data['grav'] * q_l[0,:])
303
304     # Compute Einfeldt speeds
305     s_index = np.empty((4,num_rp))
306     s_index[0,:] = ubar+cbar
307     s_index[1,:] = ubar-cbar
308     s_index[2,:] = u_l + c_l
309     s_index[3,:] = u_l - c_l
310     s[0,:] = np.min(s_index,axis=0)
311     s_index[2,:] = u_r + c_r
312     s_index[3,:] = u_r - c_r
313     s[1,:] = np.max(s_index,axis=0)
314
315     # Compute middle state
316     q_hat = np.empty((2,num_rp))
317     q_hat[0,:] = ((q_r[1,:] - q_l[1,:] - s[1,:] * q_r[0,:])
318                 + s[0,:] * q_l[0,:]) / (s[0,:] - s[1,:])
319     q_hat[1,:] = ((q_r[1,:]**2/q_r[0,:] + 0.5 * problem_data['grav'] * q_r
320 [0,:]**2
321                 - (q_l[1,:]**2/q_l[0,:] + 0.5 * problem_data['grav'] * q_l
[0,:]**2)
322                 - s[1,:] * q_r[1,:] + s[0,:] * q_l[1,:]) / (s[0,:] - s
[1,:]))

```



```

322
323 # Compute each family of waves
324 wave[:,0,:] = q_hat - q_l
325 wave[:,1,:] = q_r - q_hat
326
327 # Compute variations
328 s_index = np.zeros((2,num_rp))
329 for m in range(num_eqn):
330     for mw in range(num_waves):
331         s_index[0,:] = s[mw,:]
332         amdq[m,:] += np.min(s_index,axis=0) * wave[m,mw,:]
333         apdq[m,:] += np.max(s_index,axis=0) * wave[m,mw,:]
334
335 return wave, s, amdq, apdq

```

6.5 f-wave formulation

6.5.1 Principle

The f-wave method consists of decomposing the flux jump into f-waves (see §):

$$f(Q_i) - f(Q_{i-1}) = \sum_{k=1}^{m_w} Z_{k,i-1/2},$$

where the f-wave $Z_{k,i-1/2}$ can be related to the right eigenvector $\hat{w}_{k,i-1/2}$ of the Roe matrix:

$$Z_{k,i-1/2} = \beta_{k,i-1/2} \hat{w}_{k,i-1/2}$$

where the coefficient $\beta_{k,i-1/2}$ is the linear solution (see § 6.5.1):

$$\beta_{i-1/2} = L \cdot (f(Q_i) - f(Q_{i-1})).$$

with $L = R^{-1}$. We find that

$$\beta_{i-1/2} = \frac{1}{2\hat{c}} \begin{pmatrix} \Phi_l - \phi_r + (\hat{u} + \hat{c})(q_r - q_l) \\ \Phi_r - \phi_l - (\hat{u} - \hat{c})(q_r - q_l) \end{pmatrix}, \quad (6.38)$$

where Φ is the shorthand notation: $\Phi = u^2 h + gh^2/2$. The f-waves are then:

$$Z_{1,i-1/2} = \beta_{1,i-1/2} w_1 = \frac{\Phi_l - \Phi_r + (\hat{u} + \hat{c})(q_r - q_l)}{2\hat{c}} \begin{pmatrix} 1 \\ \hat{u} - \hat{c} \end{pmatrix} \quad (6.39)$$

and

$$Z_{2,i-1/2} = \beta_{2,i-1/2} w_2 = \frac{\Phi_r - \Phi_l - (\hat{u} - \hat{c})(q_r - q_l)}{2\hat{c}} \begin{pmatrix} 1 \\ \hat{u} + \hat{c} \end{pmatrix}. \quad (6.40)$$

6.5.2 Implementation in Pyclaw

```

336 def shallow_water_fwave_1d(q_l, q_r, aux_l, aux_r, problem_data):
337     r"""Shallow water Riemann solver using fwaves
338
339     *problem_data* should contain:

```

```

340     - *grav* - (float) Gravitational constant
341     - *dry_tolerance* - (float) Set velocities to zero if h is below this
342       tolerance.
343     """
344
345     g = problem_data['grav']
346     dry_tolerance = problem_data['dry_tolerance']
347
348
349     num_rp = q_l.shape[1]
350     num_eqn = 2
351     num_waves = 2
352
353     # initializing f-waves
354     fwave = np.empty( (num_eqn, num_waves, num_rp) )
355     # right eigenvectors
356     r1 = np.empty( (num_waves, num_rp) )
357     r2 = np.empty( (num_waves, num_rp) )
358     # initializing fluctuations and shock speeds
359     amdq = np.zeros( (num_eqn, num_rp) )
360     apdq = np.zeros( (num_eqn, num_rp) )
361     s = np.empty( (num_waves, num_rp) )
362
363
364     # Extract state
365     hl = q_l[0, :]
366     ql = q_l[1, :]
367     ul = np.where(hl > dry_tolerance, ql/hl , 0.0)
368     hr = q_r[0, :]
369     qr = q_r[1, :]
370     ur = np.where(hr > dry_tolerance, qr/hr, 0.0)
371
372     phi_l = hl * ul**2 + 0.5 * g * hl**2
373     phi_r = hr * ur**2 + 0.5 * g * hr**2
374     h_bar = 0.5 * (hr + hl)
375
376     # Speeds
377     u_hat = (np.sqrt(hl) * ul + np.sqrt(hr) * ur) / (np.sqrt(hl) + np.sqrt(
hr) )
378     c_hat = np.sqrt(g * h_bar)
379     lambda1 = u_hat - c_hat
380     lambda2 = u_hat + c_hat
381
382
383     beta1 = (phi_l - phi_r + lambda2*(qr-ql))/2/c_hat
384     beta2 = (phi_r - phi_l - lambda1*(qr-ql))/2/c_hat
385
386
387
388     r1[0, :] = 1.
389     r1[1, :] = u_hat - c_hat
390     r2[0, :] = 1.
391     r2[1, :] = u_hat + c_hat
392
393     s[0, :] = u_hat - c_hat
394     s[1, :] = u_hat + c_hat
395
396     # 1st f-wave

```

```

397     fwave[0,0,:] = beta1*r1[0,:]
398     fwave[1,0,:] = beta1*r1[1,:]
399     # 2nd f-wave
400     fwave[0,1,:] = beta2*r2[0,:]
401     fwave[1,1,:] = beta2*r2[1,:]
402
403     for m in range(num_eqn):
404         for mw in range(num_waves):
405             amdq[m, :] += (s[mw, :] < 0.0) * fwave[m, mw, :]
406             apdq[m, :] += (s[mw, :] > 0.0) * fwave[m, mw, :]
407
408             amdq[m, :] += (s[mw, :] == 0.0) * fwave[m, mw, :] * 0.5
409             apdq[m, :] += (s[mw, :] == 0.0) * fwave[m, mw, :] * 0.5
410
411     return fwave, s, amdq, apdq

```

6.6 Example: dam break

We consider a dam break problem with the following initial conditions:

- $h_l = 10$ m et $u_l = 0$ for $x \leq 0$, and
- $h_r = 0.5$ m et $u_r = 0$ for $x > 0$.

We compare the three solvers: Roe (with or without the entropy fix), the HLLE solver, and the f-wave formulation. Figures 6.11 and 6.12 show the comparison.

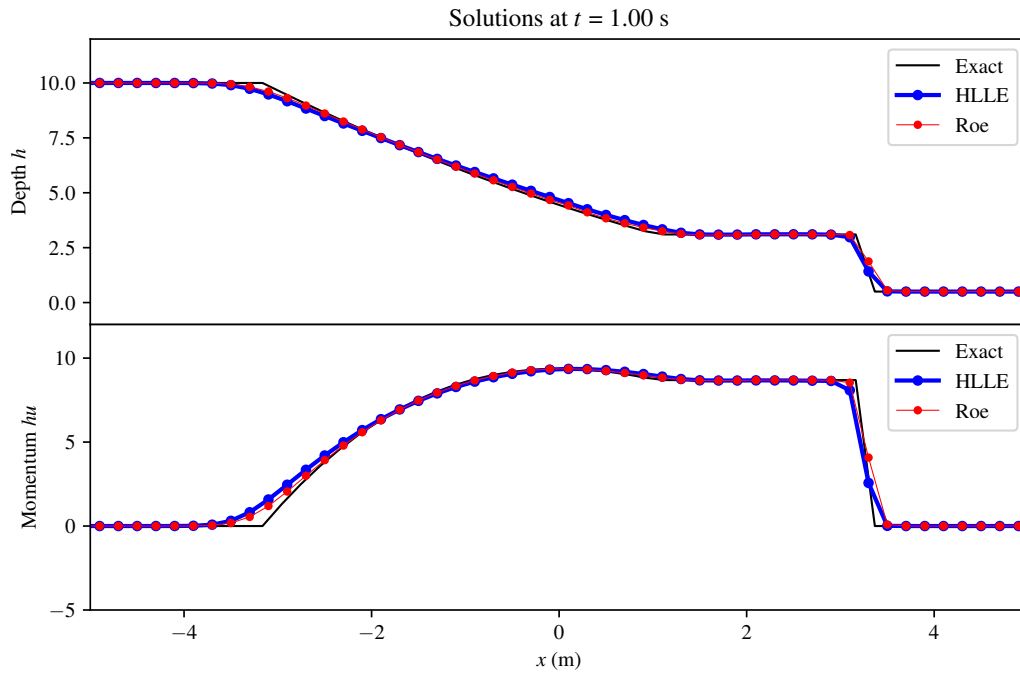


Figure 6.11 Comparison between the analytical solution, the Roe (with entropy fix) and HLLE solution.

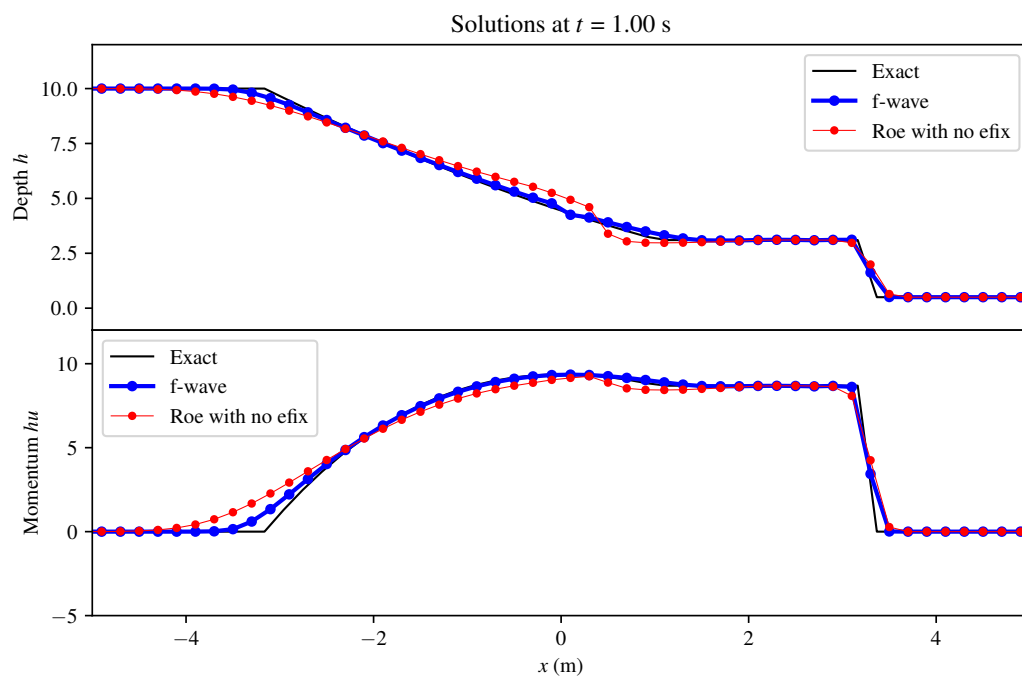


Figure 6.12 Comparison between the analytical solution, the Roe (with no entropy fix) and f-wave solution.

Solving the Saint-Venant equations with scalar transport

7.1 Theory

Let us consider the shallow water equations seen in Chap. 2, supplemented with an equation representing the advection of a scalar quantity $\phi(x, t)$ (for instance, the concentration of a pollutant that does not interplay with the water flow):

$$\frac{\partial h}{\partial t} + \frac{\partial q}{\partial x} = 0, \quad (7.1)$$

$$\frac{\partial q}{\partial t} + \frac{\partial hu^2}{\partial x} + gh \frac{\partial h}{\partial x} = 0, \quad (7.2)$$

$$\frac{\partial h\phi}{\partial t} + \frac{\partial q\phi}{\partial x} = 0, \quad (7.3)$$

where h denotes the flow depth, $q = hu$ is the flow rate, and u the depth-averaged velocity. where g is gravitational acceleration, and the unknowns are q , h and ϕ . In a matrix form, Eqs. (7.1)-(7.3) takes the form:

$$\frac{\partial}{\partial t} Q + \frac{\partial}{\partial x} f(Q) = 0, \quad (7.4)$$

where

$$f = \begin{pmatrix} q \\ q^2/h + gh^2/2 \\ q\phi \end{pmatrix} \text{ and } Q = \begin{pmatrix} h \\ q \\ h\phi \end{pmatrix}. \quad (7.5)$$

The Jacobian is

$$f' = \begin{pmatrix} 0 & 1 & 0 \\ -u^2 + gh & 2u & 0 \\ -u\phi & \phi & u \end{pmatrix}, \quad (7.6)$$

whose eigenvalues are

$$\lambda_1 = u - \sqrt{gh}, \lambda_2 = u \text{ and } \lambda_3 = u + \sqrt{gh}, \quad (7.7)$$

associated with the right eigenvectors:

$$w_1 = \begin{pmatrix} 1 \\ u - \sqrt{gh} \\ \phi \end{pmatrix}, w_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \text{ and } w_3 = \begin{pmatrix} 1 \\ u + \sqrt{gh} \\ \phi \end{pmatrix}. \quad (7.8)$$

The scalar quantity ϕ is decoupled from the water flow, and its speed depends only on the water flow velocity: $\lambda_2 = u$. The associated field is said to be *linearly degenerate* because $\nabla \lambda_2 \cdot w_2 = 0$. This gives

rise to *contact discontinuities*: when ϕ experiences a shock, there is a discontinuity in u , and thus the characteristic speeds are equal on either side of the shock waves (the characteristic curves are parallel to the shock waves). The condition $\nabla \lambda_2 \cdot w_2 = 0$ means that the eigenvalue is unchanged when we move along the integral curve $w_2(\zeta)$.

7.2 Roe solver

7.2.1 Derivation

The Roe solver is close to the version derived in Chap. 2. The only difference lies in the adding of a third wave. We need the following equations to write the Roe solver's algorithm:

- The velocities associated with the intermediate state

$$\hat{u} = \frac{q_{i-1}/\sqrt{h_{i-1}} + q_i/\sqrt{h_i}}{\sqrt{h_{i-1}} + \sqrt{h_i}} \text{ and } \bar{c} = \sqrt{\frac{1}{2}(h_{i-1} + h_i)}. \quad (7.9)$$

- The waves W_k :

$$W_k = \alpha_k w_k, \quad k = 1, 3 \quad (7.10)$$

where α_k are the components of the α vector obtained by inverting the matrix R

$$\alpha = R^{-1} \cdot \Delta Q = \frac{1}{2\bar{c}} \begin{pmatrix} (\hat{u} + \bar{c})\Delta Q_1 - \Delta Q_2 \\ \Delta Q_3 - \phi \Delta Q_1 \\ (-\hat{u} + \bar{c})\Delta Q_1 + \Delta Q_2 \end{pmatrix} \quad (7.11)$$

where $\bar{c} = \sqrt{gh}$ and $\Delta Q = (\Delta Q_1, \Delta Q_2, \Delta Q_3)$. For the second wave, we impose that there is no jump ΔQ_1 associated with the contact discontinuity, and thus we impose

$$\alpha_2 = \Delta Q_3.$$

- the characteristic speeds

$$s_1 = \hat{u} - \bar{c}, \quad s_2 = \hat{u} \text{ and } s_3 = \hat{u} + \bar{c}. \quad (7.12)$$

- The fluctuations are

$$A^+ \cdot \Delta Q_{i-1/2} = \sum_{k=1}^3 \min(\lambda_{i-1/2}^k, 0) W_{k,i-1/2},$$

$$A^- \cdot \Delta Q_{i+1/2} = \sum_{k=1}^3 \max(\lambda_{i+1/2}^k, 0) W_{k,i+1/2},$$

which gives in the present context:

- if $s_k > 0$, then $\text{amdq}(\mathbf{m}, \mathbf{i}) = \mathbf{s} * \text{wave}$.
- if $s_k < 0$, then $\text{apdq}(\mathbf{m}, \mathbf{i}) = \mathbf{s} * \text{wave}$.

7.2.2 Implementation in Clawpack

```

412 subroutine rp1(maxmx,num_eqn,num_waves,num_aux,num_ghost,num_cells, &
413               ql,qr,auxl,auxr,wave,s,amdq,apdq)
414
415 ! Solve Riemann problems for the 1D shallow water equations
416 ! with an additional passively advected tracer:
417 !   (h)_t + (u h)_x = 0
418 !   (uh)_t + ( uuh + .5*gh^2 )_x = 0
419 !   c_t + uc_x = 0
420 ! using Roe's approximate Riemann solver with entropy fix for
421 ! transonic rarefactions.
422
423 ! waves: 3
424 ! equations: 3
425
426 ! Conserved quantities:
427 !     1 depth
428 !     2 momentum
429 !     3 tracer
430
431 ! See http://www.clawpack.org/riemann.html for a detailed explanation
432 ! of the Riemann solver API.
433
434 implicit none
435
436 integer, intent(in) :: maxmx, num_eqn, num_waves, num_aux, num_ghost, &
437                        num_cells
438 real(kind=8), intent(in), dimension(num_eqn,1-num_ghost:maxmx+num_ghost)
439 :: ql, qr
440 real(kind=8), intent(in), dimension(num_aux,1-num_ghost:maxmx+num_ghost)
441 :: auxl, auxr
442 real(kind=8), intent(out) :: s(num_waves, 1-num_ghost:maxmx+num_ghost)
443 real(kind=8), intent(out) :: wave(num_eqn, num_waves, 1-num_ghost:maxmx+
444 num_ghost)
445 real(kind=8), intent(out), dimension(num_eqn,1-num_ghost:maxmx+
446 num_ghost) :: amdq,apdq
447
448 ! local variables:
449 real(kind=8) :: a1,a2,ubar,cbar,s0,s1,s2,s3,hr1,uhr1,hl2,uhl2,sfract,df
450 real(kind=8) :: delta(2)
451 integer :: i,m,mw
452 logical :: efix
453
454 data efix /.true./      ! Use entropy fix for transonic rarefactions
455
456 ! Gravity constant set in setprob.f or the shallow1D.py file
457 real(kind=8) :: grav
458 common /cparam/ grav
459
460 ! Main loop of the Riemann solver.
461 do 30 i=2-num_ghost,num_cells+num_ghost
462
463     ! compute Roe-averaged quantities:
464     ubar = (qr(2,i-1)/dsqrt(qr(1,i-1)) + ql(2,i)/dsqrt(ql(1,i)))/ &
465            ( dsqrt(qr(1,i-1)) + dsqrt(ql(1,i)) )
466     cbar=dsqrt(0.5d0*grav*(qr(1,i-1) + ql(1,i)))

```



```

464      ! delta(1)=h(i)-h(i-1) and delta(2)=hu(i)-hu(i-1)
465      delta(1) = ql(1,i) - qr(1,i-1)
466      delta(2) = ql(2,i) - qr(2,i-1)
467
468      ! Compute coeffs in the evector expansion of delta(1),delta(2)
469      a1 = 0.5d0*(-delta(2) + (ubar + cbar) * delta(1))/cbar
470      a2 = 0.5d0*( delta(2) - (ubar - cbar) * delta(1))/cbar
471
472      ! Finally, compute the waves.
473      wave(1,1,i) = a1
474      wave(2,1,i) = a1*(ubar - cbar)
475      wave(3,1,i) = 0.d0
476      s(1,i) = ubar - cbar
477
478      wave(1,2,i) = a2
479      wave(2,2,i) = a2*(ubar + cbar)
480      wave(3,2,i) = 0.d0
481      s(2,i) = ubar + cbar
482
483      wave(1,3,i) = 0.d0
484      wave(2,3,i) = 0.d0
485      wave(3,3,i) = ql(3,i) - qr(3,i-1)
486      s(3,i) = ubar
487
488      30 enddo
489
490      ! Compute fluctuations amdq and apdq
491      ! -----
492
493      if (efix) go to 110
494
495      ! No entropy fix
496      ! -----
497      ! amdq = SUM s*wave    over left-going waves
498      ! apdq = SUM s*wave    over right-going waves
499
500      do m=1,num_waves
501          do i=2-num_ghost, num_cells+num_ghost
502              amdq(m,i) = 0.d0
503              apdq(m,i) = 0.d0
504              do mw=1,num_waves
505                  if (s(mw,i) < 0.d0) then
506                      amdq(m,i) = amdq(m,i) + s(mw,i)*wave(m,mw,i)
507                  else
508                      apdq(m,i) = apdq(m,i) + s(mw,i)*wave(m,mw,i)
509                  endif
510              enddo
511          enddo
512      enddo
513
514      ! with no entropy fix we are done...
515      return
516
517      ! -----
518      110 continue
519
520      ! compute the rightgoing flux differences:
521      ! df = SUM s*wave    is the total flux difference and apdq = df - amdq

```

```

522
523 do i = 2-num_ghost, num_cells+num_ghost
524     do m=1,2
525         df = 0.d0
526         do mw=1,2
527             df = df + s(mw,i)*wave(m,mw,i)
528         enddo
529         apdq(m,i) = df - amdq(m,i)
530     enddo
531
532     ! tracer (which is in non-conservation form)
533     if (s(3,i) < 0) then
534         amdq(m,i) = amdq(m,i) + s(3,i)*wave(m,3,i)
535     else
536         apdq(m,i) = apdq(m,i) + s(3,i)*wave(m,3,i)
537     endif
538
539 enddo
540
541 return
542
543 end subroutine rp1

```

7.2.3 Implementation in Pyclaw

```

544 def shallow_roe_1D(q_l, q_r, aux_l, aux_r, problem_data):
545     r"""
546     Roe shallow water solver in 1d
547     """
548
549     # Array shapes
550     num_rp = q_l.shape[1]
551     num_eqn = 3
552     num_waves = 3
553
554     g = problem_data['grav']
555
556     # Output arrays
557     wave = np.empty( (num_eqn, num_waves, num_rp) )
558     s = np.zeros( (num_waves, num_rp) )
559     amdq = np.zeros( (num_eqn, num_rp) )
560     apdq = np.zeros( (num_eqn, num_rp) )
561
562     # Compute roe-averaged quantities
563     ubar = ( (q_l[1,:]/np.sqrt(q_l[0,:]) + q_r[1,:]/np.sqrt(q_r[0,:])) /
564             (np.sqrt(q_l[0,:]) + np.sqrt(q_r[0,:])) )
565     cbar = np.sqrt(0.5 * g * (q_l[0,:] + q_r[0,:]))
566
567     # Compute Flux structure
568     delta = q_r - q_l
569     delta1 = q_r[0,:] - q_l[0,:]
570     delta2 = q_r[1,:] - q_l[1,:]
571     alpha1 = 0.5 * (-delta2 + (ubar + cbar) * delta1) / cbar
572     alpha2 = 0.5 * ( delta2 - (ubar - cbar) * delta1) / cbar
573
574     # Compute each family of waves

```

```

575 wave[0,0,:] = alpha1
576 wave[1,0,:] = alpha1 * (ubar - cbar)
577 wave[2,0,:] = 0.
578 s[0,:]      = ubar - cbar
579
580 wave[0,2,:] = alpha2
581 wave[1,2,:] = alpha2 * (ubar + cbar)
582 wave[2,2,:] = 0.
583 s[2,:]      = ubar + cbar
584
585 wave[0,1,:] = 0.
586 wave[1,1,:] = 0.
587 wave[2,1,:] = q_r[2,:] - q_l[2,:]
588 s[1,:]      = ubar
589
590 s_index = np.zeros((3,num_rp))
591 for m in range(num_eqn):
592     for mw in range(num_waves):
593         s_index[0,:] = s[mw,:]
594         amdq[m,:] += np.min(s_index,axis=0) * wave[m,mw,:]
595         apdq[m,:] += np.max(s_index,axis=0) * wave[m,mw,:]
596
597
598 return wave, s, amdq, apdq

```

7.3 HLLC Solver

7.3.1 Principle

The HLLC solver is an extension of the HLL scheme proposed by Eleuterio Toro (Toro, 2001) to cope with the existence of a contact discontinuity. The HLL solver defines an intermediate state separating the left and right initial states. The HLLC introduces two distinct intermediate states split by the second characteristic $x = \lambda_2 t$ (see Fig. 7.1). The fluxes associated with the two intermediate states are defined using the Rankine-Hugoniot equation:

$$F_{*,l} - F_l = s_1(Q_{*,l} - Q_l), \quad (7.13)$$

$$F_{*,r} - F_r = s_3(Q_{*,r} - Q_r) \quad (7.14)$$

where $F_{*,l} = f(Q_{*,l})$ and $F_{*,r} = f(Q_{*,r})$.

In the absence of the advection equation (7.3), there would be only one intermediate state. As tracer advection does not interplay with water flow, we impose that the first two components (those associated with the water flow) of $F_{*,r}$ and $F_{*,l}$ are identical:

$$F_{*,l,1} = F_{*,r,1} = \frac{\lambda_3 F_{l,1} - \lambda_1 F_{r,1}}{\lambda_3 - \lambda_1} - \lambda_1 \lambda_3 \frac{\lambda_3 - \lambda_1}{h_l - h_r}, \quad (7.15)$$

$$F_{*,l,2} = F_{*,r,2} = \frac{\lambda_3 F_{l,2} - \lambda_1 F_{r,2}}{\lambda_3 - \lambda_1} - \lambda_1 \lambda_3 \frac{\lambda_3 - \lambda_1}{h_l u_l - h_r u_r}, \quad (7.16)$$

where $F_1 = hu$ and $F_2 = hu^2 + gh^2/2$. For the third component, we impose that there is no jump across the 1- and 3-characteristics. The only jump in $hu\phi$ is across $x = \lambda_2 t$. Because the third component $hu\phi$ is the product of ϕ and the first F component hu , then we can write

$$F_{*,l,3} = F_{*,l,1}\phi_l, \quad (7.17)$$

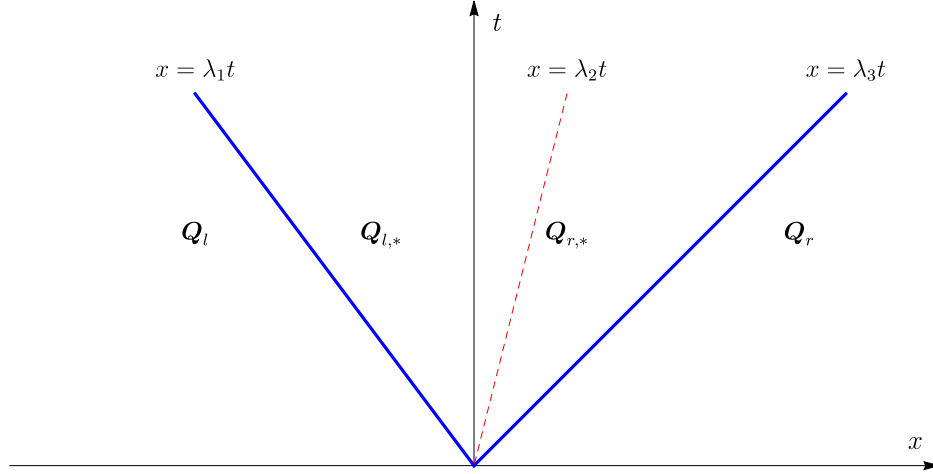


Figure 7.1 The three waves separating the left and right initial states.

$$F_{*,r,3} = F_{*,r,2} \phi_r. \quad (7.18)$$

The flux at the interface $x = 0$ is thus $F_{*,l,3}$ if $\lambda_2 > 0$, and $F_{*,r,3}$ if $\lambda_2 < 0$.

An estimate of the wave speed λ_2 is (Toro, 2001):

$$\lambda_2 = \frac{\lambda_1 h_r (u_r - \lambda_3) - \lambda_3 h_l (u_l - \lambda_1)}{h_r (u_r - \lambda_3) - h_l (u_l - \lambda_1)}. \quad (7.19)$$

We consider three waves

$$W_1 = Q_{l,*} - Q_l, \quad W_2 = Q_{r,*} - Q_{l,*} \text{ and } W_3 = Q_r - Q_{r,*}. \quad (7.20)$$

In § 3.6 and 6.4.1, we have shown that the intermediate state for the water flow is:

$$Q_*^\dagger = \frac{s_3 Q_r^\dagger - s_1 Q_l^\dagger}{s_3 - s_1} - \frac{F_r^\dagger - F_l^\dagger}{s_3 - s_1},$$

where $Q_*^\dagger = (h, hu)$ and $F^\dagger = (q, \Phi)$ (with $q = hu$ and $\Phi = hu^2 + gh^2/2$) are the first two components of Q and F . We thus have

$$h_* = \frac{s_3 h_r - s_1 h_l}{s_3 - s_1} - \frac{s_3 q_r - s_1 q_l}{s_3 - s_1},$$

and

$$q_* = (hu)_* = \frac{s_3 q_r - s_1 q_l}{s_3 - s_1} - \frac{s_3 \Phi_r - s_1 \Phi_l}{s_3 - s_1}.$$

We then deduce:

$$W_1^\dagger = \begin{pmatrix} h_* - h_l \\ q_* - q_l \end{pmatrix}, \quad W_2^\dagger = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \text{ and } W_3^\dagger = \begin{pmatrix} h_r - h_* \\ q_r - q_* \end{pmatrix}.$$

For the third component, we have

$$W_{1,3} = 0, \quad W_{2,3} = \phi_r - \phi_l, \text{ and } W_{3,3} = 0.$$

7.3.2 Implementation in Pyclaw

```

599 def shallow_hllc_1D(q_l,q_r,aux_l,aux_r,problem_data):
600     r"""
601     HLLC shallow water solver ::
602     """
603     # Array shapes
604     num_rp    = q_l.shape[1]
605     num_eqn   = 3
606     num_waves = 3
607
608     g = problem_data['grav']
609
610     # Output arrays
611     wave = np.empty( (num_eqn, num_waves, num_rp) )
612     s     = np.empty( (num_waves, num_rp) )
613     amdq  = np.zeros( (num_eqn, num_rp) )
614     apdq  = np.zeros( (num_eqn, num_rp) )
615
616     h_l   = q_l[0,:]
617     h_r   = q_r[0,:]
618     hu_l  = q_l[1,:]
619     hu_r  = q_r[1,:]
620     u_r   = hu_r/h_r
621     c_r   = np.sqrt(g * h_r)
622     u_l   = hu_l/h_l
623     c_l   = np.sqrt(g * h_l)
624     Phi_l = u_l**2*h_l+0.5*g*h_l**2
625     Phi_r = u_r**2*h_r+0.5*g*h_r**2
626
627     # Compute Roe and right and left speeds
628     u_hat = (hu_l/np.sqrt(h_l) + hu_r/np.sqrt(h_r))/(np.sqrt(h_l) + np.sqrt(h_r))
629     c_hat = np.sqrt(0.5 * g * (h_r + h_l))
630
631     # Compute Einfeldt speeds
632     s_index = np.empty((2,num_rp))
633     s_index[0,:] = u_hat - c_hat
634     s_index[1,:] = u_l - c_l
635     s[0,:] = np.min(s_index,axis=0)
636     s_index[0,:] = u_r + c_r
637     s_index[1,:] = u_hat + c_hat
638     s[2,:] = np.max(s_index,axis=0)
639
640     lambda_1 = u_hat - c_hat
641     lambda_3 = u_hat + c_hat
642     u_toro = (lambda_1*h_r*(u_r-lambda_3) - lambda_3*h_l*(u_l-lambda_1) )
643             \
644             /(h_r*(u_r-lambda_3) - h_l*(u_l-lambda_1))
645     s[1,:] = u_hat
646
647     # Compute middle state
648     h_star = (h_r * s[2,:] - h_l * s[0,:]- (hu_r-hu_l))/(s[2,:]-s[0,:])
649     hu_star = (hu_r * s[2,:] - hu_l * s[0,:]- (Phi_r-Phi_l))/(s[2,:]-s[0,:])
650
651     # Compute each family of waves
652     wave[0,0,:] = h_star - h_l
653     wave[1,0,:] = hu_star - hu_l

```

```

653 wave[2,0,:] = 0.
654 wave[0,1,:] = 0.
655 wave[1,1,:] = 0.
656 wave[2,1,:] = q_r[2,:]-q_l[2,:]
657 wave[0,2,:] = h_r - h_star
658 wave[1,2,:] = hu_r - hu_star
659 wave[2,2,:] = 0.
660
661 # Compute variations
662 s_index = np.zeros((3,num_rp))
663 for m in range(num_eqn):
664     for mw in range(num_waves):
665         s_index[0,:] = s[mw,:]
666         amdq[m,:] += np.min(s_index,axis=0) * wave[m,mw,:]
667         apdq[m,:] += np.max(s_index,axis=0) * wave[m,mw,:]
668
669 return wave, s, amdq, apdq

```

7.4 F-wave formulation

7.4.1 Principle

The f-wave method consists of decomposing the jump in the flux (7.5) into three f-waves

$$f(Q_i) - f(Q_{i-1}) = \sum_{k=1}^3 Z_{k,i-1/2},$$

where the f-wave $Z_{k,i-1/2}$ can be related to the right eigenvector $\hat{w}_{k,i-1/2}$ of the Roe matrix:

$$Z_{k,i-1/2} = \beta_{k,i-1/2} \hat{w}_{k,i-1/2}$$

where the coefficient $\beta_{k,i-1/2}$ is the linear solution (see § 6.5.1):

$$\beta_{i-1/2} = L \cdot (f(Q_i) - f(Q_{i-1})).$$

with $L = R^{-1}$. We find that:

$$\beta_{i-1/2} = \frac{1}{2\hat{c}} \begin{pmatrix} \Phi_l - \Phi_r + (\hat{u} + \hat{c})(q_r - q_l) \\ 2\hat{c}(\phi_r q_r - \phi_l q_l + \phi(q_r - q_l)) \\ \Phi_r - \Phi_l - (\hat{u} - \hat{c})(q_r - q_l) \end{pmatrix}, \quad (7.21)$$

where $\Phi = hu^2 + gh^2/2$. The f-waves are then:

$$Z_{1,i-1/2} = \beta_{1,i-1/2} w_1 = \frac{\Phi_l - \Phi_r + (\hat{u} + \hat{c})(q_r - q_l)}{2\hat{c}} \begin{pmatrix} 1 \\ \hat{u} - \hat{c} \\ \phi \end{pmatrix},$$

$$Z_{2,i-1/2} = \beta_{2,i-1/2} w_2 = (\phi_r q_r - \phi_l q_l - \phi(q_r - q_l)) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

and

$$Z_{3,i-1/2} = \beta_{3,i-1/2} w_3 = \frac{\Phi_r - \Phi_l - (\hat{u} - \hat{c})(q_r - q_l)}{2\hat{c}} \begin{pmatrix} 1 \\ \hat{u} + \hat{c} \\ \phi \end{pmatrix}.$$

As for the Roe solver, we assume that there is no jump in ϕ for the 1- and 3- shock waves while for the 2-wave, there is no jump in h and hu (and so $q_r = q_l = \hat{q} = \hat{u}\bar{h}$), and so the correct f-waves are:

$$Z_{1,i-1/2} = \beta_{1,i-1/2} w_1 = \frac{\Phi_l - \Phi_r + (\hat{u} + \hat{c})(q_r - q_l)}{2\hat{c}} \begin{pmatrix} 1 \\ \hat{u} - \hat{c} \\ 0 \end{pmatrix}, \quad (7.22)$$

$$Z_{2,i-1/2} = \beta_{2,i-1/2} w_2 = (\phi_r - \phi_l) \hat{q} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \quad (7.23)$$

and

$$Z_{3,i-1/2} = \beta_{3,i-1/2} w_3 = \frac{\Phi_r - \Phi_l - (\hat{u} - \hat{c})(q_r - q_l)}{2\hat{c}} \begin{pmatrix} 1 \\ \hat{u} + \hat{c} \\ 0 \end{pmatrix}. \quad (7.24)$$

7.4.2 Implementation in Pyclaw

```

670 def shallow_hllc_fwave_1d(q_l, q_r, aux_l, aux_r, problem_data):
671     r"""Shallow water Riemann solver using fwaves
672     """
673
674     g = problem_data['grav']
675     dry_tolerance = problem_data['dry_tolerance']
676
677     num_rp = q_l.shape[1]
678     num_eqn = 3
679     num_waves = 3
680
681     # Initializing arrays
682     fwave = np.empty( (num_eqn, num_waves, num_rp) )
683     s = np.empty( (num_waves, num_rp) )
684     amdq = np.zeros( (num_eqn, num_rp) )
685     apdq = np.zeros( (num_eqn, num_rp) )
686     r1 = np.zeros( (num_waves, num_rp) )
687     r2 = np.zeros( (num_waves, num_rp) )
688     r3 = np.zeros( (num_waves, num_rp) )
689
690     # Extract state
691     h_l = q_l[0, :]
692     h_r = q_r[0, :]
693     hu_l = q_l[1, :]
694     hu_r = q_r[1, :]
695     u_l = np.where(h_l > dry_tolerance, hu_l / h_l, 0.0)
696     u_r = np.where(h_r > dry_tolerance, hu_r / h_r, 0.0)
697
698     # Flux and Roe depth
699     phi_l = h_l * u_l**2 + 0.5 * g * h_l**2
700     phi_r = h_r * u_r**2 + 0.5 * g * h_r**2
701     h_bar = 0.5 * (h_l + h_r)
702
703     # Speeds
704     u_hat = (np.sqrt(h_l)*u_l + np.sqrt(h_r)*u_r) / (np.sqrt(h_l) + np.sqrt(h_r))
705     c_hat = np.sqrt(g * h_bar)
706

```

```

707     s[0, :] = np.amin(np.vstack((u_l - np.sqrt(g * h_l), u_hat - c_hat)),
708                        axis=0)
709     s[1, :] = u_hat
710     s[2, :] = np.amax(np.vstack((u_r + np.sqrt(g * h_r), u_hat + c_hat)),
711                        axis=0)
712
713     beta1 = (phi_l - phi_r + (u_hat+c_hat)*(hu_r-hu_l))/2./c_hat
714     beta2 = (q_r[2, :]- q_l[2, :])*u_hat
715     beta3 = (phi_r - phi_l - (u_hat-c_hat)*(hu_r-hu_l))/2./c_hat
716
717     r1[0, :] = 1.
718     r1[1, :] = u_hat - c_hat
719     r1[2, :] = 0.
720
721     r2[0, :] = 0.
722     r2[1, :] = 0.
723     r2[2, :] = 1.
724
725     r3[0, :] = 1.
726     r3[1, :] = u_hat + c_hat
727     r3[2, :] = 0.
728
729     fwave[0, 0, :] = beta1 * r1[0, :]
730     fwave[1, 0, :] = beta1 * r1[1, :]
731     fwave[2, 0, :] = beta1 * r1[2, :]
732
733     fwave[0, 1, :] = beta2 * r2[0, :]
734     fwave[1, 1, :] = beta2 * r2[1, :]
735     fwave[2, 1, :] = beta2 * r2[2, :]
736
737     fwave[0, 2, :] = beta3 * r3[0, :]
738     fwave[1, 2, :] = beta3 * r3[1, :]
739     fwave[2, 2, :] = beta3 * r3[2, :]
740
741     for m in range(num_eqn):
742         for mw in range(num_waves):
743             amdq[m, :] += (s[mw, :] < 0.0) * fwave[m, mw, :]
744             apdq[m, :] += (s[mw, :] > 0.0) * fwave[m, mw, :]
745
746     return fwave, s, amdq, apdq

```

7.5 Example: dam break

We consider a dam break problem with the following initial conditions: $h_l = 3$ m et $u_l = 0$ for $x \leq 0$, and $h_l = 1$ m et $u_l = 0$ for $x > 0$. We compare the two solvers: Roe (with no entropy fix) and the f-wave formulation of the HLLC solver. Figures 7.2 shows the comparison.

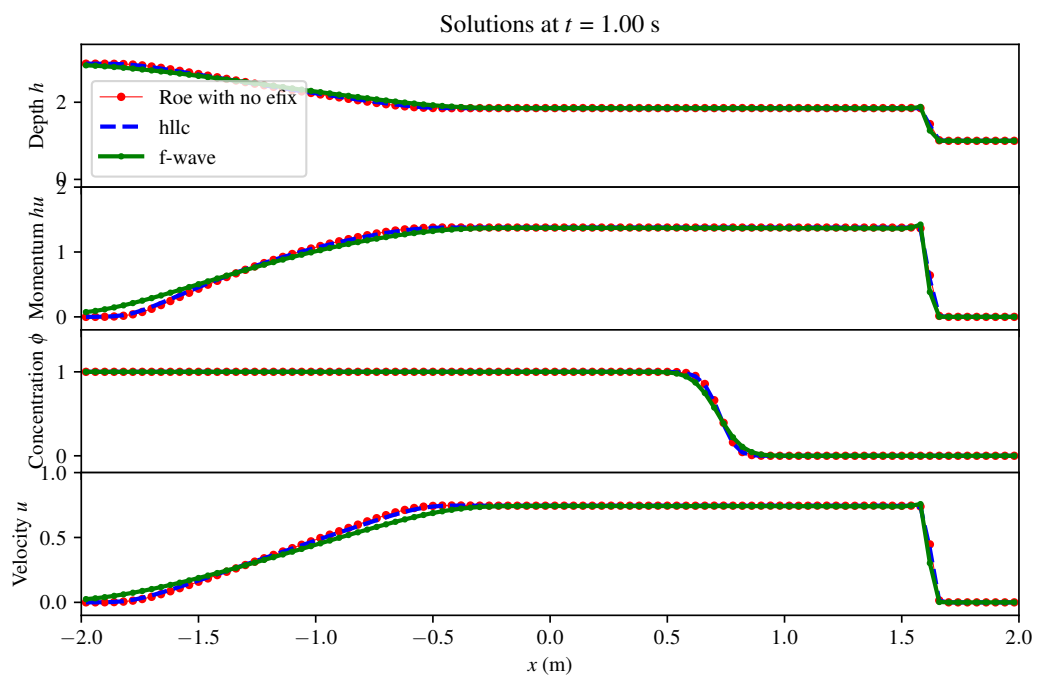


Figure 7.2 Comparison of the three methods: Roe solver, HLLC, and the f-wave variant of the HLLC algorithm. Computations done with $g = 1 \text{ m/s}^2$.

Solving the Saint-Venant equations with a source term

8.1 Dam-break problem on a sloping bed

8.1.1 Governing equations

We consider the Saint-Venant equations for a frictionless on a sloping bed¹ (see Fig. 8.1). The depth-averaged equations take the nonconservative form:

$$\frac{\partial h}{\partial t} + \frac{\partial h\bar{u}}{\partial x} = 0, \quad (8.1)$$

$$\frac{\partial \bar{u}}{\partial t} + \bar{u} \frac{\partial \bar{u}}{\partial x} + g \cos \theta \frac{\partial h}{\partial x} = g \sin \theta, \quad (8.2)$$

where h denotes the flow depth, $\bar{u}$ is the depth-averaged velocity, θ is the slope angle, and g is the gravitational acceleration.

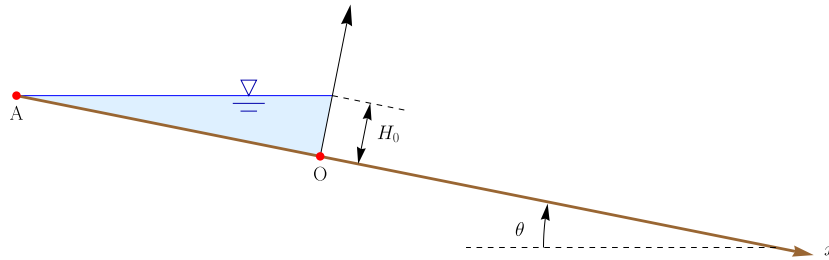


Figure 8.1 The initial flow configuration.

8.1.2 Dimensionless Saint-Venant equations

We make equations (8.1)–(8.2) dimensionless using the following dimensionless variables:

$$\hat{x} = \frac{x}{H_0}, \quad \hat{t} = \frac{t}{T_*}, \quad \hat{u} = \frac{\bar{u}}{U_*}, \quad \text{and} \quad \hat{h} = \frac{h}{H_0}, \quad (8.3)$$

¹I expand the calculations of my WRR paper ([Ancey et al., 2008](#)).

where H_0 is the initial depth at the gate, and the time and velocity scales are defined from it:

$$T_* = \sqrt{\frac{H_0}{g \cos \theta}} \text{ and } U_* = \sqrt{g H_0 \cos \theta}.$$

Let us consider the Saint-Venant equations (8.1)–(8.2) in a dimensionless and nonconservative form:

$$\frac{\partial h}{\partial t} + \frac{\partial h u}{\partial x} = 0, \quad (8.4)$$

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \frac{\partial h}{\partial x} = \tan \theta. \quad (8.5)$$

The hat decoration has been removed.

8.1.3 Characteristic form in the fixed frame

In a characteristic form, these equations are:

$$\frac{d}{dt}(u + 2c) = \tan \theta \text{ along } \frac{dx}{dt} = u + c, \quad (8.6)$$

$$\frac{d}{dt}(u - 2c) = \tan \theta \text{ along } \frac{dx}{dt} = u - c, \quad (8.7)$$

where $c = \sqrt{h}$. Let us integrate these equations and introduce the Riemann variables ρ and σ that label the ρ -characteristics ($\rho = \text{constant}$) and the σ -characteristics ($\sigma = \text{constant}$):

$$u + 2c = \rho + t \tan \theta \text{ along } \frac{dx}{dt} = u + c, \quad (8.8)$$

$$u - 2c = \sigma + t \tan \theta \text{ along } \frac{dx}{dt} = u - c. \quad (8.9)$$

8.1.4 Change of frame

To make the equations homogeneous, we can use the following change of variables (considering a change of frame, with the frame moving with the flow front):

$$\xi = x - \frac{\tan \theta}{2} t^2 \text{ and } v = u - t \tan \theta, \quad (8.10)$$

and thereby obtain:

$$\frac{\partial h}{\partial t} + \frac{\partial h v}{\partial \xi} = 0, \quad (8.11)$$

$$\frac{\partial v}{\partial t} + v \frac{\partial v}{\partial \xi} + \frac{\partial h}{\partial \xi} = 0. \quad (8.12)$$

8.1.5 Characteristic form in the moving frame

In a characteristic form, Eqs. (8.11)–(8.12) lead to:

$$\frac{d}{dt}(v + 2c) = 0 \text{ along } \frac{d\xi}{dt} = v + c, \quad (8.13)$$

$$\frac{d}{dt}(v - 2c) = 0 \text{ along } \frac{d\xi}{dt} = v - c. \quad (8.14)$$

Let us introduce the Riemann variables:

$$r = v + 2c \text{ and } s = v - 2c, \quad (8.15)$$

and the inverse change of variables:

$$v = \frac{r + s}{2} \text{ and } h = \frac{(r - s)^2}{16}. \quad (8.16)$$

8.1.6 Saint-Venant equations in the hodograph plane

We will use the hodograph method by inverting $t(r, s)$ and $\xi(r, s)$. First, we start with the hodograph transformation $(t, \xi) \rightarrow (v, h)$:

$$\begin{pmatrix} dt \\ d\xi \end{pmatrix} = A \cdot \begin{pmatrix} dv \\ dh \end{pmatrix} \text{ where } A = \begin{pmatrix} \partial_v t & \partial_h t \\ \partial_v \xi & \partial_h \xi \end{pmatrix}.$$

The Jacobian of A is

$$J = \partial_v t \partial_h \xi - \partial_v \xi \partial_h t,$$

and the inverse of A is thus:

$$A^{-1} = \frac{1}{J} \begin{pmatrix} \partial_h \xi & -\partial_h t \\ -\partial_v \xi & \partial_v t \end{pmatrix}.$$

from which we deduce the inverse change of variables:

$$\frac{\partial v}{\partial t} = \frac{1}{J} \frac{\partial \xi}{\partial h}, \quad \frac{\partial v}{\partial \xi} = -\frac{1}{J} \frac{\partial t}{\partial h}, \quad \frac{\partial h}{\partial t} = -\frac{1}{J} \frac{\partial \xi}{\partial v} \text{ and } \frac{\partial h}{\partial \xi} = \frac{1}{J} \frac{\partial t}{\partial v}.$$

The Saint-Venant equations (8.11)–(8.12) then become:

$$-\frac{\partial \xi}{\partial v} + v \frac{\partial t}{\partial v} - h \frac{\partial t}{\partial h} = 0, \quad (8.17)$$

$$\frac{\partial \xi}{\partial t} - v \frac{\partial t}{\partial h} + \frac{\partial t}{\partial v} = 0. \quad (8.18)$$

By differentiating Eq. (8.17) with respect to h , Eq. (8.18) with respect to v , and taking the sum, we obtain a second-order partial differential equation for t :

$$-2 \frac{\partial t}{\partial h} - h \frac{\partial^2 t}{\partial h^2} + \frac{\partial^2 t}{\partial v^2} = 0. \quad (8.19)$$

Second, we make the change of variables $(v, h) \rightarrow (r, s)$. The chain rule provides:

$$\begin{aligned} \frac{\partial t}{\partial h} &= \frac{\partial t}{\partial r} \frac{\partial r}{\partial h} + \frac{\partial t}{\partial s} \frac{\partial s}{\partial h}, \\ &= \frac{1}{c} \left(\frac{\partial t}{\partial r} - \frac{\partial t}{\partial s} \right), \\ \frac{\partial t}{\partial v} &= \frac{\partial t}{\partial r} \frac{\partial r}{\partial v} + \frac{\partial t}{\partial s} \frac{\partial s}{\partial v}, \\ &= \frac{\partial t}{\partial r} + \frac{\partial t}{\partial s}. \end{aligned}$$

The second-order derivative is:

$$\frac{\partial^2 t}{\partial h^2} = -\frac{1}{2c^3} \left(\frac{\partial t}{\partial r} - \frac{\partial t}{\partial s} \right) + \frac{1}{c^2} \left(\frac{\partial^2 t}{\partial r^2} - 2 \frac{\partial^2 t}{\partial r \partial s} + \frac{\partial^2 t}{\partial s^2} \right).$$

In the end, we obtain the governing equation for $t(r, s)$:

$$L[t] = \frac{\partial^2 t}{\partial r \partial s} - \frac{3}{2(r-s)} \left(\frac{\partial t}{\partial r} - \frac{\partial t}{\partial s} \right) = 0. \quad (8.20)$$

Alternatively, this equation can be obtained more easily using the characteristic equations (8.13) and (8.14) (Garabedian, 1964). Along an r -characteristics $r = u + 2c = cst$, then

$$d\xi = \frac{\partial \xi}{\partial r} dr + \frac{\partial \xi}{\partial s} ds = (v + c)dt = \frac{3r + s}{4} \left(\frac{\partial t}{\partial r} dr + \frac{\partial t}{\partial s} ds \right),$$

and since $dr = 0$, then we obtain:

$$\frac{\partial \xi}{\partial s} = \frac{3r + s}{4} \frac{\partial t}{\partial s} \quad (8.21)$$

Similarly, from the s -characteristics, we obtain:

$$\frac{\partial \xi}{\partial r} = \frac{3s + r}{4} \frac{\partial t}{\partial r} \quad (8.22)$$

By taking the r -differential of Eq. (8.21) and subtracting the s -differential of Eq. (8.22), we obtain Eq. (8.20).

8.1.7 Riemann problem

Initial conditions

We want to solve the Riemann problem composed of the Saint-Venant equations (8.4) and (8.5), and subject to the initial conditions:

$$u(x, 0) = 0 \text{ and } h(x, 0) = \begin{cases} H_0 \left(1 - \frac{x}{x_a} \right) & \text{for } x \leq 0, \\ 0 & \text{for } x > 0, \end{cases} \quad (8.23)$$

where $x_a = -H_0 \cot \theta$ the abscissa of point A (see Fig. 8.1). In a dimensionless form and for the moving frame, these equations read:

$$v(\xi, 0) = 0 \text{ and } h(\xi, 0) = \begin{cases} 1 - \frac{\xi}{\xi_a} & \text{for } \xi \leq 0, \\ 0 & \text{for } \xi > 0, \end{cases} \quad (8.24)$$

where $\xi_a = -\cot \theta$ the abscissa of point A (see Fig. 8.1) and the hat has been removed from the dimensionless variables.

Boundary conditions

Like for the classic Riemann problem, the flow is bounded by two points (see Fig. 8.2):

- the front F of coordinate $\xi_f(t)$ at which:

$$h(\xi_f, t) = 0, \quad (8.25)$$

for $t > 0$. The initial condition is: $\xi_f(0) = 0$. The front is a material point, and thus $u_f = u(\xi_f, t) = \dot{\xi}_f$.

- the back point B of coordinate $\xi_b(t)$ at which:

$$h(\xi_b, t) = 1 - \xi_b \text{ and } u(\xi_b, t) = 0, \quad (8.26)$$

for $0 \leq t \leq t_b$, where t_b denotes the time at which the back front reaches the end point A. The initial condition is: $\xi_b(0) = 0$.

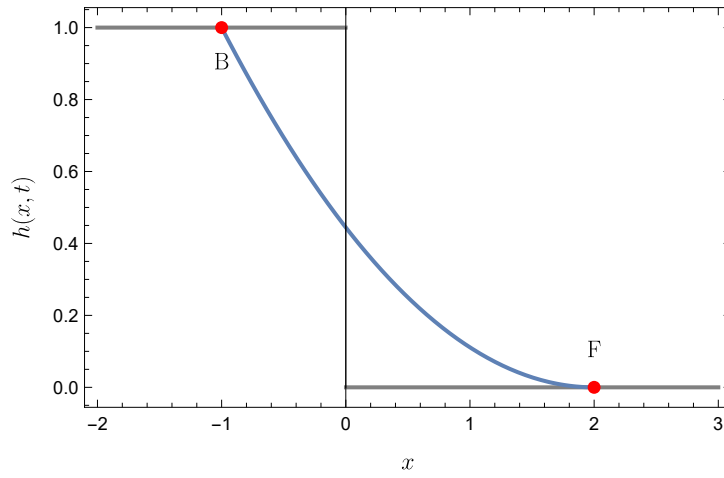


Figure 8.2 Flow depth profile and notation. The flow is bounded by points B and F.

The two moving boundaries are characteristics, and thus their equation is given by the characteristic equations (8.6) and (8.7). For the front F ($h(x_f, t) = 0$), the boundary is an r - and an s -characteristics associated with $r = 2$ and $s = 2$ (or equivalently $\rho = \sigma = 2$):

$$\frac{dx_f}{dt} = u = 2 + t \tan \theta \Rightarrow x_f = 2t + \frac{t^2}{2} \tan \theta, \quad (8.27)$$

and in the moving frame, we have:

$$\xi_f = 2t.$$

The backward wave B is an s -characteristic with $s = -2$:

$$\frac{dx_b}{dt} = -c = -1 + \frac{t}{2} \tan \theta \Rightarrow x_b = -t + \frac{t^2}{4} \tan \theta,$$

up to time:

$$t_b = 2 \cot \theta.$$

For $t > t_b$, we have $c = 0$ and $u = -2 + t \tan \theta$, and thus:

$$\frac{dx_b}{dt} = u = -2 + t \tan \theta \Rightarrow x_b = -2t + \frac{t^2}{2} \tan \theta + a,$$

where a is a constant of integration satisfying $x_b(t_b) = x_a$, and thus $a = \cot \theta$. In the moving frame, we have:

$$\xi_b = \begin{cases} -t - \frac{t^2}{4} \tan \theta & \text{for } t \leq t_b, \\ -2t + \cot \theta & \text{for } t > t_b, \end{cases}$$

while in the fixed frame, we have

$$x_b = \begin{cases} -t + \frac{t^2}{4} \tan \theta & \text{for } t \leq t_b, \\ -2t + \frac{t^2}{2} \tan \theta + \cot \theta & \text{for } t > t_b. \end{cases} \quad (8.28)$$

8.1.8 Hodograph plane

In the hodograph plane (Fig. 8.3), the computational domain is the triangle OFC. The moving boundaries are stationary points or lie on the segment OC:

- The front F coordinates are $s = r = 2$, and thus is a stationary point in the hodograph plane.
- For $t \leq t_b$, the rear point B lies on the segment OC with $s = -2$ and $-2 \leq r \leq 2$. For this segment, we have $r = v + 2c = 2 - 2t \tan \theta$ since $c = 1 - t \tan \theta/2$, $u = 0$, $v = u - t \tan \theta$. We thus have:

$$t = \left(1 - \frac{r}{2}\right) \cot \theta.$$

- For $t \geq t_b$, the rear point B stays at the same position (point C) $s = r = -2$ in the hodograph plane.

In the hodograph plane, the segment OF represents the early instants of flow after removal of the wall. Indeed, the velocity and depth follow the Ritter solution at very short times ($t \rightarrow 0$) regardless of the bottom slope:

$$u = \frac{2}{3}(\zeta + 1) \text{ and } h = \frac{(2 - \zeta)^2}{9}$$

where $\zeta = x/t$ is the similarity variable varying in the range $-1 \leq \zeta \leq 2$. For the Ritter solution in the fixed frame, we have:

$$\rho = u + 2c - t \tan \theta = 2 - t \tan \theta \text{ and } \sigma = u - 2c - t \tan \theta = \frac{2}{3}(2\zeta - 1) - t \tan \theta,$$

and thus in the limit of $t \rightarrow 0$, we have

$$r = 2 \text{ and } -2 \leq s \leq 2.$$

8.1.9 Characteristic diagram

Figure 8.4 shows the family of s and r characteristics. I also plotted the back and front curves, $x_b(t)$ given by Eq. (8.28) and $x_f(t)$ given by Eq. (8.27), respectively.

I ran GeoClaw to compute the characteristic curves. The depth and velocity data were interpolated, then the characteristic curves (8.8) and (8.9) were integrated numerically. I also determined the back and front positions in the GeoClaw data. There is a good agreement between theory and numerical simulations. The only significant difference concerns the s -characteristics associated with $s = -2$, which departs from the theoretical solution for $t > 10$.

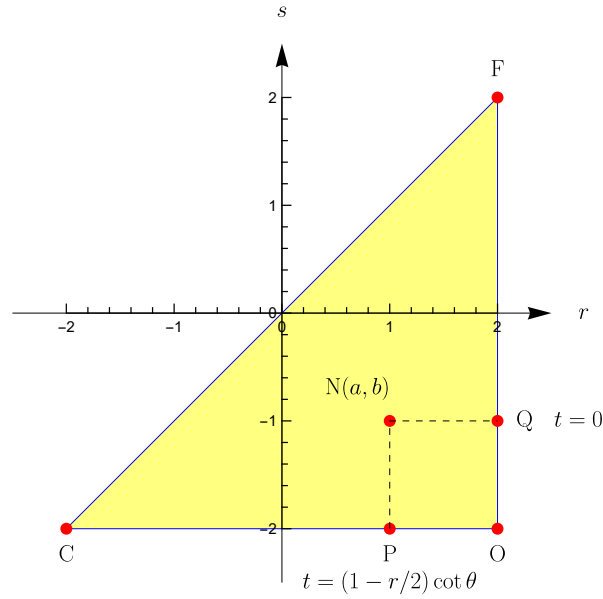


Figure 8.3 Domain of integration $\mathcal{D}$. Equation (8.20) is solved over the OPNQ rectangle. The integration domain is bounded by the initial condition $-2 \leq s \leq 2$ and for which $t = 0$. The front point F remains at the same place $(s, r) = (2, 2)$ over time. The segment OC corresponds to the rear; the boundary conditions impose $t = (1 - r/2) \cot \theta$. The segment OF represents the early time solution, and thus $t = 0$ on this segment.

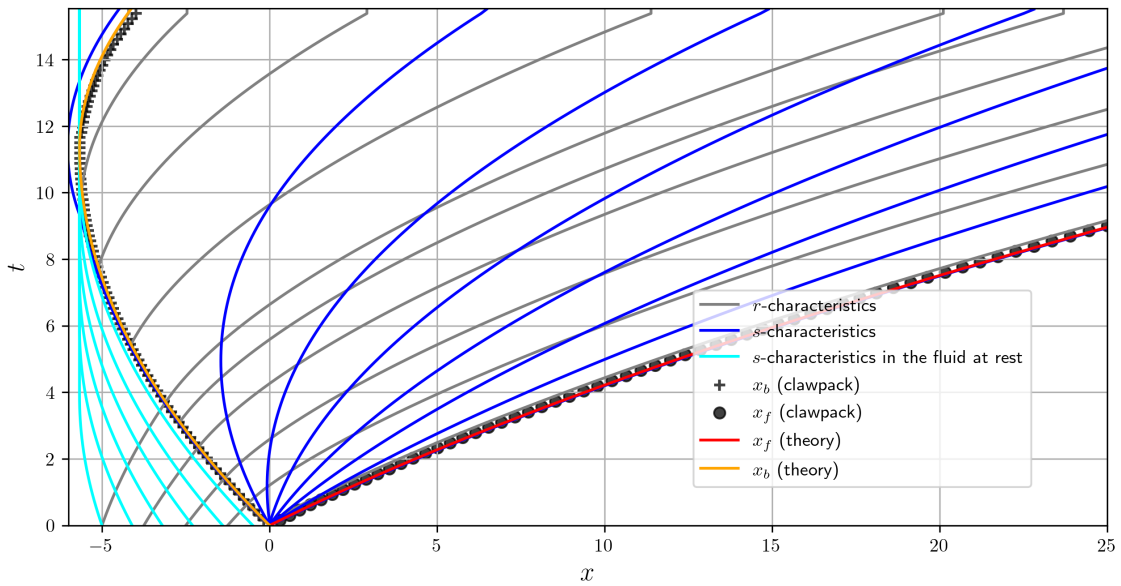


Figure 8.4 r and s -characteristics in the characteristic plane $x-t$. The back and front curves $x_b(t)$ and $x_f(t)$ are given by Eq. (8.28) given by Eq. (8.27), respectively.

8.1.10 Riemann function

Formulation

Equation (8.20) can be solved using the Riemann function technique (Garabedian, 1964, see pp. 127–134). Let us define the linear operator:

$$L[t] = t_{sr} + \alpha t_r + \beta t_s, \quad (8.29)$$

with the functions:

$$\alpha = -\frac{3}{2(r-s)} \text{ and } \beta = \frac{3}{2(r-s)}.$$

We want to solve:

$$L[t] = 0 \quad (8.30)$$

We introduce the adjoint operator M (applied to a function $\tau(r, s)$) in such a way that we can express the difference $\tau L[t] - tM[\tau]$ as the divergence of a vector field $U = (U, V)$

$$\tau L[t] - tM[\tau] = \nabla \cdot U = \frac{\partial U}{\partial r} + \frac{\partial V}{\partial s}.$$

By doing so, we can then use Green's theorem to transform the integral over the domain $\mathcal{D}$ into an integral of the oriented contour Γ of $\mathcal{D}$:

$$\int_{\mathcal{D}} (\tau L[t] - tM[\tau]) dr ds = \int_{\Gamma} U \cdot n d\eta = \int_{\Gamma} (U ds - V dr),$$

where n is the outward normal vector along the contour $\mathcal{D}$ and $d\eta$ is a dummy variable of integration defined such that $nd\eta = (ds, -dr)$. The adjoint operator of L is:

$$M[\tau] = \tau_{sr} - (\alpha\tau)_r - (\beta\tau)_s. \quad (8.31)$$

As a consequence, the vector field $(U(s, r), V(s, r))$ is:

$$U = \alpha\tau t + \frac{\tau}{2}t_r - \frac{1}{2}\tau_r t, \quad (8.32)$$

$$V = \beta\tau t + \frac{\tau}{2}t_s - \frac{1}{2}\tau_s t. \quad (8.33)$$

The domain of integration $\mathcal{D}$ is the rectangle OPNQ, and thus the oriented (counterclockwise) contour Γ is OQNP. We want to compute the solution at point N with coordinates (a, b) . This domain of integration is selected so that point N is connected to the initial and boundary conditions (in other words, $\mathcal{D}$ is also the domain of dependence of the solution for the Cauchy problem). The contour involves the vertical line PN ($r = cst$) and the horizontal segment NQ ($s = cst$). By integrating $\int_{\Gamma} (U ds - V dr)$ by parts, we can remove the dependency on the t derivatives:

$$\begin{aligned} \int_{\Gamma} (U ds - V dr) &= \int_{NP} (\alpha\tau - \tau_s)t ds - \int_{QN} (\beta\tau - \tau_r)t dr \\ &\quad + \tau(a, b)t(a, b) - \frac{1}{2}\tau(a, -2)t(a, -2) - \frac{1}{2}\tau(2, b)t(2, b) + \int_{POQ} B[t, \tau] \end{aligned} \quad (8.34)$$

where (a, b) are the coordinates of point N in Fig. 8.3, and the differential operator B is:

$$B[t, \tau] = U[t, \tau]ds - V[t, \tau]dr.$$

The next step is to remove the dependency on τ along the segments NP and QN. We therefore seek a function τ such that:

$$M[\tau] = 0 \quad (8.35)$$

subject to

$$\tau_r = \beta\tau, \quad (8.36)$$

$$\tau_s = \alpha\tau, \quad (8.37)$$

$$\tau(a, b) = 1. \quad (8.38)$$

Riemann function for the Saint-Venant equations

The Riemann function R can be computed as follows (Garabedian, 1964, see problem 9, § 5.1, p. 150). Let us consider a partial differential equation of the form:

$$v_{xy} + \frac{\lambda}{2} \frac{1}{x+y} (v_x + v_y) = 0, \quad (8.39)$$

whose adjoint operator is

$$N[v] = 0, \text{ with } N[v] = v_{xy} - (av)_x - (bv)_y + cv, \text{ and } a = b = \frac{\lambda}{2} \frac{1}{x+y},$$

and where $c = 0$. Following Garabedian (1964), we pose

$$v = \frac{(x+y)^\lambda}{(x+\eta)^{\lambda/2}(x+\eta)^{\lambda/2}} W(\zeta), \text{ with } \zeta = \frac{(x-\xi)(y-\eta)}{(x+\eta)(y+\xi)}.$$

We find that W satisfies the equation

$$-\lambda^2 W(\zeta) + 4(1 - (\lambda + 1)\zeta)W'(\zeta) + \zeta(1 - \zeta)W''(\zeta) = 0,$$

whose solution is

$$W(\zeta) = F\left[\frac{\lambda}{2}, \frac{\lambda}{2}, 1, \zeta\right],$$

where F is the hypergeometric function. With $\lambda = 3$, $x = r$ and $y = -s$, we find the solution to the adjoint problem (8.35) with M given by Eq. (8.31). Note that alternative representations (in particular, in terms of Legendre functions) can be obtained using properties of F (Abramowitz & Stegun, 1964, see pp. 559–562):

$$F(a, b, c, z) = (1-z)^{-b} F\left(c-a, b, c, \frac{z}{z-1}\right),$$

which gives

$$F\left[\frac{3}{2}, \frac{3}{2}, 1, z\right] = (1-z)^{-\frac{3}{2}} F\left[-\frac{1}{2}, \frac{3}{2}, 1, \frac{z}{z-1}\right].$$

A new transformation makes it possible to use the Legendre function (Abramowitz & Stegun, 1964, see p. 562)

$$F\left[-\frac{1}{2}, \frac{3}{2}, 1, z\right] = P\left[\frac{1}{2}, 0, 1-2z\right].$$

where P is the Legendre function of degree $1/2$ and order 0 . The solution then becomes

$$v(x, y) = \frac{(x+y)^{3/2}}{(a+b)^{3/2}} P\left[\frac{1}{2}, 0, 1-2\frac{(a-x)(b-y)}{(a+b)(x+y)}\right].$$

8.1.11 Solution to the governing equation for t

The solution to Eq. (8.35) is thus the Riemann function:

$$\tau(r, s) = R(r, s; a, b) = \frac{(r-s)^3}{(r-b)^{3/2}(s-a)^{3/2}} F\left(\frac{3}{2}, \frac{3}{2}, 1, \frac{(r-a)(s-b)}{(r-b)(s-a)}\right). \quad (8.40)$$

Making use of equations (8.34) and (8.35), we find that the $t(a, b)$ at point N depends solely on what happens on the boundary POQ:

$$t(a, b) = \frac{1}{2}\tau(P)t(P) + \frac{1}{2}\tau(Q)t(Q) + \int_{\text{QOP}} B. \quad (8.41)$$

where point P has coordinates $(a, -2)$ and point Q has coordinates $(2, b)$.

Since on the boundaries PO and OQ we have $t_r = -\cot\theta/2$ and $t_s = 0$, respectively, we can reduce the equation for t to

$$t(a, b) = \cot\theta \int_2^a R(r, -2; a, b) \frac{2-5r}{4(r+2)} dr. \quad (8.42)$$

The variable ξ is then computed by integrating the s -characteristic equation (8.22)

$$\xi(r|s = cst) = \frac{1}{4}(3s+r)t(r, s) + \frac{1}{4} \int_r^2 t(\zeta, s) d\zeta, \quad (8.43)$$

where we have taken into account the boundary condition $\xi = 0$ at $t = 0$.

Figure 8.5 compares the theoretical solutions with the numerical solutions obtained using GeoClaw.

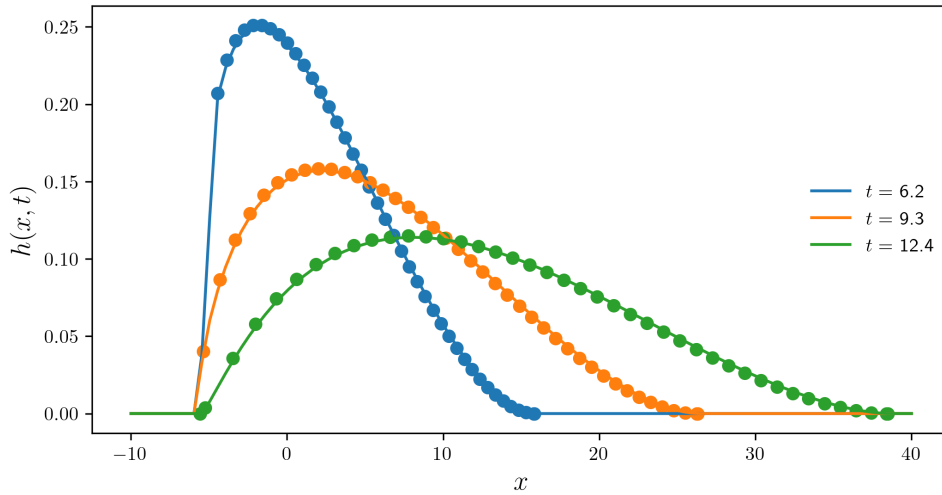


Figure 8.5 Flow depth profiles. The dots show the theoretical profiles. The solid lines show the GeoClaw solutions.

8.2 Coulomb dam-break problem on a horizontal bottom

8.2.1 Governing equations

I closely follow the derivation made by Kerswell (2005). The depth-averaged equations take the non-conservative form:

$$\frac{\partial h}{\partial t} + \frac{\partial h \bar{u}}{\partial x} = 0, \quad (8.44)$$

$$\frac{\partial \bar{u}}{\partial t} + \bar{u} \frac{\partial \bar{u}}{\partial x} + K g \cos \theta \frac{\partial h}{\partial x} = g \sin \theta - \mu g \cos \theta, \quad (8.45)$$

where h denotes the flow depth, $\bar{u}$ is the depth-averaged velocity, K is the earth pressure coefficient, θ is the slope angle, and μ is the Coulomb friction coefficient.

8.2.2 Dimensionless Saint-Venant equations

We will consider a horizontal bottom ($\theta = 0$), and make equations (8.44)–(8.45) dimensionless using the following dimensionless variables:

$$\hat{x} = \frac{x}{X_*}, \quad \hat{t} = \frac{t}{T_*}, \quad \hat{u} = \frac{\bar{u}}{U_*}, \quad \text{and} \quad \hat{h} = \frac{h}{L_*}, \quad (8.46)$$

where L_* is a vertical length scale, and the abscissa, time and velocity scales are defined from it:

$$X_* = \frac{K}{\mu} L_*, \quad T_* = \frac{1}{\mu} \sqrt{K \frac{L_*}{g}} \quad \text{and} \quad U_* = \sqrt{K g L_*}.$$

Let us consider the Saint-Venant equations (8.44)–(8.45) in a dimensionless and nonconservative form:

$$\frac{\partial h}{\partial t} + \frac{\partial h u}{\partial x} = 0, \quad (8.47)$$

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \frac{\partial h}{\partial x} = -1. \quad (8.48)$$

The hat decoration has been removed. In a characteristic form, these equations are:

$$\frac{d}{dt}(u + 2c) = -1 \quad \text{along} \quad \frac{dx}{dt} = u + c, \quad (8.49)$$

$$\frac{d}{dt}(u - 2c) = -1 \quad \text{along} \quad \frac{dx}{dt} = u - c, \quad (8.50)$$

where $c = \sqrt{h}$. Let us integrate these equations and introduce the Riemann variables r and s that label the r -characteristics ($r = \text{constant}$) and the s -characteristics ($s = \text{constant}$):

$$u + 2c = 2r - t \quad \text{along} \quad \frac{dx}{dt} = u + c, \quad (8.51)$$

$$u - 2c = -2s - t \quad \text{along} \quad \frac{dx}{dt} = u - c. \quad (8.52)$$

8.2.3 Saint-Venant equations in the hodograph plane

We will use the hodograph method by inverting $t(r, s)$ and $x(r, s)$. First let us note:

$$u = r - s - t \text{ and } h = \frac{(r + s)^2}{4}. \quad (8.53)$$

We can now express the characteristic equations (8.51) and (8.52) in the form:

$$\begin{aligned} \frac{\partial x}{\partial s} &= \frac{\partial x}{\partial t} \frac{\partial t}{\partial s} \text{ along } r = \text{constant}, \\ &= (u + c) \frac{\partial t}{\partial s}, \\ &= \left(r - s - t + \frac{r + s}{2} \right) \frac{\partial t}{\partial s}, \\ &= \left(\frac{3}{2}r - \frac{1}{2}s - t \right) \frac{\partial t}{\partial s}, \end{aligned} \quad (8.54)$$

and

$$\begin{aligned} \frac{\partial x}{\partial r} &= \frac{\partial x}{\partial t} \frac{\partial t}{\partial r} \text{ along } s = \text{constant}, \\ &= (u - c) \frac{\partial t}{\partial r}, \\ &= \left(r - s - t - \frac{r + s}{2} \right) \frac{\partial t}{\partial r}, \\ &= \left(\frac{1}{2}r - \frac{3}{2}s - t \right) \frac{\partial t}{\partial r}. \end{aligned} \quad (8.55)$$

By taking the r -differential of Eq. (8.54) and subtracting the s -differential of Eq. (8.55), we obtain the following second-order linear partial differential equation for t :

$$\frac{\partial^2 t}{\partial r \partial s} + \frac{3}{2(r + s)} \left(\frac{\partial t}{\partial r} + \frac{\partial t}{\partial s} \right) = 0. \quad (8.56)$$

8.2.4 Riemann problem

Initial conditions

We want to solve the Riemann problem composed of the Saint-Venant equations (8.47) and (8.48), and subject to the initial conditions:

$$u(x, 0) = 0 \text{ and } h(x, 0) = \begin{cases} 1 & \text{for } x \leq 0, \\ 0 & \text{for } x > 0. \end{cases} \quad (8.57)$$

We assume that the box containing the material is sufficiently large for the back wall not to create a wave that affects the flow dynamics. This case is addressed by ?, and the reader is referred to.

Boundary conditions

Like for the classic Riemann problem, the flow is bounded by two points (see Fig. 8.6):

- the front F of coordinate $x_f(t)$ at which:

$$h(x_f, t) = 0, \quad (8.58)$$

for $t > 0$ and $x_f(0) = 0$. Note that $\dot{x}_f(2) = 0$ (see below).

- the back point B of coordinate $x_b(t)$ at which

$$h(x_b, t) = 1 \text{ and } u(x_b, t) = 0, \quad (8.59)$$

for $t > 0$ and $x_b(0) = 0$.

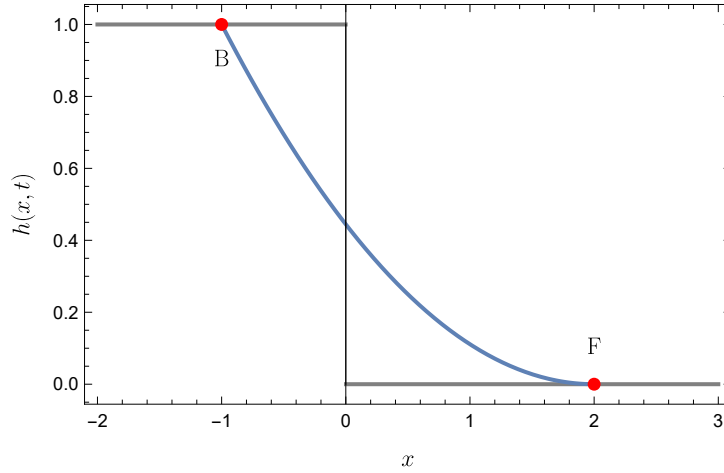


Figure 8.6 Flow depth profile and notation. The flow is bounded by points B and F.

Figure 8.7 shows the family of s and r characteristics. Note that for $t < 0$, we must consider that the source term is 0 otherwise the initial conditions (8.57) cannot hold for $t < 0$. Under this condition, the characteristics are in the form

$$r = 1 \text{ along } x = t + x_0 \text{ and } s = 1 \text{ along } x = -t + x_0$$

for any $x_0 < 0$. At time $t = 0$, a fan of s -characteristics develops with $-1 \leq s \leq 1$. The front curve is the characteristic curve associated with $r = 1$ and $s = 1$. The front position is thus given by:

$$\frac{d}{dt}x_f(t) = u + c = 2 - t. \quad (8.60)$$

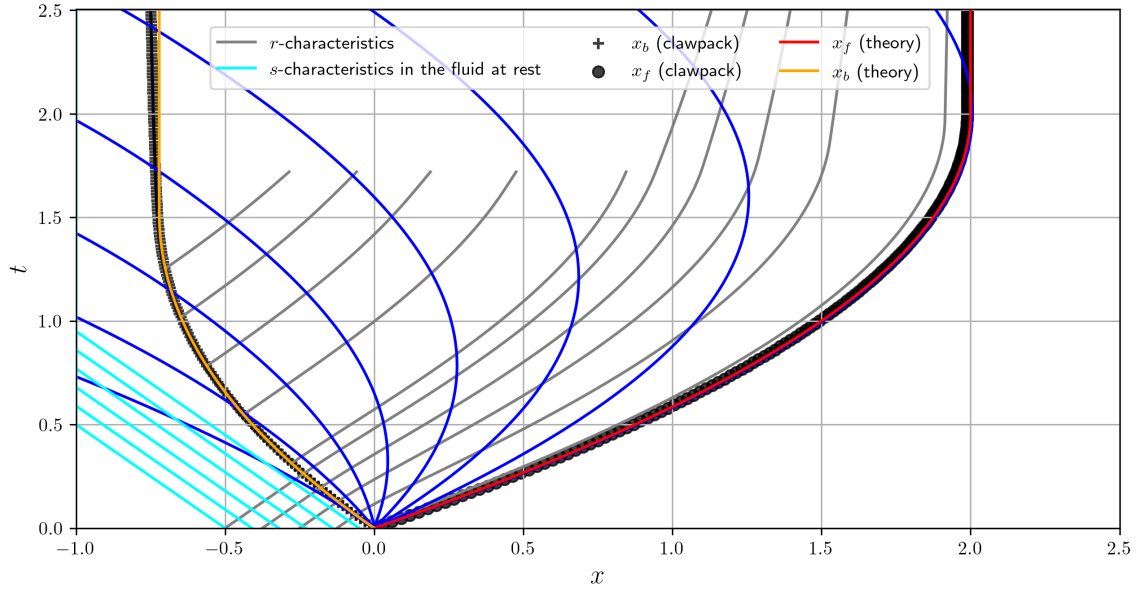
As a consequence, the front position is given by:

$$x_f(t) = 2t - \frac{t^2}{2}. \quad (8.61)$$

Arrested state

There is a limiting position corresponding to the arrest of the front when $\dot{x}_f = 0$:

$$x_{arrest} = 2 \text{ at time } t_{arrest} = 2. \quad (8.62)$$

Figure 8.7 r and s characteristics.

Characteristic diagram

We can plot the boundary conditions in the hodograph plane. Initially at $t = 0$, we have for $x \leq 0$:

$$r = \frac{u + 2c}{2} = 1 \text{ and } s = -\frac{u - 2c}{2} = 1. \quad (8.63)$$

At short times ($t \ll 1$), a wave forms between points B and F. At point B, we have

$$r = \frac{u + 2c}{2} = 1 \text{ and } s = -\frac{u - 2c}{2} = 1 \quad (8.64)$$

while at point F moving at velocity $\dot{x}_f = 2$

$$r = \frac{u + 2c}{2} = 1 \text{ and } s = -\frac{u - 2c}{2} = -1. \quad (8.65)$$

There is thus a s -characteristic fan with s in the $[-1, 1]$ range. In Fig. 8.8, the initial condition $t = 0$ corresponds thus to the segment FO for which $r = 1$ and $-1 \leq s \leq 1$. The equations of the s -characteristics in the fan are:

$$\frac{dx}{dt} = u - c = c - 2s - t, \quad (8.66)$$

since $u = 2c - 2s - t$.

For the rear at time t , we have $h = 1$ and $u = 0$, and thus:

$$r = \frac{u + 2c + t}{2} = 1 + \frac{t}{2} \text{ and } s = -\frac{u - 2c + t}{2} = 1 - \frac{t}{2}, \quad (8.67)$$

from which we deduce

$$r + s = 2 \text{ and } t = r - s = 2(r - 1). \quad (8.68)$$

In Fig. 8.8, the rear is represented by the curve OB, which is a segment on the straight line $r = 2 - s$. Contrary to the classic Riemann problem, this curve is not a characteristic curve, and thus we cannot

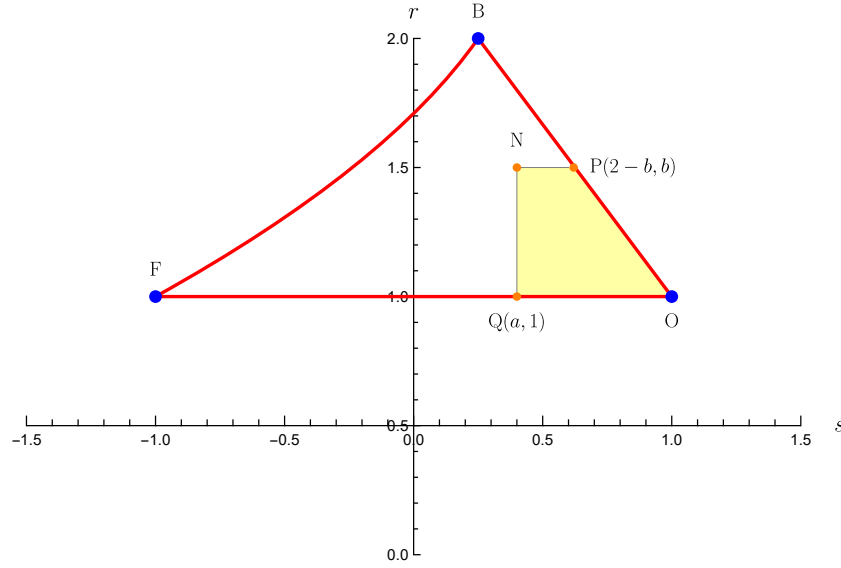


Figure 8.8 Domain of integration $\mathcal{D}$. Equation (8.56) is solved over the NPOQ polygon. The integration domain is bounded by the initial condition $-1 \leq s \leq 1$ and for which $t = 0$. The front point F remains at the same place $(s, r) = (1, 1)$ over time. The segment OB corresponds to the rear; the boundary conditions $u = 0$ and $h = 1$ impose that $r + s = 2$ and $t = 2(r - 1)$. The curve FB represents the arrested time ($u = 0$). As point B has not yet been determined, this figure is purely conceptual.

calculate it. It shall be computed numerically in the course of solving Eq. (8.56). Note that at point B, the velocity is always 0, and thus does not depend on r or s : $t_r = t_s = 0$. Making use of the definition of the velocity in Eq. (8.53), we deduce that:

$$\frac{\partial u}{\partial r} = 1 - \frac{\partial t}{\partial r} \text{ and } \frac{\partial u}{\partial s} = -1 - \frac{\partial t}{\partial s}, \quad (8.69)$$

which implies that at point B, we have:

$$\frac{\partial t}{\partial r} = 1 \text{ and } \frac{\partial t}{\partial s} = -1. \quad (8.70)$$

For the front at time t , the velocity is given by $u = 2 - t$, and thus we have:

$$r = \frac{u + 2c + t}{2} = 1 \text{ and } s = -\frac{u - 2c + t}{2} = 1 - \frac{t}{2}, \quad (8.71)$$

8.2.5 Riemann function

Formulation

Equation (8.56) can be solved using the Riemann function technique (Garabedian, 1964, see pp. 127–134). Let us define the linear operator:

$$L[t] = t_{sr} + \alpha t_s + \beta t_r, \quad (8.72)$$

with the functions:

$$\alpha = \beta = \frac{3}{2(r + s)}.$$

We want to solve:

$$L[t] = 0 \quad (8.73)$$

The adjoint operator of L is:

$$M[v] = v_{sr} - (\alpha v)_s - (\beta v)_r. \quad (8.74)$$

We seek a function v such that:

$$M[v] = 0 \quad (8.75)$$

subject to

$$v_r = \alpha v, \quad (8.76)$$

$$v_s = \beta v, \quad (8.77)$$

$$v(a, b) = 1, \quad (8.78)$$

where (a, b) are the coordinates of point N in Fig. 8.8.

Riemann function for the Saint-Venant equations

The solution to Eq. (8.75) is the Riemann function:

$$R(s, r, a, b) = \frac{(s+r)^3}{(r+a)^{3/2}(s+b)^{3/2}} F\left(\frac{3}{2}, \frac{3}{2}, 1, \frac{(r-b)(s-a)}{(r+a)(s+b)}\right) \quad (8.79)$$

where F is the hypergeometric function.

The adjoint operator M has been defined such that:

$$vL[t] - tM[v] = U_r + V_s,$$

where $(U(s, r), V(s, r))$ is the vector field:

$$U = \alpha vt + \frac{1}{2}vt_r - \frac{1}{2}v_r t, \quad (8.80)$$

$$V = \beta vt + \frac{1}{2}vt_s - \frac{1}{2}v_s t. \quad (8.81)$$

Green's theorem shows that over an integration domain $\mathcal{D}$ bounded by $\mathcal{C}$ (oriented counterclockwise), we have the equivalence:

$$\int_{\mathcal{D}} (U_s + V_r) dr ds = \int_{\mathcal{C}} U dr - V ds.$$

Here we have

$$\int_{\mathcal{D}} (U_r + V_s) dr ds = \int_{\mathcal{C}} \left(\alpha vt + \frac{1}{2}vt_r - \frac{1}{2}v_r t \right) dr - \left(\beta vt + \frac{1}{2}vt_s - \frac{1}{2}v_s t \right) ds.$$

The integration domain is the NQOP polygon. We can rearrange the terms above by integrating the integrands by parts and thereby, we obtain:

$$\begin{aligned} \int_{\text{NQOP}} (U_r + V_s) dr ds &= \int_{\text{NQ}} (\alpha v - v_r) t dr - \int_{\text{PN}} (\beta v - v_s) t ds \\ &\quad - v(N)t(N) + \frac{1}{2}v(P)t(P) + \frac{1}{2}v(Q)t(Q) + \int_{\text{QOP}} B[t, v] \end{aligned} \quad (8.82)$$

where

$$B[t, v] = \left(\alpha vt + \frac{1}{2} vt_r - \frac{1}{2} v_s t \right) dr - \left(\beta vt + \frac{1}{2} vt_s - \frac{1}{2} v_r t \right) ds$$

Making use of equations (8.73) and (8.75) together with (8.77) and (8.78), we find that the $t(a, b)$ at point N depends on what happens on the boundary QOP:

$$t(a, b) = \frac{1}{2} v(P)t(P) + \frac{1}{2} v(Q)t(Q) + \int_{\text{QOP}} B. \quad (8.83)$$

where point P has coordinates $(2 - b, b)$ and point Q has coordinates $(a, 1)$.

Solution to the dam-break problem

Equation (8.83) can be cast in the following form:

$$t(a, b) = (b - 1)R(2 - b, b, a, b) + \int_{\text{QOP}} B. \quad (8.84)$$

since $t(Q) = 0$ and $t(P) = t(2 - b, b) = 2(b - 1)$. Since we have $t = 0$ on the segment QO, then Eq. 8.84 simplifies to:

$$t(a, b) = (b - 1)R(2 - b, b, a, b) + \int_{\text{OP}} B[t, R]. \quad (8.85)$$

Along OP, we have $r = 2 - s$, and $ds = -dr$. The operator B becomes:

$$B[t, R] = \left(\frac{3}{2(r + s)} Rt + \frac{1}{2} Rt_r - \frac{1}{2} R_s t + \frac{3}{2(r + s)} Rt + \frac{1}{2} Rt_s - \frac{1}{2} R_r t \right) dr \quad (8.86)$$

$$= \left(\frac{3}{(r + s)} Rt + \frac{1}{2} R(t_r + t_s) - \frac{1}{2} (R_s + R_r) t \right) dr \quad (8.87)$$

Over the segment OP, we have seen that $t = 2(r - 1)$ and $r + s = 2$. This implies:

$$\frac{dt}{dr} = 2,$$

and at the same time:

$$\frac{dt}{dr} = \frac{\partial t}{\partial r} + \frac{\partial t}{\partial s} \frac{ds}{dr} = \frac{\partial t}{\partial r} - \frac{\partial t}{\partial s}$$

and thus, the following relationship holds over this segment:

$$\frac{\partial t}{\partial r} - \frac{\partial t}{\partial s} = 2. \quad (8.88)$$

The situation to Eq. (8.56) is thus:

$$t(a, b) = (b - 1)R(2 - b, b, a, b) + \int_1^b B(r, a, b) dr \quad (8.89)$$

where now B takes the form:

$$\begin{aligned} B(r, a, b) = & \frac{3}{2} R(2 - r, r, a, b) t \\ & + R(2 - r, r, a, b) (t_r(2 - r, r) - 1) \\ & - \frac{R_s(2 - r, r, a, b) + R_r(2 - r, r, a, b)}{2} t(2 - r, r), \end{aligned} \quad (8.90)$$

and after substituting $t = 2(r - 1)$ into the integral term of Eq. (8.90), we end up with the expression:

$$\begin{aligned} B(r, a, b) = & 3R(2 - r, r, a, b)(r - 1) \\ & + R(2 - r, r, a, b) (t_r(2 - r, r) - 1) \\ & - [R_s(2 - r, r, a, b) + R_r(2 - r, r, a, b)](r - 1). \end{aligned} \quad (8.91)$$

Evaluation of the derivative t_r

We note that Eq. (8.91) involves the r -derivative t_r . This derivative is not known. Admittedly we know that the r - and s -derivatives of t are related by Eq. (8.88) because $t = 2(r - 1)$ along the line $r + s = 2$, but these equations are not sufficient to determine t_r . We can estimate this derivative using an iterative method (Picard's method).

At point P of coordinates $(2 - b, b)$, the solution is:

$$t(P) = 2(b - 1). \quad (8.92)$$

Equation (8.89) tells us that the solution to the Saint-Venant equations is:

$$t(P) = (b - 1)R(2 - b, b, 2 - b, b) + \int_1^b B(r, 2 - b, b) dr = b - 1 + \int_1^b B(r, 2 - b, b) dr, \quad (8.93)$$

since $R(2 - b, b, 2 - b, b) = 1$ while the integrand B simplifies to:

$$\begin{aligned} B(r, 2 - b, b) = & R(2 - r, r, 2 - b, b)(3r - 4) \\ & + R(2 - r, r, 2 - b, b)t_r(2 - r, r) \\ & - (R_s(2 - r, r, 2 - b, b) + R_r(2 - r, r, 2 - b, b))(r - 1). \end{aligned} \quad (8.94)$$

Combining Eqs. (8.92) and (8.93), we obtain an equation for t_r :

$$\int_1^b R(2 - r, r, 2 - b, b)t_r(2 - r, r) dr = b - 1 + \int_1^b [(4 - 3r)R + (R_s + R_r)(r - 1)] dr, \quad (8.95)$$

where R and its derivatives in the integral are evaluated at $(2 - r, r, 2 - b, b)$. This is a Volterra equation of the first kind for t_r .

To estimate $t_r(2 - r, r)$ numerically, we can differentiate Eq. (8.95) with respect to b and obtain a Volterra equation of the second kind where t_r appears out of the integral term. Picard's iterative method can then be used to solve the resulting equation (Zwillinger, 1992, see pp. 560–562).

Differentiating the left-hand member of Eq. (8.95) with respect to b , we obtain:

$$\frac{\partial}{\partial b} \int_1^b R(2 - r, r, 2 - b, b)t_r(2 - r, r) dr = t_r(2 - b, b) + \int_1^b t_r(2 - r, r) \frac{\partial}{\partial b} R(2 - r, r, 2 - b, b) dr, \quad (8.96)$$

while differentiating the right-hand member of Eq. (8.95) with respect to b yields:

$$f(b) = 1 + 4 - 3b + \frac{3}{2}(b - 1) + \int_1^b \frac{\partial}{\partial b} [R(4 - 3r) + (R_s + R_r)(r - 1)] dr$$

since

$$R_s(2 - b, b, 2 - b, b) = R_r(2 - b, b, 2 - b, b) = \frac{3}{4}$$

After rearranging the terms, we end up with:

$$f(b) = \frac{1}{2}(7 - 3b) + \int_1^b \frac{\partial}{\partial b} [R(4 - 3r) + (R_s + R_r)(r - 1)] dr, \quad (8.97)$$

where R and its derivatives in the integral are evaluated at $(2 - r, r, 2 - b, b)$. Matching Eqs. (8.96) and Eqs. (8.97) leads to a Volterra equation of the second kind:

$$t_r(2 - b, b) = f(b) - \int_1^b t_r(2 - r, r) \frac{\partial}{\partial b} R(2 - r, r, 2 - b, b) dr, \quad (8.98)$$

We solve this equation iteratively:

$$t_r^{(k+1)} = f(b) - \int_1^b t_r^{(k)}(2-r, r) \frac{\partial}{\partial b} R(2-r, r, 2-b, b) dr,$$

starting from:

$$t_r^{(0)} = f(b).$$

The function $t_r^{(k+1)}$ converges towards the solution quickly (after three iterations). Figure 8.9 shows this function. If we approximate the numerical solution by a third-order polynomial, we find:

$$t_r(2-r, r) = 3.4779 - 1.328r - 0.2887r^2 + 0.1379r^3. \quad (8.99)$$

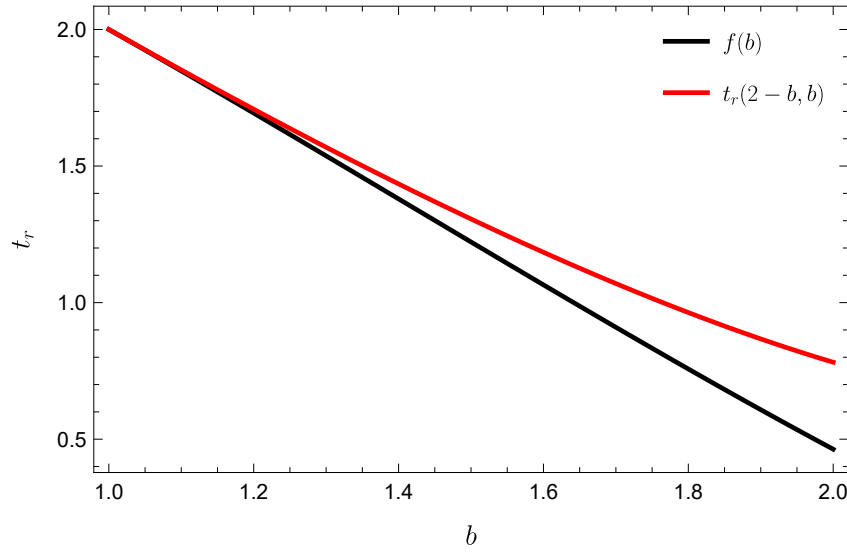


Figure 8.9 Plot of the r -derivative $t_r(2-b, b)$ as a function of b for $1 \leq b \leq 2$.

We can now determine the position of point B by using the condition (8.70):

$$t_r = 1 \Rightarrow r(B) = 1.76481.$$

Point B reaches its asymptotic position at time:

$$t_B = 2(r(B) - 1) = 1.52962. \quad (8.100)$$

Note that we can now determine how x_b varies with time. Indeed, by integrating (x_s, x_r) given by Eqs. (8.54) and (8.55) along the line segment $r + s = 2$, we obtain:

$$\begin{aligned} dx_b &= \frac{\partial x}{\partial r} dr + \frac{\partial x}{\partial s} ds, \\ \frac{dx_b}{dr} &= \left(\frac{1}{2}r - \frac{3}{2}s - t \right) \frac{\partial t}{\partial r} + \left(\frac{3}{2}r - \frac{1}{2}s - t \right) \frac{\partial t}{\partial s} \frac{ds}{dr}, \\ &= \left(\frac{1}{2}r - \frac{3}{2}(2-r) - t \right) t_r - \left(\frac{3}{2}r - \frac{1}{2}(2-r) - t \right) (t_r - 2), \\ &= 2 - 2t_r, \end{aligned}$$

since Eq. (8.88) yields $t_s = t_r - 2$ and we have $ds = -dr$. If we use the third-order approximation (8.99) and the relation $t = 2(r - 1)$ on OB, then we obtain:

$$x_b(t) = 0.034475 \left(\frac{t}{2} + 1 \right)^4 - 0.0962333 \left(\frac{t}{2} + 1 \right)^3 - 0.664 \left(\frac{t}{2} + 1 \right)^2 + 3.4779 \left(\frac{t}{2} + 1 \right). \quad (8.101)$$

The position of B at time t_B is

$$x_B = -0.721577.$$

Figure 8.10 shows the comparison between theory and Clawpack (Geoclaw). There is a good agreement for the front position $x_f(t)$. For the back point $x_b(t)$, there is a good agreement up to time $t = 1.5$ when the back point should reach its final position $x_B = -0.721577$. Geoclaw does not predict an arrested state, but a slow creeping motion.

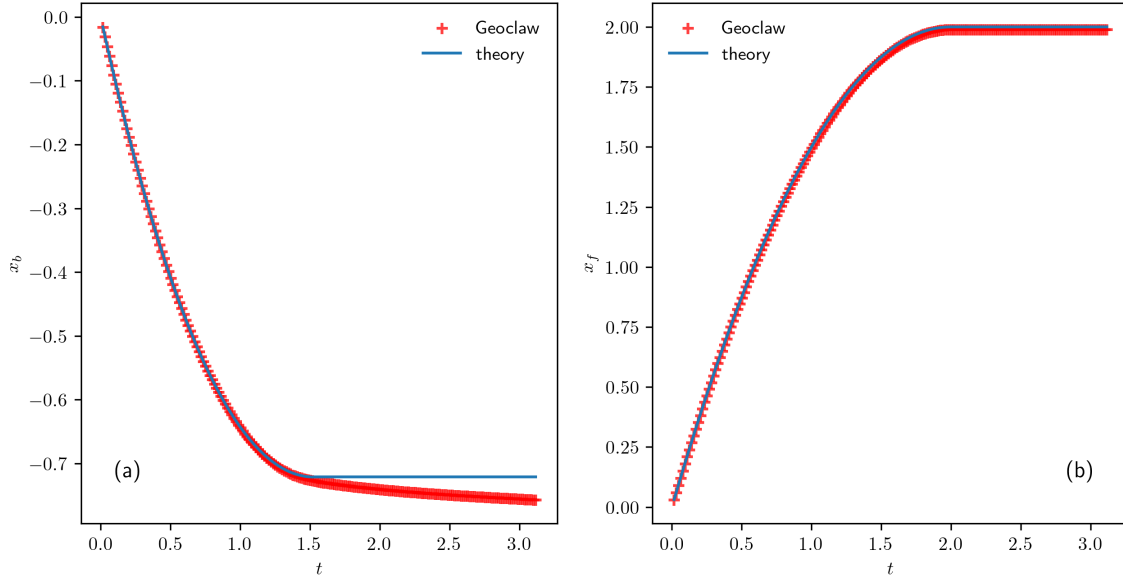


Figure 8.10 (a) Time evolution of $x_b(t)$. (b) Time evolution of $x_f(t)$.

8.2.6 Summary

Time $t(a, b)$ is given by Eq. (8.89):

$$t(a, b) = (b - 1)R(2 - b, b, a, b) + \int_1^b B(r, a, b) dr$$

where now B is given by Eq. (8.91):

$$B(r, a, b) = R(2 - r, r, a, b)[3r - 4 + t_r(2 - r, r)] - [R_s(2 - r, r, a, b) + R_r(2 - r, r, a, b)](r - 1).$$

We can now deduce $x(a, b)$ by integrating the s -characteristic Eq. (8.54):

$$x(a, b) = \int_1^b \frac{\partial x}{\partial r} dr = \int_1^b \left(\frac{1}{2}r - \frac{1}{2}a - t(a, r) \right) \frac{\partial t}{\partial r}(a, r) dr, \quad (8.102)$$

since $x(a, 1) = 0$. We obtain by integrating this equation by parts:

$$\begin{aligned} x(a, b) &= \frac{1}{2}[rt]_1^b - \frac{1}{2} \int_1^b t(a, r) dr - \frac{3a}{2}[t]_1^b - \frac{1}{2}[t^2]_1^b, \\ &= \frac{b - 3a}{2}t(a, b) - \frac{1}{2} \int_1^b t(a, r) dr - \frac{1}{2}t^2(a, b). \end{aligned} \quad (8.103)$$

By solving the equation

$$r - s - t(r, s) = 0$$

numerically, we can determine the equation $r = r(s)$ of the curve FB in Fig. 8.8. A numerical approximation is:

$$r(s) = 1.544 + 0.871s + 0.273s^2 + 0.029s^3 + 0.090s^4 + 0.010s^5. \quad (8.104)$$

Figure fig:C8:coulomb-u-h shows the theoretical flow-depth and velocity profiles and compares them with the numerical solutions obtained using Geoclaw.

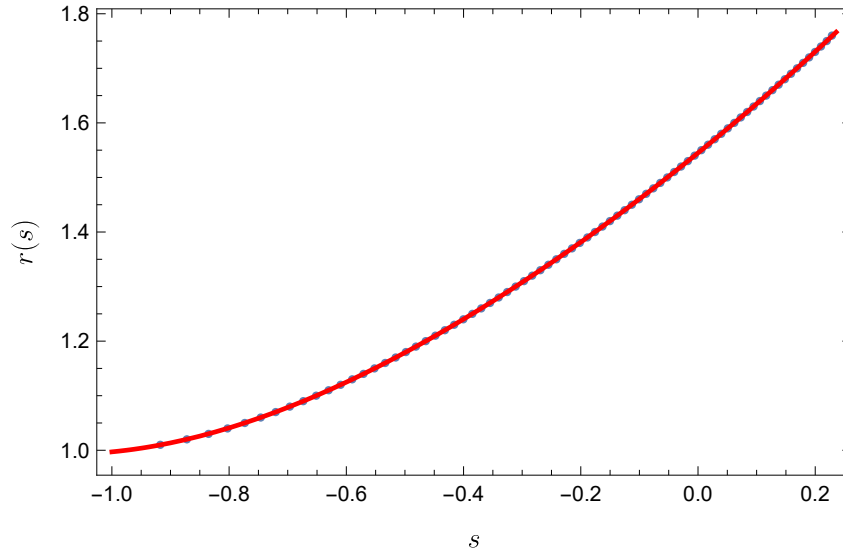


Figure 8.11 Plot of the curve $r = r(s)$ of the curve FB. The dots are the numerical solution to $r - s - t(r, s) = 0$. The red line shows the numerical approximation (8.104).

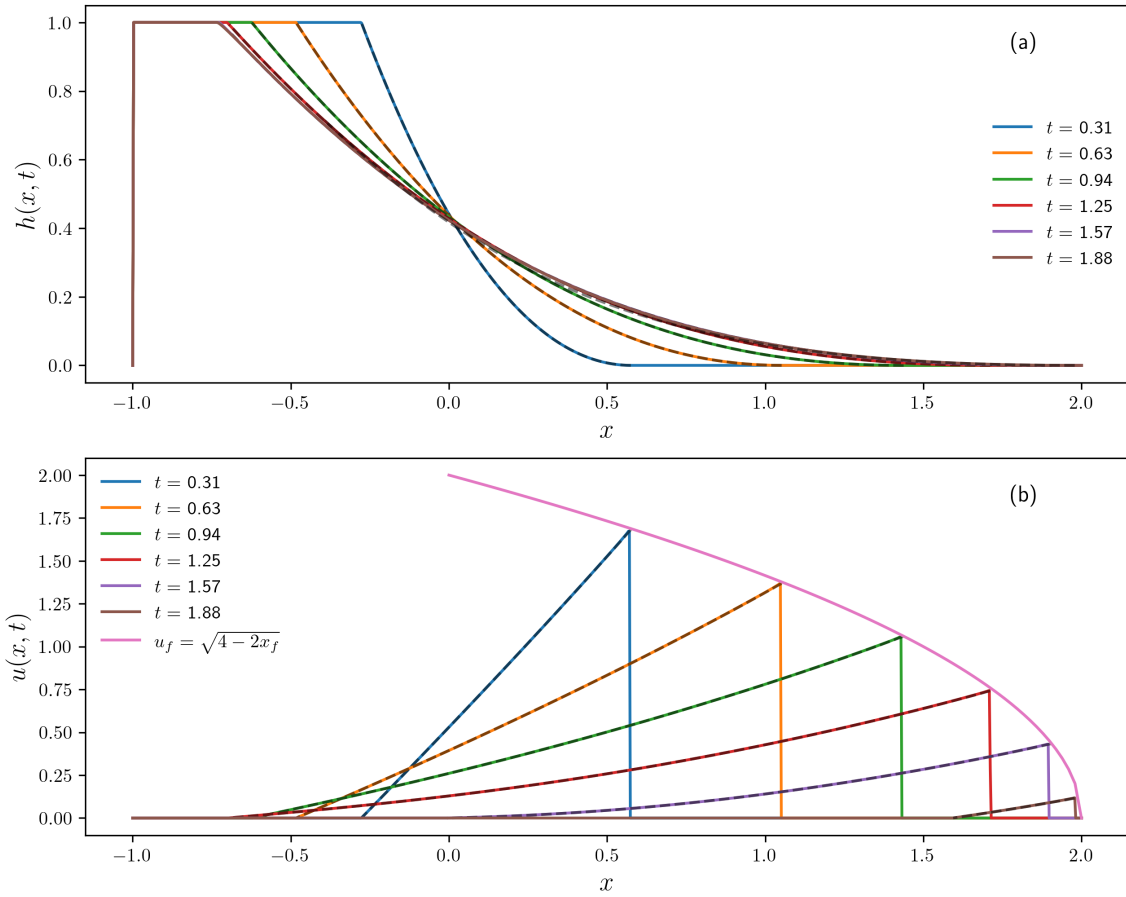


Figure 8.12 Flow depth (a) and velocity (b) profiles at times $t = 0.31, 0.63, 0.94, 1.25, 1.57, 1.88$. The solid line shows the curve $u(x_f) = \sqrt{4 - 2x_f}$. Simulations made with a mesh size $\delta x = 10^{-3}$ m, final computational time $t = 10$ s, domain $[-10, 30]$, initial depth $h_0 = 1$ m. Results were recorded every $dt = 5 \times 10^{-2}$ s. The solid lines are the theoretical solutions, while the dashed lines are the Geoclaw solutions.

8.3 Flow resistance

In an one-dimensional fixed Cartesian frame, the Saint-Venant equations take the tensorial form

$$\frac{\partial}{\partial t}Q + \nabla f(Q) = S, \quad (8.105)$$

where $Q = (h, hu)$ is the unknown, and $S = (0, S)$ is the source term. The computation strategy involves first solving the homogenous problem (LeVeque, 2002) :

$$\frac{\partial}{\partial t}Q + \nabla f(Q) = 0, \quad (8.106)$$

then correcting the solution by taking the effect of the source term on the momentum $q = hu$:

$$\varrho \frac{d}{dt}q = S(Q), \quad (8.107)$$

where $S(Q)$ takes the following form if we consider a flow experiencing flow resistance:

$$S(U) = -\frac{\varrho g}{K^2 h^{1/3}} |u|u, \quad (8.108)$$

$$= -\frac{\varrho g}{K^2 h^{7/3}} |q|q, \quad (8.109)$$

where K is the Manning-Strickler coefficient.

Let us assume that we have computed the solution q_* to the homogenous equation (8.106), and we are now seeking the solution at time $k + 1$. Using a semi-implicit discretization of (8.107) leads to

$$q^{k+1} = q^* - dt \frac{g}{K^2 h^{7/3}} |q^*| q^{k+1}, \quad (8.110)$$

$$q^* = q^{k+1} \left(1 + \frac{g dt}{K^2 h^{7/3}} |q^*| \right), \quad (8.111)$$

$$q^{k+1} = \frac{q^*}{1 + dt \frac{g}{K^2 h^{7/3}} |q^*|}. \quad (8.112)$$

- ABRAMOWITZ, M. & STEGUN, I. A. 1964 *Handbook of Mathematical Functions*. Washington: National Bureau of Standards.
- ANCEY, C., IVERSON, R. M., RENTSCHLER, M. & DENLINGER, R. P. 2008 An exact solution for ideal dam-break floods on steep slopes. *Water Resources Research* **44**, W01430.
- ANTUONO, M. 2010 A shock solution for the nonlinear shallow water equations. *Journal of Fluid Mechanics* **658**, 166–187.
- BERGER, M. J., GEORGE, D. L., LEVEQUE, R. J. & MANDLI, K. T. 2011 The GeoClaw software for depth-averaged flows with adaptive refinement. *Advances in Water Resources* **34**, 1195–1206.
- BERGER, M. J. & LEVEQUE, R. J. 1998 Adaptive mesh refinement using wave-propagation algorithms for hyperbolic systems. *SIAM Journal on Numerical Analysis* **35** (6), 2298–2316.
- BLADÉ, E., CEA, L., CORESTEIN, G., ESCOLANO, E., PUERTAS, J., VÁQUEZ-CENDÓN, E., DOLZ, J. & COLL, A. 2014 Iber: herramienta de simulación numérica del flujo en ríos. *Revista Internacional de Métodos Numéricos para Cálculo y Diseño en Ingeniería* **30**, 1–10.
- CEA, L. & BLADÉ, E. 2015 A simple and efficient unstructured finite volume scheme for solving the shallow water equations in overland flow applications. *Water Resources Research* **51**, 5464–5486.
- CUNGE, J. A., HOLLY, F. M. & VERWEY, A. 1980 *Practical Aspects of Computational River Hydraulics*. Boston: Pitman publishing.
- EINFELDT, B. 1988 On Godunov-type methods for gas dynamics. *SIAM Journal on Numerical Analysis* **25** (2), 294–318.
- FERRICK, M. G. 2005 Simple wave and monoclinal wave models: River flow surge applications and implications. *Water Resources Research* **41**, W11402.
- FOWLER, A. C. 2011 *Mathematical Geoscience*. Berlin: Springer.
- GARABEDIAN, P. R. 1964 *Partial Differential Equations*. New York: John Wiley & Sons.
- GEORGE, D. L. 2006 Finite Volume Methods and Adaptive Refinement for Tsunami Propagation and Inundation. PhD thesis, University of Washington.
- GEORGE, D. L. 2008 Augmented Riemann solvers for the shallow water equations over variable topography with steady states and inundation. *Journal of Computational Physics* **227**, 3089–3113.
- GEORGE, D. L. 2011 Adaptive finite volume methods with well-balanced Riemann solvers for modeling floods in rugged terrain: Application to the Malpasset dam-break flood (France, 1959). *International Journal for Numerical Methods in Fluids* **66**, 1000–1018.

- GODUNOV, S. K. 1959 Finite difference method for numerical computation of discontinuous solutions of the equations of fluid dynamics (English translation by I. Bohachevsky). *Matematicheskij sbornik* 47, 271–376.
- GODUNOV, S. K. 1962 The problem of a generalized solution in the theory of quasilinear equations and in gas dynamics. *Uspekhi Matematicheskikh Nauk* 17 (3), 147–158.
- GUINOT, V. 2010 *Wave Propagation in Fluids—Models and Numerical Techniques*. Hoboken: John Wiley & Sons.
- HARTEN, A. 1983 High resolution schemes for hyperbolic conservation laws. *Journal of computational physics* 49, 357–393.
- HARTEN, A., LAX, P. D. & VAN LEER, B. 1983 On upstream differencing and Godunov-type schemes for hyperbolic conservation laws. *SIAM Review* 25, 35–61.
- HOLDEN, H. & RISEBRO, N. H. 2015 *Front Tracking for Hyperbolic Conservation Laws*, vol. 152. Berlin: Springer.
- KALISCH, H., MITROVIC, D. & TEYEPITI, V. 2017 Delta shock waves in shallow water flow. *Physics Letters A* 381 (13), 1138–1144.
- KERSWELL, R. R. 2005 Dam break with Coulomb friction: a model for granular slumping? *Physics of Fluids* 17, 057101.
- KETCHESON, D. I., GOTTLIEB, S. & MACDONALD, C. B. 2011 Strong stability preserving two-step Runge–Kutta methods. *SIAM Journal on Numerical Analysis* 49 (6), 2618–2639.
- KETCHESON, D. I., LEVEQUE, R. J. & DEL RAZO, M. 2020 *Riemann Problems and Jupyter Solutions*. Philadelphia: Society for Industrial and Applied Mathematics.
- KETCHESON, D. I., MANDLI, K. T., AHMADIA, A. J., ALGHAMDI, A., QUEZADA DE LUNA, M., PARSANI, M., KNEPLEY, M. G. & EMMETT, M. 2012 PyClaw: Accessible, extensible, scalable tools for wave propagation problems. *SIAM Journal on Scientific Computing* 34 (4), C210–C231.
- KETCHESON, D. I., PARSANI, M. & LEVEQUE, R. J. 2013 High-order wave propagation algorithms for hyperbolic systems. *SIAM Journal on Scientific Computing* 35 (1), A351–A377.
- VAN LEER, B. 1979 Towards the ultimate conservative difference scheme. V. A second-order sequel to Godunov’s method. *Journal of computational Physics* 32 (1), 101–136.
- LEVEQUE, R. J. 1992 *Numerical Methods for Conservation Laws*. Basel: Birkhäuser.
- LEVEQUE, R. J. 2002 *Finite Volume Methods for Hyperbolic Problems*. Cambridge: Cambridge University Press.
- MANDLI, K. T., AHMADIA, A. J., BERGER, M., CALHOUN, D., GEORGE, D. L., HADJIMICHAEL, Y., KETCHESON, D. I., LEMOINE, G. I. & LEVEQUE, R. J. 2016 Clawpack: building an open source ecosystem for solving hyperbolic PDEs. *PeerJ Computer Science* 2, e68.
- PAULSEN, M. O. & KALISCH, H. 2020 Admissibility conditions for Riemann data in shallow water theory. *Zeitschrift für Naturforschung* 75A (7), 637–648.
- PERUZZETTO, M., MANGENEY, A., BOUCHUT, F., GRANDJEAN, G., LEVY, C., THIERY, Y. & LUCAS, A. 2021 Topography curvature effects in thin-layer models for gravity-driven flows without bed erosion. *Journal of Geophysical Research: Earth Surface* 126, e2020JF005657.

- RAYLEIGH, J. W. S. 1914 On the theory of long waves and bores. *Proceedings of the Royal Society of London series A* **90** (619), 324–328.
- RICHARD, G. L. & GAVRILYUK, S. L. 2013 The classical hydraulic jump in a model of shear shallow-water flows. *Journal of Fluid Mechanics* **725**, 492–521.
- ROE, P. L. 1981 Approximate Riemann solvers, parameters vectors, and difference schemes. *Journal of Computational Physics* **44**, 357–372.
- SAINT-VENANT, A. B. D. 1871 Théorie du mouvement non permanent des eaux, avec application aux crues des rivières et à l'introduction des marées dans leur lit. *Comptes Rendus de l'Académie des Sciences, série I* **173**, 147–154– 237–240.
- SHU, C.-W. 1998 Essentially Non-oscillatory and Weighted Essentially Non-oscillatory Schemes for Hyperbolic Conservation Laws. In *Advanced Numerical Approximation of Nonlinear Hyperbolic Equations, Lectures given at the 2nd Session of the Centro Internazionale Matematico Estivo (C. I. M. E.) held in Cetraro, Italy, June 23-28, 1997* (ed. A. Quarteroni), , vol. 1697. Berlin: Springer.
- SHU, C.-W. 2020 Essentially non-oscillatory and weighted essentially non-oscillatory schemes. In *Acta Numerica*, , vol. 29, pp. 701–762. Cambridge University Press.
- SMOLLER, J. 1982 *Shock Waves and Reaction-Diffusion Equations*. New York: Springer.
- STOKER, J. J. 1957 *Water Waves*. New York: Interscience Publishers.
- SWEBY, P. K. 1984 High resolution schemes using flux limiters for hyperbolic conservation laws. *SIAM journal on numerical analysis* **21** (5), 995–1011.
- TORO, E. F. 2001 *Shock-Capturing Methods for Free-Surface Shallow Flows*. Chichester: Wiley.
- TORO, E. F. 2009 *Riemann Solvers and Numerical Methods for Fluid Dynamics—A Practical Introduction*. Dordrecht: Springer.
- TORO, E. F. 2019 The HLLC Riemann solver. *Shock Waves* **29**, 1065–1082.
- TORO, E. F. & GARCIA-NAVARRO, P. 2007 Godunov-type methods for free-surface shallow flows: A review. *Journal of Hydraulic Research* **45**, 736–751.
- WHITHAM, G. B. 1974 *Linear and Nonlinear Waves*. New York: John Wiley & Sons.
- ZWILLINGER, D. 1992 *Handbook of differential equations*. Boston: Academic Press.

- accuracy, 50, 69, 77, 88
- AMRClaw, 88, 97
- animation, 153
- averaging, 47, 50, 52
- beam warming, 69
- boundary, 79
 - absorbing, 80
 - condition, 79
 - Dirichlet, 83
 - extrapolation, 80
 - inflow, 83
 - non-reflecting, 80
 - periodic, 80
 - transmissive, 80
 - wall, 80
- cavity, 36
- cell, 47, 79
- CFL, 48
 - see Courant–Friedrichs–Lewy, 50
- characteristic
 - curve, 6, 9, 20, 51
 - form, 4, 9, 20
 - variable, 1
- Clawpack, 7, 51, 58–60, 84, 86, 94, 108, 179
- Cole–Hopf
 - transformation, 144
- condition
 - depth positivity, 35, 168
 - entropy, 58
 - Lax, 12
 - Lax entropy, 14
 - Rankine–Hugoniot, 39
- conservative, 8
- contact
 - discontinuity, 38
- contact discontinuity, 191
- convexity, 8
- Courant–Friedrichs–Lewy, 50, 55
- curve
 - characteristic, 14
 - integral, 15, 17, 29
- dam break, 39, 189, 201
- dclaw, 90
- decomposition, 6, 51
- degenerate, 191
- diffusion, 78, 144
- discontinuity
 - contact, 191, 196
- dry
 - bed, 36, 168
 - cavity, 36, 168
- eigenvalue, 1
- eigenvector, 2
 - left, 1
 - right, 1, 6
- Einfeldt, 183
- energy, 20, 24
 - specific, 20
- entropy, 14, 21, 24, 58
 - condition, 22, 24
 - fix, 59, 61, 76, 135, 178, 179
- environment
 - variable, 85
- equation
 - advection, 8
 - Burgers, 133
 - conservative spatially-varying advection, 121
 - diffusion, 144
 - homogenous, 224
 - nonlinear, 8
 - Rankine–Hugoniot, 9, 14, 22
 - Saint-Venant, 19, 82, 167, 191, 224
 - shallow water, 19, 167, 191
 - spatially-varying advection, 117
 - tracer, 191
- f-wave, 63
- f2py, 88, 123
- factor
 - integrating, 14
- flow resistance, 224
- fluctuation, 53, 58, 60, 69

- flux, 8
 - interface, 54
- form
 - conservative, 8, 12
- Fromm, 69
- GeoClaw, 88
- ghost, 47, 79
- Godunov
 - method, 50, 51
 - see also solver, 50
- Harten
 - theorem, 71
- heteroclinic, 27
- high-resolution, 70
- homogenous, 1
- Hugoniot
 - locus, 14, 23
- hyperbolic, 1
- kernel, 144
- LAPACK, 145
- Lapack, 86
- Lax, 11, 14
 - condition, 12
 - entropy, 11, 33, 34, 58
- limiter, 51, 74, 76
 - maxmod, 69
 - MC, 70
 - minmod, 69, 70
 - superbee, 69, 70
- linearisation, 63
- linearity
 - degenerate, 15
- Makefile, 86
- Manning–Strickler, 224
- maxmod, 69
- mesh, 47, 88
 - refinement, 88
- method
 - Adams–Moulton, 160
 - backward Euler, 160, 163
 - beam-warming, 70
 - Brent, 163
 - Crank–Nicolson, 78, 145, 148
 - Euler, 77
 - f-wave, 63
 - flux limiter, 70
 - fractional step, 77
 - Fromm, 70
 - high-resolution, 70
 - implicit, 78
 - Lax–Wendroff, 69, 70, 73
 - of characteristics, 140
 - Runge–Kutta, 78
 - slope limiter, 69
 - Strang splitting, 78
 - total-variation diminishing, 71
 - TR-BDF2, 78
 - trapezoidal, 78, 160
 - upwind, 48, 77
 - well-balanced, 79
- minmod, 69
- modal
 - matrix, 2
- MUSCL, 69
- nonlinearity
 - genuine, 15, 30
- orbit, 27
- PETSc, 88
- phase
 - plane, 7, 27
 - portrait, 31
- problem
 - Cauchy, 4
 - Riemann, 1, 6, 11, 33, 121, 133, 168
- Pyclaw, 88, 99, 109, 148, 155, 182, 184, 186, 195, 198
- PyWENO, 88
- rarefaction
 - fan, 40
- rarefaction wave
 - admissible, 31
- regularisation, 10, 25
- Riemann
 - invariant, 12, 14, 16, 29
 - problem, 6, 11, 33, 51, 168
 - variable, 1, 12, 14, 19
- Roe, 61
 - matrix, 177, 184, 186, 199
- Saint-Venant, see equation
- shallow water, see equation
- Sharpclaw, 76, 88, 103
- shock
 - admissible, 22, 24, 25
 - wave, 22
- slope
 - downwind, 69, 70

- upwind, 69, 70
- smoothness, 74
- solution
 - Ritter, 44
 - similarity, 44
- solver
 - approximate, 57
 - exact, 39
 - f-wave, 63, 135, 186, 199
 - HLL, 55, 61, 63, 135, 183
 - HLLC, 196
 - HLLE, 63, 183
 - linearised, 61
 - Roe, 61, 62, 135, 175, 192, 201
 - two-wave, 61, 183
- source
 - diffusion, 144
 - term, 1, 64, 77, 117, 121, 144, 148, 224
- spectral
 - matrix, 2, 53
- stability, 51
- state
 - dry, 33
 - intermediate, 7, 33, 34, 39
- stiff, 78
- system
 - nonlinear, 12
- theorem
 - Harten, 71
- total variation, 71
- transonic, see wave
- TVD, 71
- Visclaw, 110
- viscosity, 25, 144
- wave, 6, 51, 53
 - acoustic, 107
 - advection, 93
 - monoclinal, 25
 - rarefaction, 11, 12, 16, 23, 29, 30, 40, 57, 60, 76, 133
 - shock, 9, 11, 12, 14, 22, 39, 57, 60, 133
 - simple, 5, 15, 16
 - transonic, 51, 57, 59–61, 76, 133, 135, 178
 - travelling, 10, 26, 145
- well-balanced, 64
- WENO, 103
- wet
 - bed, 40, 168